One Identity Manager 8.0

# Synchronization Technology

## A walkthrough and reference

**Legend**

**WARNING: A WARNING icon indicates a potential for property damage, personal injury, or death.**

**CAUTION: A CAUTION icon indicates potential damage to hardware or loss of data if instructions are not followed.**

IMPORTANT, NOTE, TIP, MOBILE, or VIDEO: An information icon indicates supporting information.

# Contents

# Introduction

One of the key features of a One Identity installation is synchronization and provisioning with IT systems in the company. For a lot of systems, One Identity Manager (referred to as OneIM in this document) provides out of the box pre-configured synchronization templates. When it comes to modifying the synchronization settings or if you have to create a new synchronization project from scratch, it really helps to have some in-depth knowledge about the OneIM synchronization technology. This document will give a detailed step by step introduction to this technology explaining what happens inside the synchronization "engine".

# Structure

The document will explain the parts of the synchronization technology by showing how to create a synchronization project from scratch. As target system, we will use LDAP for various reasons:

1. There are a variety of LDAP servers available for free. This allows you to setup a "free" environment to experiment with.
2. LDAP synchronization projects can get quite complex. Thus, they are using a lot of features of the OneIM synchronization technology.
3. Once you understand how to create a synchronization project for a free LDAP server, other LDAP based services/services that have a LDAP interface will be easy to understand as well (Active Directory, Novell eDirectory, etc.).

When progressing through the configuration steps, the details for each item can be reviewed in the reference section, the explanation of the core synchronization framework concepts or how the synchronization engine is incorporated in OneIM. Screenshots are based on product version 8.0 but the majority of the functionality/UI is already available in previous versions. A troubleshooting section is also available.

> Hint: If you are familiar with Docker, you can use the osixia OpenLDAP container under: https://git.io/vbp4G

# Walkthrough LDAP configuration

This section will be a rather quick walkthrough of a LDAP synchronization project creation from scratch.

## Step 1: Create a new synchronization project

Use the "Synchronization Editor" to create a new synchronization project. It is the main UI component for configuring data synchronization and the first application we will launch in this walkthrough. Configurations are called synchronization projects.

This is what the application looks like when it is started for the first time in a new environment.

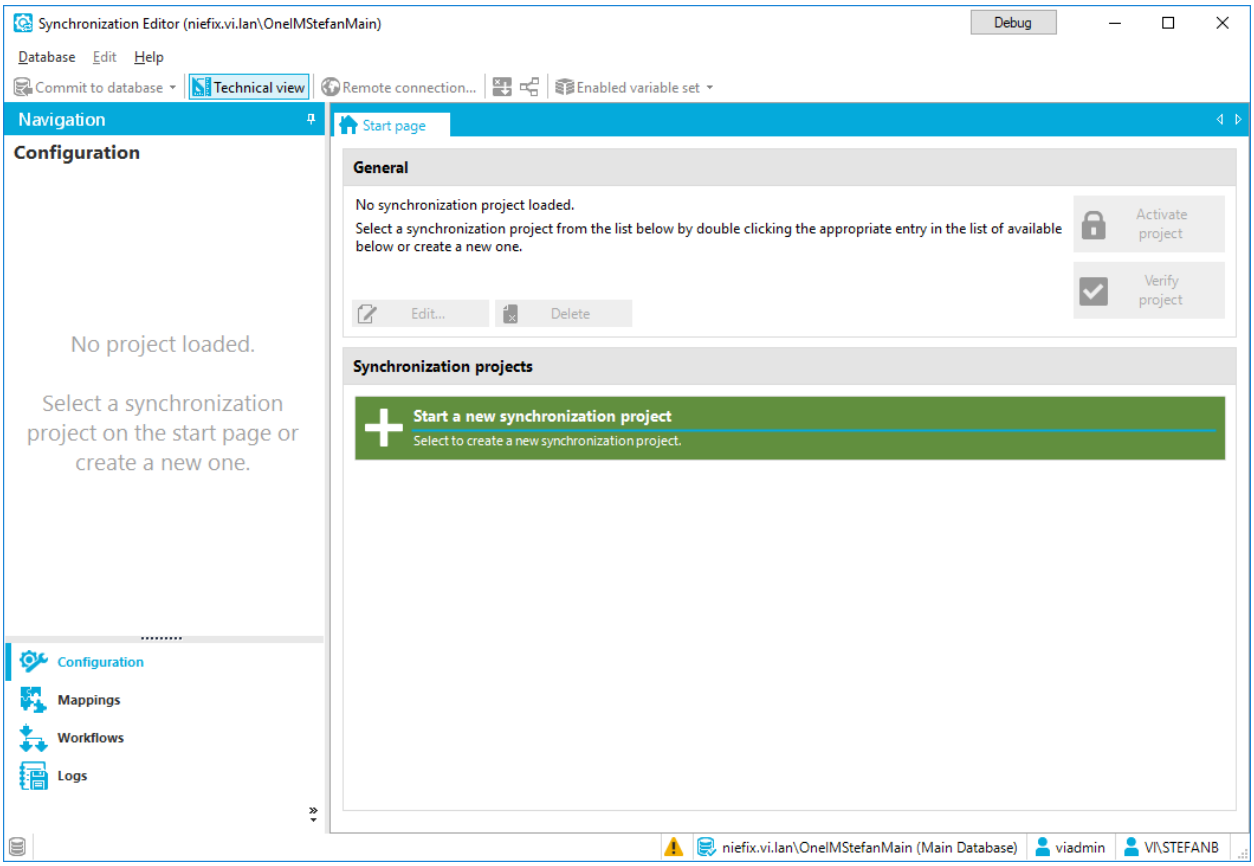



*Figure 1 Synchronization Editor*

In order to see every option this document is referring to, please active the expert mode ("Database" menu → "Settings")

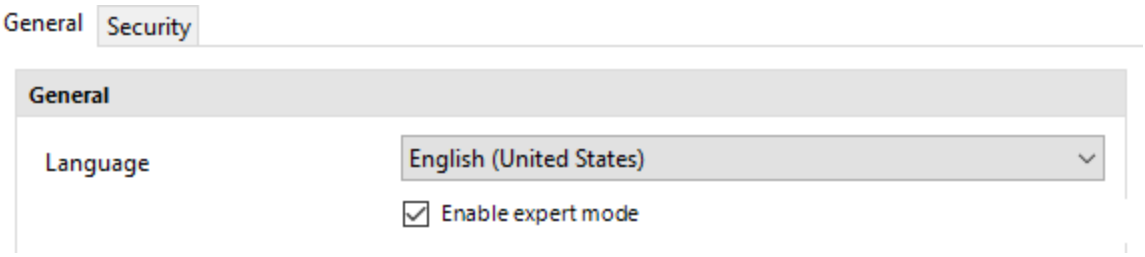



*Figure 2 Expert mode setting*

If you have already worked with the Synchronization Editor before, you know that there is one target system connection per synchronization project. That is actually not entirely true because, from a synchronization engine point of view, there are two connections. The first one, is (always) the connection to the One Identity Manager database. The second connection is the one to the system you want to synchronize with One Identity Manager.

A connection is established by a target system connector. This is basically a library that implements a common interface that is known by the synchronization engine. To keep it simple for now, the methods of this connector interface provide information about the target system and exposes methods to read and write data from and to the system.

The connection to the OneIM side is no exception. The One Identity Manager connector needs to implement the common connector interface too so that the synchronization engine does not have to treat it differently.

Finally, the Synchronization Editor user assumes that only one connection is being established to the target system although there are actually two connections being established. The following image shows the connections of the synchronization project we are going to create.

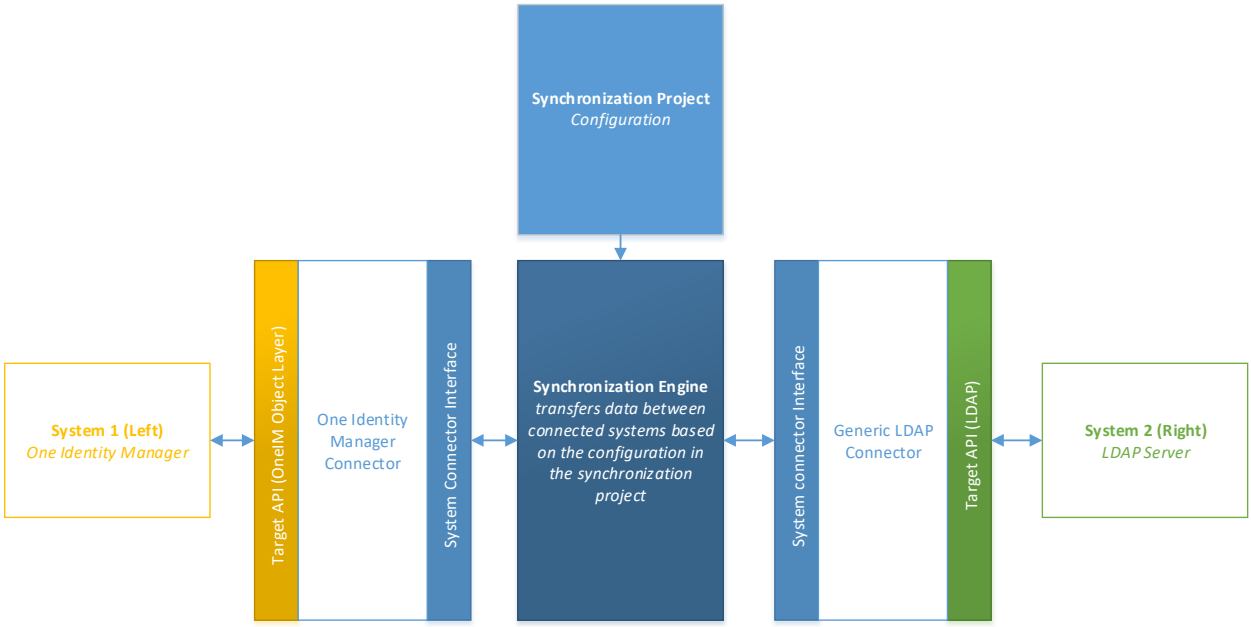



*Figure 3 Connections*

Now it is time to launch the "Start new synchronization project" wizard in the Synchronization Editor. Confirm the "Welcome" page and proceed to target system selection. Choose the "Generic LDAP Connector".

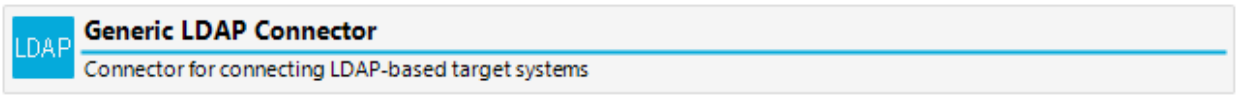



*Figure 4 Generic LDAP Connector*

If you do not see this item in the list, you might not have installed the LDAP module. Once selected, the wizard needs to know, if the machine you are running on can directly access the LDAP system. If not, you can use the remote connection server. What this is and how to configure it, can be found in the documentation of the DPR module.

Once you click next, the LDAP connection wizard appears. Check "Expert mode" to see the advanced connection settings and click "Next".

Enter the network settings for your LDAP and proceed. Once you reach the page "Select project template", choose "Create blank project".

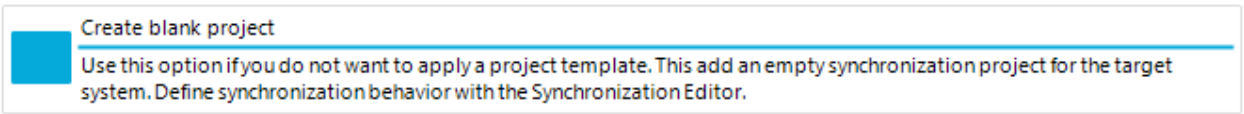



*Figure 5 Template selection*

The next step is to choose a name and a scripting language for your synchronization project. In this document, Visual Basic .NET language will be used because all 'out of the box' templates are created for this language. There are several places in a synchronization project where scripts can be applied. Those places will be described in detail as this walk-through progresses.

Hint: The scripting environment of the synchronization engine has nothing in common with the rest of the OneIM scripting environment. Scripts used in the synchronization framework are compiled on demand so compiling the OneIM database is not required when changing scripts within the Synchronization Editor.

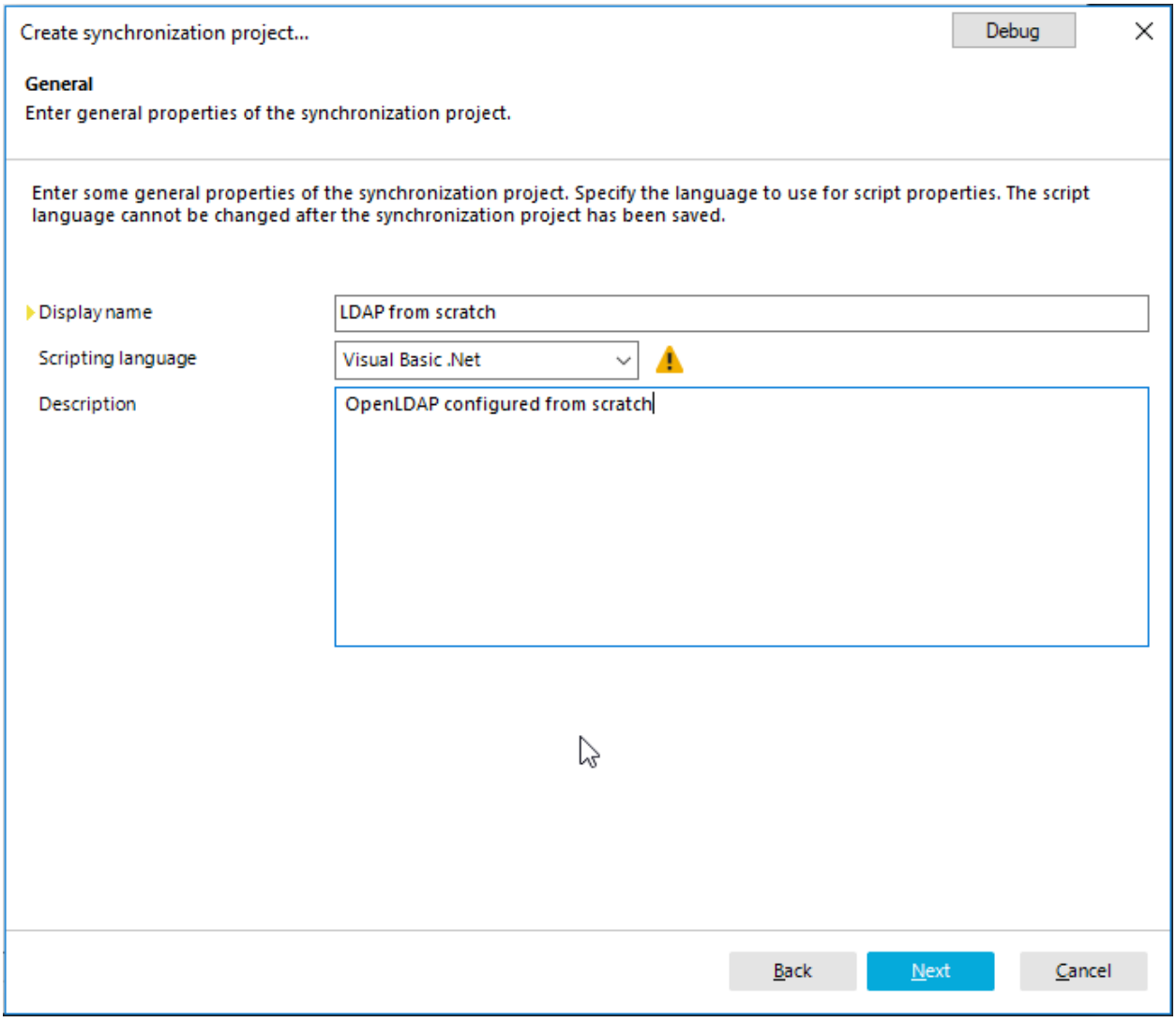
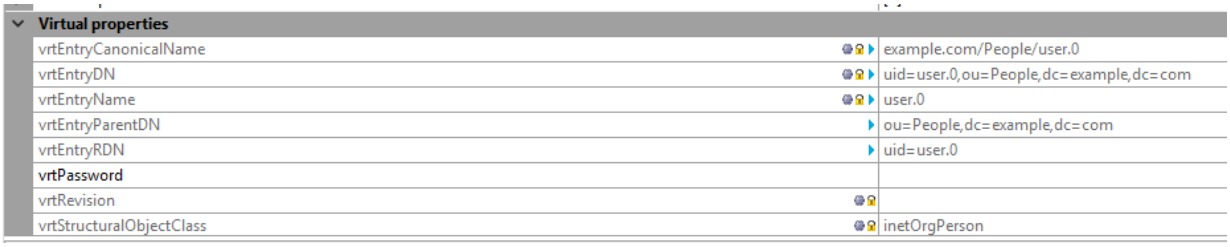



*Figure 6 Synchronization project properties*

## Browse your Target System using the Target System Browser

You should now use the Target System Browser to locate your LDAP Server. This is important to verify your connectivity and permissions.

It also allows you to review the automatically generated, so called, virtual attributes, of the connector. These are attributes, generated by the connector to make your life easier—for example an LDAP entry does not typically have an attribute storing its DN or RDN or the DN of its parent entry. All these values are needed to define mappings and synchronization behavior, so the connector generates these for later use. The vrtStructuralObjectClass, as the final in the list of inherited structural object classes, can be used as the definitive type of the object, for example.



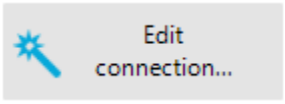

# Step 2: Convert connection information to variables

Once finished with the wizard, you will be able to access the connections to OneIM and LDAP in the "Configuration" section. Internally, the connection is stored as a connection string containing name/value pairs. For LDAP, the connection string will look like this:

**authenticationtype**=Basic;**cacheschema**=False;**clienttimeout**=3600;**guid attribute**=entryUUID;**ismovesupported**=True;**isrenamesupported**=True;**pagesize**=500;**password**=<yourpassword>;**passwordattribute**=userPassword;**port**=389;**protocolversion**=3;**revisionattributes**=createTimestamp%0007modifyTimestamp;**rootentry**="dc=OneIM,dc=local";**server**=vidrn303.vi.lan;**setpwdmethod**=Default;**usepagedsearch**=True;**usepartitionedsearch**=False;**username**="cn=admin,dc=OneIM,dc=local";**usessl**=False;**usestarttls**=False

To modify your connection settings, there are 2 options.

1. Re-run the connection wizard by clicking the "Edit connection" button




2. Convert certain items into a variable and change the variable values. This option is not available for all connection parameters. The available parameters are determined by the connector. To convert variables, navigate to the connection in the configuration section and expand the "Connection parameters" tab. Then click the "Convert" button.

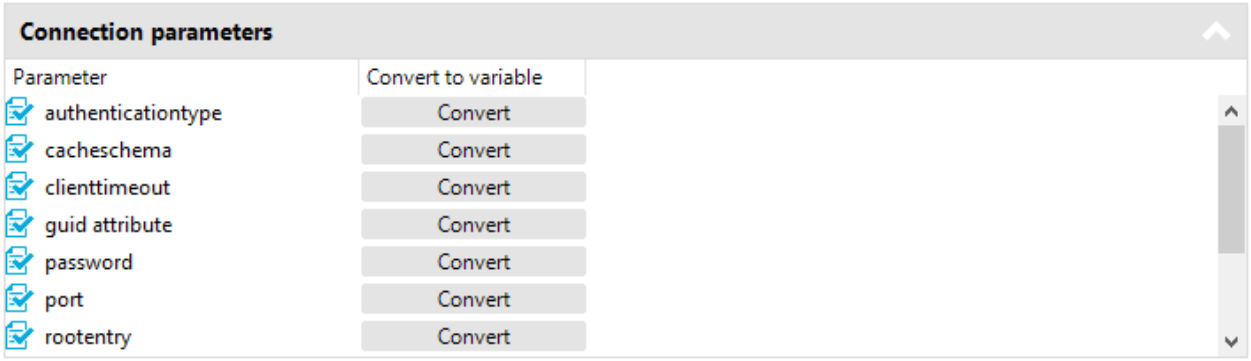



*Figure 7 Connection parameters before conversion*

For our walkthrough we will convert all of the connection parameters to variables.

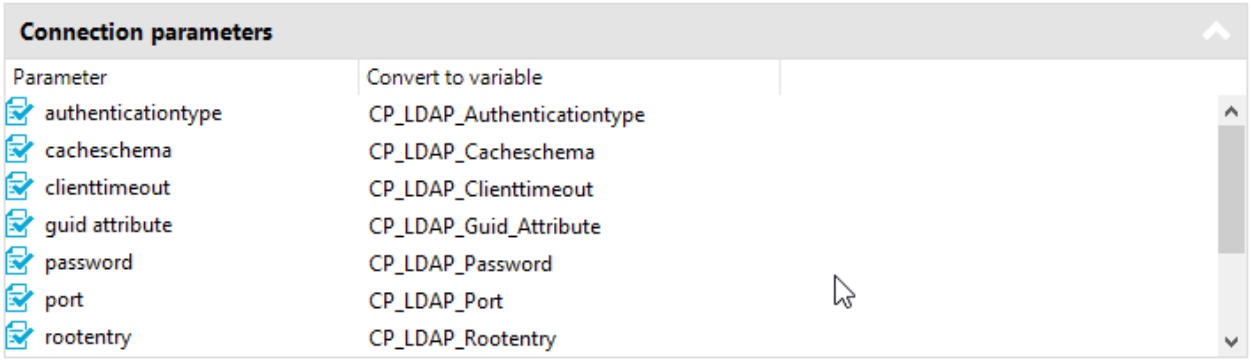
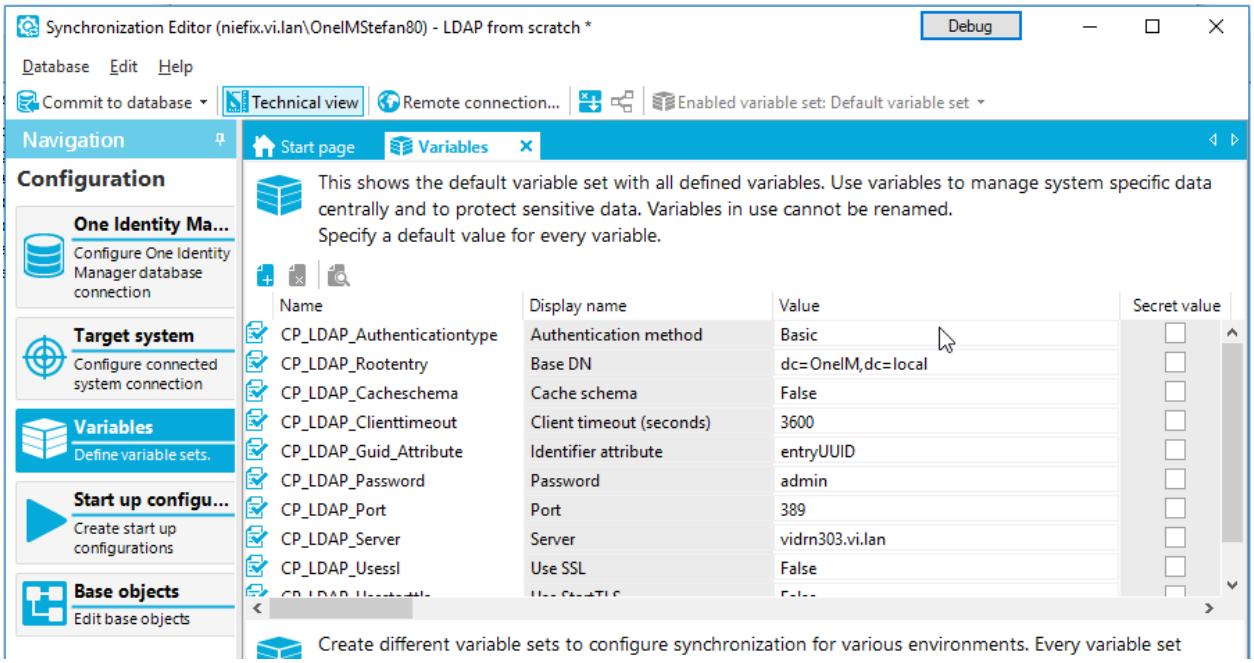



*Figure 8 Connection parameters after conversion*

After you have done this, the variables can be edited in the "Configuration" → "Variables" section.




More information about variables can be found in the variables section.

This will also modify the connection string to the following value:

**authenticationtype**=$CP_LDAP_Authenticationtype$;**cacheschema**=$CP_LDAP_Cacheschema$;**clienttimeout**=$CP_LDAP_Clienttimeout$;**guid attribute**=$CP_LDAP_Guid_Attribute$;**ismovesupported**=True;**isrenamesupported**=True;**pagesize**=500;**password**=$CP_LDAP_Password$;**passwordattribute**=userPassword;**port**=$CP_LDAP_Port$;**protocolversion**=3;**revisionattributes**=createTimestamp%0007modifyTimestamp;**rootentry**=$CP_LDAP_Rootentry$;**server**=$CP_LDAP_Server$;**setpwdmethod**=Default;**usepagedsearch**=True;**usepartitionedsearch**=False;**username**=$CP_LDAP_Username$;**usessl**=$CP_LDAP_Usessl$;**usestarttls**=$CP_LDAP_Usestarttls$

# Step 3: Create the LDAP Domain in OneIM

When dealing with systems where OneIM can handle multiple instances, you need to create an “anchor” or “base” object in the OneIM database first. This is because some tables (like LDAPAccount) store accounts from multiple Domains…and we need to be very clear how to distinguish objects from different Domains.  To do so, open the Manager application and navigate to the “LDAP” section (1). Click “Domains” (2) and then the “New” (3) icon in the toolbar.

Hint: When creating a synchronization project with an out of the box synchronization template, the template takes care of creating the required objects in the OneIM database.

The property you need to pay special attention to is the "Distinguished name" (4) property. For our synchronization example, it has to equal the "Root Entry" setting of your LDAP connection. If you have converted the connection parameters as described in step 2, you can copy the distinguished name from the variable "CP_LDAP_Rootentry".

Since we do not directly modify the OneIM domain object during the synchronization, the other values can be entered as you like.

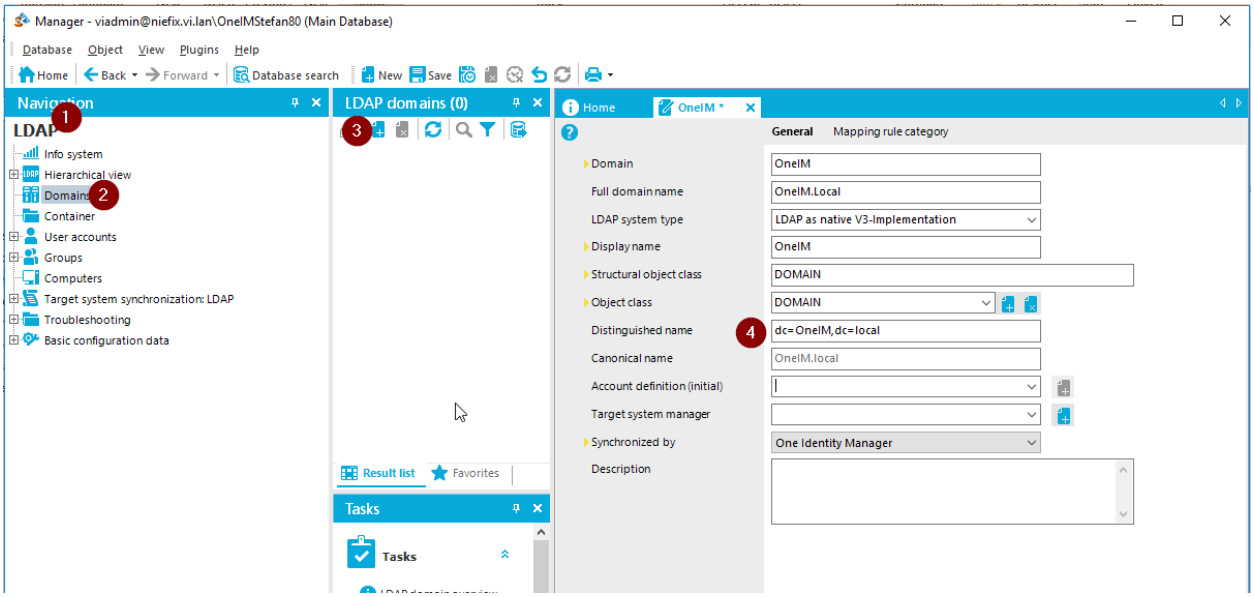



*Figure 9 OneIM LDAP object*

The next thing for this step is to register this OneIM object as "Base object" in the Synchronization Editor. Open your synchronization project in the Synchronization Editor and navigate to the "Configuration → Base objects" section. Then click the "New" button in the toolbar.

Hint: Although available, do **not** use the wizard button here. Its functionality will be covered in the "Base objects" section.

Now enter the following values:

| Setting | Description |
|---|---|
| Base table | The OneIM object type of the base object (in out example, a LDAP domain that is stored in "LDPDomain") |

| | |
|---|---|
| Base object | The instance of the object (select the one that you have just created in the "Manager" application) |
| Synchronization server | Select a synchronization server (if a server is not present yet, you can register one by using the "new" button next to the list. Make sure you actually install it too) |
| Variable set | Leave this empty. The default variable set will be used. |

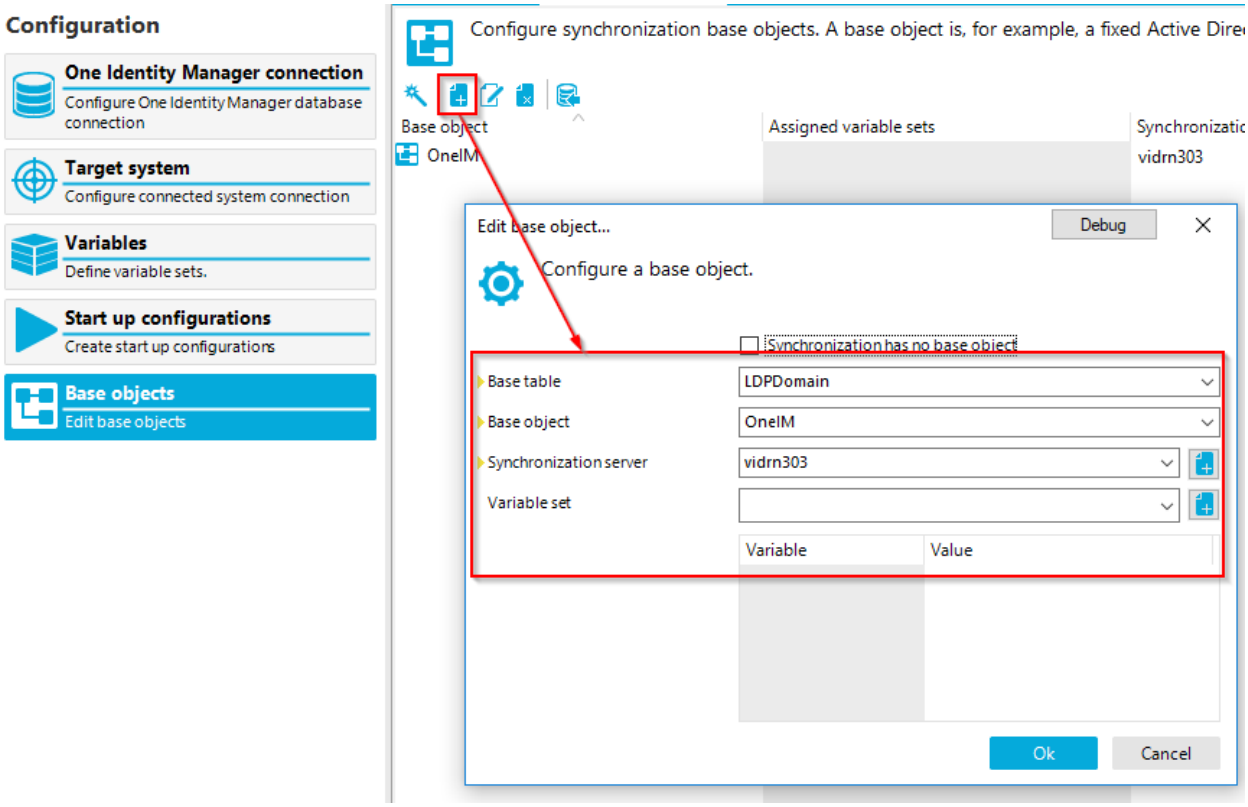



*Figure 10 Create a new base object*

By defining a base object the OneIM connector will not automatically limit the data it loads to this domain. A base object is just a configuration aid that is very closely tied to the OneIM processing engine as well as the OneIM data model. Therefore there is no generic way to calculate a filter based on that. In fact, there can even be scenarios, where you use a whole OneIM table as base object, but need to filter synchronization objects based on an attribute value.

In conclusion, in addition to a base object, a synchronization scope needs to be defined for the OneIM connection.

To do this, navigate to the OneIM connection (1) and click the "Edit Scope" (3) button in the "Scopes" section (2). As you can see in the screen shot, we are configuring here the OneIM side of the project. By convention, this is always the "Left-Hand-Side" of the project.

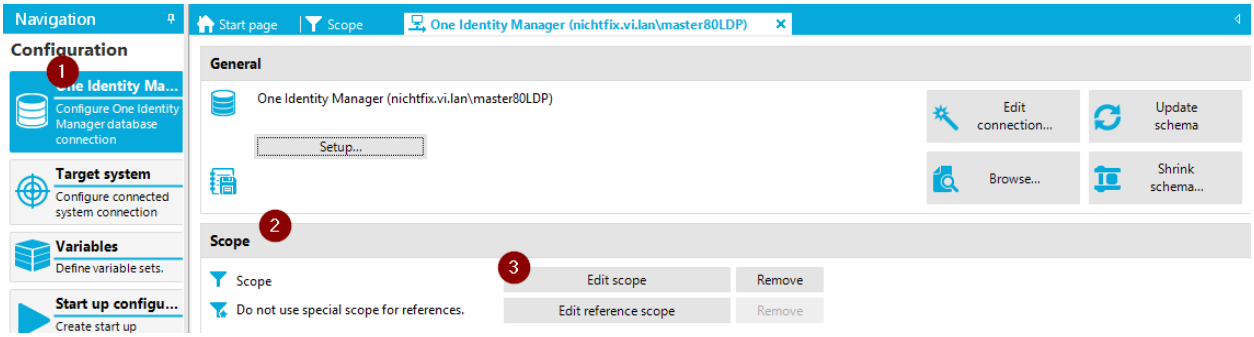



*Figure 11 Open scope definition*

Now scroll to LDPDomain in the scope hierarchy list and enter the following values.

*System filter (SQL Whereclause for OneIM):*

```
upper(DistinguishedName) = upper('$CP_LDAP_Rootentry$')
```

*Object filter*

```
DistinguishedName='$CP_LDAP_Rootentry$'
```

As you can see, the scope also uses the variable *"$CP_LDAP_RootEntry$"* containing the root entry DN. In case you add an additional LDAP domain for synchronization to this synchronization project by defining a new base object in the future, the scope is automatically be adjusted by using the given variable set.

To check if your scope is working, open the target system browser (2) for the OneIM connection (1) and check, if the domain created in the Manager application, appears in the LDPDomain list (3).

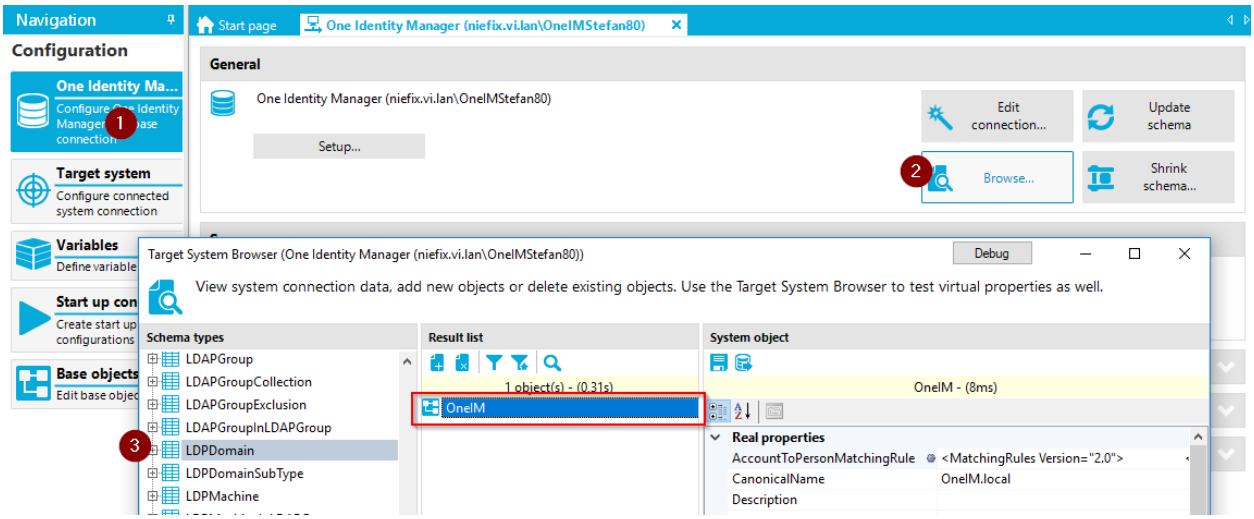



*Figure 12 Check scope*

Hint: When creating a synchronization project with an out of the box synchronization template, the template takes care of configuring the scope for the OneIM connection if required.

## Step 4: Create mappings between OneIM and LDAP objects

Once the connections and scopes are in place, it is time to configure how data is transferred between the two systems and how object pairs can be identified. Those configuration items are called mappings. They can be found in the mapping section of the Synchronization Editor.

When creating synchronization projects from scratch, the first thing that needs to be thought of when it comes to mapping creation is, which schema types do you want to handle in the target system and what will be the corresponding object in OneIM. For our walkthrough, those pairs will be:

| **OneIM** | **LDAP** |
|---|---|
| LDAPAccount | inetOrgPerson |
| LDAPGroup | groupOfNames |
| LDAPGroup | groupOfUniqueNames |
| LDAPContainer | Organization |
| LDAPContainer | OrganizationalUnit |

The number of those defined pairs is also the number of required mappings. So in our example we will need (at least) 5 mappings.

Now it is time to determine, if additional schema classes are required. That is usually the case, when a schema type appears in multiple pairs. In our example, the OneIM schema types “LDAPGroup” and “LDAPContainer” are used multiple times. So for “LDAPGroup” we need to define schema classes that distinguish between “groupOfNames” and “groupOfUniqueNames”: instances of both these object types are mapped to the same OneIM table in the same domain, but we will distinguish them via their schema class. For “LDAPContainer”, we need to define schema classes that distinguish between “organization” and “organizationalunit”.

To do this, check the schema type for a property that might be suited for filtering. The OneIM LDAP* schema types have a property called “Structural object class” that contains the structural object class of the LDAP entry. So, in our case, this is suited to create the generic schema classes for the mappings.

The following sections will demonstrate, how to create each of the mappings. The general approach to creating a mapping will be explained in detail for the first mapping and should be used for the others accordingly.

## LDAPContainer to OrganizationalUnit mapping

The first mapping that we are going to configure is "LDAPContainer ←→OrganizationalUnit". Navigate to the mapping section and click the "New" button. Enter the following values:

| | |
|---|---|
| Mapping name | "**OrganizationalUnit**" (we use the name of the LDAP schema type here since it is unique according to the mapping pairs we've defined) |
| Base mapping | <empty> |
| Mapping direction | "**Both directions**" (since we want to transfer data in either direction) |
| Description | LDAPContainer/organizationalUnit |
| Hierarchy synchronization | **Yes** (since organizations are used mainly for defining the directory tree structure in LDAP) |
| Only suitable for updates | **No** (we can and will create a mapping that works for all CRUD operations) |
| Maps objects referenced by multiple references | **Yes** (although we will not read/write organizations as multi-valued references, it is possible to do this in theory with the mapping we define) |

Now we need to define a generic schema class for OneIM that contains only those LDAPContainer instances that have the "StructuralObjectClass" property set to "organizationalUnit". If you have not done this already, press the "new" icon next to the "One Identity Manager schema class" list. And create the class as follows:

| | |
|---|---|
| Class type | Generic schema class |
| Schema type | LDAPContainer |
| Display name | LDAPContainer(OrganizationalUnit) |
| Class name | LDAPContainer_OrganizationalUnit |
| System filter | upper(StructuralObjectClass)='ORGANIZATIONALUNIT' |
| Select objects (ignore case = true) | StructuralObjectClass='OrganizationalUnit' |

As target system schema class use the "organization (all)" class.

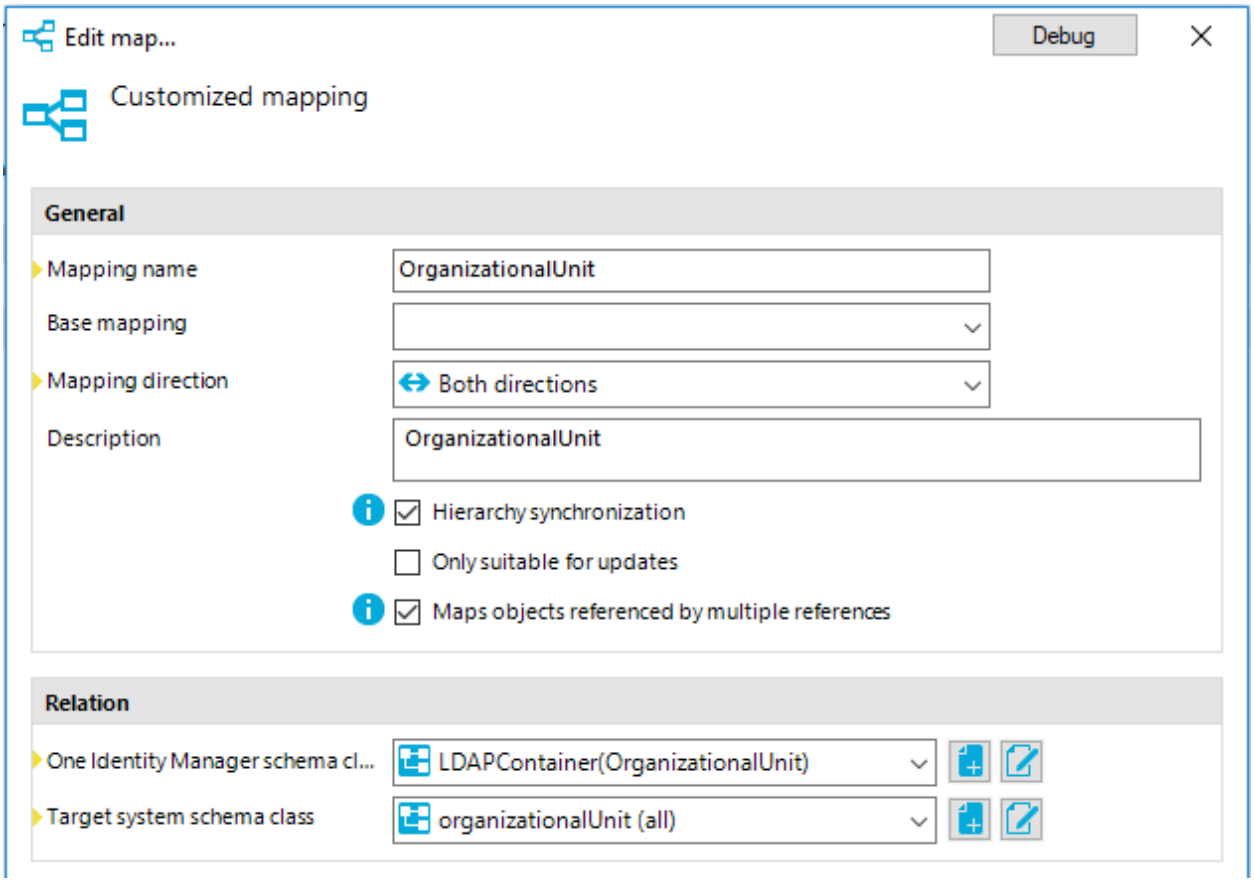



*Figure 13 OrganizationalUnit mapping*

Once you click okay a wizard appears that allows you to select different ways of creating the mapping for each attribute in the targeted object types. Cancel the wizard for now (it can be run at any time), this then opens the mapping definition window.

Now it is time to define the matching rules that are used to identify pairs of objects from both systems. To do so, try to find properties on both sides that have (or are supposed to have) equal values and are unique. In our example we have multiples of those properties that we can use. The most obvious is the distinguishedName property. It exists on both sides (exposed as vrtEntryDN in LDAP) and identifies an object uniquely in LDAP because there cannot be two entries sharing a single distinguished name. However, since a distinguished name can be modified (i.e. when an object is moved) it would be great to have another property that is not alterable. During the LDAP connection wizard, you were asked to specify a unique identifier if you were running the wizard with advanced features turned on. Since the connector tries to detect the name of the attribute automatically, chances are good that it was preselected. However, in step 2, we converted some of the connection parameter contents into variables. The name of the identifier attribute was among them. So, search the project variables for the existence of a variable called "CP_LDAP_Guid_Attribute". If present, it contains the name of the schema property that

contains the unalterable identifier property of the system. This property is exposed by the LDAP connector as virtual property "vrtUniqueIdentifier". So, in conclusion we can match the contents of vrtUniqueIdentifer with the objectGUID property in OneIM. Since this property is more reliable since it is not alterable, it should be preferred over the distinguished name. The distinguished name however should also be used since the guid is read only in LDAP and can therefore not be prepopulated by OneIM which means it is empty when creating a new object in the OneIM database.

These considerations lead to the following two matching rules.

| Prio | OneIM property | LDAP property | Remarks |
|---|---|---|---|
| 1 | ObjectGUID | vrtUniqueIdentifier | Prio 1 since not alterable |
| 2 | Distinguished name | vrtEntryDN | Additional rule because it is always present in both systems |

To actually create the rules, click the "new" icon in the matching rule section and create the appropriate "Value comparison rules". Make sure they are **not** marked as "Case sensitive".

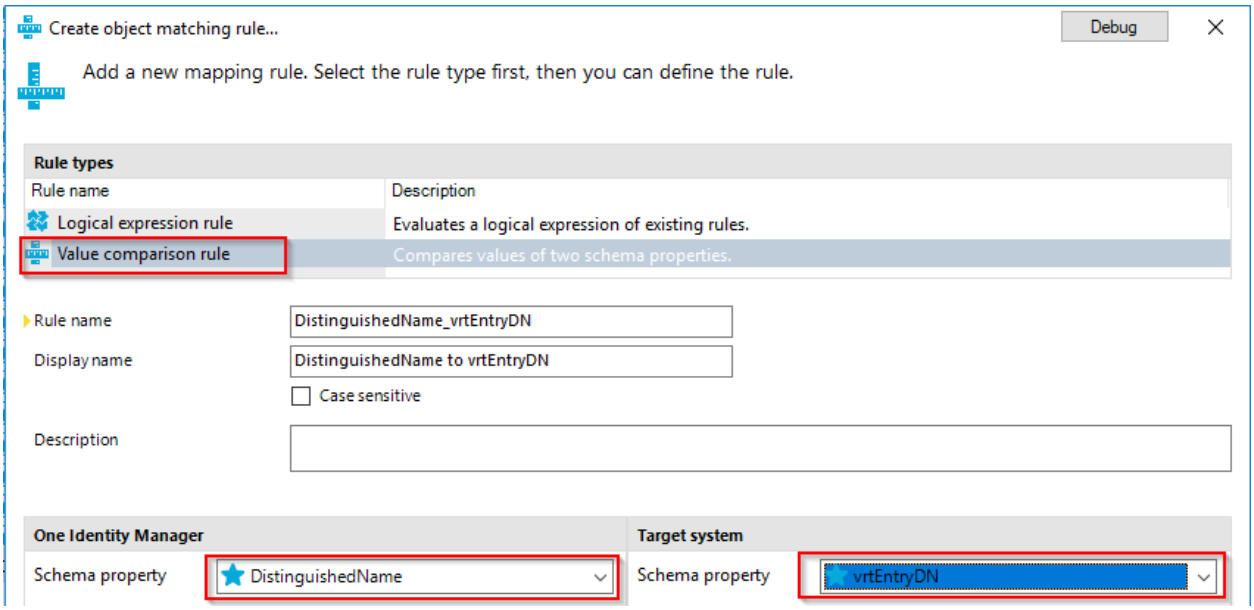



*Figure 14 value comparison rule*

The result looks like this:

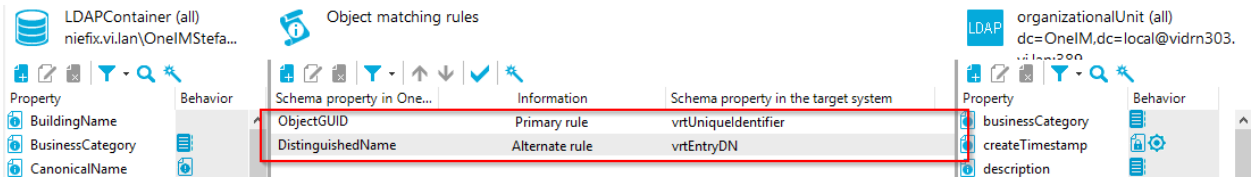


*Figure 15 Matching rules*

The next step is to define the mapping rules for each of the attributes in the target objects. This will determine how data is transferred between the systems. As a first approach focus on all mandatory properties, properties used in the filter of the defined generic schema classes as well as the properties that were used for matching for each side. We only focus on schema properties that have the following attributes:

- Autofillbehavior: Never
- Mandatorybehavior: Always

To evaluate this, you can look at the behavior column next to the property.

> Hint: The behavior column contains icons for the most commonly attributes of a property. By hovering the mouse of an icon, you'll get a tooltip with a brief explanation.

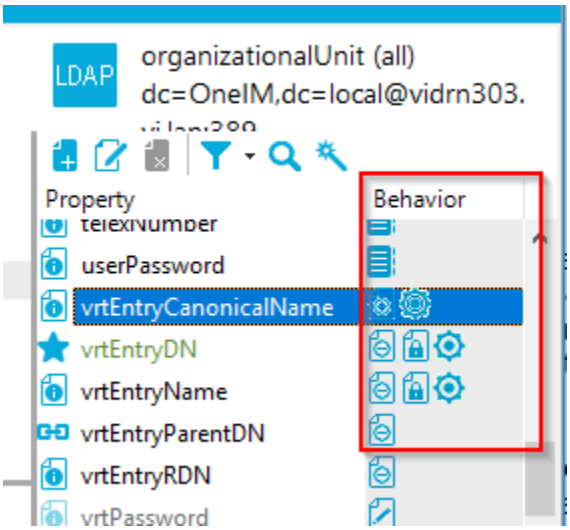



*Figure 16 Behavior column*

For "LDAPContainer" (OneIM) those properties are "cn", "DistinguishedName", "ObjectClass", "UID_LDPDomain" and "ObjectGUID".

For "OrganizationalUnit" (LDAP) they are "objectClass", "vrtEntryRDN", "vrtEntryParentDN" and "ou".

So now create mapping rules for those properties. In most cases, you can use drag and drop to do this. This will create "Value comparison mapping rules". However, the default settings for drag and drop create a case sensitive mapping rule that has the "Ignore mapping direction restriction on adding" option set and inherits the mapping direction from

the mapping. Those options need to be adjusted on demand. In LDAP, case sensitive comparison is very uncommon. Therefore, this setting should be removed. Also, remove the **Error! Reference source not found.** when creating rules with drag and drop.  Please refer t o the link for details on the meaning of this option.

The most obvious rule is the one for "**objectClass**". Both properties are multi-valued strings. Drag and drop them to map those properties together. Do not forget to remove the case sensitivity option.

Next, create a rule that maps "**DistinguishedName**" from LDAPContainer with "**vrtEntryDN**" from LDAP. The created rule will automatically have the mapping direction set to "One Identity Manager" since the property is read only in LDAP. Also set the "Force mapping against direction of synchronization" to enable the instant reload of this property during provisioning. Map LDAPContainer "**ObjectGUID**" with "**vrtUniqueIdentifier"** with the same settings. (Note, if vrtUniqueIdentifier is not a string, you need to create an additional virtual property that converts the native value to string. Sometimes those identifiers are of type "binary" or "Integer")

The OneIM property "**cn"** can be mapped to LDAP "**ou"** since they have the same meaning. Those properties are not compatible since cn is single valued and ou is multi valued. When you drag and drop, a Property mapping rule conflict wizard will open. Choose the first option, which will set the "Handle first property value as single value" option of the value comparison rule. The other option would create virtual MVP converter property.

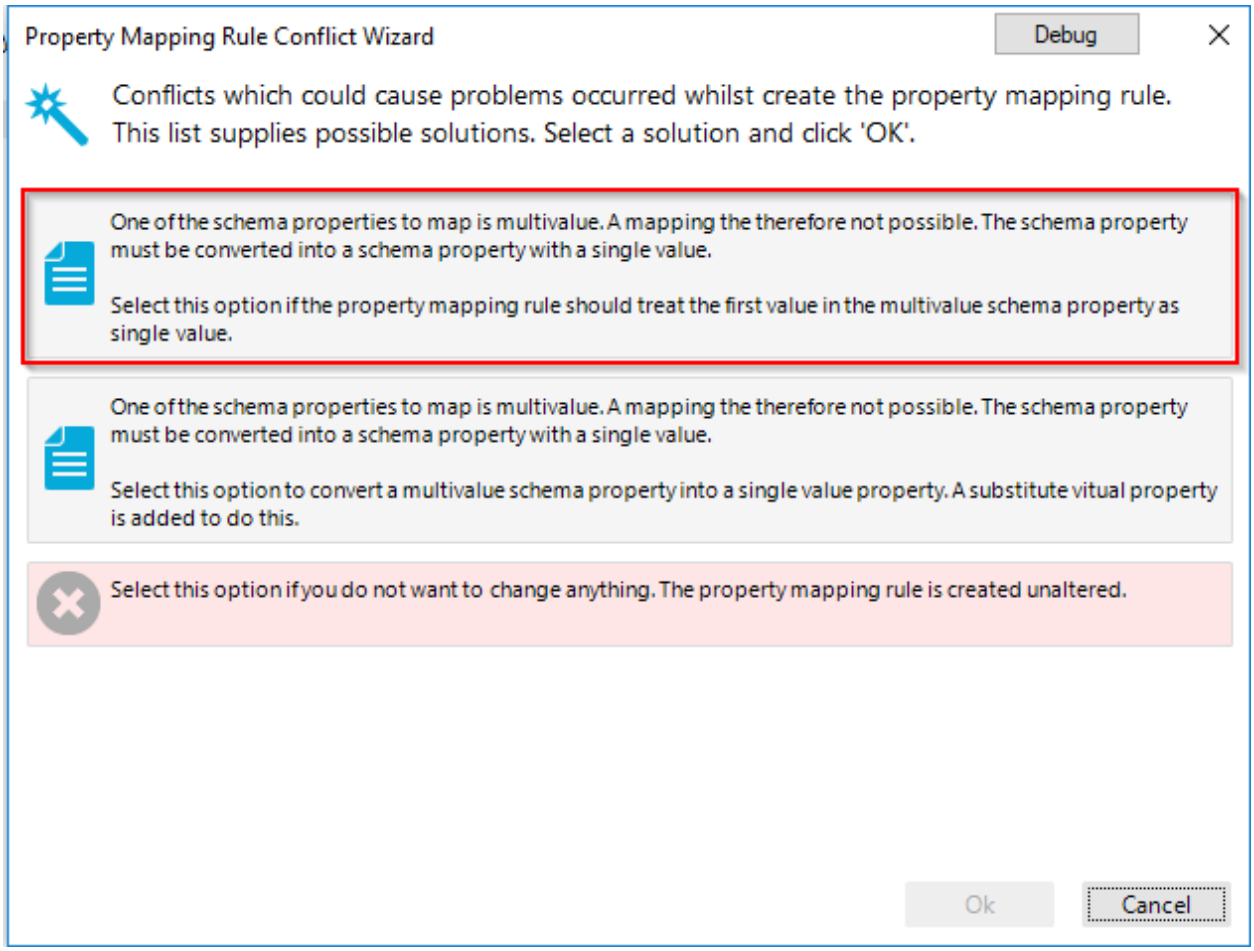



*Figure 17 Conflict resolution single vs. multi valued*

The last mandatory property that is left for the OneIM side is UID_LDPDomain. To populate this property, we need to create a virtual property on LDAP side that contains a value that "somehow" can identify the LDPDomain record in OneIM. Fortunately, such a value is available. It is the distinguished name of the base entry that we already used to define the synchronization scope. This value is available via the variable $CP_LDAP_Rootentry$. So all we need to do, is to create a virtual fixed value property "vrtDomainDN" of datatype string (3) that uses the variable as data (4). To add this property click the "new" button (1) in the property list on LDAP side, choose "Fixed value" (2) and enter the settings as shown on the screenshot.

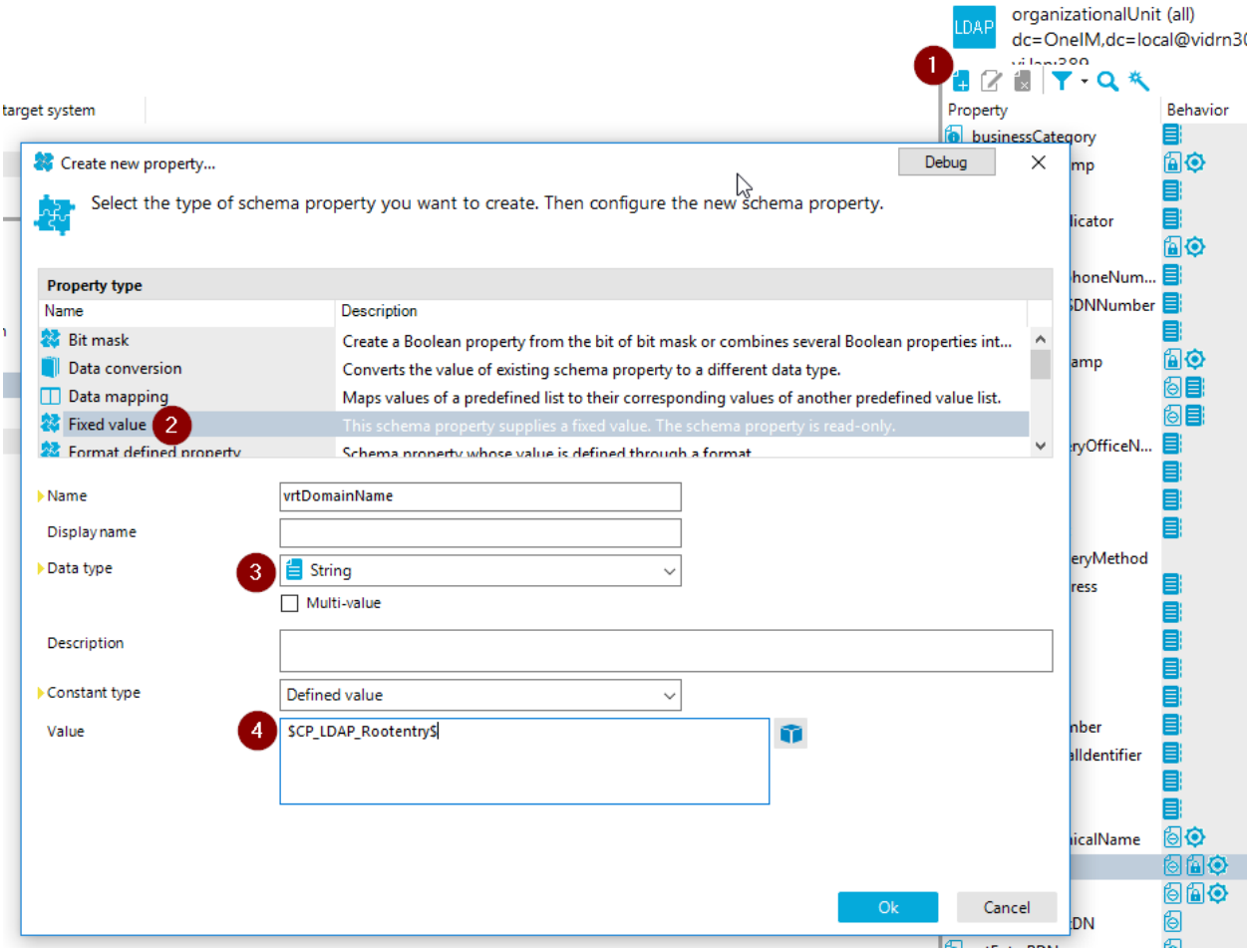
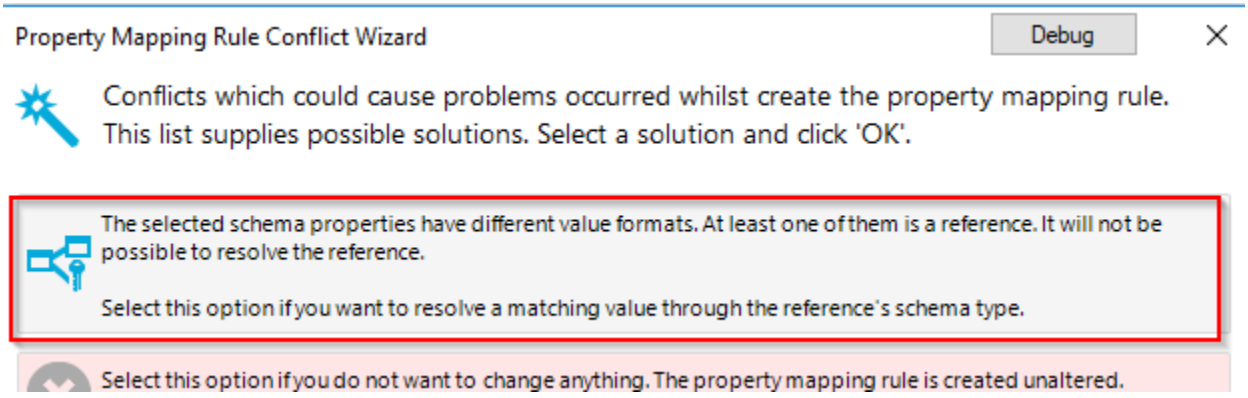
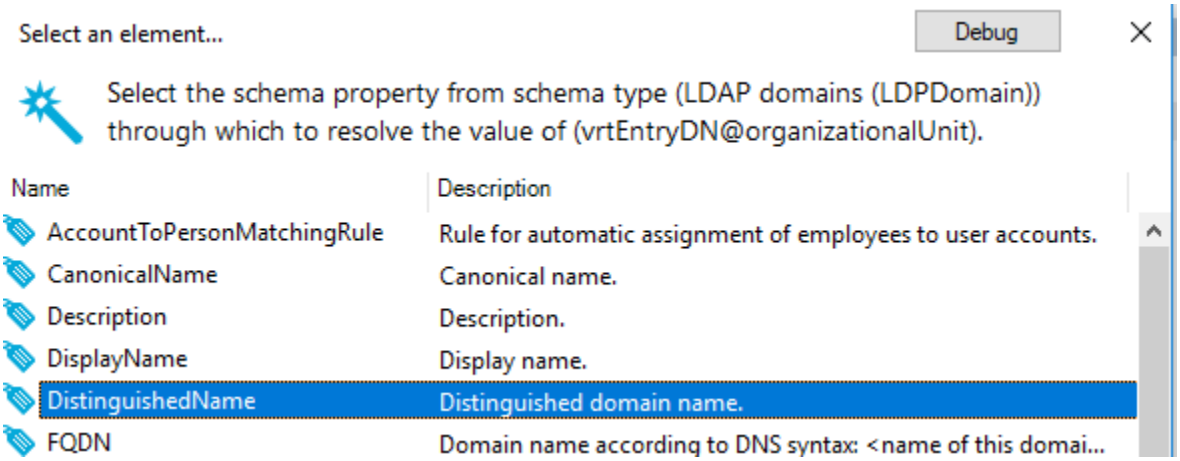



*Figure 18 Fixed value property vrtDomainDN*

Now drag vrtDomainDN to UID_LDPDomain. The conflict resolution wizard will pop up again and notify you that you are trying to map a reference property (UID_LDPDomain) with a simple string (vrtDomainDN). Choose the option to create a reference resolution property.



The wizard will then ask you to specify the search property of the resulting reference resolution property. In our case, this is "DistinguishedName".




The warning regarding "Distinguished name is not a unique key" that might pop up can be ignored. Once confirmed, the preconfigured property appears. Hit "Ok" and you should see a new mapping rule "**VRT_UID_LDPDomain** ← vrtDomainDN".

Now it is time to take care of the LDAP side. Things get a bit tricky here. The properties "**vrtEntryRDN**" and "**vrtEntryParentRDN**" still need to be mapped.

The property **"vrtEntryRDN"** contains the first distinguished name component. For a distinguished name "ou=Sales,dc=OneIM,dc=local" the RDN is "ou=Sales". Since we mapped **"cn"** from LDAPContainer with **"ou"** in LDAP, we should also use "cn" to build a virtual property on OneIM side that constructs the rdn. For demonstration purposes, we will use a format defined virtual property (1) here. However since it does not take care of proper distinguished name escaping, you should use the script property approach in production as the out of the box templates do. The configuration of the new virtual **"vrtEntryRDN"** property on OneIM uses the format: OU=%cn% as shown on the screenshot.

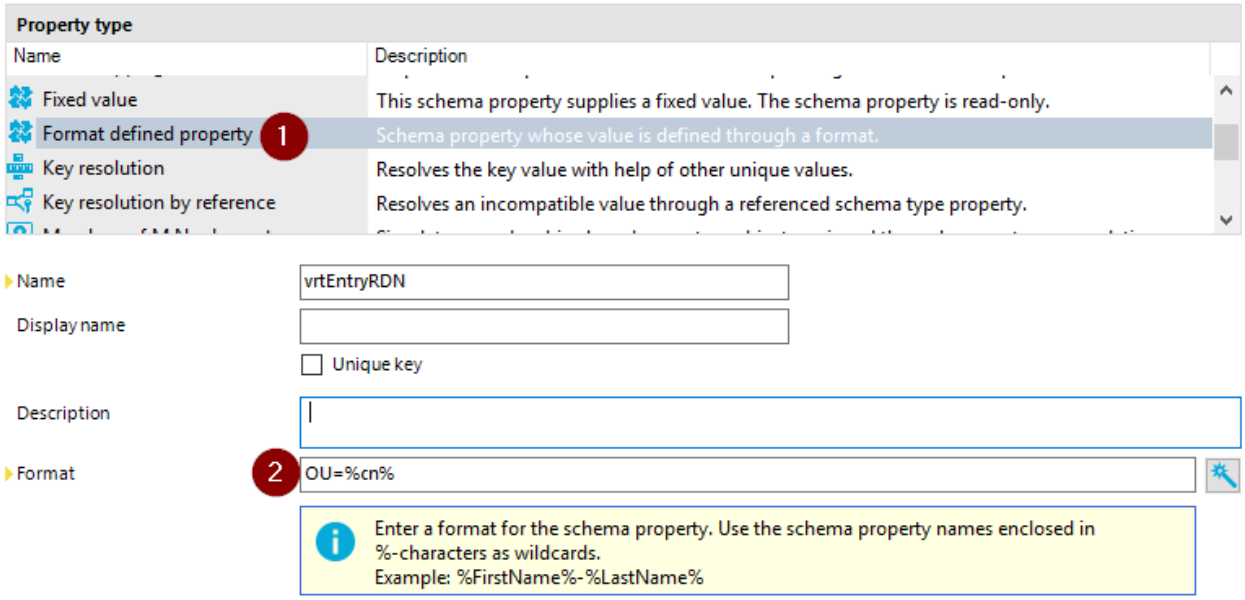



*Figure 19 Format defined vrtEntryRDN property*

Now drag and drop the vrtEntryRDN properties and the mapping rule is done.

The more tricky part is the vrtEntryParentDN property. It is basically the non RDN part. To stick with the example, the parent dn of "ou=Sales,dc=OneIM,dc=local" is "dc=OneIM,dc=local". When looking at OneIM, we have two options, where this value can come from. If the object is directly located under the base dn, the distinguished name of the LDPDomain reference must be used. If the object is located within another container, the parent containers reference has to be used. The distinguished name of the domain is already available through the virtual "**VRT_UID_LDPDomain**" property that was created earlier. Now, we still need distinguished name of a potential parent container in case the object is not directly located under the base entry. For this purpose, we create a virtual "key resolution by reference" property "**VRT_UID_ParentLDAPContainer**" with base column "UID_ParentLDAPContainer", search property "DistinguishedName". The settings for "Save unresolvable keys" and "Handle failure to resolution as error" can be left turned off. Their meaning is described in the "key resolution by reference" property section later in this document.

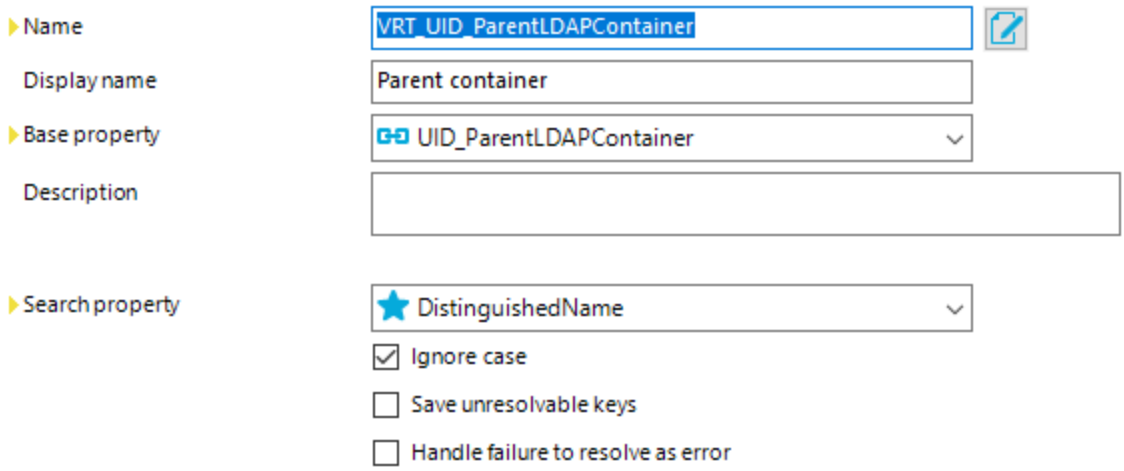



*Figure 20 VRT_UID_ParentLDAPContainer property*

Now we have 2 potential properties in OneIM ("VRT_UID_LDPDomain" and "VRT_UID_ParentLDAPContainer") that need to be mapped to vrtEntryParentDN depending on the data situation. To implement this, we can use the mapping condition setting of the mapping rules. We to use VRT_UID_LDPDomain if UID_ParentLDAPContainer is empty otherwise we want to use VRT_UID_ParentLDAPContainer). So first, we drag and drop bot VRT* properties to vrtEntryParentDN. Then we modify the rules as follows:

VRT_UID_LDPDomain to vrtEntryParentDN

- Switch mapping direction to "Target system"
- Set mapping condition to: `Left.UID_ParentLDAPContainer=''`

VRT_UID_ParentLDAPContainer to vrtEntryParentDN

- Set mapping condition to: `(ProjectionDirection='ToTheRight' and Left.UID_ParentLDAPContainer<>'' or ProjectionDirection='ToTheLeft')`

Now the basic set of rules is set up. Those rules allow basic CRUD operations as well as matching objects. Now we will add additional "simple" rules using the wizard that we initially skipped. To start it, click the rule wizard in the mapping section.


*Figure 21 Rule wizard*

Proceed with the option "Create mapping rules dynamically based on similarity" and click next. Then set the precision slider to about 80% and click next. The result should look like this:

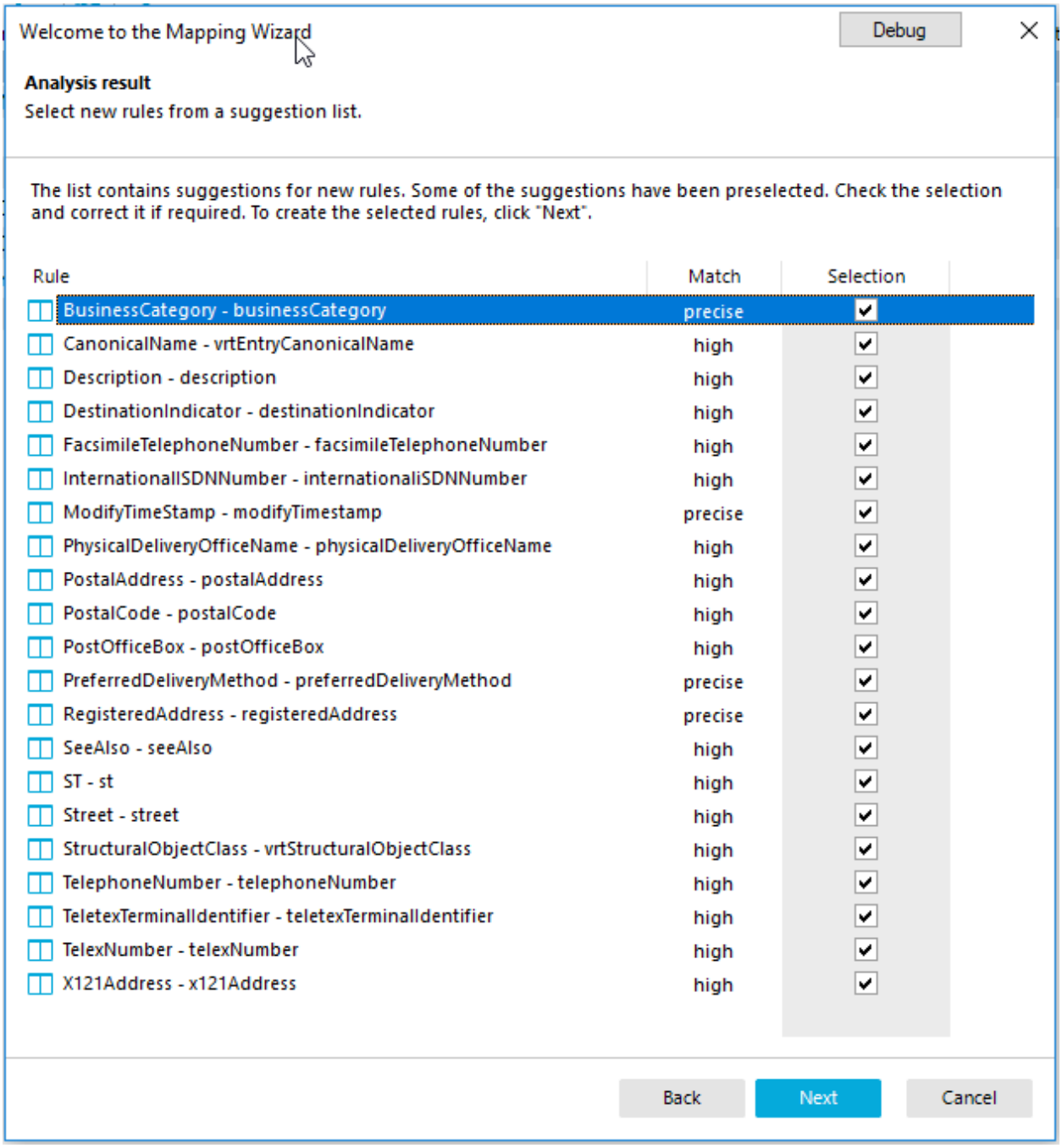



*Figure 22 Auto mapping result*

Review the rules and uncheck the ones you do not want to use. Then click next. The new mapping rules will appear with a yellow background. This is just a visual information for you to identify the rules that were automatically created in case you want/need to review them. The last thing to do is check and, if required, map the remaining properties. To filter the

property lists for those, click the filter icon and select the option “Hide used schema properties”.

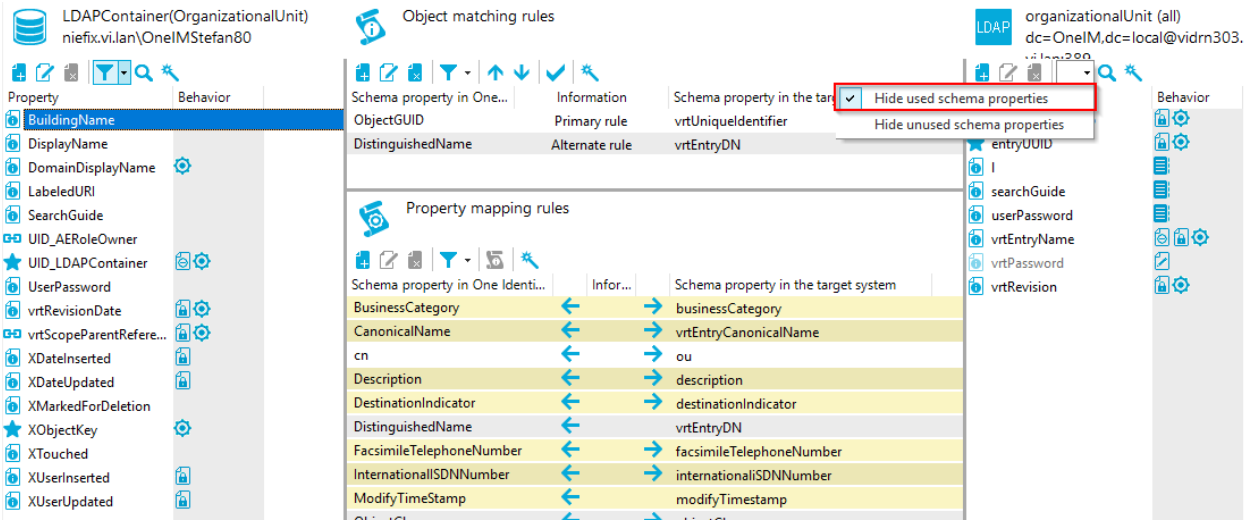



*Figure 23 Hide used schema properties*

We will consider the first mapping as completed.

## LDAPContainer to Organization mapping

The LDAPContainer to Organization mapping is very similar to the “OrganizationalUnit” mapping described in the previous section. So we will focus on the convenience features during mapping creation here. However, the first thing to do is create the mapping definition. This has to be done in the same way as the OrganizationalUnit mapping. So create a new mapping with the following settings:

Mapping settings:

| Setting | **Value** |
|---|---|
| Mapping name | Organization |
| Base mapping | <empty> |
| Mapping direction | Both directions |
| Description | LDAPContainer/organization |
| Hierarchy synchronization | Yes |
| Only suitable for updates | No |
| Maps objects referenced by multiple references | Yes |
| One Identity Manager schema class | <New> (see below) |
| Target system schema class | organization (all) |

The One Identity Manager schema class for Organization has to be configured as follows:

| | |
|---|---|
| Class type | Generic schema class |
| Schema type | LDAPContainer |
| Display name | LDAPContainer(Organization) |
| Class name | LDAPContainer_Organization |
| System filter | upper(StructuralObjectClass)='ORGANIZATION' |
| Select objects (ignore case = true) | StructuralObjectClass='Organization' |

This time, we **will** actually use the mapping wizard and not cancel it. Run it with the "Create rules based on template" mode. This mode allows you to select an existing mapping as template. The wizard will then try to create the rules and virtual properties, based on the selected mappings.

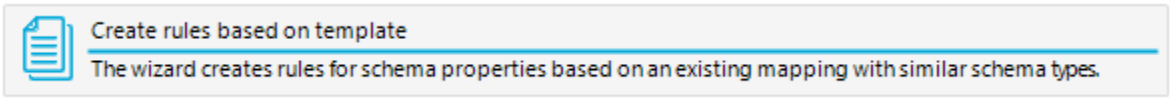
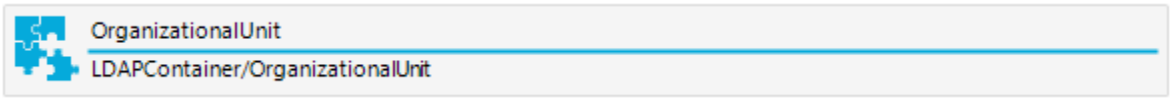



*Figure 24 Rule wizard template mode option*

Then, select the existing mapping for "OrganizationalUnit"




Confirm the list of mapping rules and finish the wizard. Now we need to do two minor adjustments. The first one is, create a mapping rule between "**cn**" in OneIM and "**o**" in LDAP as you did for "cn" to "ou" in the OrganizationalUnit mapping.

Then you need to change the format pattern of the virtual "**vrtEntryRDN"** property on OneIM side from OU=%cn% to O=%cn%.

Congratulations, you are done with the "Organization" mapping.

## LDAPAccount to InetOrgPerson mapping

Create a mapping with the following settings

| Setting | **Value** |
|---|---|
| Mapping name | InetOrgPerson |
| Base mapping | <empty> |
| Mapping direction | Both directions |
| Description | LDAPAccount/organization |
| Hierarchy synchronization | No (not a "container") |
| Only suitable for updates | No |
| Maps objects referenced by multiple references | Yes |
| One Identity Manager schema class | LDAPAccount (all) |

| Target system schema class | inetOrgperson (all) |
|---|---|

Again, use the "Create rules based on template" wizard and select the "OrganizationalUnit" mapping. Make sure you remove the suggested rule "cn ←→ ou". This is a false "friend".

This time, we need to do some more adjustments.

Since the name of the container reference property of LDAPAccount is UID_LDAPContainer instead of UID_LDAPParentcontainer, the wizard was unable to create several elements. This needs to be done manually now. Create a "key resolution by reference" property "**VRT_UID_LDAPContainer**" that uses "**UID_LDAPContainer"** as base property and "DistinguishedName" as search property. Then drag and drop VRT_UID_LDAPContainer to vrtEntryParentDN and VRT_UID_LDPDomain also to vrtEntryParentDN. Then we modify the rules as follows:

VRT_UID_LDPDomain to vrtEntryParentDN

- Switch mapping direction to "Target system"
- Set mapping condition to: `Left.UID_LDAPContainer=''`

VRT_UID_LDAPContainer to vrtEntryParentDN

- Set mapping condition to: `(ProjectionDirection='ToTheRight' and Left.UID_LDAPContainer<>'' or ProjectionDirection='ToTheLeft')`

The next task is the modification of the "**vrtEntryRDN"** format setting. Change it to CN=%cn%. Also map "**cn**" to "**cn**".

Since inetOrgPerson is very different from organizationalUnit there are a lot of unmapped properties left. You can either map them by hand or re-run the mapping rule wizard in "Create mapping rules dynamically based on similarity" mode. Make sure you review the wizard suggestions.

You will recognize that the wizard will not be able to find all rules based on property names. Here some examples that need to be created manually:

UserID ← → UID
PagerTelephoneNumber ← → pager
MobileTelephoneNumber ← → mobile

In addition to those "simple" properties, there are some references in OneIM that can be populated.
UID_LDAPAccountManager: Create a "key resolution by reference"
**"VRT_UID_LDAPAccountManager"** property that searches LDAPaccount for the given distinguished name and map this property to "**manager**" on LDAP side.

UID_LDAPAccountSecretary: Create a "key resolution by refrence"
**"VRT_UID_LDAPAccountSecretary"** property that searches LDAPAccount for the given distinguished name and map this property to "**secretary**" on LDAP side.

The one thing that is left is the password. The LDAP connector exposes a write only property called "**vrtPassword"**. In OneIM, there is a "UserPassword" field. At the first glance, it seems that this is an easy mapping but especially with passwords you need to pay special attention on the OneIM side. Passwords are often only temporarily stored in OneIM password fields. They are cleared, once the password is successfully written to the target system. Therefore create a mapping with of **"UserPassword"** to **"vrtPassword"** with the condition: `Left.UserPassword<>''`.

## LDAPGroup to groupOfUniqueNames mapping

The groupOfUniqueNames mapping settings are as follows:

| Setting | **Value** |
|---|---|
| Mapping name | GroupOfUniqueNames |
| Base mapping | <empty> |
| Mapping direction | Both directions |
| Description | LDAPGroup/groupOfUniqueNames |
| Hierarchy synchronization | No (not a "container") |
| Only suitable for updates | No |
| Maps objects referenced by multiple references | Yes |
| One Identity Manager schema class | <new> See below |
| Target system schema class | groupOfUniqueNames (all) |

The One Identity Manager schema class for LDAPGroups of type groupOfUniqueNames needs to be created with the following settings:

| Class type | Generic schema class |
|---|---|
| Schema type | LDAPGroup |
| Display name | LDAPGroup (groupOfUniqueNames) |
| Class name | LDAPGroup _GroupOfUniqueNames |
| System filter | upper(StructuralObjectClass)='GROUPOFUNIQUENAMES' |
| Select objects (ignore case = true) | StructuralObjectClass='GROUPOFUNIQUENAMES' |

This time, we use the InetOrgPerson mapping as a template. This will save you from configuring the vrtEntryParentDN property mappings again because they are the same for both schema types.

Looking at the LDAP side, there are only a few relevant properties left to map. The "**owner**" property can be used to populate **"UID_LDAPAccountOwner"** the same way as the manager or secretary properties were handled in the InetOrgPerson mapping.

In addition, we have a multi-valued reference for the first time. The "**member**" property contains the group members and needs to be synchronized to the corresponding M:N tables in OneIM. The current mapping however is "just" for LDAPGroup and not LDAPAccountInLDAPGroup etc. So firstly, we need make those "records" available. This can be achieved by creating a virtual "members of M:N schema types" property.

We need to determine which M:N tables in OneIM we can populate. Let us look at the OneIM schema types of the mappings that we already did or that we still want to handle. Those types are, LDAPAccount, LDAPGroup and LDAPContainer. Now we check OneIM for M:N schema types (tables) that store references to those types. You will find LDAPAccountInLDAPGroup, LDAPGroupInLDAPGroup and LDPMachineInLDAPGroup. Since we create(d) mappings for LDAPAccount and LDAPGroup we want to import those types of memberships to OneIM.

So what we need to do, is create "members of M:N schema types" property that exposes the objects of LDAPAccountInLDAPGroup and LDAPGroupInLDAPGroup on the OneIM side.

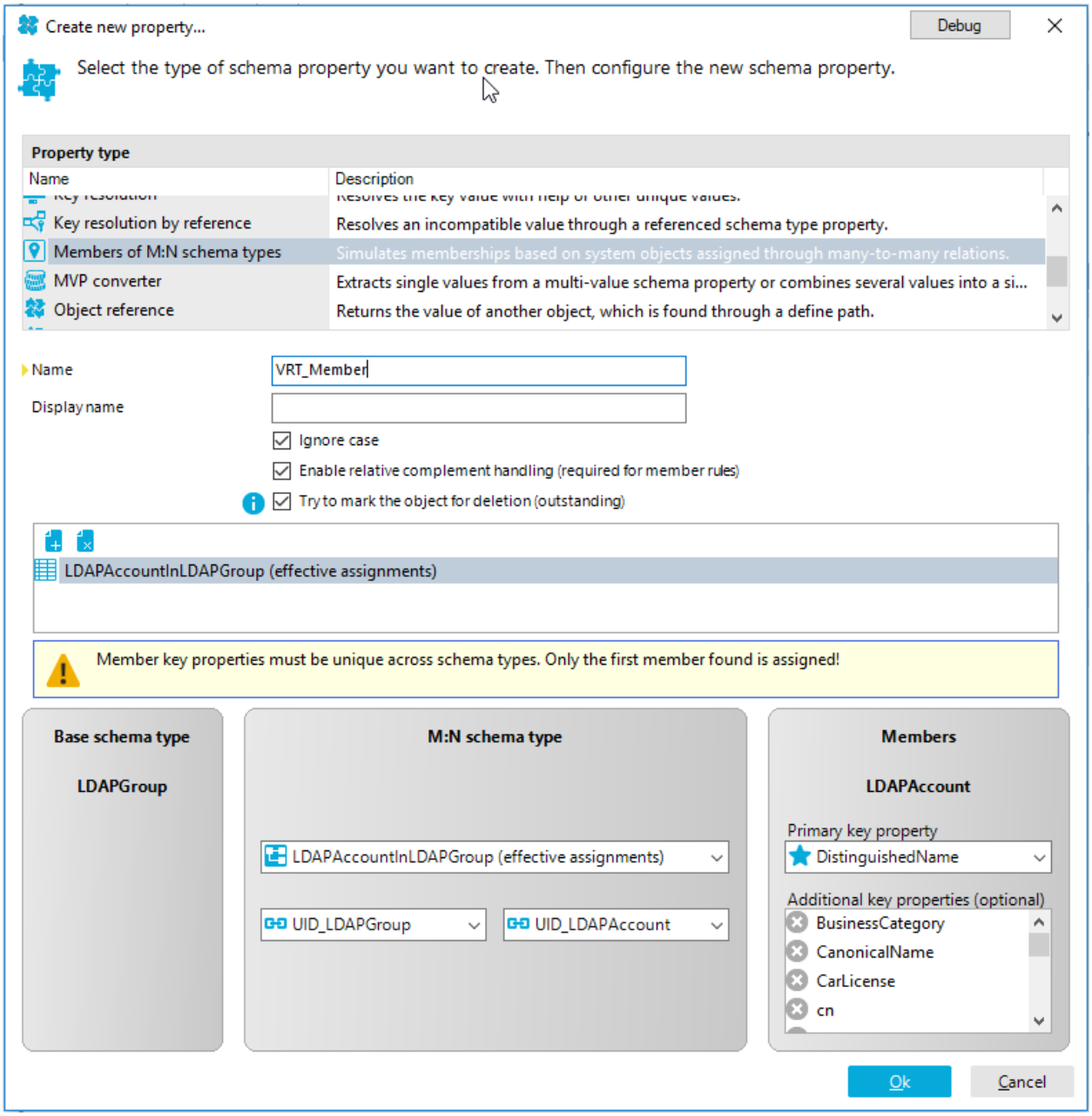



*Figure 25 VRT_Member*

As you can see, the LDAPAccountInLDAPGroup (effective assigments) schema class was chosen. As a rule of thumb you should use the "effective assignments" class if present. As "primary key property" we use the distinguished name since this is the value, which has actually to be returned, when the value of this virtual property is read or to be used when searching in the LDAPAccount table. The meaning of the settings "Enable relative complement handling (required for member rules) as well as the "Try to mark the object for

deletion (outstanding)" are described in the "members of m:n schema types" property section.

Add the same configuration for LDAPGroupInLDAPGroup.

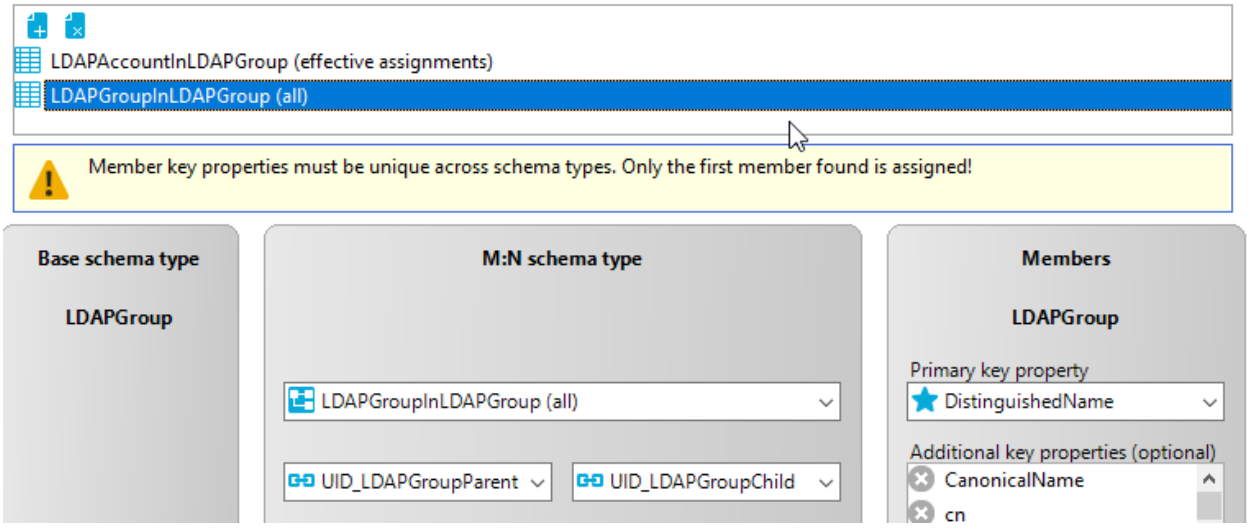



*Figure 26 LDAPGroupInLDAPGroup settings*

Once done with the virtual property configuration, "VRT_Member" needs to be mapped with the "uniqueMember" property. This time, we do not drag and drop since instead of a value comparison rule we need a multi-reference mapping rule.

So click the "new" button (1) in the mapping section and select the "multi-reference mapping rule" (2) as rule type. Give it a name (3) and set the mapping direction to "both directions" (4). Then select VRT_Member (5) for the One Identity Manager side and uniqueMember (6) for LDAP. The last thing to do, is to add the member filter for the LDAP connection (7). It specifies that we will only want to handle references that are pointing to the object types of the given list. All other references will be ignored. This setting needs to be set, since in LDAP there might be references to objects that we do not handle in our synchronization project. According to the configuration of the VRT_Member property, we only handle objects that are synchronized to LDAPAccount and LDAPGroup. To those types, we import inetOrgPerson, groupOfNames and groupOfUniqueNames. So those are the types we want set for our filter.

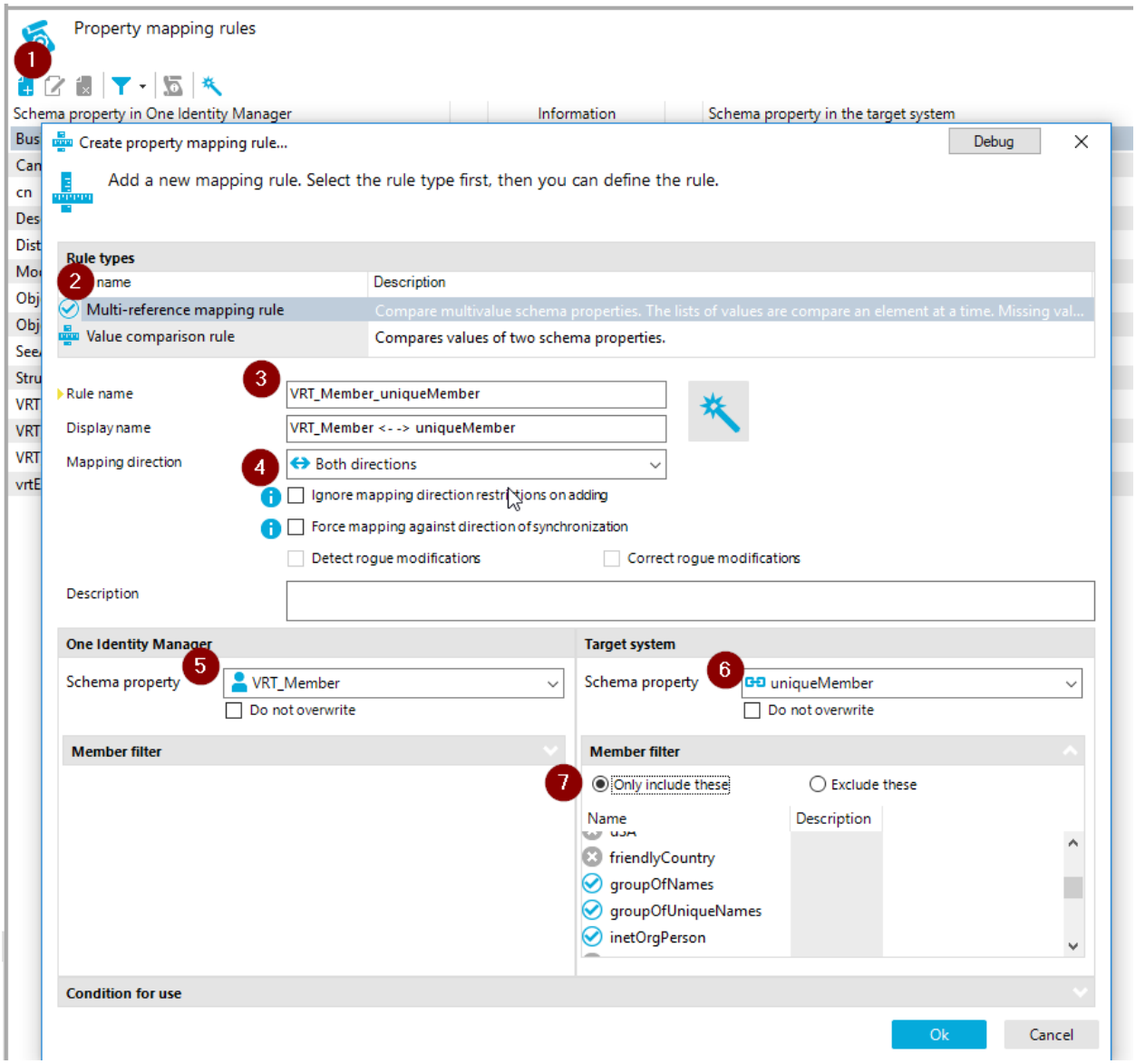



*Figure 27 member mapping rule configuration*

That's it for the groupOfUniqueNames mapping.

## LDAPGroup to groupOfNames mapping

For groupOfNames create a mapping with the following settings:

| Setting | **Value** |
|---|---|
| Mapping name | GroupOfNames |
| Base mapping | <empty> |
| Mapping direction | Both directions |
| Description | LDAPGroup/groupOfNames |
| Hierarchy synchronization | No (not a "container") |
| Only suitable for updates | No |
| Maps objects referenced by multiple references | Yes |
| One Identity Manager schema class | <new> See below |
| Target system schema class | groupOfNames (all) |

The One Identity Manager schema class for LDAPGroups of type groupOfNames is created with the following settings:

| Class type | Generic schema class |
|---|---|
| Schema type | LDAPGroup |
| Display name | LDAPGroup (groupOfNames) |
| Class name | LDAPGroup_GroupOfNames |
| System filter | upper(StructuralObjectClass)='GROUPOFNAMES' |
| Select objects (ignore case = true) | StructuralObjectClass='GROUPOFNAMES' |

When going through the mapping wizard, use the groupOfUniqueNames mapping as a template. Since groupOfNames and groupOfUniqueNames are very similar, you will find that the only mapping rule that is missing is the rule for **VRT_Member** to **member**. Unfortunately, the member property of groupOfNames is a mandatory property in contrast to uniqueMember of groupOfUniqueNames. Now we have to get a little bit creative. A new LDAPGroup created in OneIM always has no members (LDAP***InLDAPGroup) because the LDAPGroup object must be created first. Another case is when the last membership is removed from OneIM the group will remain.

So what are the options?

1) Somehow prevent the provisioning of new "empty groups"?
   ➔ Not a very good idea because the next synchronization would delete the group from OneIM (or tag it as missing) since it does not exist in LDAP.

2) Ensure that there is always some kind of "default" member present as member.

So how can 2) be achieved? It is basically a combination of the techniques that were already used for other mapping use cases. We need a conditional mapping rule that defines a default member that has to be mapped to "member" in case VRT_Member is empty. As a

candidate for default member, we will use the group itself a.k.a. the group is its own member. This should have no implications to security. So at first we need to create a multi-valued property on OneIM side containing the distinguished name of the group.

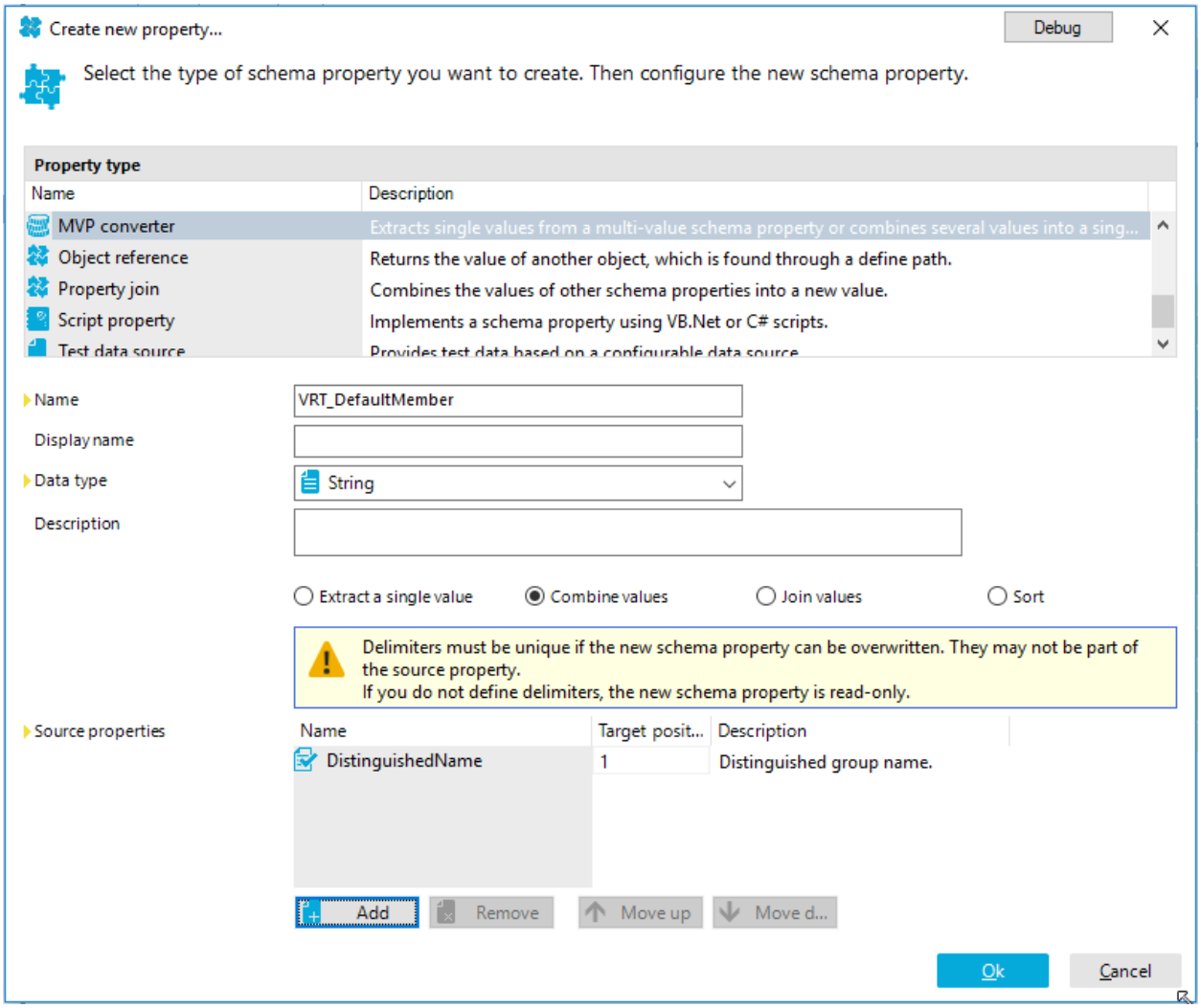



*Figure 28 VRT_DefaultMember configuration*

Then map VRT_DefaultMember to member. Configure the mapping direction as "Target system" and set the condition to `Left.VRT_Member=''`.

Then create a multi reference mapping rule with the same configuration you did for the "VRT_Member <- -> uniqueMember" rule of the groupOfUniqueNames mapping. Note that you cannot use "Drag and drop" for this! Set its mapping condition to

```
(ProjectionDirection='ToTheRight' and Left.VRT_Member<>'' or
ProjectionDirection='ToTheLeft')
```

## The result should look like this

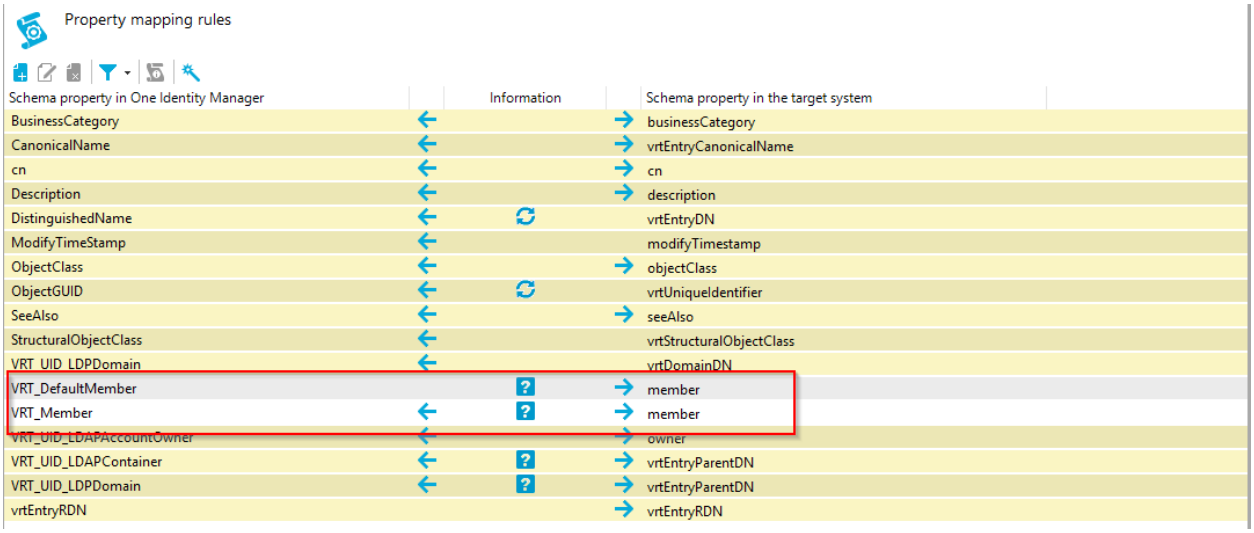



*Figure 29 groupOfNames mapping*

You have now successfully created all of the mappings.

# Step 5: Create a synchronization workflow

Synchronization workflows describe, which objects (mappings) shall be transferred in which direction (to OneIM or to target system). A workflow consists of workflow steps, each using a configured mapping.

To create a workflow for a bulk import from the target system to OneIM (a.k.a. synchronization), navigate to the "Workflows" section in the Synchronization Editor and start the workflow wizard by clicking the wand symbol.

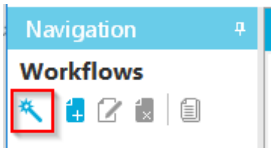



*Figure 30 Start workflow wizard*

Enter a name and a description for your workflow and proceed to the "**Assign mapping**" page. All mappings of your synchronization project are preselected which we will leave as it is for this walkthrough.

Proceed to "**Synchronization behavior for intersections**" and assign the "Update" method to the One Identity Manager side as shown on the following screenshot. This will configure the engine to pull changes from the target system (LDAP) to One Identity Manager for matching objects.

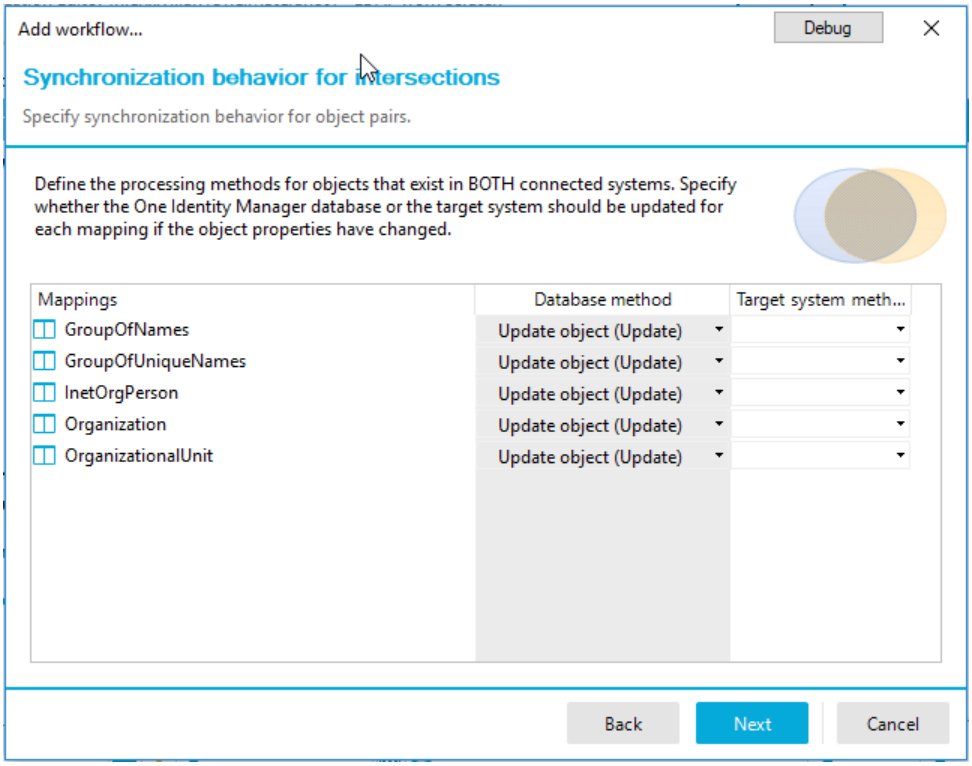



*Figure 31 Synchronization workflow intersection settings*

Proceed to "**Relative complement handling in One Identity Manager**". This page configures the actions for objects that only exist in OneIM. During a synchronization those objects shall typically be deleted from OneIM or tagged as "outstanding" (not present in the target system). In our example workflow, we will delete those objects from OneIM.

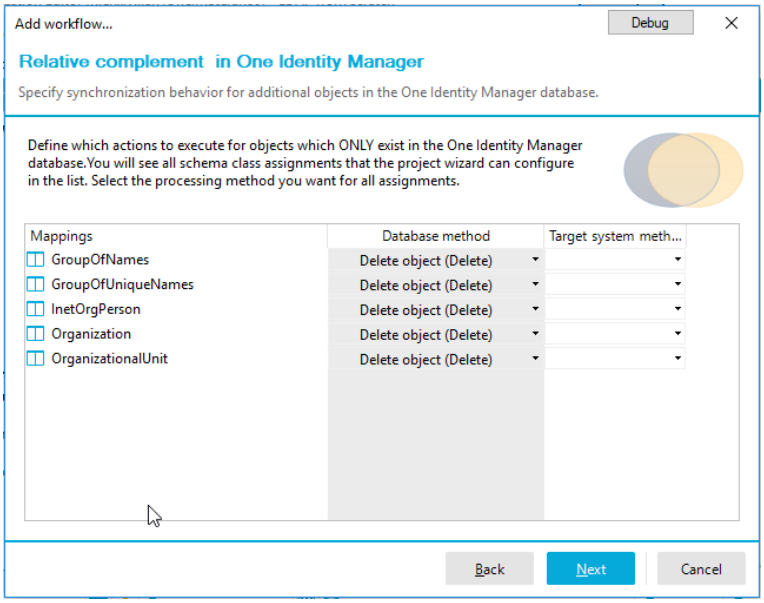



*Figure 32 One Identity Manager complement configuration*

Proceed to the "**Relative complement in target system**" page and configure the actions for objects that were only found in LDAP. Those shall be inserted in the OneIM database which is why the corresponding "insert" method is assigned for the OneIM side.

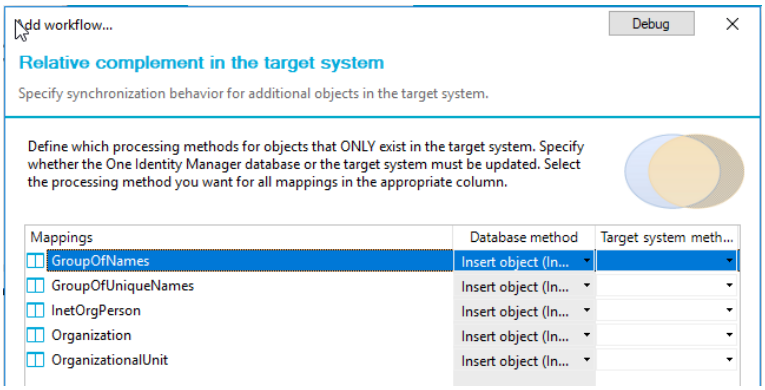



*Figure 33 Target system complement configuration*

Finish the Wizard and the workflow for the synchronization task is done.

All that is left is to create a startup configuration for the workflow. To create such a startup configuration navigate to “Configuration” → “Start up configurations” and click the new button in the toolbar.

Enter a display name and select the workflow you just created and pick the default variable set. If you like, you can also assign an OneIM schedule to this startup configuration to trigger a scheduled synchronization process automatically.

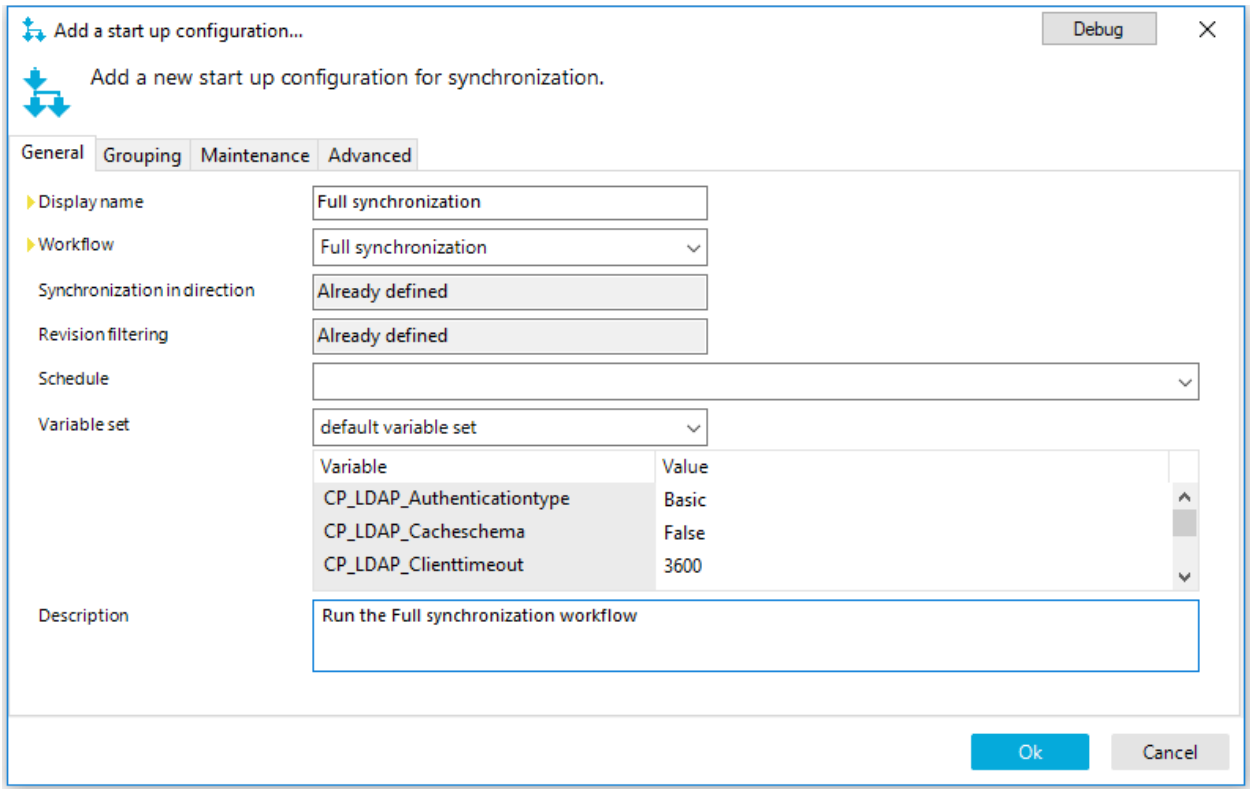



*Figure 34 Start configuration settings.*

Once confirmed, the startup configuration is listed with several options in the “Start up configurations” section.

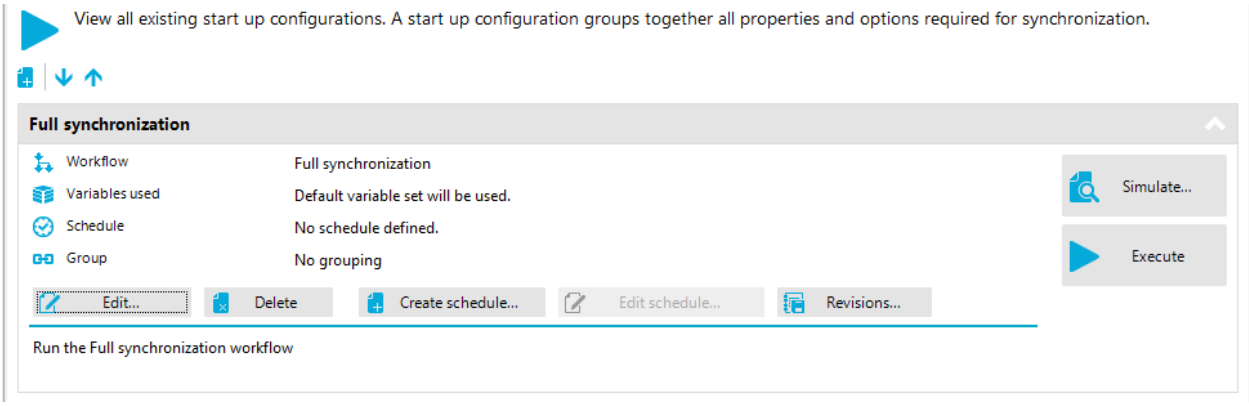



*Figure 35 Start up configuration*

We can now do a quick test of the configuration by simulating a workflow execution using the "Simulate" button. Choose "**Simulate from target system to One Identity Manager**" and click "**Start simulation**". The engine will now start to run the synchronization engine but will just log the methods it would execute instead of actually executing them. Note that this simulation runs on the machine where the Synchronization Editor was launched. When dealing with large target systems, a simulation can take a very long time and consume large amounts of CPU/memory. If the machine has significantly less resources than the actual synchronization server it is also possible that simulation runs into memory issues.

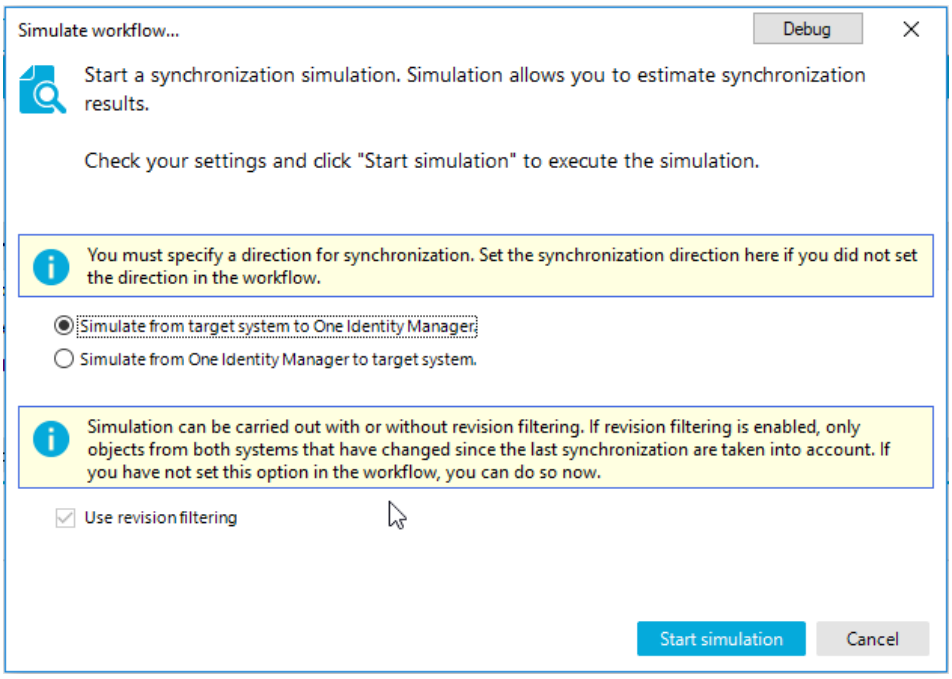



*Figure 36 Simulate a workflow*

Since the simulation uses the actual data from the systems, a simulation will give you initial feedback about the validity of your mapping rules. Let us assume you forgot to setup a proper multi-value to single value conversion. The simulation will throw an error like shown on the following screenshot.

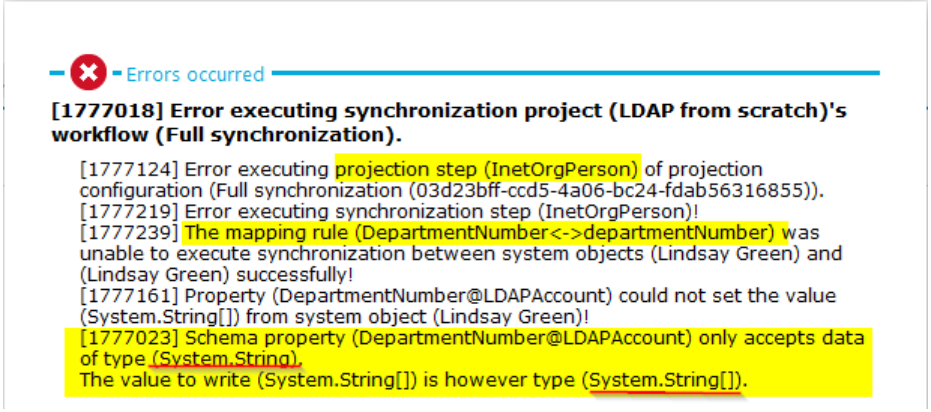



*Figure 37 Example error during simulation*

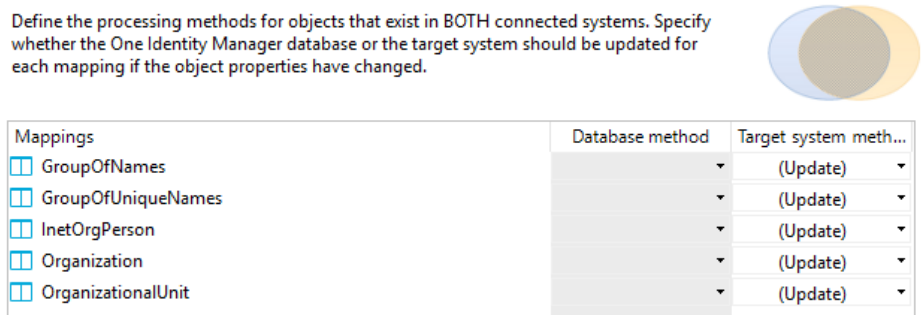
A successful simulation will output a report with information about the number of processed objects and potential called methods.

To run the workflow use the execute button. When running the Synchronization Editor in expert mode, you can run the workflow "live" on the local machine instead starting a process on the OneIM processing server configured at the base object.

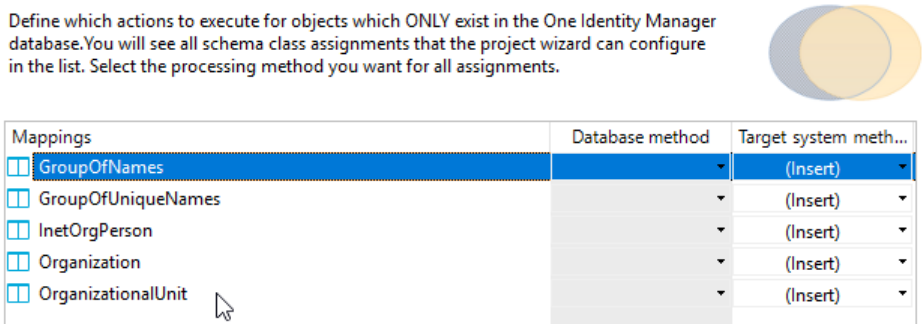
# Step 6: Configure ad hoc provisioning

In the previous step we configured a full synchronization workflow that is triggered based on a schedule or run manually in the Synchronization Editor. This section is dedicated to provisioning single changes to the target system. We will use a dedicated "Provisioning" workflow that we will configure first. The workflow wizard is used for the configuration as done in the previous step. Just modify the method assignments as follows:

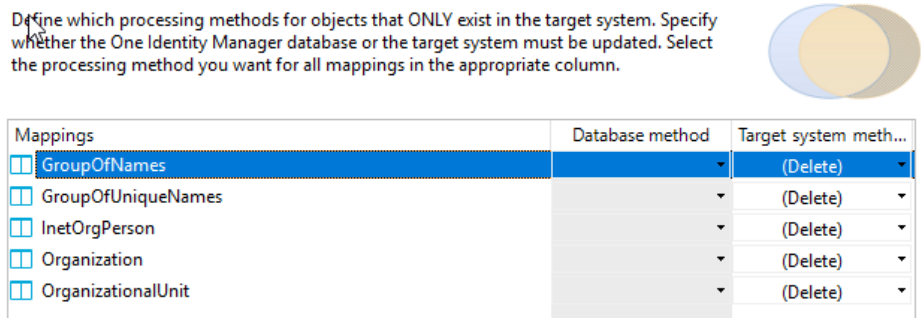
"Synchronization behavior for intersections"



“Relative complement in One Identity Manager”




“Relative complement in target system”




Commit your synchronization project to the database before you continue with the next step.

Now we need to link the workflow to the processing engine of OneIM. To do so, open the Designer and navigate to the “Process orchestration” section (1) and navigate the “Provisioning process operations” node (2). If you have existing synchronization projects that were created with out of the box templates, there might be already some items in the overview.

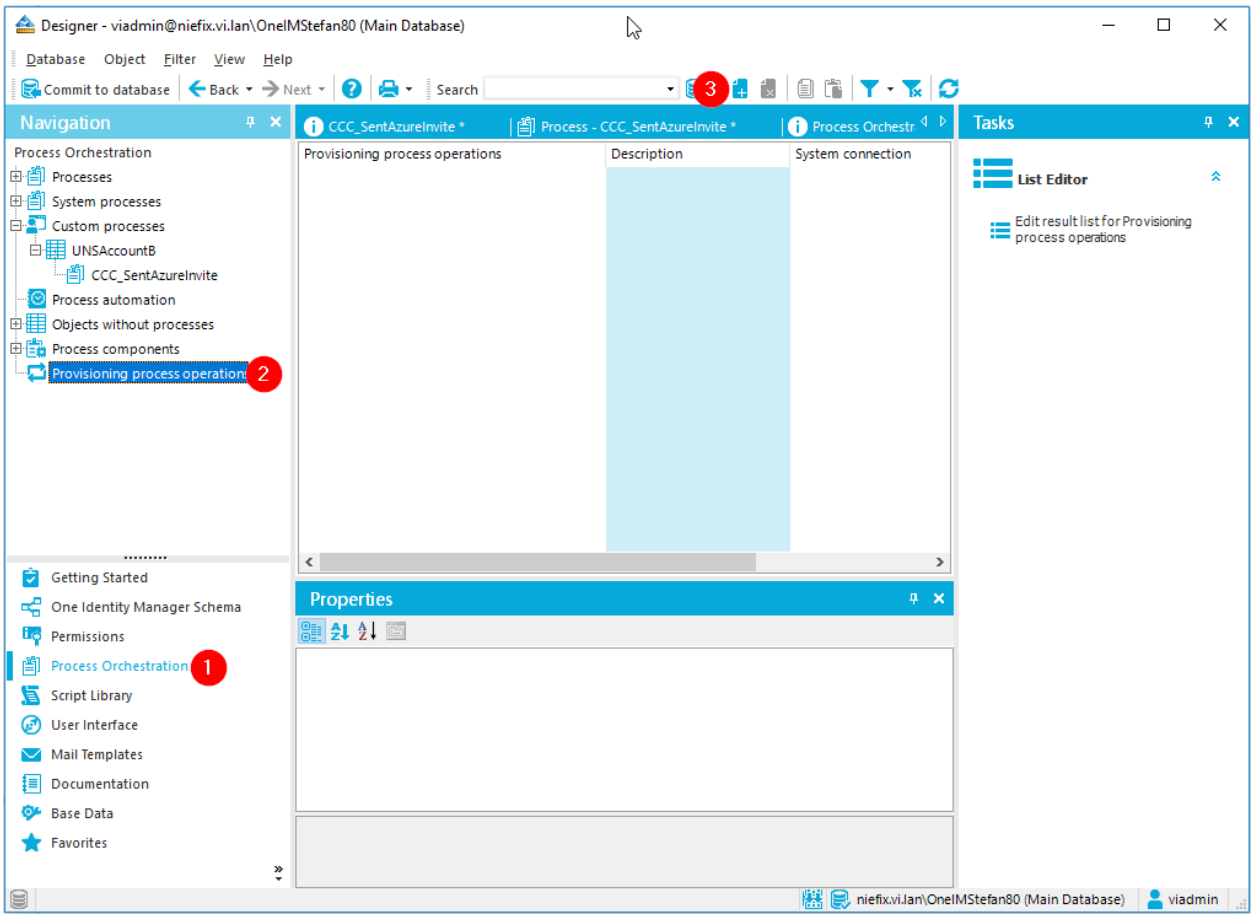



*Figure 38 Configure DPRObjectOperations*

Since we want to execute the provisioning workflow in our example, create a link for each OneIM schema type (table) you want to provision. Those are LDAPAccount, LDAPGroup and LDAPContainer. We will now create a “Provisioning process operation” for each of those tables, pointing to the provisioning workflow and the LDAP target system. As a name for those operations we will use “AdHoc” as shown on the following screenshot:

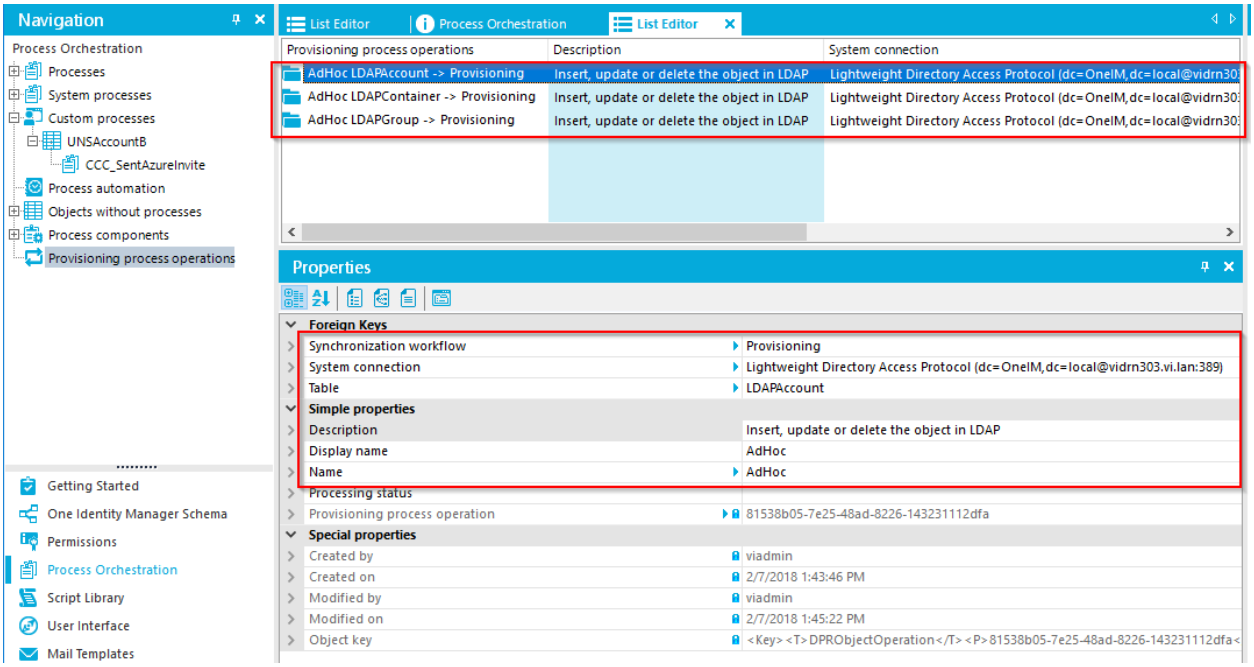



*Figure 39 Provisioning object operation definition*

Hint: Up to OneIM version 8.0, the display of a workflow did not contain the synchronization project of the workflow. This made it hard to identify the correct workflow in the selection list. If you already have multiple synchronization projects that contain workflows with similar or equal names you can copy their UID (i.e. by selecting them in the Object Browser) and paste them into the Designer.

The next thing to do is to create OneIM processes that actually execute the provisioning task. For demonstration purposes, disable the out of the box OneIM process for the LDAP table LDAPAccount in the “Designer” by setting the “Do not generate” flag. We will recreate a basic version of it next.

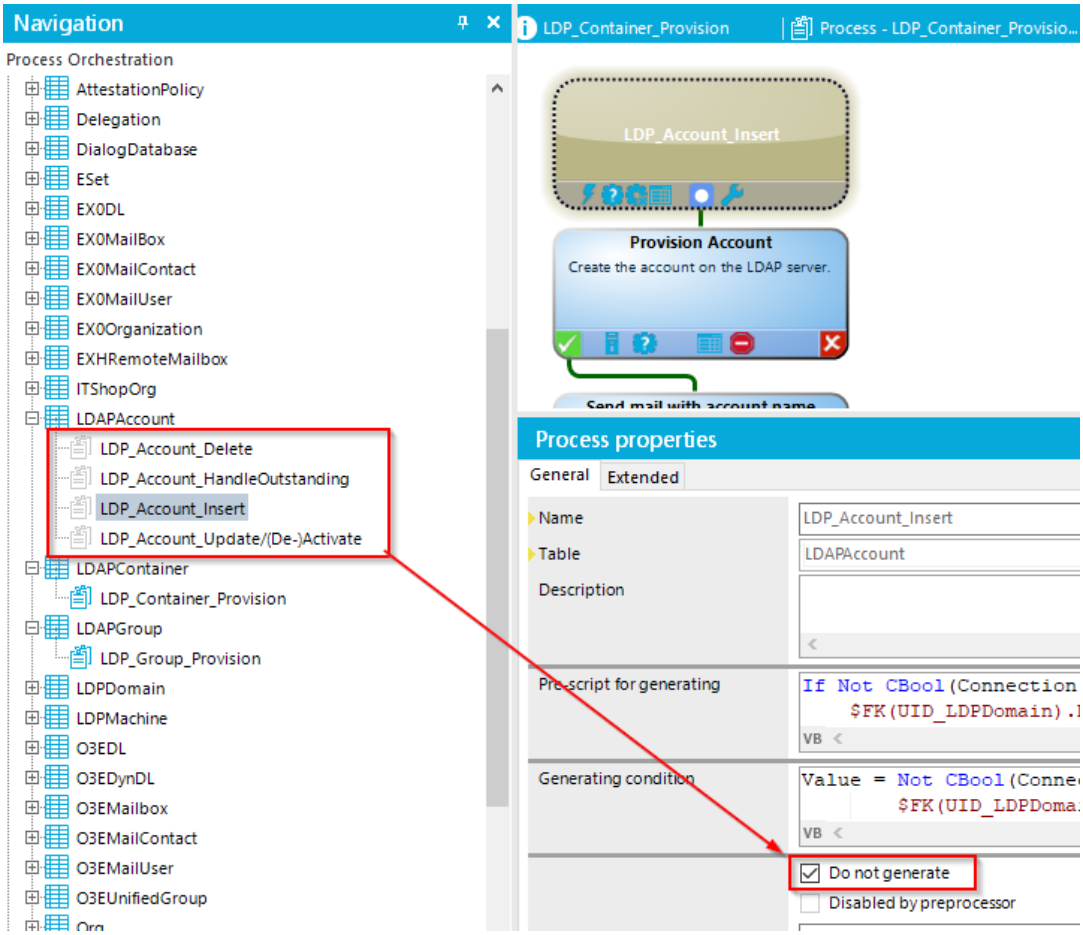



*Figure 40 Disable out of the box processes for LDAPAccount*

Instead of separate processes for Insert/Update/Delete, we will create a single process that executes the provisioning workflow each time a record is inserted, updated or deleted from the LDAPAccount table. To do so, create a new process "**CCC_LDAPAccount_Provision**" and select "LDAPAccount" as a base table.

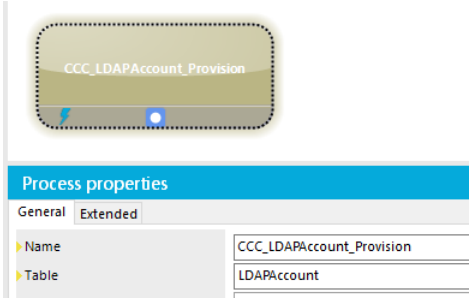



*Figure 41 New OneIM process*

Now add the OneIM events Insert, Update and Delete to the process.

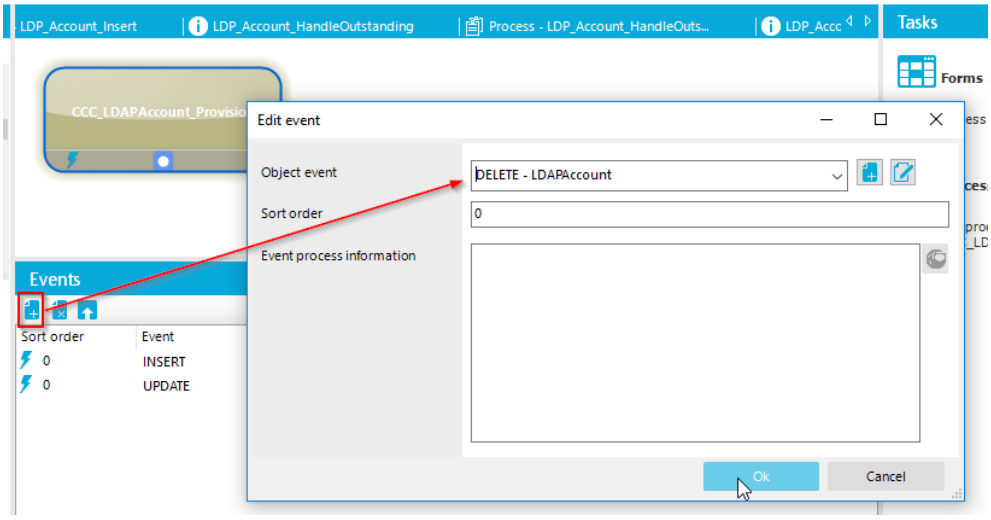
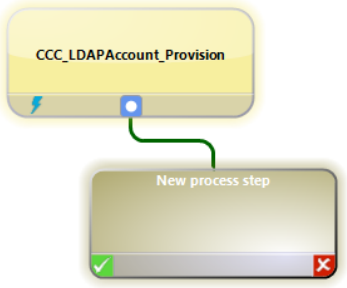
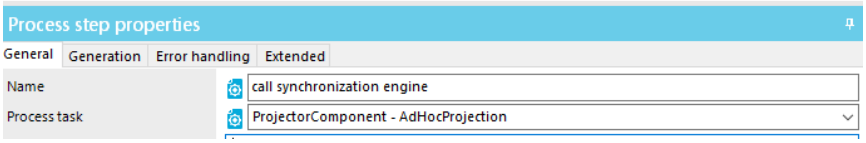



*Figure 42 Setting events listeners*

Next thing to do is to add a process step that actually calls the synchronization engine to do its work. So we are adding a new Job called "Call synchronization engine" that is using the process task "ProjectorComponent – AdHocProjection".




Make sure to set the generating condition of the process to the following value:

```
Value = Not CBool(Connection.Variables("FULLSYNC"))
```

This will prevent the execution of the provisioning workflow when the synchronization workflow saves data to the OneIM tables.

Let's have a look at parameters we still have to set that are not prepopulated. Those are **UID_DPRProjectionConfiguration** (the UID of the provisioning workflow) and **UID_DPRSystemVariableSet** (the variable set to use). Those values can be obtained by

calling the DialogScript DPR_GetAdHocData. This is usually done once in the pre script of the process and looks like this (see the DPR_GetAdHocData section in this document for information about the parameters and their meaning).

```
' Only execute this pre-script when the object is changed outside a full synchonization

If Not CBool(Connection.Variables("FULLSYNC")) Then

	Try
		'variable that indicates if a provisioning config was found
		values("AdHocDataFound") = False
		'variable that indicates, if there are any changes that need to be published
		values("NeedExecute") = False

		Imports System.Collections.Generic
		' try to obtain the id of the workflow, variableset
		' and processing server by looking up a provisioning
		' process operation (DPRObjectOperation) and the corresponding
		' base object (DPRRootObjectConnectionInfo). This is done by
		' DPR_GetAdHocData
		' Parameters:
		'	$FK(UID_LDPDomain).XObjectKey$ = XObject key of db record configured in root object
		'	"LDAP" = the systemtype (identifier of the connector)
		'	"" = system version (if available/required)
		'	"AdHoc" = the Name of the DPR_ObjectOperation

		Dim data As IDictionary(Of String,string) = DPR_GetAdHocData( _
			$FK(UID_LDPDomain).XObjectKey$, "LDAP", "", "AdHoc" )

		' If data was found, transfer it to variables
		' that can be used by the process steps of this process
		If Not data is Nothing
			' data was found
			values("AdHocDataFound") = True

			' call script that evaluates if the changes done to the OneIM object are relevant
			' in the synchronization project (i.e. are changed fields used in mapping/matching)
			values("NeedExecute") = DPR_NeedExecuteWorkflow(data("ProjectionConfigUID").ToString(),entity)
			' Construct an object key of the database object to be processed
			values("ObjectKey") = New DbObjectKey(base.Tablename,$UID_LDAPAccount$).ToXmlString()
			' the UID variable set to use
			values("UID_DPRSystemVariableSet") = data("VariableSetUID")
			' the UID of the workflow to run
			values("UID_DPRProjectionConfiguration") = data("ProjectionConfigUID")
			' the server to process the record
			values("UID_QBMServer") = data("ExecutionServerUID")
		End if

	Catch ex as AdHocDataException
		values("AdHocDataFound") = False
	End Try

End If
```

With the data returned by DPR_GetAdHocData you can now add the missing settings to the process step such as the execution server, workflow and variable set.

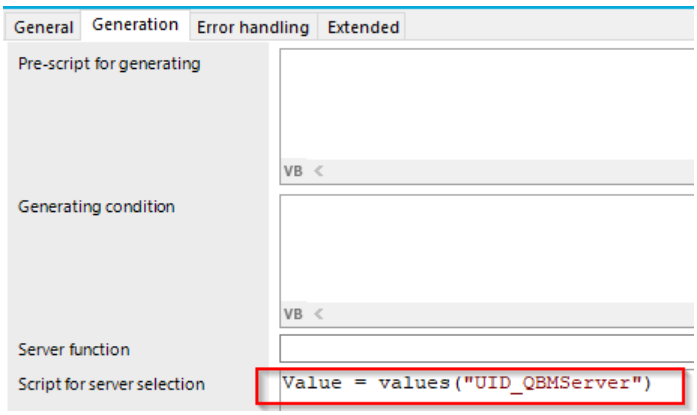



*Figure 43 Execution server setting*

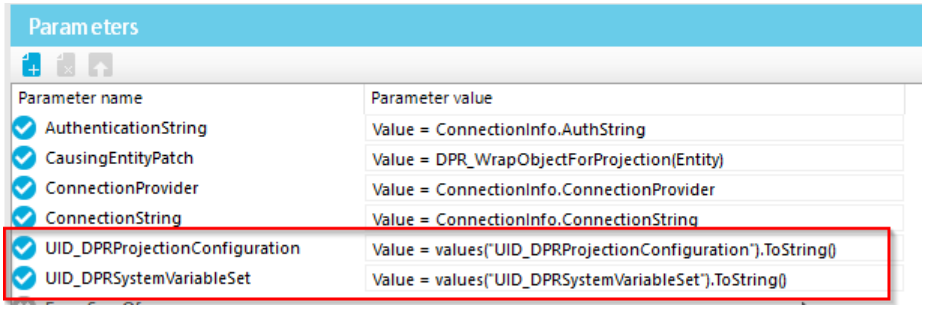



*Figure 44 Workflow and variable set id*

Once edited, commit the processes to the database and compile it. The last thing left to do is to activate the synchronization project in the Synchronization Editor. If a synchronization project is not activated, the processing engine will not execute any process related to it.

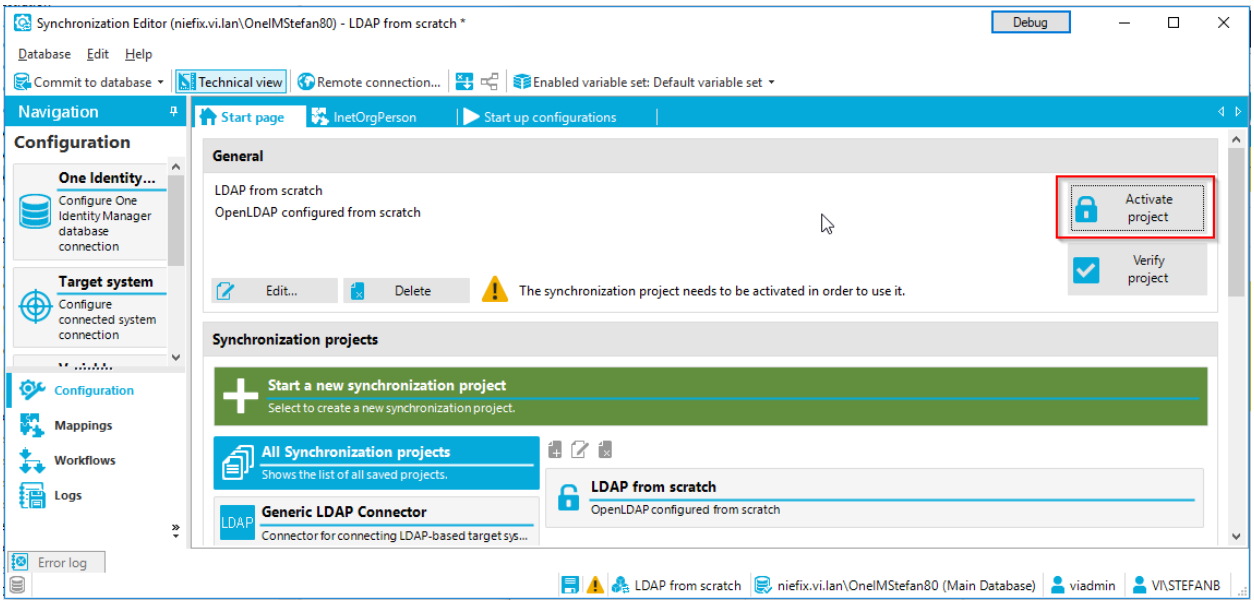



*Figure 45 Activate synchronization project*

Now your synchronization project is fully configured. You can and should now test your synchronization projects by running test synchronizations and reviewing the logs. Once the synchronization runs without errors you can start testing out the ad hoc processes by creating objects in the Manager application.

# Synchronization Framework Concepts

This section covers concepts that are used throughout the whole synchronization framework and did not fit into a specific section of the walkthrough.

## Schema

The schema is one of the most important items of a synchronization project. It describes the system, its entities and methods and is provided by the system connector.

The following screenshot shows, how to open the Schema browser for the target system. Make sure you have "Expert mode" enabled otherwise you will not see the browser.

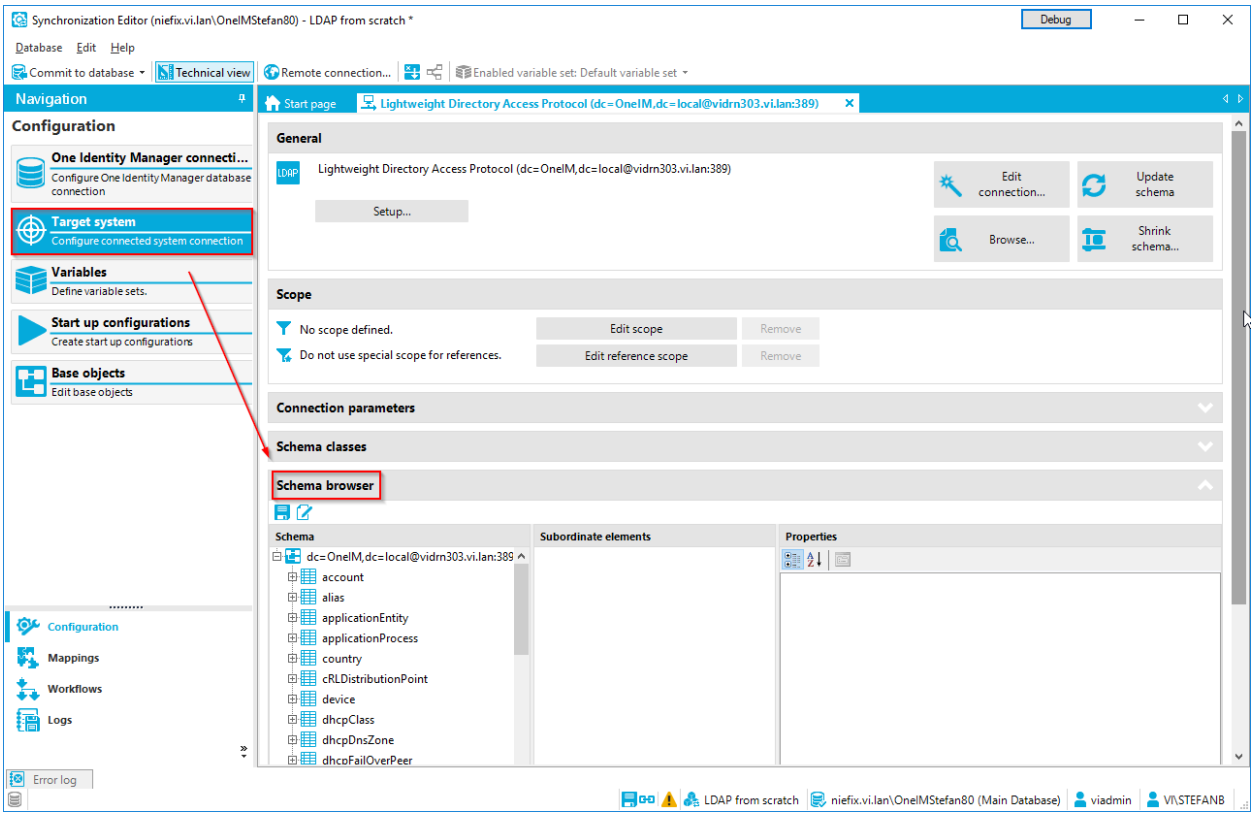



*Figure 46 Schema browser*

As stated, a schema is provided by each connector and contains the description of "elements" that can be found in the system. Those "elements" are (schema) types with their (schema) properties and optionally (schema) methods and (schema) classes. The schema that was returned by the connector will be stored within the synchronization project. The synchronization engine can send this stored schema back to the connector upon connecting. This allows the connector to access the schema information without actually doing calls to

the system. However, this is an option for the connector developer to speed up certain processes/calls. Some connectors will still need access to the schema.

To refresh the persisted schema, select the system you want to refresh (OneIM or target system) in the "Configuration" pane and click the "Update schema" button.

## Schema types

Schema types are, as the name implies, the types of objects that can be found in a system like users, groups, containers etc. In the OpenLDAP default schema, inetOrgPerson is a schema type. Each schema type has attributes (not to be confused with schema properties) that describe several properties of the type itself. There is a base set of attributes and each connector can add its own attributes. Those are not relevant for the engine itself but can be leveraged, when the connector uses the persisted schema passed by the engine. The following base attributes are available for each schema type.

| **Attribute** | **Description** |
|---|---|
| Name | The name of the attribute returned by the system |
| ScriptSafeIdentifier | The name of the schema type with replaced special characters to be used in scripts etc. |
| Displayname | A more user friendly name |
| DisplayPattern | The connector can expose a DisplayPattern for a schema type which specifies, which schema properties are used to construct a display. Example: "%cn% - (%sn%, %givenName%)" |
| IsVirtual | A connector can expose additional schema types that not directly refer to a type in the target system. The connector developer should mark those types as virtual when exposing them in the schema. |
| IsAdditional | (for internal use) |
| IsMNType | (for internal use) |
| IsObsolete | Indicates that objects of this type should no longer be used because they might not be available anymore in a new version of the target system (or its connector) |
| IsReadOnly | Indicates that objects of this type cannot be created/modified/deleted by the connector |
| MetaData | Can be used by the connector store additional data. |
| SchemaKey | A full qualified object identifier (auto generated) |
| Uid | Element id in the persisted schema (auto generated) |
| IsLocked | Indicates that this element was exposed by the connector |

## Schema properties

The properties that are available for a certain schema type are called schema properties. “givenName” and “sn” for example, are properties of the schema type “inetOrgPerson”. A schema property has a base set of attribute to specify its behavior. Additional attributes can be added by the connector. Here, the same rules as for additional schema type attributes apply. They are not used by the engine, but passed back to the connector for optimization purposes.

The following table contains the common base attributes for schema properties.

| **Attribute** | **Description** |
|---|---|
| Name | The name of the property |
| ScriptSafeIdentifier | The name of the attribute with replaced special characters to be used in scripts etc. |
| Description | A brief description of the property |
| DisplayName | A more user friendly name |
| AccessConstraint | Read/Write constraints<br>- ReadOnly<br>- ReadAndInsertOnly (read/write only on insert, otherwise only read)<br>- WriteOnly (i.e. passwords)<br>- None (read/write) default |
| AutoFillBehavior | Indicates, if the Property is calculated by the system.<br>- Always (the system always calculates a value)<br>- Never (the system does not calculate a value)<br>- Conditional (the system may calculate a value) |
| DataType | Data type of the property contents<br>- Binary<br>- Boolean<br>- DateTime<br>- Decimal<br>- Float<br>- Integer<br>- String<br>- SystemObjectData (this datatype can be exposed for references which will be described later. It represents an instance of an object from the target system) |
| DataTypeSize | Optional information how big the contents can be i.e. a max. number of bytes if DataType is “Binary” |
| IsDisplayPart | Calculated automatically based on the “DisplayPattern” attribute of the schema type the property belongs to |
| IsHierarchySortCriterion | Indicates that this property contains a value that, when entries are sorted by it, will return a hierarchically sorted list i.e. canonicalname in Active Directory. |
| IsLargeObject | Indicates that this property contains a big amount of data or is expensive to load |
| IsLocked | Indicates that this element was exposed by the connector |

| | |
|---|---|
| IsMethodParameter | Indicates that this property is actually only a parameter for a schema method exposed by the connector. That means that this "property" has no real counterpart in the actual target system. |
| IsMultivalue | Indicates that the property contains an array of values |
| IsObsolete | Indicates that this property should no longer be used because it might not be available anymore in a new version of the target system (or its connector) |
| IsPreferredKey | Indicates that this property is a unique key that identifies an object (i.e. like a GUID or a distinguishedName) and that the connector will prefer this key over other potential keys. There can only be one preferred key per schema type |
| IsReference | Indicates that this property contains a reference (or multiple references in case it is also marked as IsMultivalue). |
| IsRevisionCounter | The property acts as a revision that can be used for synchronization optimization. There can only be one revision counter property per schema type. If a combination is required (i.e. the calculated maximum of createTimeStamp and modifyTimeStamp, the connector needs to expose this, as a single virtual property) |
| IsSecretValue | Properties that contain sensitive data are marked with this attribute. |
| IsUniqueKey | Indicates that this property is a unique key that identifies an object (i.e. like a GUID or a distinguishedName). In contrast to IsPreferredKey, a schema type can have multiple unique keys. |
| IsVirtual | A connector can expose additional schema properties for a schema type that not directly refer to a property in the target system. The connector developer should mark those types as virtual when exposing them in the schema. They are usually prefixed with "vrt" i.e. "vrtRevision" |
| MandatoryBehavior | Determines, if the property is mandatory for an object of the schema type. Possible values are:<br>- Never (optional property)<br>- Always (mandatory property)<br>- RuleBased (mandatory depending on the data situation)<br><br>While the "Always" behavior is enforced by the engine during data synchronization, "RuleBased" is used as an UI indicator and for creating the synchronization execution plans. |
| MaximumLength | The max. length of the content (i.e. for Strings) |
| MetaData | Can be used by the connector store additional data. |

## References

A very complex topic is the definition of relations between objects. Usually, target system objects have properties that store keys to other objects in some form. This information is also part of the schema exposed by the connector. A reference description contains, which property points to which schema property of which schema type.

Example(s):

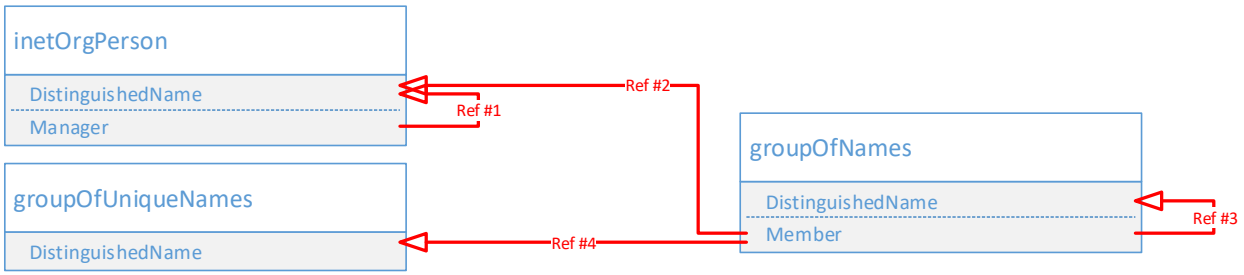



*Figure 47 Reference example*

| Ref# | SchemaType | Property | | Target schemaType | Target property |
|---|---|---|---|---|---|
| 1 | inetOrgPerson | Manager | → | inetOrgPerson | DistinguishedName |
| 2 | groupOfNames | Member | → | inetOrgPerson | DistinguishedName |
| 3 | groupOfNames | Member | → | groupOfNames | DistinguishedName |
| 4 | groupOfNames | Member | → | groupOfUniqueNames | DistinguishedName |
| | … | … | → | … | … |

Since distinguishedName(s) in LDAP can point to objects of any schema type, the potential reference targets are basically all available schema types. You can view those targets in the schema browser of the Synchronization Editor.

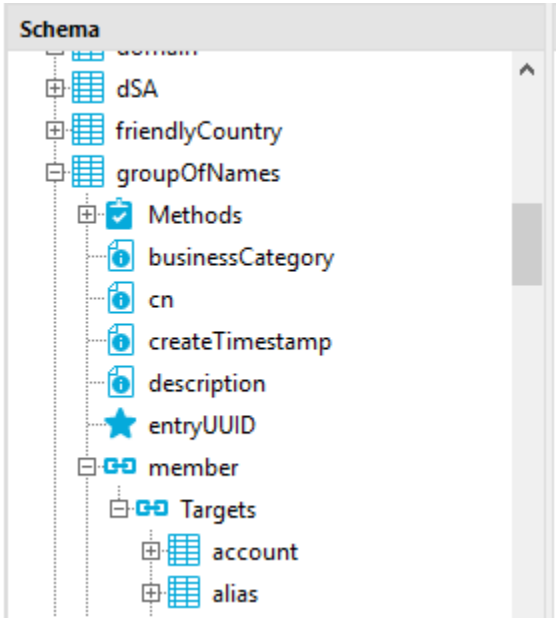



*Figure 48 References in schema browser*

## Virtual Properties

In addition to the properties that are exposed by the connector, virtual properties can be defined at the engine level. These can either be based on other (native) properties of a schema type to perform conversions, be calculated by scripts or represent a fixed value. They can be created either in the mapping editor, shown later in this document, or the schema Browser (starting with OneIM version 8.0). To add properties, navigate to the schema browser and click the edit icon:

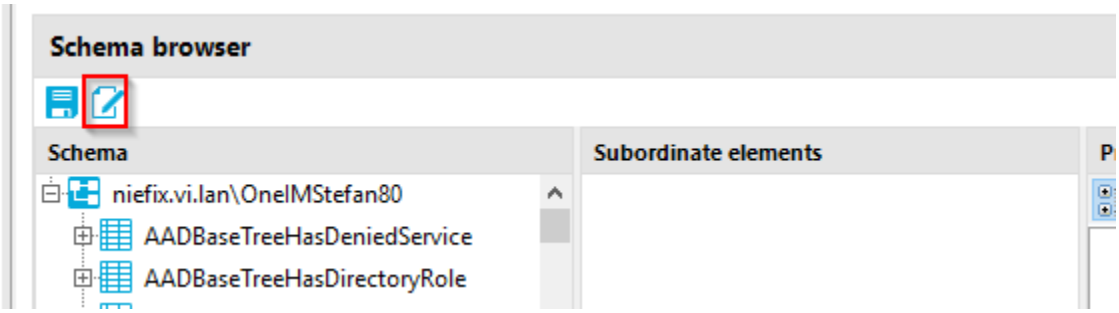



*Figure 49 Open schema editor*

New schema properties can be created by selecting the schema type and clicking the "Add property" action. Then choose the desired type of property.

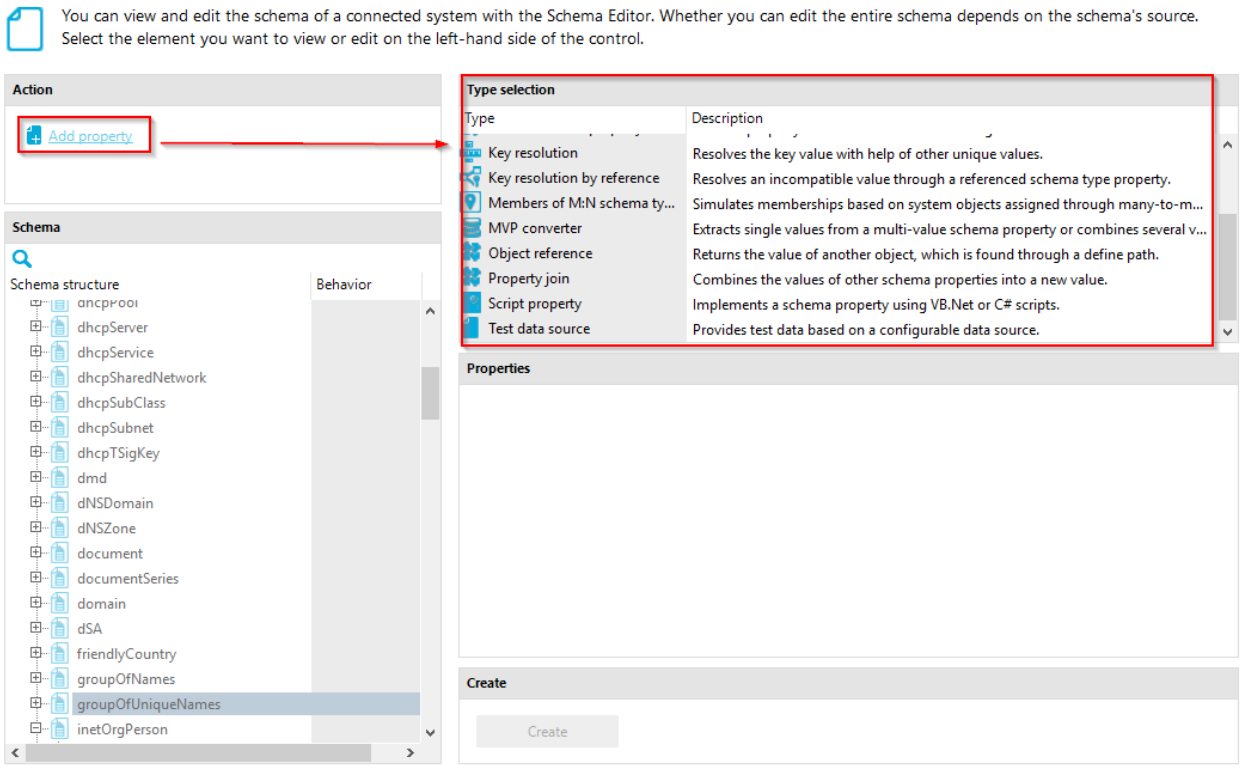



*Figure 50 Adding a schema property*

The different types of properties will be explained during the chapter dealing with the mapping creation.

## Schema methods

Schema methods describe common actions that can be performed with objects of a schema type. Common methods that are supported by most of the connectors are “Insert”, “Update” and “Delete”. But a connector can expose additional methods. The OneIM connector exposes MarkAsOutstanding or UnMarkAsOutstanding as schema methods.

When a method is executed, a SystemObjectData is passed to the connector along with the name of the method that shall be executed. The SystemObjectData instance itself is the only parameter the method gets. Therefore, any data required to execute the method needs to be present in the SystemObjectData. That means, if properties other than the native object (schema) properties are required to perform the method, the connector must expose additional virtual schema properties that are marked as “IsMethodParameter”. They are then available for mapping like regular properties, which will be described later.

Hint: The set of schema methods is defined by the connector. It is not possible to add your own virtual schema methods in the Synchronization Editor.

## Schema classes

Schema classes are used to group/distinguish the objects of a schema type based on attributes. You can see all schema classes of your current synchronization project by navigating to “Configuration” → “<Connection>” and opening the section “Schema classes”.

As you can see, there is already at least one schema class available for each schema type. This is the so called “master” class that is a class without a filter containing all objects of the schema type. It is usually named “<schema type name> (all)”.

To add a new schema class, click the “new” icon as shown on the screenshot.

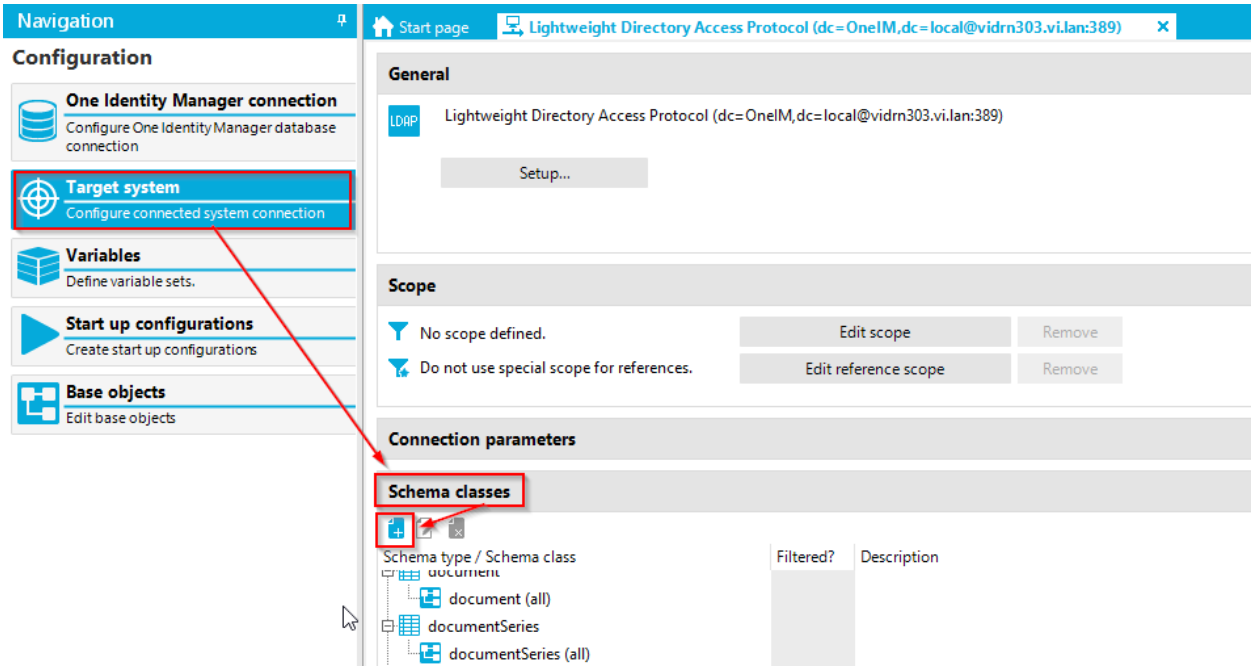



*Figure 51 Create a new schema class*

There are currently two class implementations available for creating schema classes, each having a different purpose.

## Unique objects

The "Unique objects" schema class can be used, to group objects by one or more properties and then return one "random" element of that group.

**Example**

Suppose you want to populate your OneIM Department table with all departments that are found in the "departmentNumber" property of your inetOrgPerson objects. You would create a schema class "Departments (from inetOrgPerson)" that uses the departmentNumber property as grouping property. The following table shows some example data. The objects with the departmentNumber marked in blue would be returned, all others are omitted.

| givenName | sn | departmentNumber |
|---|---|---|
| Mary | Mc Donald | Sales |
| ~~Jack~~ | ~~Homes~~ | ~~Sales~~ |
| Bob | Meyer | Development |
| Gary | Grey | HR |
| Lindsay | Green | Support |
| Stefan | Börner | <empty> |

Note: The element returned from the group can vary from call to call. So you should not use properties in your mapping configuration that are not part of the grouping properties

The configuration of that class would look like this. The “Filter” section will be explained in the section regarding the “Generic schema class”

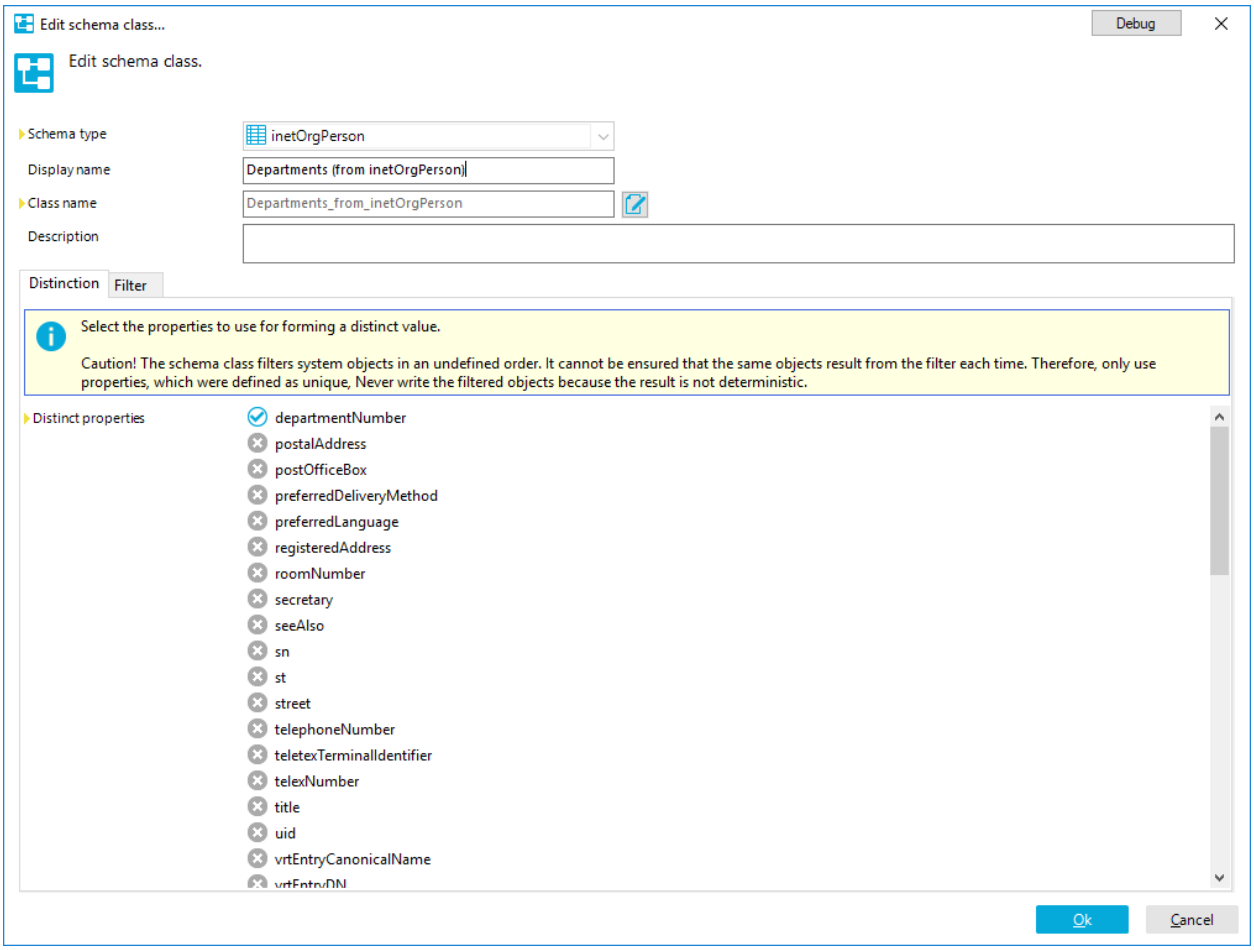



*Figure 52 Unique schema class example*

In order to check the results, you can open the target system browser. The “schema types” tree will contain the configured schema classes such as the “Departments from inetorgPerson” class. Click it to see the resulting objects.

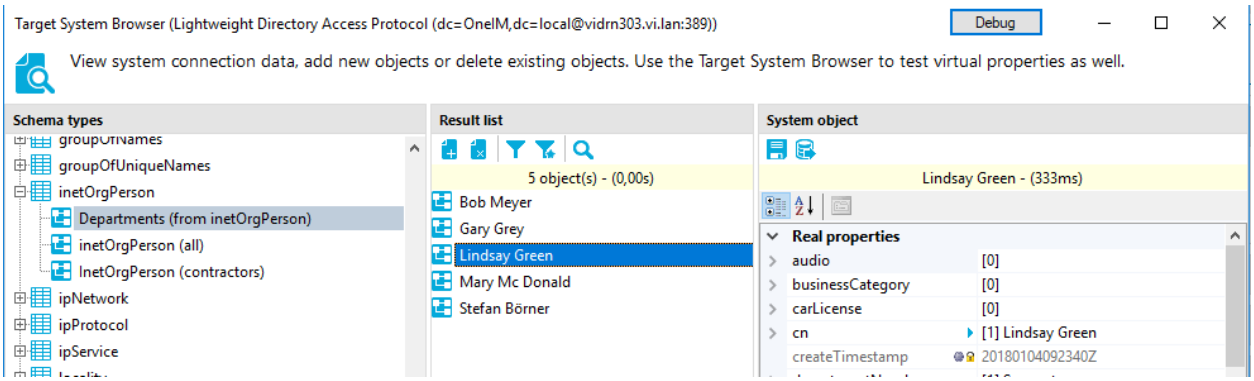



*Figure 53 Unique schema class example*

## Generic Schema class

The generic schema class allows you to select a subset of the objects of a schema type based on the objects attributes. This is done by a two phased filter mechanism. The first filter is the system filter that is handled entirely by the connector. Depending on the connector, the syntax varies. For the OneIM connector, the syntax is SQL, for the LDAP connector the syntax needs to be a valid LDAP query. Since not all connectors (or the APIs that they use) support system filters, this first filter is optional. However, if system filtering is supported, it should be used to increase the synchronization speed. The second filter (the select objects filter) is applied to resulting objects of the system filter to determine the final set of objects belonging to the schema class. It can be more restrictive than the system filter and must be specified when creating a schema class (exception to the rule: the out of the box master class).

**Example:**

If you want to just handle LDAP inetOrgPerson objects that have a certain value in employeeType you would have to create a schema class "inetOrgPerson (Contractors only)". To do that, open the "Configuration" pane, and select the system you want to create a schema class for (LDAP in our example). Then open the "Schema classes" section. To create our "Contractors" class, click the "new" icon. As class type, select "Generic schema class".

Enter the following values:

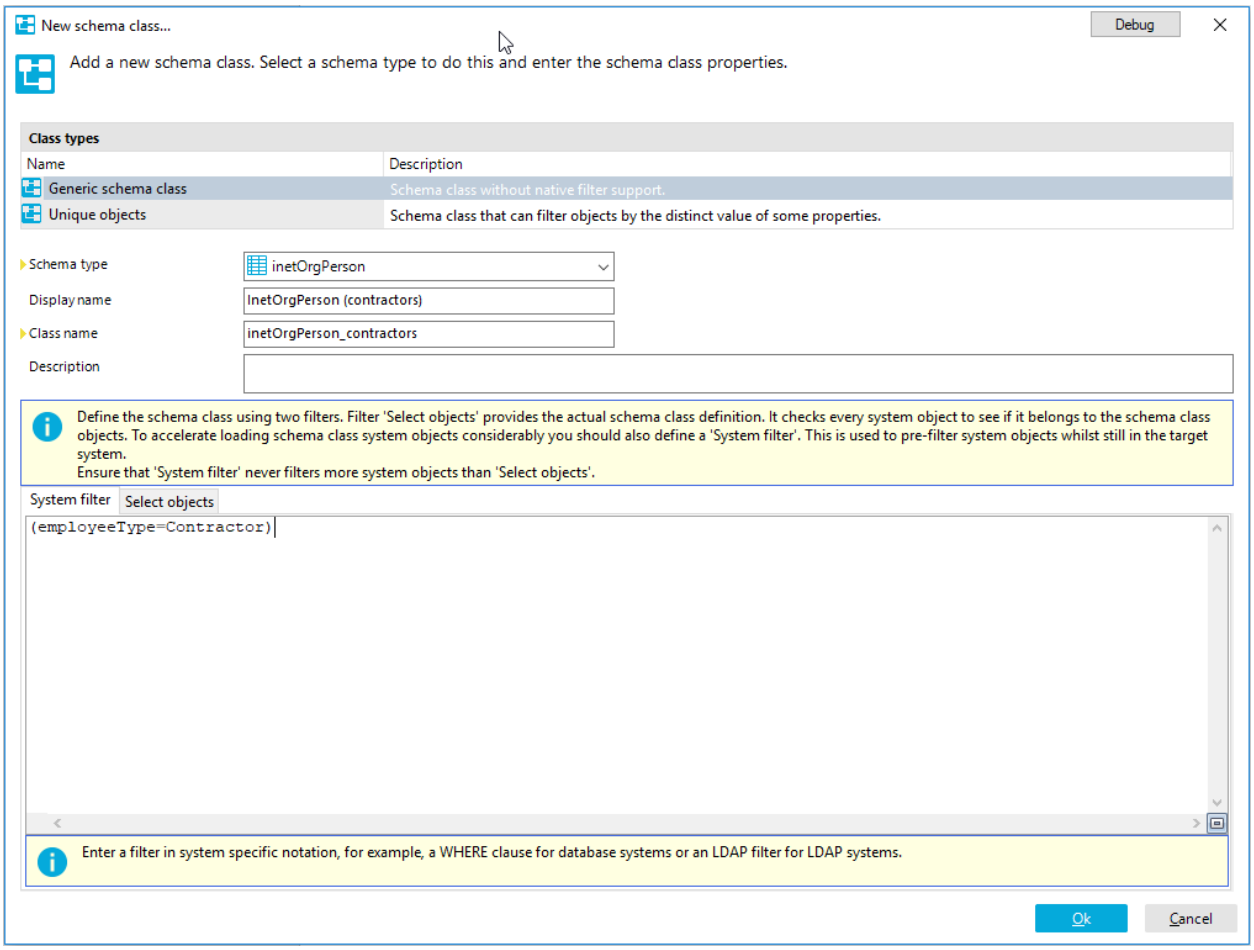



*Figure 54 Generic schema class example*

The system filter is added as an AND condition to the internal filter (objectclass=inetOrgPerson) for querying the schema type. So the resulting query will be (&(objectclass=inetOrgPerson)(employeeType=contractor)).

As described, the select objects filter needs to be specified as well. This filter is applied to the objects that is returned by the connector or are newly created in memory during the execution of a workflow. In order to match exactly the same objects as the system filter enter the following value:

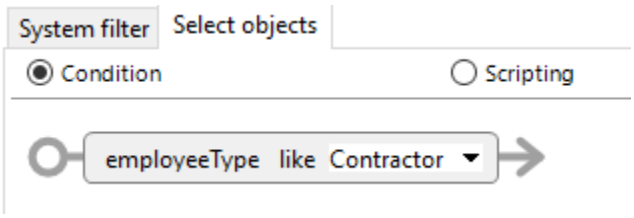



*Figure 55 Select object Filter*

You might wonder why the operator "like" is used. The reason is: employeeType is a multi-valued property and the "like" operator is applied to each value. If employeeType was a single valued property you could have used "employeeType = Contractor" as filter. Let us view the results in the target system browser:

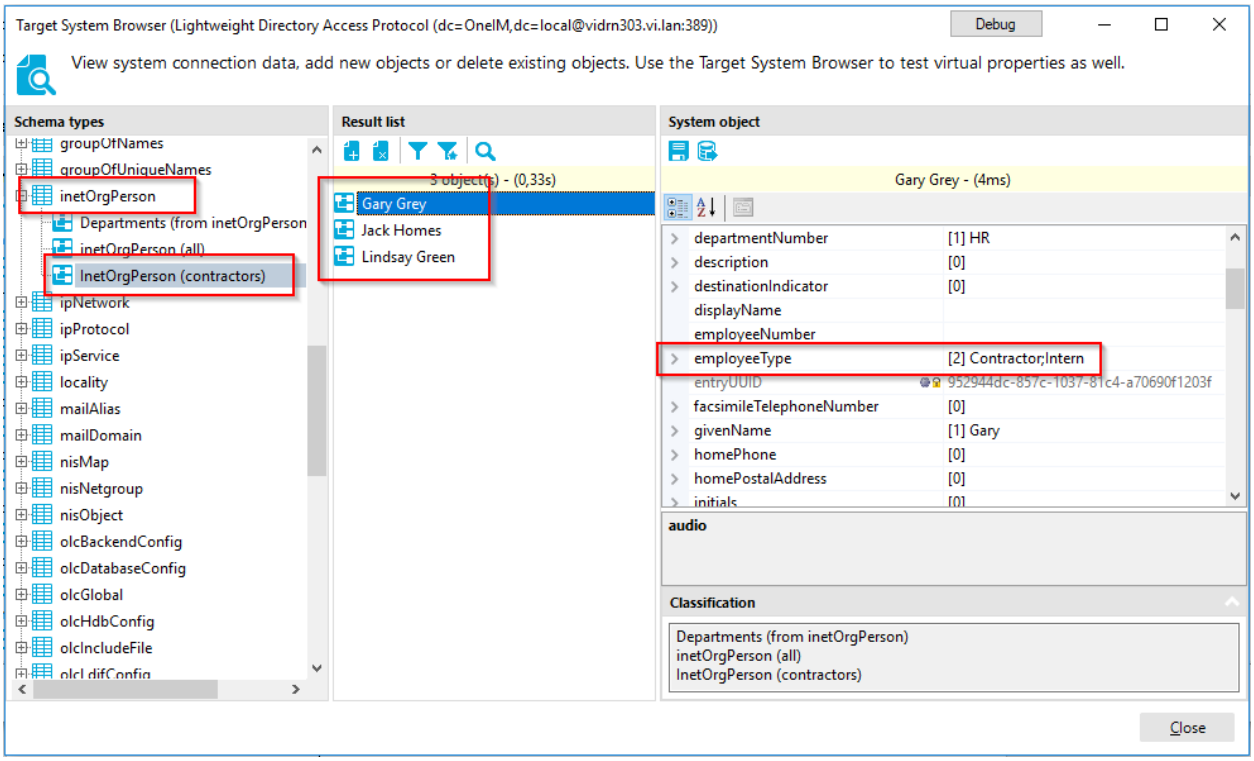
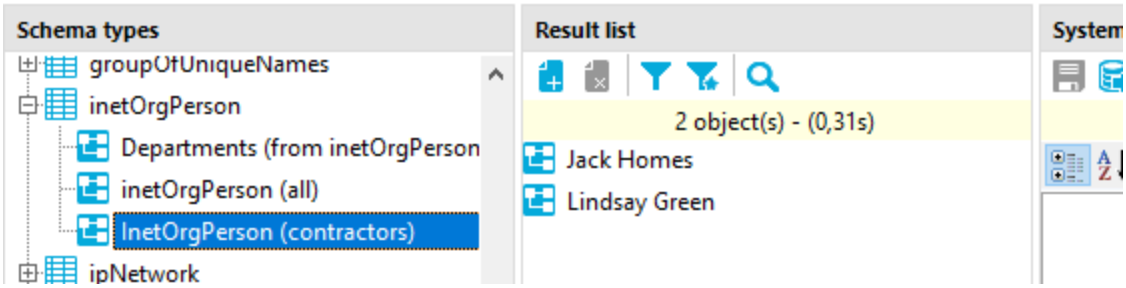



*Figure 56 Generic schema class result*

As you can see, three objects are returned. The selected object does not only have "Contractor" but also "Intern" as value for employeeType. To demonstrate that the object filter can be more restrictive, edit the schema class and set the system objects filter to the following value while leaving the system filter unaltered:


*Figure 57 Altered system objects filter*

The result in the target system browser will look like this




As you can see, "Gary Grey" is no longer in that list.

So as you can see, the system objects filter was applied on the result returned by the connector as the following illustration shows:

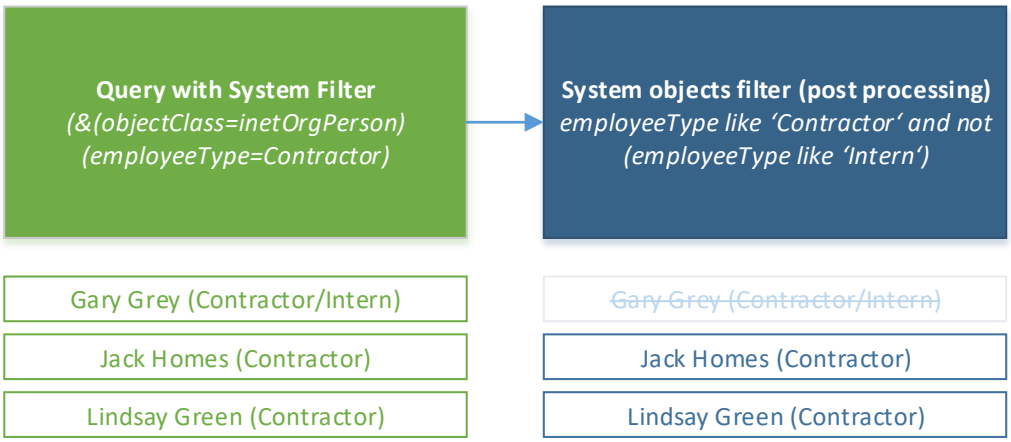



*Figure 58 Filtering with system and system objects filter*

In theory, you can also just specify the system objects filter and would get the same result. But this is considered to be a configuration flaw because in this scenario, the following illustration applies:

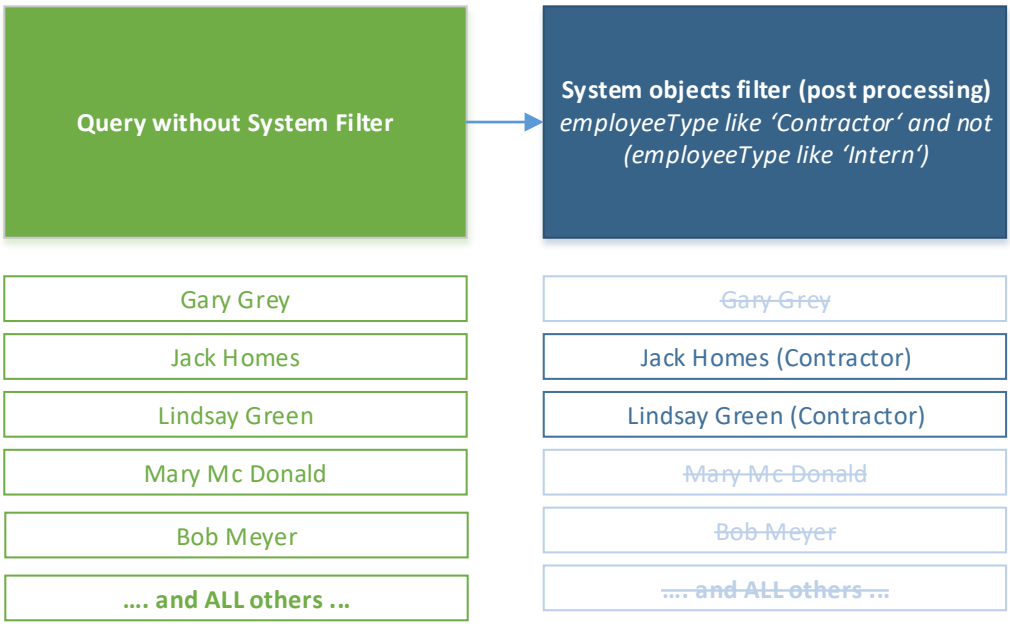



*Figure 59 Class without system filter*

As you can see, the connector would load the complete list of all inetOrgPerson objects from LDAP which will take a lot longer than just querying a reduced list and then the engine has to apply the filter to each of this returned objects. The overhead is huge!

Hint: If available, use system filters when designing schema classes. This will speed up the synchronization.

You can change a schema class after its creation. However, once synchronization was fully configured and run, you have to be very careful because a change in filters can lead to unwanted create, update or delete operations during the next synchronization.

Hint: Be careful when changing an existing schema class. It is safe as long as it is not used yet.

# Mappings

To configure a synchronization, the engine needs to know, how to transfer data from one system to the other (OneIM to LDAP / LDAP to OneIM) and how to identify matching objects in both systems. Those are the two main tasks of the mapping configuration.

A mapping is always configured between two schema classes (not types!) of different systems (in our example OneIM and LDAP). A synchronization project can contain multiple mappings for the same pair of schema classes. Their usage depends on the workflow configuration which will be covered in a later section.

To create a new mapping, click the "new" button in the mapping section.

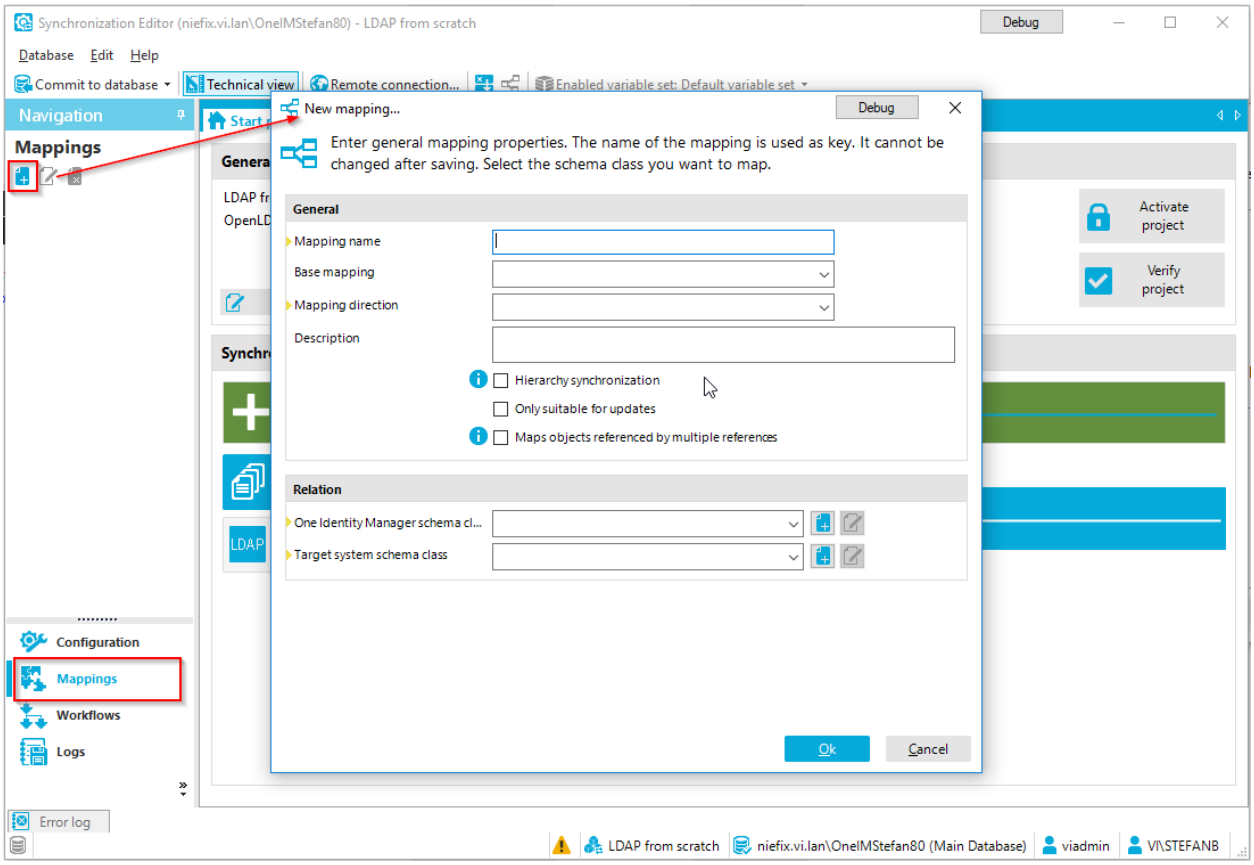



*Figure 60 Insert a new mapping*

Specify values for the following fields:

# Mapping Fields

## Mapping name

This is the name of your mapping. As this will appear in the list in the Synchronization Editor, log messages etc., consider using a name that cannot be confused with other mappings that might handle the same classes.

## Base mapping

You can extend an existing mapping by creating a new mapping and set "base mapping" to your existing mapping. Be aware that this "derived" mapping must use the same schema classes as the base mapping. This can be used when creating very different mappings for different workflows for the same schema classes. In those scenarios, a mapping with the very basic set of common rules (i.e. matching rules) can be defined as base. The mappings that inherit this base mapping then just need the use case specific rules. Changes to matching etc. can then be done in one place (base mapping).

## Mapping direction

This direction specifies, in which direction the mapping is able to transfer data. It is used as the default for all rules that will later be configured in the mapping. This default can be overwritten per rule. The following values are available:

| | |
|---|---|
| Both directions | The mapping can transfer data in either direction |
| One Identity Manager | Data transfer is only allowed/available from the target system to OneIM |
| Target system | Data transfer is only allowed/available from OneIM to the target system |
| Inherit from base mapping | Use the configuration of the base mapping. This option can only be used if a base mapping was actually specified. |

The mapping direction that is effective for a mapping rule is the one that is configured closest to the rule which means a direction configured on rule level will override the direction of the mapping. However, there are other features besides the mapping direction that will prevent data from being transferred. Those features are the "Do not overwrite" setting of a mapping rule or mapping conditions configured for a rule.

## Description

This provides a brief optional description for the mapping. A good practice is to name the schema classes that are handled by the mapping and in case of multiple mappings for the same schema classes a little hint in which context they are used.

## Hierarchy synchronization

This option can be used in systems that have some kind of container structure. By marking a mapping with this option, the engine will try to build the synchronization execution plan in

a way that those objects are handled early in the synchronization process. This speeds up the synchronization by avoiding retries that might occur during the synchronization of objects that are contained in those containers.

## Only suitable for updates

This option indicates that the mapping will be able to transfer data between existing pairs of objects but not create new ones. Deletion of objects is also possible. The engine needs to know this during the evaluation of the mappings when creating the synchronization execution plan. During this evaluation, it is checked, if all mandatory properties are actually mapped by the rules of the mapping. If this is not the case, the engine will throw an error. However, if existing object pairs just need to be updated, there is often no need to map/populate each mandatory property.

## Maps objects referenced by multiple references

When the synchronization engine handles references e.g. the "member" property of "groupOfNames" LDAP objects it needs to compare the list of members from LDAP with the list of members that are stored in OneIM. Depending on the mapping rule configuration, those lists contain SystemObjectData instances.

Example:

| OneIM member list (display – schema type) | LDAP member list display – schema type |
|---|---|
| Doe, Jon (LDAPAccount) | Jon Doe (inetOrgPerson) |
| Development (LDAPGroup) | Development (groupOfNames) |
| | Gary Grey (inetOrgPerson) |

Now, the engine needs to find all mappings in the synchronization project that handle schema classes. These schema classes use the schema types that occur in the list. For the above sample the engine would need to find the following mappings:

A mapping that (Jon Doe):

… uses a schema class that uses the schema type LDAPAccount in OneIM
    … uses a schema class that uses the schema type inetOrgPerson in LDAP

A mapping that (Development):

    … uses a schema class that uses the schema type LDAPGroup in OneIM
    … uses a schema class that uses the schema type groupOfNames in LDAP

All mappings that (Gary Grey)

    … use a schema class that uses the schema type groupOfNames in LDAP

With those mappings the engine is able to find and populate the reference attribute in the system to be handled according to the current synchronization.

Since you can have a large variety of mappings, use the option “Maps objects referenced by multiple references” to mark those mappings that the engine should consider when dealing with those references.

Hint: You can safely mark your default mappings as “Maps objects referenced by multiple references”. Just omit mappings that were created for special workflows that i.e. do a “fire and forget” type of action.

## One Identity Manager schema class

The schema class of the OneIM connection you want to use. You can also create a new one by clicking the “add” button next to the selection list.

## Target system schema class

The schema class of the target system connection you want to use. You can also create a new one by clicking the “add” button next to the selection list.

After you specified the data mentioned above, the mapping wizard starts. This wizard helps you to create the mapping rules for your mapping. It offers the following two options

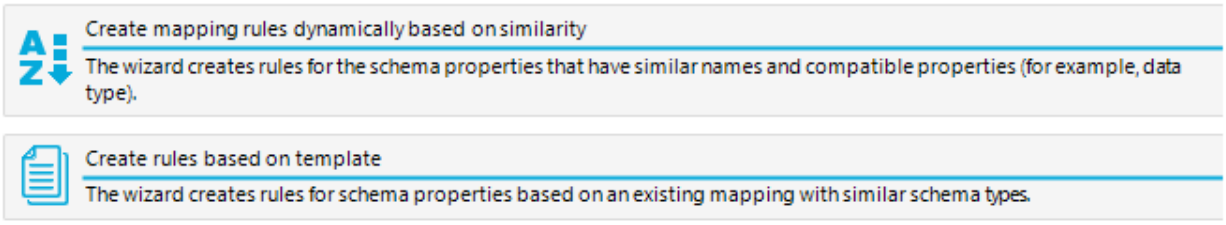



*Figure 61 Mapping editor options*

Since we want to create the mapping and its rules from scratch, we will cancel the wizard at this stage.

A mapping consists of mapping and matching rules which will be described in the following sections

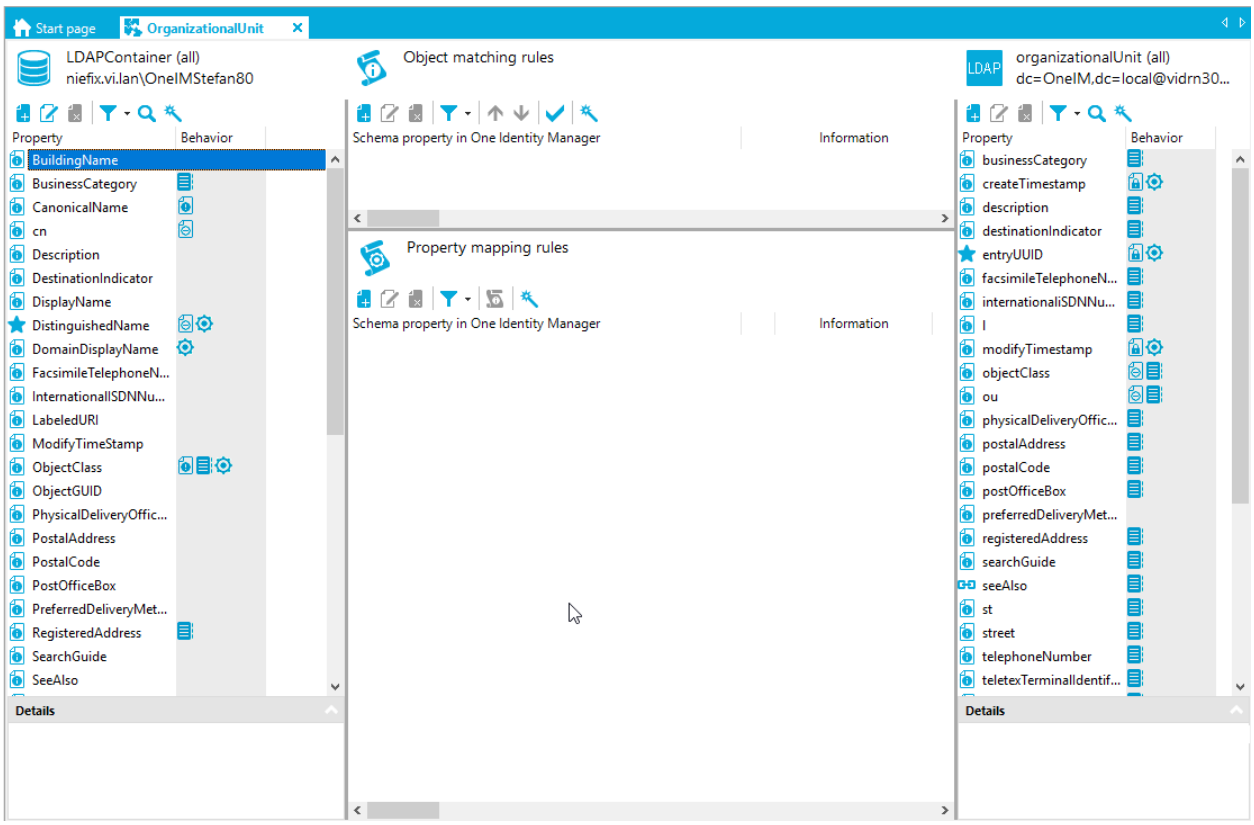



*Figure 62 Mapping editor*

## Mapping rules

Mapping rules describe how data can be transferred from one system to the other. To be more precise, it allows you to configure, which property value from one schema class of the mapping, can be written to which property of the other schema class of the mapping. Those rules are also responsible for comparing the two values to check, if a data transfer is required at all. To be able to do this, both properties need to have the same data type and setting for "IsMultiValue".

Since this is not always the case, some kind of data conversion is required. At this point, virtual schema properties are used. There is a whole variety of implementations available out of the box OneIM module can add new ones. Details about the different available virtual properties can be found in the virtual schema properties section.

The rules on how to transfer data of a property on the one side to the property on the other side, are determined by the settings of the mapping rule. The common settings for all types of mapping rules are the mapping direction. It describes, in which direction the engine **is allowed** to transfer data using this rule. This is dependent on multiple things:

- Are there any access constraints in place for one or both properties? If so, they need to be taken into account when specifying the mapping direction. Read-only properties cannot be written and write-only properties cannot be read.
- Do you want one system to be the master for a particular property? If so, the direction should point "away" from the side that you want to be the master.
- If the mapping rule shall be able (allowed) to transfer data in either direction, choose "Both directions".
- To prevent (disable) data transfer via a rule, set its mapping direction to "Do not map".
- By selecting the "Use mapping" option, the rule will be configured like the mapping it is contained in.

The next settings influence the mapping rule behavior during execution time. During execution time the workflow sets the synchronization direction (either "to the left" or "to the right") and all mapping rules that are allowed to transfer data in the given direction will be executed.

## Value comparison rule

The value comparison rule is the most commonly used mapping rule. Therefore, it is automatically used when properties are mapped via drag and drop in the Synchronization Editor. It is designed to transfer the value of a property in one system to a property of the matched object in the other system if the values are not equal yet (therefore the "comparison" part in the rule name). This rule has some additional settings that very useful in certain scenarios.

In addition to the common mapping rule settings, the value comparison rule offers the following settings:

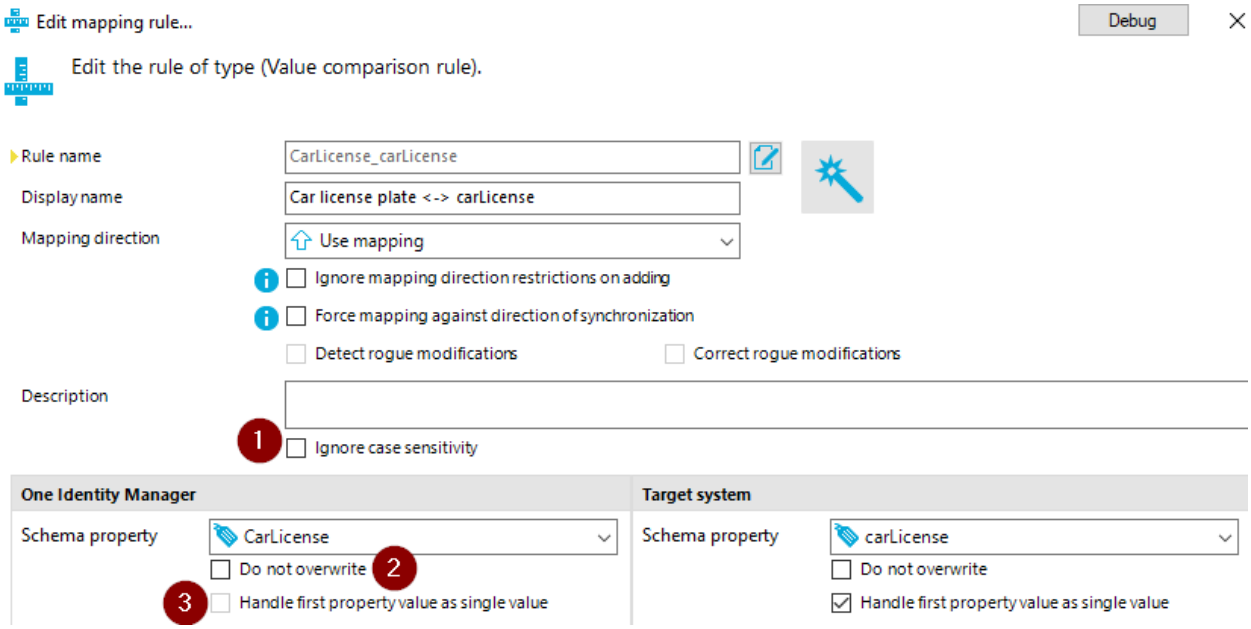



*Figure 63 Value comparison rule properties*

**Ingore case sensitivity (1)**
As stated, the rule only transfers data after a comparison for property value equality stated that the mapped properties have different values in the systems. The "Ignore case sensitivity" setting instructs the rule, to compare string values case insensitive.

**Do not overwrite (2)**
This can be set for either side of the mapping rule and changes the behavior of the rule as follows. The property will only be set, if the target object's property is not yet set. If you set this option for the One Identity Manager property "CarLicense" in the rule shown in the screenshot, a synchronization "Target system" → "OneIM" would only update those objects in OneIM that do not have a value set for "CarLicense" yet. In contrast to the "Ignore mapping restriction on adding" setting, "Do not overwrite" is also effective for update.

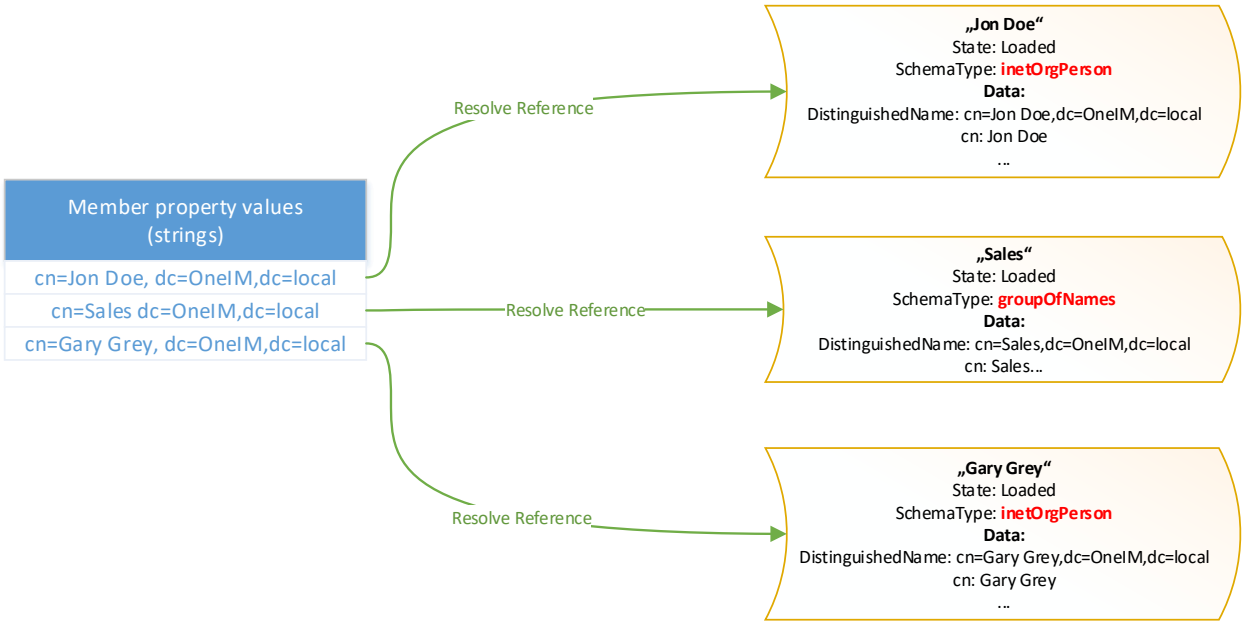
**Handle first property value as single value (3)**
Especially in LDAP based systems, properties are often defined as multi-valued although they only contain one value most of the time. This option causes the rule to only handle (compare/read/write) the first value of a multi-valued property. Otherwise you would have to create a virtual MVP converter property for each of the properties.

## Multi-reference mapping rule

As the name implies, this rule handles the data transfer between multi-valued references. So the properties you select in this rule have to have the attributes **IsMultiValue** and **IsReference** set to true (you cannot change those attributes for properties that were natively exposed by the connector). Furthermore, the datatype that the rules operates on is

SystemObjectData. If the datatype of the selected property is not SystemObjectData, the rule implies that the values are some kind of key that can be used to load the actual object as SystemObjectData. This process uses the system connector reference resolution method.

Let us take a look at the following example. A LDAP group member property contains the distinguished name strings of its members. The connector's reference resolution method takes the distinguished name strings as input and tries to load the referenced objects and return their SystemObjectData representation.




The properties requested for the various target objects are determined as follows:

Due to the reference schema information, the potential targets of a multi-valued property are known to the synchronization engine. Based on this information, it searches for other mappings that involve the potential targets. From these mappings, all properties are gathered that are used during the matching process by inspecting the matching rules. In this process only those mappings are taken into account that are marked as "Maps objects referenced by multiple references". If the mappings use schema classes other than the master class, all required properties that are used for configuring the schema class are included as well.

If the connector returns a resolved object that has no mapping configured, the engine will stop processing with an error message. If you only want to handle a subset of reference targets, you can configure include/exclude lists of schema types that shall be handled by the rule. The following screenshots shows a rule that will ignore all resolved objects that are not of type "groupOfNames", "groupOfUniqueNames" or "inetOrgPerson"

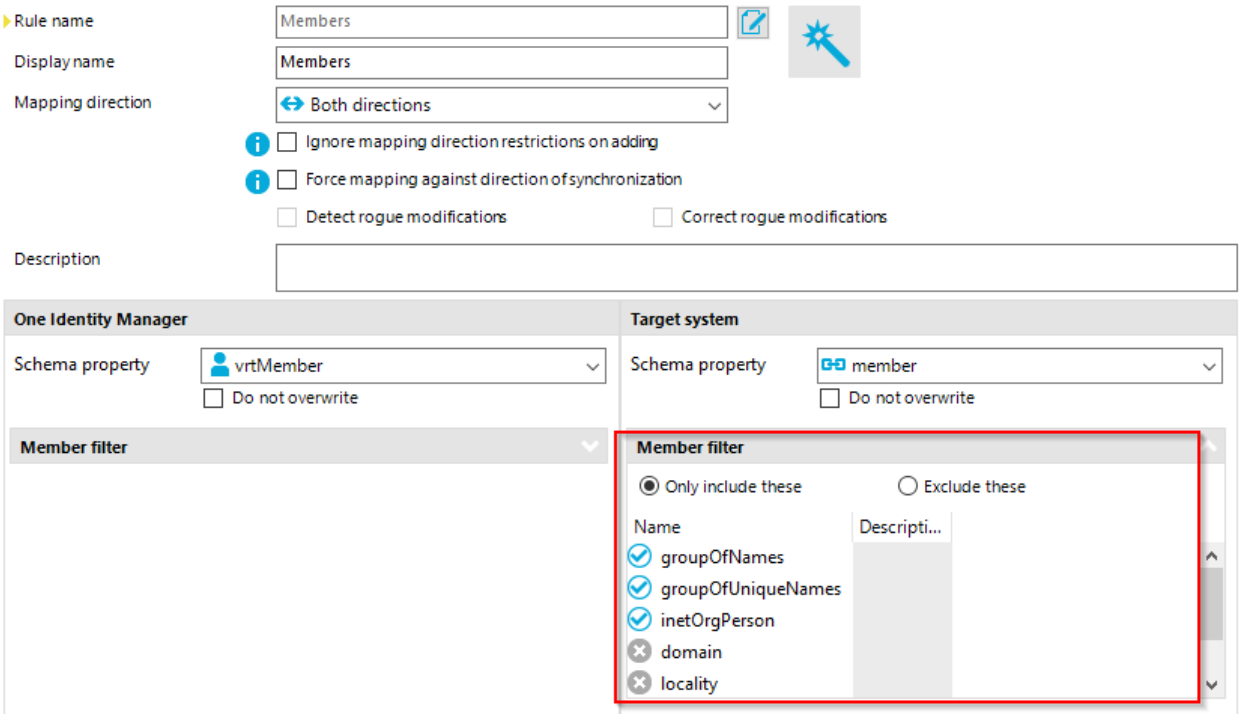



*Figure 64 Member filter*

Once the mapping rule has the contents of the two properties available in form of SystemObjectData instances, it can start performing the data transfer in the execution direction. To do that, the first task is to map check the schema type of the member objects on each side, and search for a suitable mapping that can be used for comparison.


*Figure 65 Mapping search of multi reference mapping rule*

In the illustration above, the mapping rule between OneIM property "vrtMember" and LDAP system property "member" has already loaded the property values as SystemObjectData instances for both sides. The LDAP object "Admin" was ignored during this process because it is of schema type "organizationalRole", which is not part of the include list (member filter) configured for the mapping rule. Now the rule searches for suitable objects. For the objects from OneIM side (orange) mappings are searched that are using an OneIM schema class that is based on the schema type of the object from OneIM. For objects from LDAP (green) mappings are searched that are using a LDAP schema class that is based on the schema type of the object from LDAP. Note that in the above example, the OneIM object "Sales" returns two potential mappings because the schema type "LDAPGroup" is used by the OneIM schema classes of the mappings "LDAPGroup (groupOfNames) ← → groupOfNames" as well as "LDAPGroup (groupOfUniqueNames) ← → groupOfUniqueNames".

Once the mappings are determined, the rule starts to iterate through those mappings and performs an object matching between objects from OneIM with objects from the target system (LDAP) that suit the mapping according to the schema class. If a match was found, the object pair is removed from the list of objects to be processed by the further mappings.

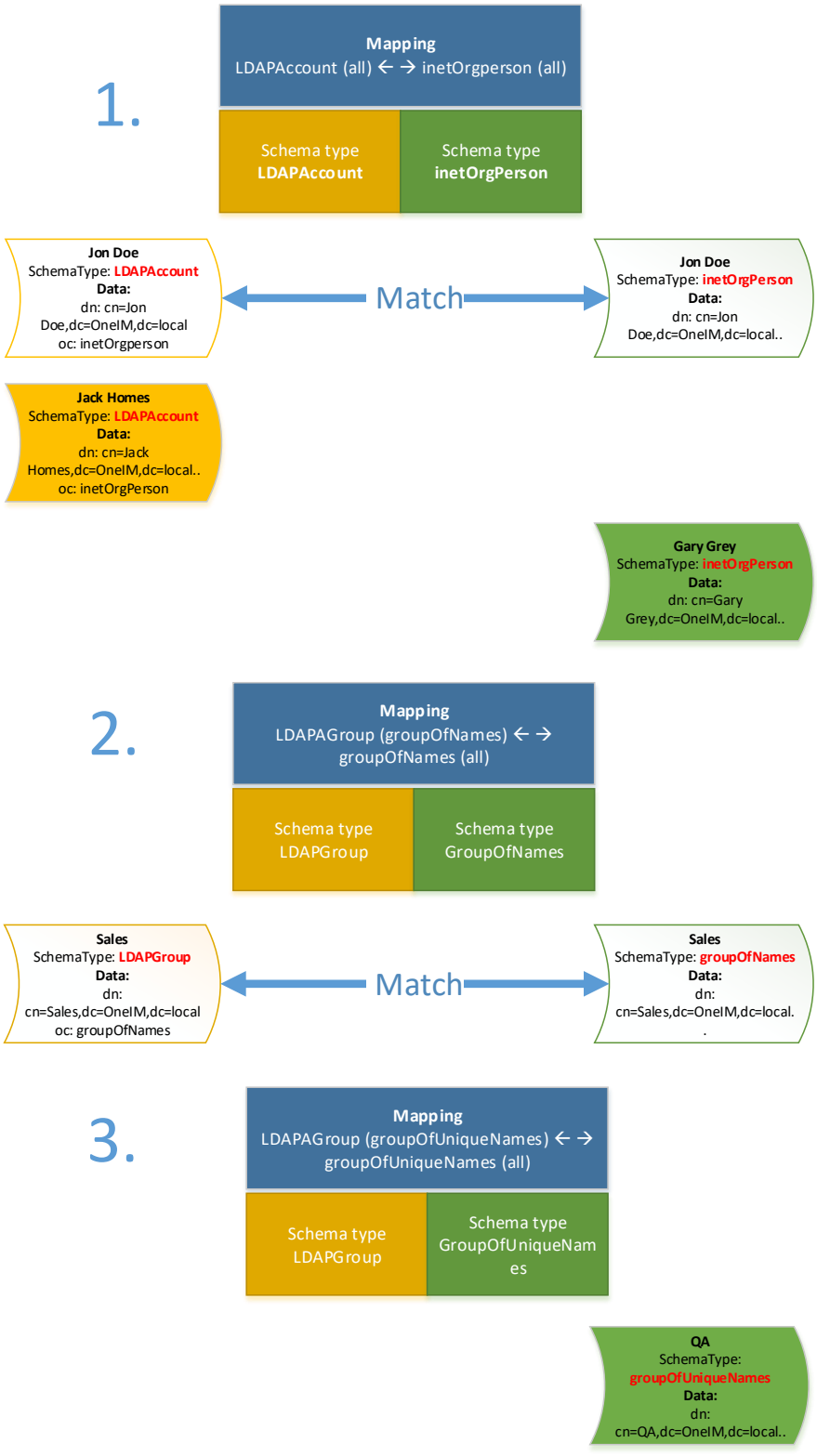



*Figure 66 Matching in multi reference mapping rule*

In the illustration above, the mapping LDAPAccount (all) ← → inetOrgPerson is executed first. The suitable objects according to their schema type (class) for this mappings are

OneIM: Jon Doe, Jack Homes
LDAP: Jon Doe, Gary Grey

While Jon Doe from OneIM matches with Jon Doe from LDAP, Jack Homes in OneIM and Gary Grey in LDAP remain on the list of unmatched objects. Since there is no other mapping in place that suits those objects, they remain on the final "delta" list.

The second mapping that is used, is "LDAPGroup (groupOfNames) ← → groupOfNames". The suitable objects from OneIM and LDAP are the matching pair Sales. So both "Sales" from OneIM and "Sales" from LDAP are removed from the list of remaining objects.

The last mapping that is executed is "LDAPGroup (groupOfUniqueNames) ← → groupOfUniqueNames". The remaining object list for this mapping just contains one object "QA" from LDAP that will also become part of the delta list.

So the remaining delta list looks as follows


*Figure 67 Delta list*

Now the rule needs to evaluate the current execution direction. If the execution direction is "Target system", "Jack homes" from OneIM needs to be added to the member property in LDAP while "Gary Grey" and "QA" need to be removed. If the direction is "OneIM", "Jack Homes" needs to be removed (or marked as outstanding) and "Gary Grey" and "QA" need to be added in OneIM.

Let us assume, the execution direction is “Target System”. The rule needs to construct modifications for the LDAP “member” property.

Add: CN=Jack Homes, dc=OneIM,dc=local
Remove: CN=Gary Grey,dc=OneIM,dc=local
Remove: CN=QA,dc=OneIM, dc=local

The following section describes, how the rule is able to translate the objects of the delta list, to the actual values that are written to the property.

At first, we look at the objects of the delta list that exist on OneIM side. “Jack Homes” is of schema type “LDAPAccount” and of schema class “LDAPAccount (all)”. We already know the mapping(s) that are used for this particular type. So what the engine now does, is use the first of those mappings to create an in memory LDAP “prototype” SystemObjectData based on the known mapping. Now we have a list of LDAP SystemObjectData instances and the information, what to do with them.


The last thing to do, is to translate those instances to the value that actually need be written to the property. Those are the distinguished name strings of the objects in case of LDAP. Translating a SystemObjectData to a value is the opposing operation of the connector’s reference resolution that was used at the very beginning of the process. This opposing operation is called ‘reference creation’ and is also done by the connector. For LDAP, the connector will just return the distinguished name stored in the SystemObjectData that was passed.

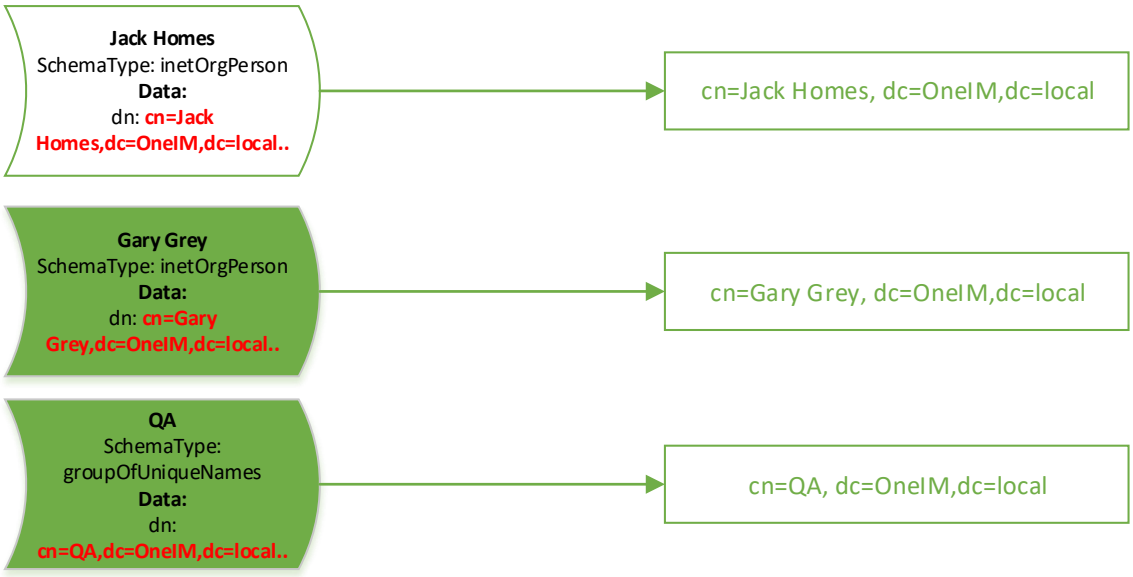



*Figure 68 Reference creation input/output*

The rule now creates the described add and remove modifications for the SystemObjectData instance of the "group" object that contains the members.

Hint: When the OneIM "Merge mode" feature is activated, the rule will not calculate the differences as described above between OneIM and target system. It will just create modifications based on the records in DPRMembershipAction. This will prevent reverting manual changes that were done in the target system but were not yet synchronized to OneIM

## Common Options

**"Ignore mapping direction restriction on adding"**
One of the common use cases for this option is a property that only can be set when creating a new object and is otherwise read only. In this scenario, you would set the mapping direction in the "read" direction only (to OneIM) but also set this option. The engine/workflow would then write the attribute only at object create time (even though the arrow for this attribute points to the left). As you would expect for a mapping rule with an arrow pointing to the left, no subsequent updates are done on this attribute.

Hint: This can be used in conjunction with the rogue change settings to allow new objects from the target system to be initially imported to OneIM

In contrast to the "Do not overwrite" setting, this setting is only effective when new objects are created and allows to set the value. "Do not overwrite" applies to insert and update scenarios and only allows setting a value for objects that do not have a property value yet.

**"Force mapping against direction of synchronization"**
Sometimes, there is a requirement to transfer data in the opposite direction of the actual workflow execution. Imagine a target system that calculates a key when creating objects and those keys shall/have to be available instantly in the other system. A common example is the "read back" of a new LDAP objects UID during provisioning. By setting this option, the rule will cause a reload of the provisioned object, read its GUID and write it back to the originating object in OneIM. Those rules have a special indicator in the list of the mapping editor

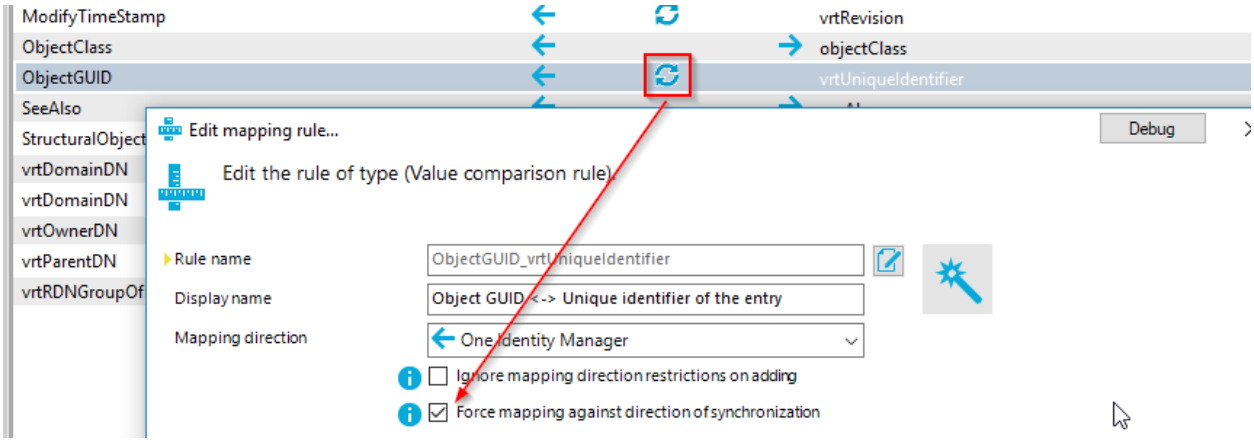



*Figure 69 "Reload" indicator*

As the name of the option implies, it works against the mapping direction during workflow execution. If you set this option for a rule that is configured to run in "Both directions", the behavior will be as follows:

a) When data provisioned from OneIM → TargetSystem, the value in OneIM will be updated with the new value from the target system.
b) When data is read into OneIM (OneIM ←TargetSystem) the value of the TargetSystem will be updated with the value from OneIM

A use case for this scenario is the calculation of the "EmailAddresses" property of a recipient in Microsoft Exchange. When EmailAddresses is not explicitly specified, Exchange will calculate its value on its own. Since EmailAddresses can be also be written, you could create one mapping rule having "both directions" set as mapping direction. But usually, you do not want b) to happen during a regular sync Exchange → OneIM. So if you like to have the calculated value available in OneIM **only** during provisioning without the write back OneIM → Exchange you need to configure two rules.

1) Rule with "Force mapping against direction of synchronization" for mapping direction OneIM → Exchange a.k.a.

2) Rule without "Force mapping against direction of synchronization" for mapping direction Exchange → OneIM (to prevent write back during synchronization Exchange)

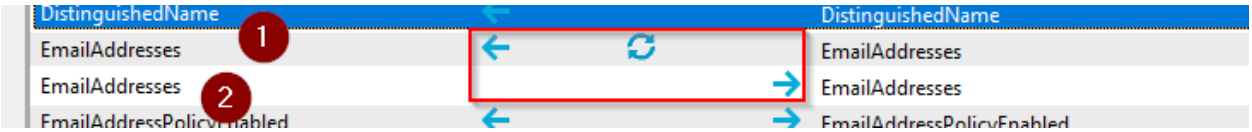



*Figure 70 Reload only during provisioning*

The settings "Detect roguemodifications" and "Correct roguemodifications" are used for the rogue modification detection feature that will be described in a later section.

A mapping condition can also be applied to any rule. This condition is evaluated during runtime and can use the all of the properties available for each side. Besides that, the current execution direction is also available for incorporation in the mapping condition. Properties of the OneIM connection are accessed via `Left.<PropertyName>` whereas properties of the target system can be accessed via `Right.<PropertyName>`. The execution direction is available via `ProjectionDirection`. Variables can be accessed as well.

The following condition example will execute the rule only, if the execution direction is directed to the target system (1) and the vrtParentDN property on the left side is not empty (2) or the execution direction points to OneIM (3). Also, evaluation of values shall be executed case insensitive:

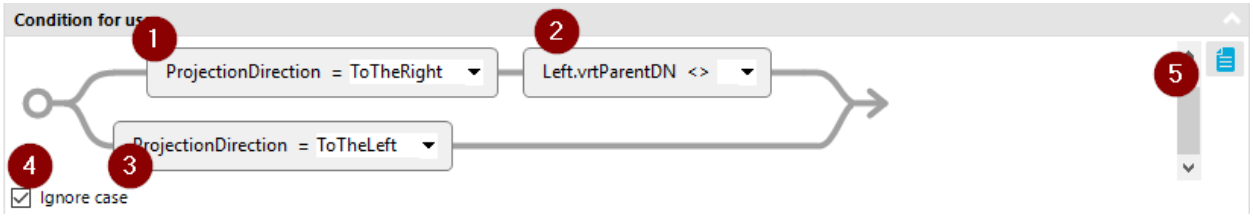



*Figure 71 Mapping condition (graphical)*

You can switch between graphical and textual representation of the mapping condition by clicking the "Mode" button (5)

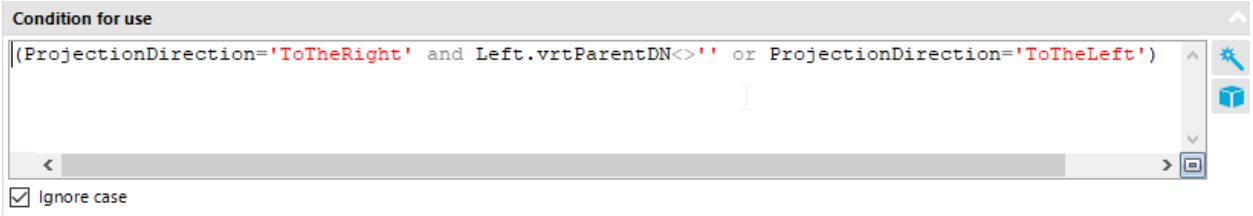



*Figure 72 Mapping condition (text mode)*

Those are the common settings that are available for each mapping rule implementation. OneIM currently ships with two types of mapping rules which will be handled in the following sections.

### Rogue modification detection/correction

The term rogue modification describes a scenario, where two systems are synchronized with each other, one of them (usually OneIM) was declared as master but a manual modification has been done in the slave system i.e. a modification by a user with administrative privileges.

The rogue modification detection can be used to identify those "manual" modifications during the scheduled synchronizations and report/revert them. This feature can be configured per property in the corresponding mapping rules.

To configure a system as master for a property, the mapping direction of the mapping rule needs to point "away" from it. If you want OneIM to be the master for a property, the direction would be "Target system".

As a example, we want to define that OneIM is master for the description property of organizationalUnit objects in LDAP. To achieve this, create a mapping rule for description that has the mapping direction "Target system" (1) and the option "Detect roguemodifications" enabled. Then configure a scheduled synchronization "Target system" → "OneIM" which includes a workflow that uses the mapping containing that rule. Theoretically the rule would be ignored, since its mapping direction is configured to only transfer data to the target system but since the rogue detection setting is enabled (2), the engine will also compare the values of description. When the values are different (master <> slave) it will log a rogue modification and (if configured (3)) automatically revert the value in the target system to the value specified in the master (OneIM).

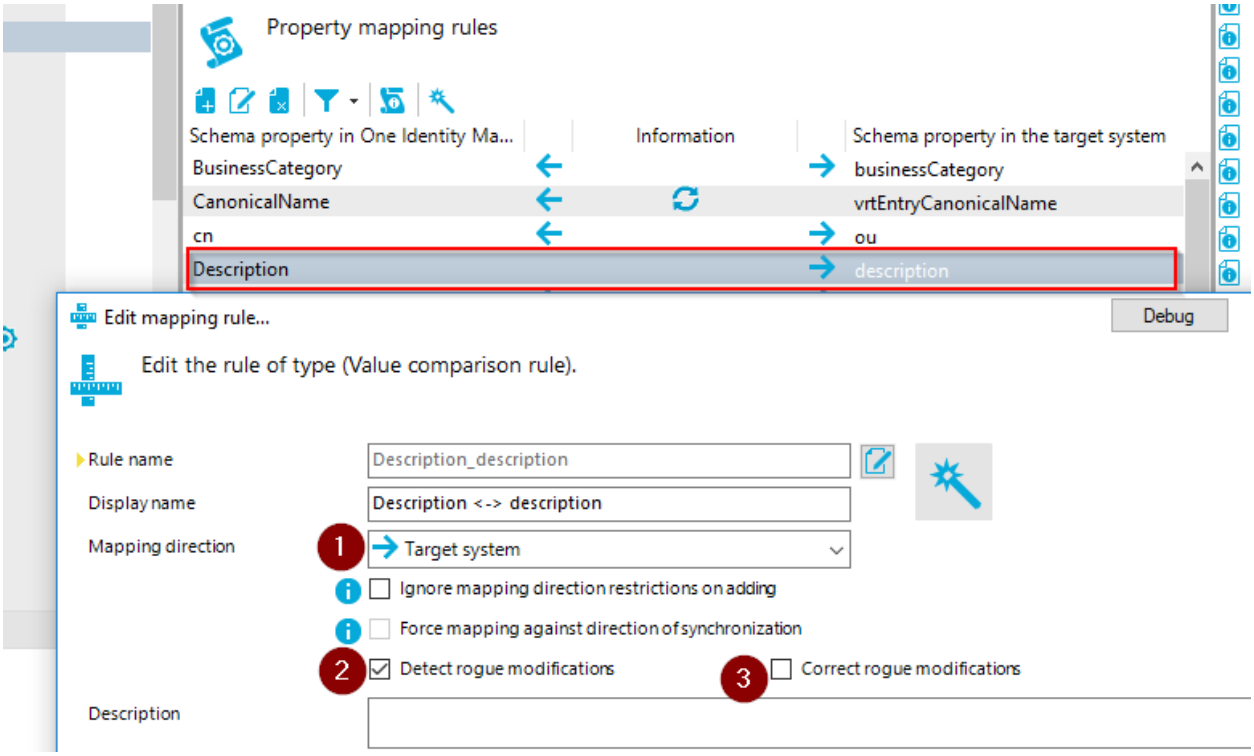



*Figure 73 Configure rouge detection*

When the description is now modified in LDAP, the next synchronization will create a new message in the synchronization log.

Hint: Synchronization processes executed by OneIM return the ID of the synchronization (UID_DPRJournal). This can be used, to add any custom rogue change handling for a completed synchronization such as sending notification mails etc. See section "Logging" for more details.

## Matching rules

Matching rules are used to create a 1:1 relation between 2 objects in OneIM and the target system. A mapping can have multiple matching rules. These rules are used in the given order to check whether two objects can be matched together. To change the order of the matching rules, use the "up" and "down" icons in the "Object matching rules" section.

A matching rule can have different implementations (rule types). The following two implementations are available out of the box. To select them, create a new matching rule by clicking the "new" button in the "Object matching rules" section.

## Logical expression rule

The logical expression rule allows you to define expression based rules for matching. These expressions use the defined mapping rules of type value comparison rule as elements that can be combined using the operators AND, OR. The following example shows a logical expression rule that will generate a match, if the value comparison rules for givenName and sn (firstname/lastname) or the rule for employeeNumber match.

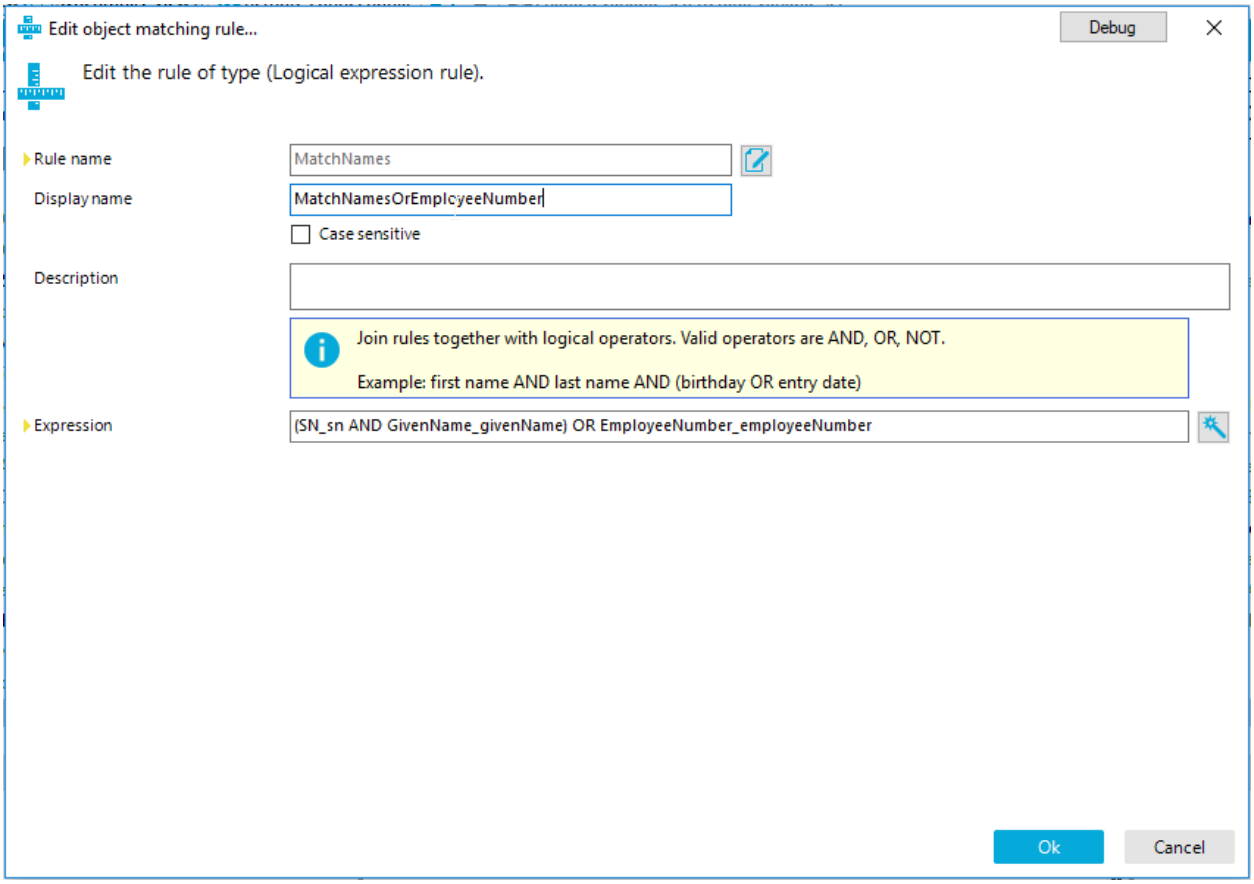



*Figure 74 Logical expression rule example*

## Value comparison matching rule

A value comparison rule performs a simple value by value comparison between a pair of properties from each system. The data type needs to be compatible and you can specify whether the values shall be compared case sensitive or not.

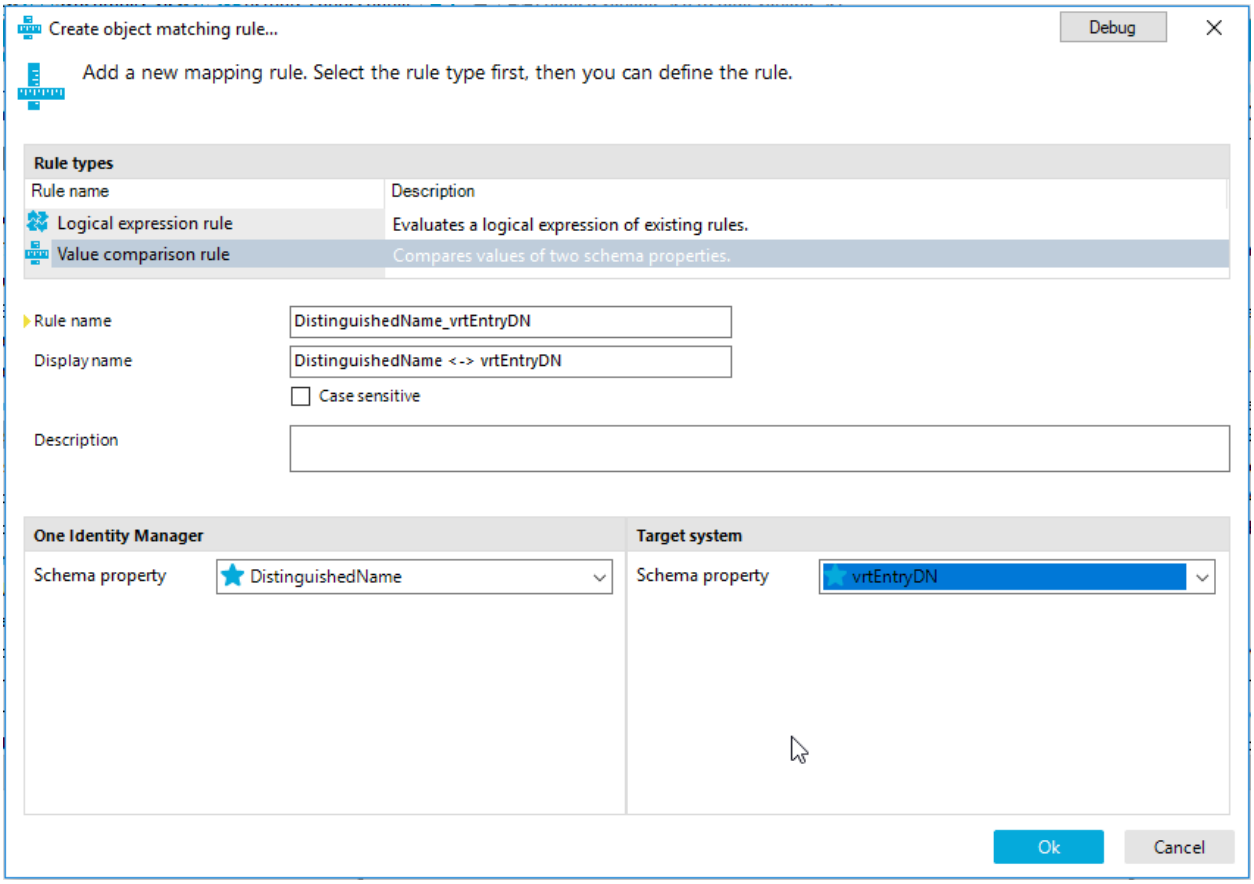



*Figure 75 Value comparison matching rule*

# Datastore for references

The datastore for references is an optional feature that can be exposed by any connector. It is used during synchronization to store values of reference properties that could not be written to the current system. Currently, the only connector that exposes a data store is the One Identity Manager connector.

As the name implies, it is used when dealing with references that cannot be resolved during a synchronization or provisioning process. The issue/use is best illustrated by the following examples.

A mapping rule shall transfer the "manager" property of an inetOrgPerson entry in LDAP to the UID_LDAPAccountManager column (foreign key) in OneIM. The property value of the "manager" entry in LDAP contains the distinguished name of the manager entry. To populate the UID_LDAPAccountManager column, a virtual schema property "key resolution by reference" was created on the OneIM side that searches the table LDAPAccount for the given distinguished name from ldap and returns its UID of the referenced LDAPAccount object in OneIM.

## Handling keys that cannot be resolved during Synchronization

The resolution of a target system key to a OneIM object key might fail due to several reasons such as:

- The LDAPAccount the distinguished name is referring to is not present in the OneIM database at all/yet.
- The manager property value in LDAP does not point to an object of type inetOrgPerson (i.e. someone put the distinguished name of a groupOfNames there instead of an inetOrgPerson)

In this scenario the rule, has several options how to deal with this situation

**Ignore the value (leave UID_LDAPAccountManager empty)**

This option is only viable if data is just synchronized to OneIM and never written back to LDAP. A write operation would clear the value in LDAP which equals data loss. To select this option uncheck the 'Save unresolvable keys' option.

**Store the value (distinguished name) that could not be resolved**

Although UID_LDAPAccountManager cannot be populated, the native value from LDAP is preserved in the data store for references. If a write operation is performed, there is an initial check to establish whether UID_LDAPAccountManager has a value. If it is empty, the value from the datastore is used (if present) to (re-)populate the property value. → **This setting (the data store) prevents data loss on "write" operations**

The following chart is the extended example illustration of the "key resolution" virtual property write process with added data store functionality

To select this option check the 'Save unresolvable keys' option.

**Cancel the execution entirely**

This option is rarely used. The only use case is when this unresolvable reference causes major failures or even data loss in other synchronization steps. For example, the vrtDomainDN virtual property is flagged as such in the LDAP Connector by default. This is because a resolution error in this particular scenario indicates a major configuration issue. The vrtDomainDN virtual property sets the mandatory property UID_LDPDomain by searching a record in LDPDomain having the given distinguished name. The source value from the target system is the content of a fixed value virtual property containing the distinguished name of the root entry and is therefore the same for each object. So if this value cannot be resolved, it cannot be resolved for any of the objects and therefore will

cause failures for **all** objects. In this scenario it is better to stop the synchronization job and review the configuration.

To select this option, check the 'Handle failure to resolve as error' option.

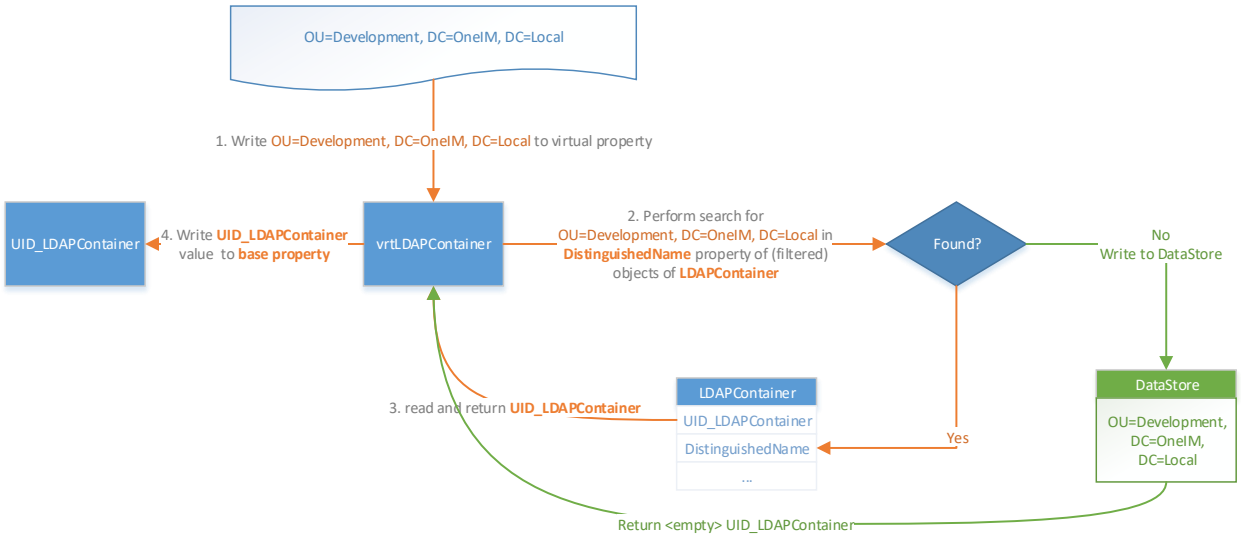



*Figure 76 Datastore writing*

And the extended read illustration

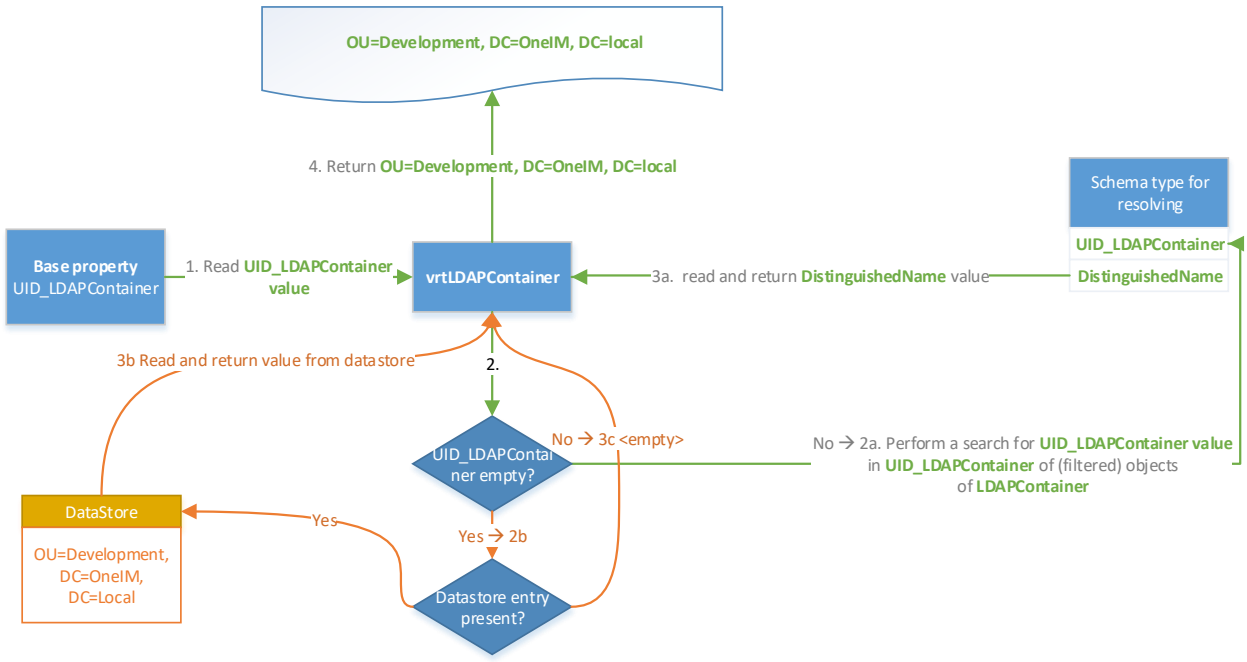


*Figure 77 Datastore read*

The same principle applies, when dealing with multi-valued references. The following components (can) use the reference data store:

| Type | Com | Configurable (yes/no) |
|---|---|---|
| Virtual property | Key resolution | Yes |
| Virtual property | Key resolution by reference | Yes |
| Mapping rule | Multi-reference mapping rule | No (always active) |

The OneIM connector datastore implementation stores its data in the DPRAttachedDataStore table. The structure of this table is

| Column | Description |
|---|---|
| OwnerObject | XObjectKey of the OneIM object that contains the property that has a value which could not be resolved |
| UID_DataQBMClrType | The data type of the stored data |
| DataFull | The value that could not be resolved |
| DataShort | The truncated value that could not be resolved |
| HasDataFull | If the property value does not fit in the DataShort field, it is truncated, stored in DataFull and the flag "HasDataFull" is set. (→ for performance reasons while handling BLOBs) Usually happens when a value of DataQBMClrType VI.*.SystemObjectData is stored. |
| OwnerProperty | The identifier of the property that contains the unresolvable data |
| SourceInfo | The identifier of the component that created the record. |

## Datastore maintenance

Since reference values that were stored as unresolvable during a process must not be unresolvable in subsequent processes, it might be advantageous to check the contents of the data store from time to time in case some of the values have become resolvable in the meantime. If so, the target property value will be populated with the correct reference and the datastore entry will be removed. There are two different approaches to do that.

a) Force the next synchronization to process objects that have datastore entries. This will cause a retry of the reference resolution during the next synchronization. This approach is useful, if the referenced objects are imported by another synchronization. A common use case for this approach is synchronizing multiple Active directory domains that have groups with cross domain memberships. This approach is the default when creating new start up configurations.

b) Try to resolve every datastore entry immediately at the end of a synchronization. This mode basically loads the object that contains the unresolved reference and writes the stored value to the virtual property again to trigger a resolution attempt. You can also specify how many retry cycles per datastore entry you want to execute.

This approach is useful, if unresolvable references can occur due to the order of the steps in a synchronization workflow that cannot be automatically resolved. The OOTB OneIM Exchange synchronization projects favor this approach for performance reasons over the automatic dependency resolution that leads to more synchronization steps.

The type of approach you want to use in your synchronization workflow can be specified per start configuration on the "Maintenance" tab.

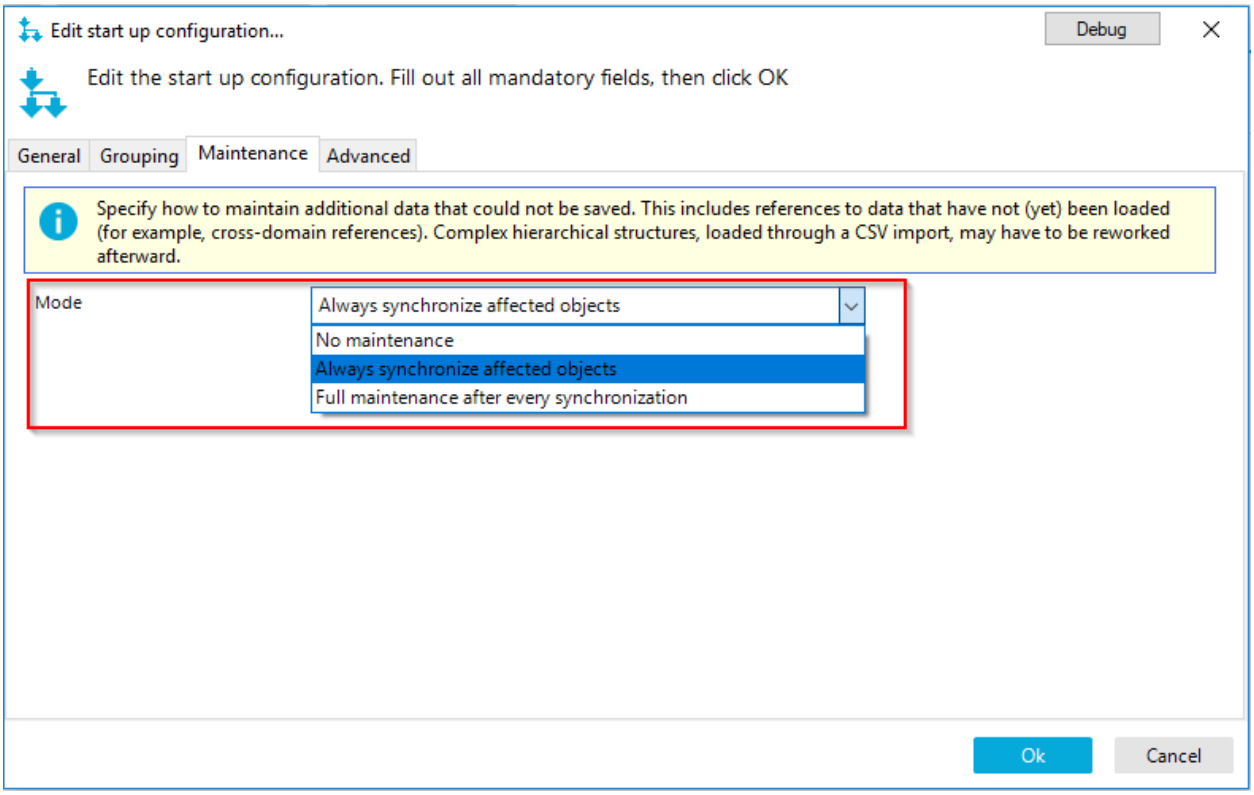



*Figure 78 Datastore maintenance configuration*

# Synchronization

This section describes the basic principle of how the synchronization engine transfers data from one system to the other when performing a full synchronization of two schema types based on a given mapping.

A full synchronization is divided into multiple phases.

## Revision determination

During this first phase, the synchronization engine determines the maximum revision value of the all schema types that will be handled during the synchronization. This value is stored for upcoming synchronizations to allow certain optimizations. A example is shown on illustration “Matching phase”.

## Matching phase

The second phase is called “Matching” phase. Its task is to compare the lists of objects from the first system (usually OneIM) to the list of objects from the target system based on the configured matching rules of the used mapping. The outcome are three sets.

A) A set of object pairs that could be matched together according to the matching rules
B) A set of objects from the target system that do not exist in OneIM
C) A set of objects in OneIM that do not exist in the target system

To prevent runtime and or memory issues, the engine will query only a reduced set of properties from each system. To be more precise, it will query the properties that…

… are used by the mappings matching rules

… are used in the schema class filter

… are marked as unique keys

… form the display of the object

… are marked as revision property

Once again, those lists returned by the connectors contain **all objects** of a schema type **with a reduced set of properties** and are often referred to as **“slim lists”**. At this point, the lists have to complete to ensure a proper handling of the different sets.

Once the three sets are created, the processing or mapping phase starts.

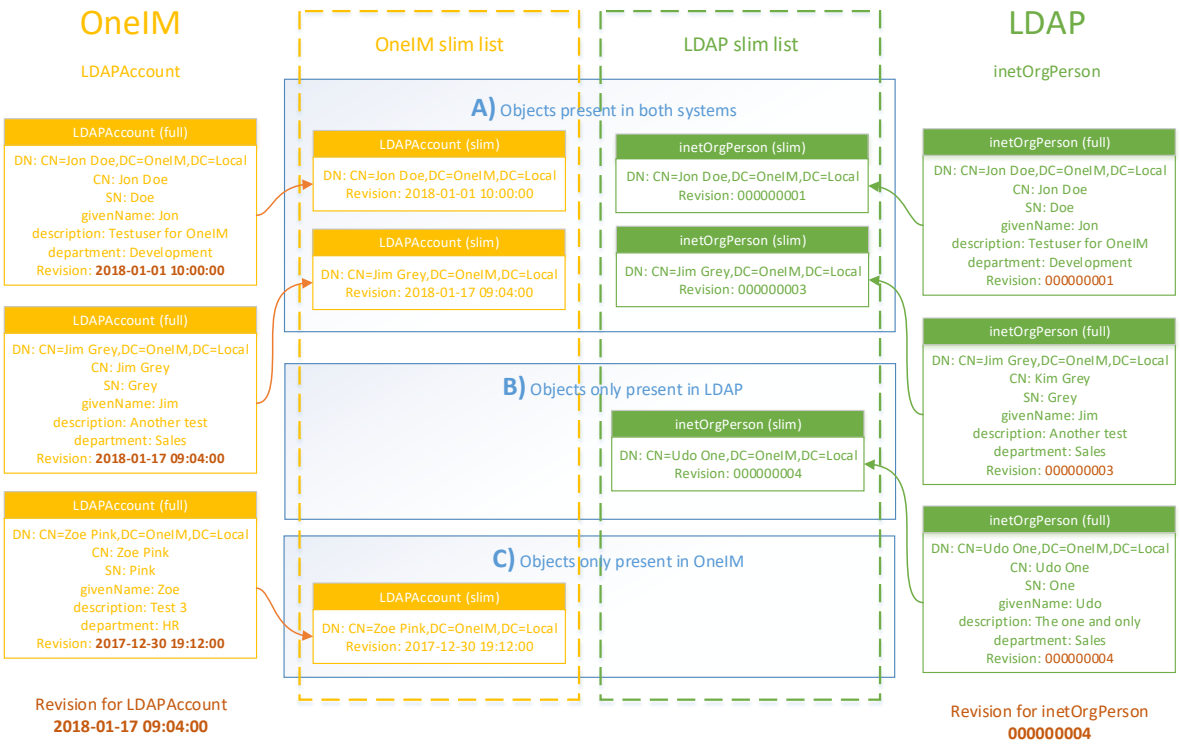



*Figure 79 Matching phase*

## Mapping phase

During the mapping phase, data is transferred between the systems and schema methods are executed. Which schema method shall be executed for which set is configured in the corresponding workflow step. In order to execute the method, the synchronization engine must have access to all properties of the involved objects. During the matching phase, only a reduced set of properties was loaded. To execute the configured mapping rules the engine has to reload all involved properties. Since those fully loaded objects can be quite big, the engine will process the datasets in partitions.

The default partition size is 1024 objects. This can be adjusted in the workflow configuration.

The following illustration shows the sets A, B and C created by the matching step on the left side. The synchronization engine divides the elements of those sets into partitions A1 – A5, B1, B2, C1 and C2 of 6 elements each. For each of those partitions, the objects involved are reloaded and the configured methods are executed, which are "Update" for the elements of set A, Insert for the elements of Set B and Delete for the elements of set C. The "<unmatched>" items represent a non-existing object. The partitions are handled sequentially and revision optimization is not applied in this example.

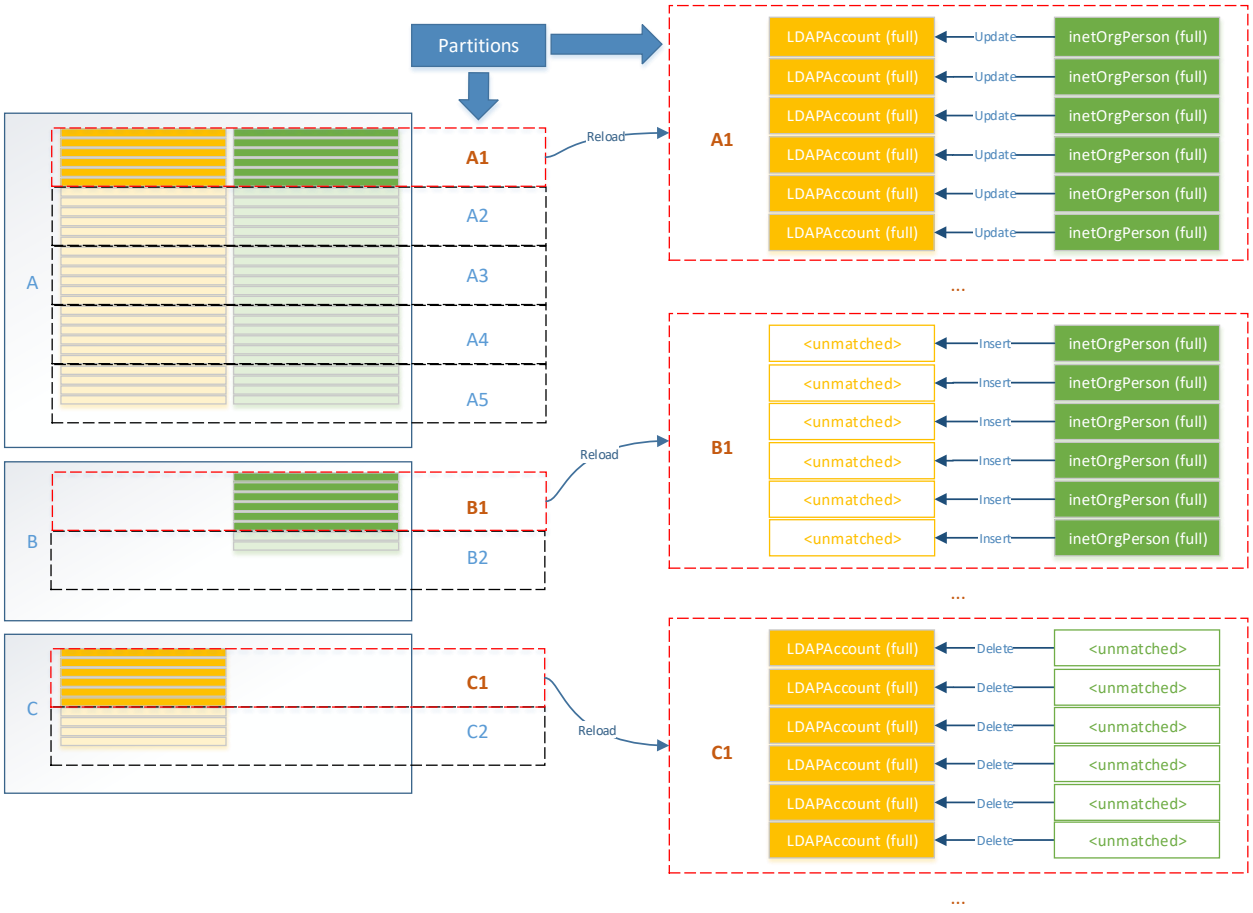



*Figure 80 Mapping phase*

As stated, reloading all objects from both systems can be very time consuming. In a regular synchronized production environment, the number of items in set B and C is typically pretty small compared to the items of A and they need to be reloaded anyway.

The items of set A is a bit different. In a synchronized environment, there should be only a few objects that actually differ. For those items, calling an "update" method is not required because the objects on both sides are already "equal" according to the defined mapping rules. However, to check **if** there are differences, the object need to be reloaded **unless**, we can somehow check, if the object was changed since the last synchronization at all. This is what revision handling or revision filtering does.

## Revision filtering

A revision is a property value of a target system object that gives information when or if the object was changed. Revisions can be available in different forms.

- LDAP: the maximum value of createTimeStamp and modifyTimeStamp
- ActiveDirectory: usnChanged
- OneIM: maximum value of XDateInserted, XDateUpdated and XDateSubItem

The idea is that each time a modification is performed, the value of the revision is **increased**.

Revisions are loaded along with the small (reduced) list during the matching phase and are then compared to the revisions of the last synchronization run. If the revision property was not "increased", a full load is not required. In order to compare the revision, to the previous run, the engine determines the highest revision per schema type at the beginning of a workflow execution and stores it into an internal revision store. This revision will be used the next time the workflow runs. Revision filtering can only be applied when both objects of set A support revisions.

The following illustration shows two subsequent synchronizations. The first synchronization has no revision information at all and therefore needs to process all items of set A. This first synchronization also determines the revision values for each system and stores them.

Between the first and the second synchronization the LDAP object "Gary Grey" and the OneIM object "Jon Doe" are modified which lead to an increase of their revision property value. After the second synchronization is done with the matching phase, it filters out items (pairs) that were modified in one of the two systems by comparing the current revision property value to the revision value for the schema type stored in the first synchronization. Only those objects that were actually modified are reloaded and passed to the mapping phase.

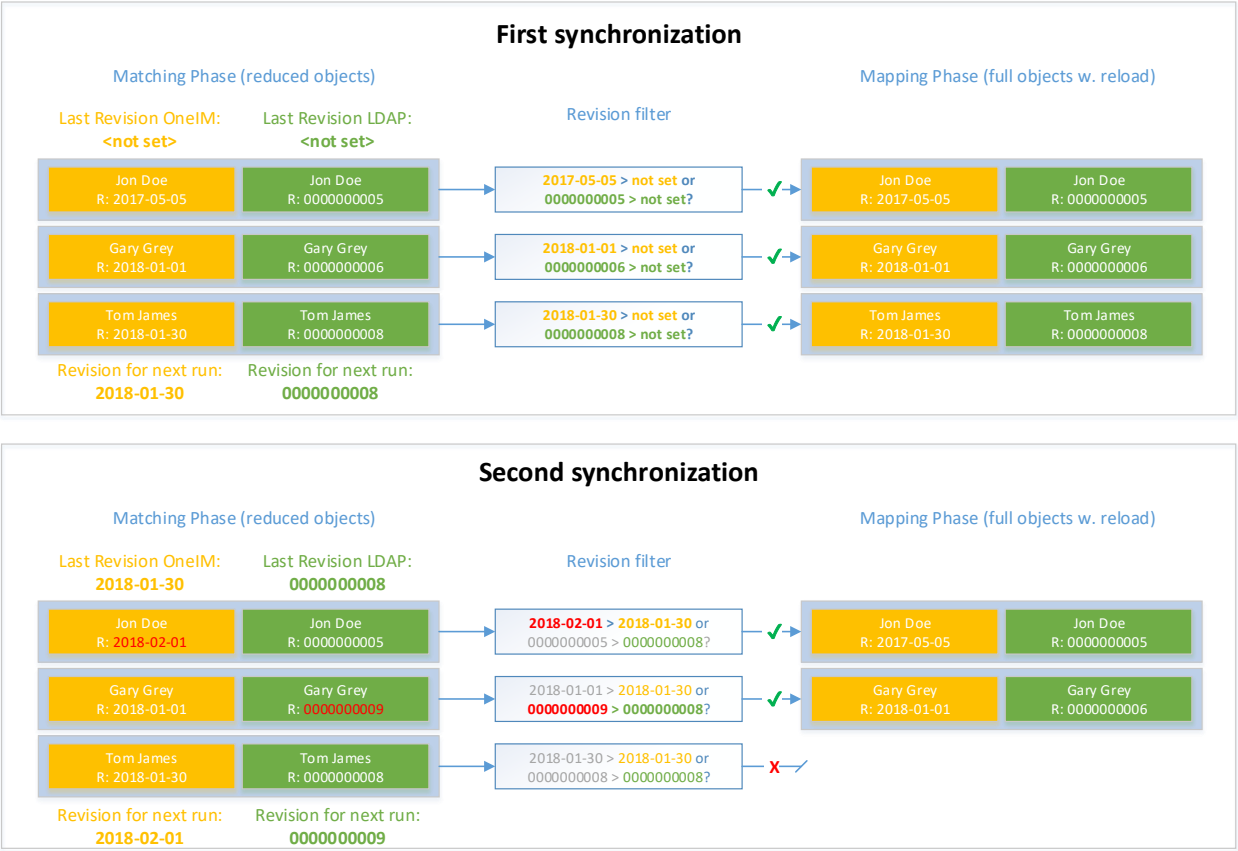



*Figure 81 Revision filtering*

## Reload Threshold

The reload threshold is an optimization for schema types that have very few properties. It impacts the number of properties that are requested during the matching phase of a synchronization. As stated in the matching phase section, the engine calculates a minimum set of properties that need to be loaded to perform its task. If the number of those properties is less than the configured reload threshold, the engine checks if the number of remaining properties before reaching the threshold, is sufficient to completely load the objects with the properties required during the mapping phase. If so, the objects are loaded accordingly. This prevents the requirement to reload an object during the mapping phase which can speed up the synchronization process.

Hint: Properties that are marked as "**IsLargeObject**" will prevent reload threshold optimization attempts because the object lists required during the matching phase are supposed to be delivered fast and be lightweight.

The Threshold can either be configured for a complete workflow at the start up configuration or per workflow step.

# Workflows

Synchronization workflows describe, which objects (mappings) shall be transferred in which direction (to OneIM or to target system). The term synchronization usually describes a workflow execution with a full data comparison (matching + mapping) whereas the term (ad hoc) provisioning refers to a workflow execution for a single object.

A workflow consists of workflow steps (4), each using a configured mapping. The direction of data transfer is determined for each step by either the caller (i.e. a schedule that instructs the workflow to transfer data to OneIM), the workflow default or the configuration of the step itself.

Workflows can be found in the "Workflow" (1) section of the Synchronization Editor. The workflow list (2) contains all workflows that are configured in the current project. The general settings of the workflow can be edited in the "General" section (3). The workflow steps of the workflow are listed in the "workflow" panel (4).

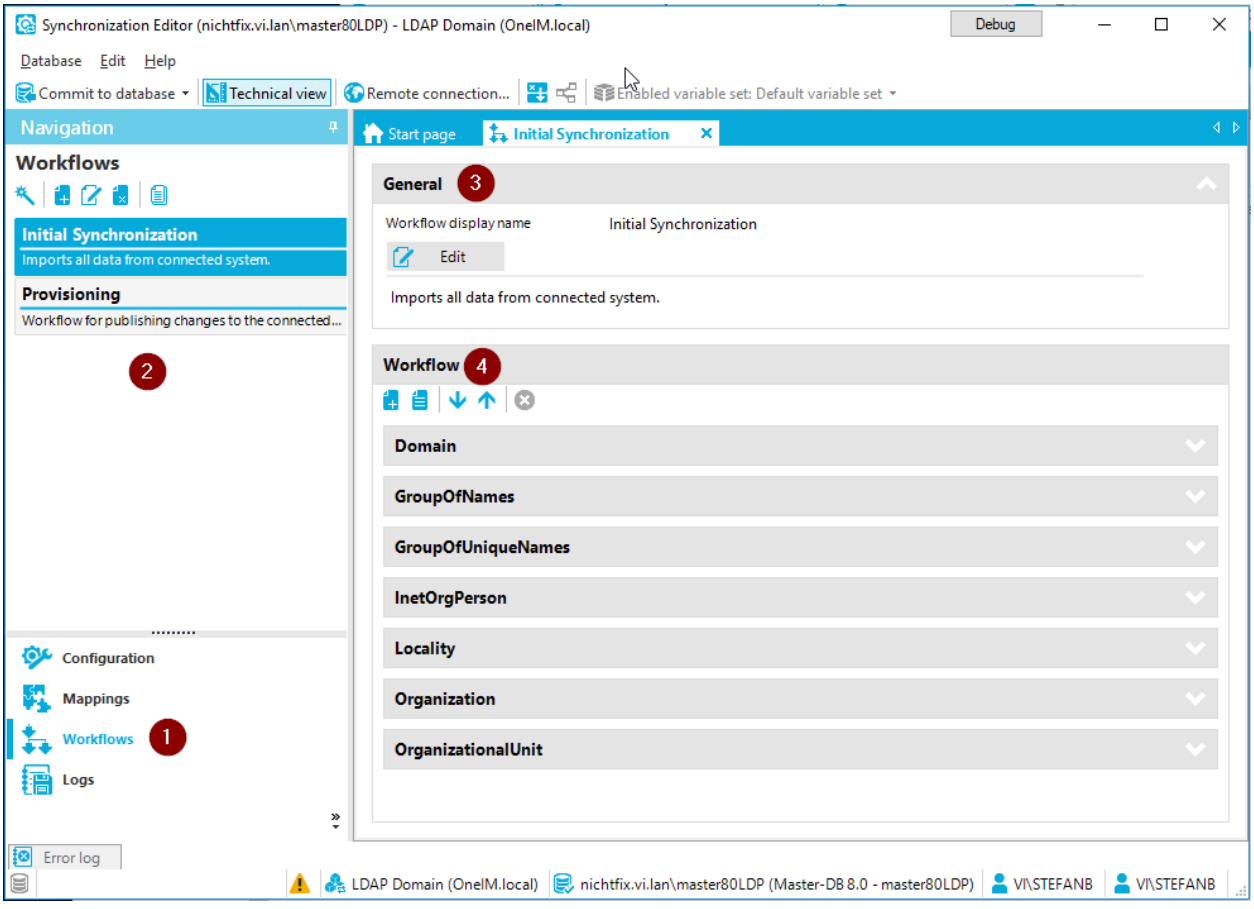



*Figure 82 Workflow UI*

## General settings

The general settings contain basic information such as the workflow name as well as some settings that describe some defaults for the workflow steps that are contained in the workflow.

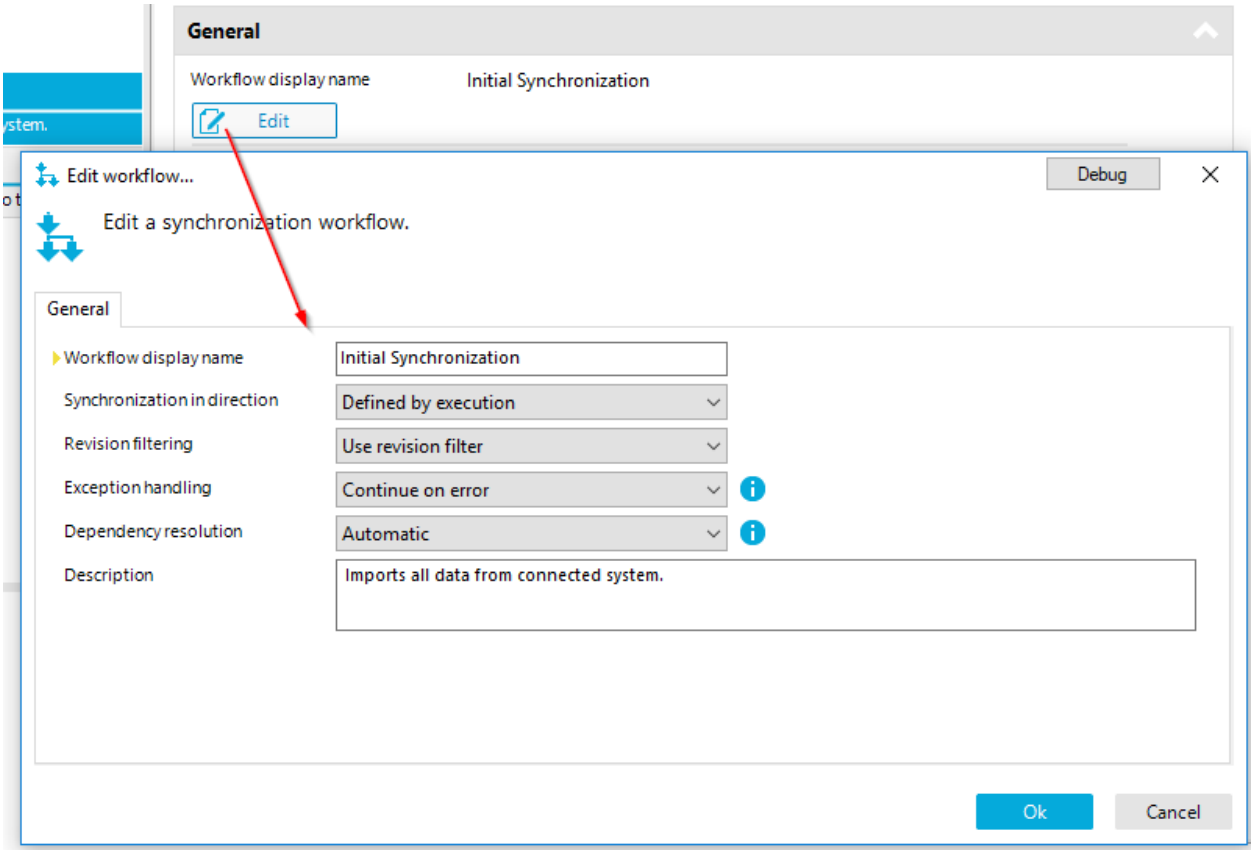



*Figure 83 General workflow settings*

| Setting | Description |
|---|---|
| Workflow display name | Display name for the workflow |
| Synchronization direction | Default synchronization direction (can be overwritten by the workflow step) |
| Revision filtering | Default revision filtering setting (can be overwritten by the workflow step) |
| Exception handling | Default exception handling (can be overwritten by the workflow step) |
| Dependency resolution | Turn automatic dependency resolution on or off (see following section) |

## Dependency Resolution

The Dependency resolution feature is one of the core features of the synchronization engine. Based on the schema information it calculates the optimal workflow step execution order. If required, the engine also splits a workflow step into multiple sub steps. Those are then executed as "Phase 2" steps during the synchronization. The result of the workflow calculation is called execution plan. The execution plan is calculated based on the target synchronization directions of the workflow steps and optimized for data creation.

In the following example we have two mappings (workflow steps) with several relations. An organization can be located within another organization. InetOrgPersons are also assigned to an organization and may refer to other inetOrgPerson objects as managers or owners.

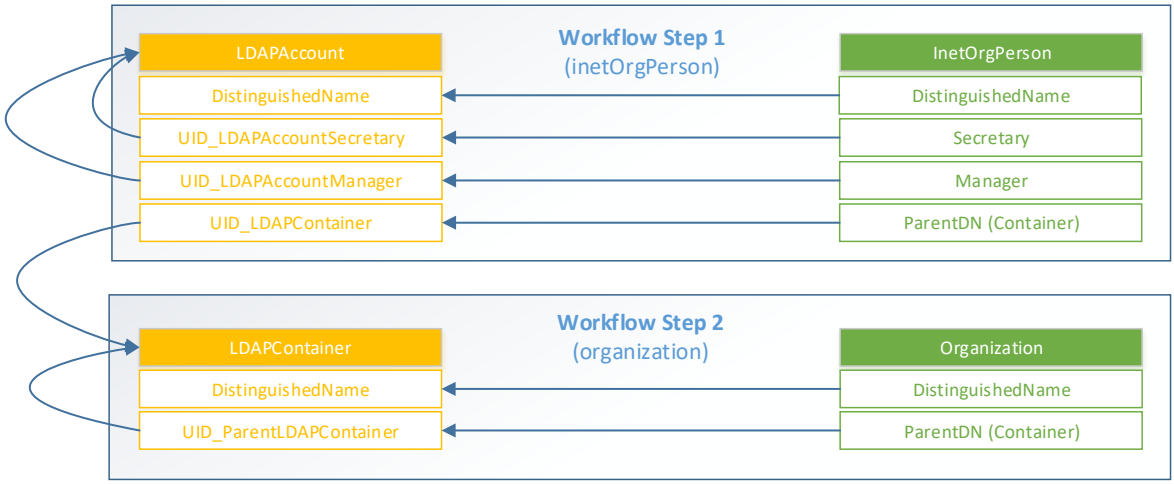



*Figure 84 Original workflow order*

As you can see, there are several issues with this workflow execution order. First of all, in an initial import scenario, the engine would not be able to populate the UID_LDAPContainer properties for the inetOrgPerson objects of workflow step 1. To fix this, the mappings have to be reordered so that organization objects are synchronized before inetOrgPerson.

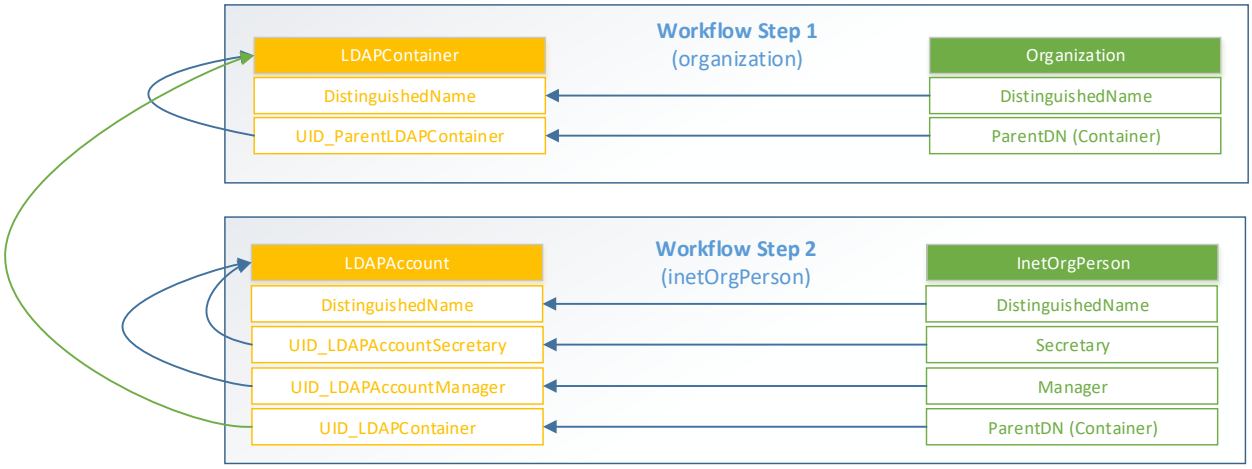



*Figure 85 Dependency resolution interation 1*

As you can see, by fixing the execution order the conflict regarding the container reference could be solved. But there are three other relations that might be problematic. The import of the "Manager" and/or "Secretary" fields will only work, if the referenced LDAPAccounts were synchronized before. There may also be circular relations within such fields due to bad data quality in the target system. Same goes for the "ParentDN" reference of the organization. To resolve this conflicts, the engine will split the two workflow steps into four as shown on the next illustration.

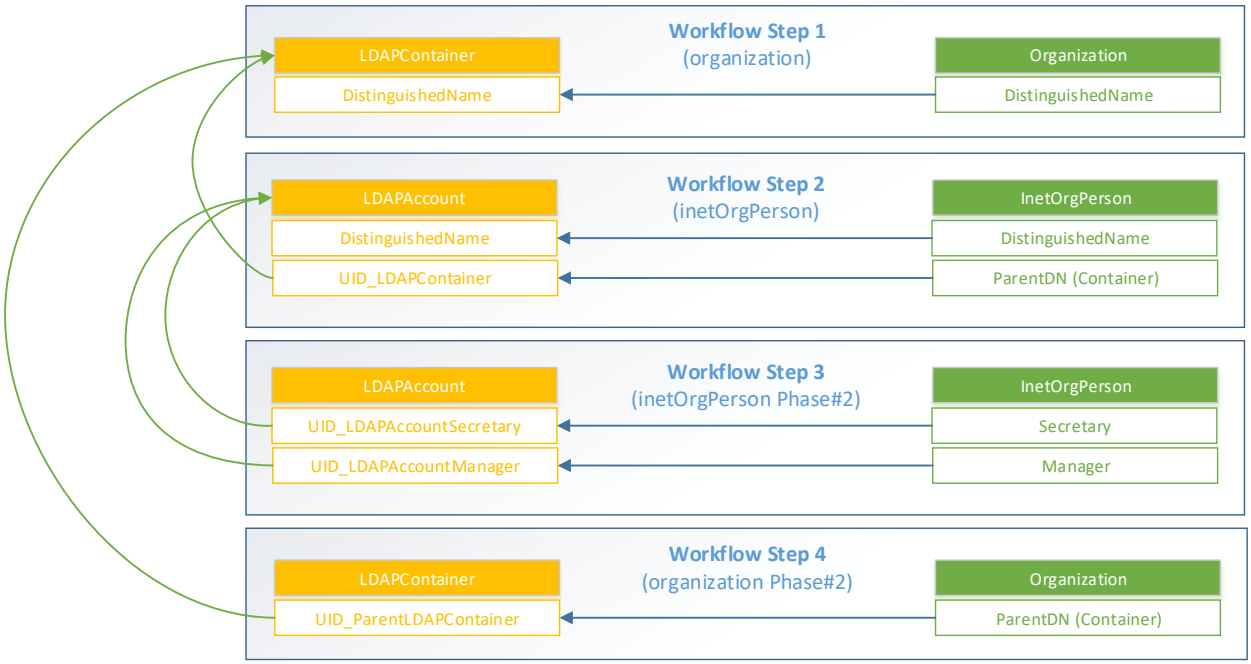



*Figure 86 Dependency resolution iteration 2*

Besides the reference information, the synchronization engine also relies on information like the synchronization direction, mandatory properties and the “hierarchy synchronization” setting of the involved mappings. However, sometimes it is not possible to resolve conflicts i.e. when a mandatory self-reference is involved. In those scenarios the synchronization engine will save objects that it was not able to process as “failure”. The list of those failures will be reprocessed at the end of the synchronization and be iterated until none of those failures could be resolved by retries. The execution plan can be rendered as a report by clicking the “execution plan” icon in the workflow section.

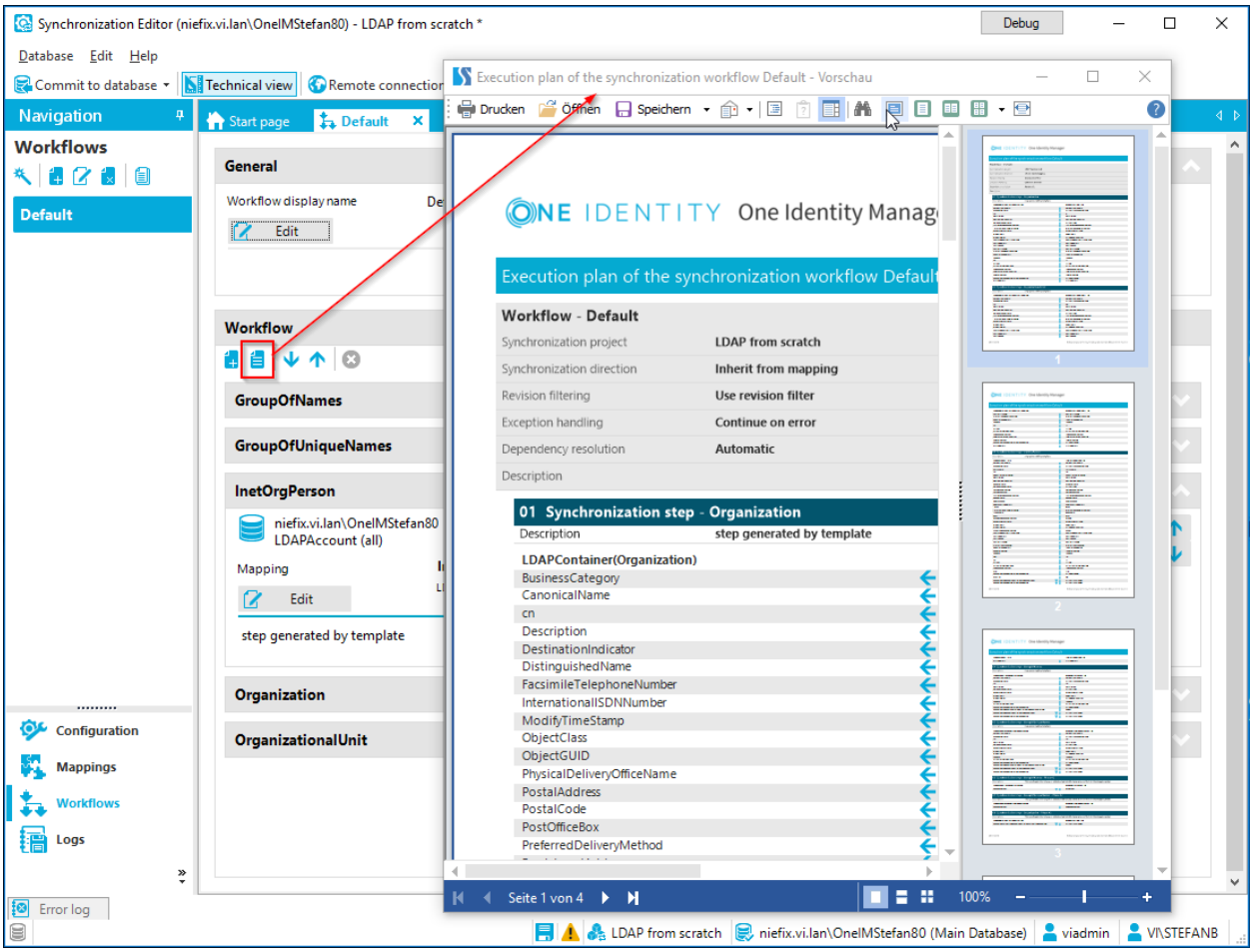



*Figure 87 Show execution plan*

## Workflow Steps

As stated, workflows consist of multiple workflow steps. Each step uses a mapping as a source of information how to match objects from both systems and to get the rules, how data can be transferred. When a step is executed within a workflow, it actually transfers data based on the processing settings.

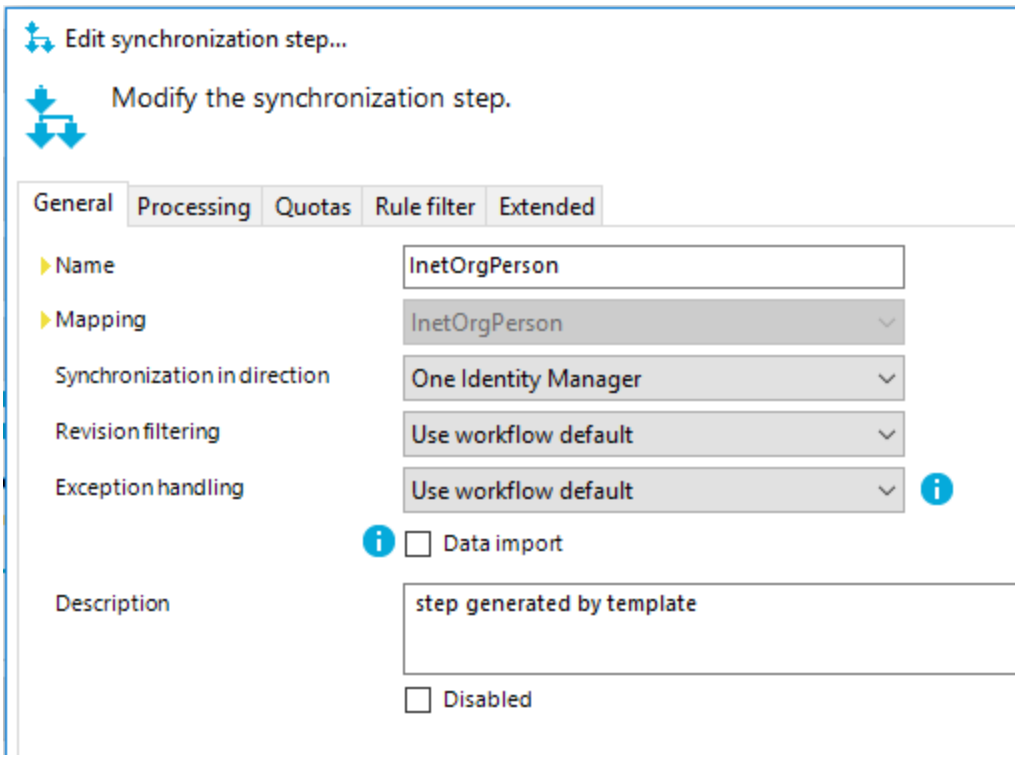



*Figure 88 Workflow step general settings*

## General workflow step settings

| Setting | Description |
|---|---|
| Name | The name of the workflow |
| Mapping | The mapping to use in the workflow step |
| Synchronzation direction | The synchronization direction is the direction in which the workflow step is going to transfer data for objects that are present in both systems (matched object pairs). Data transfer for objects that only exist in one of the systems will be performed according to the side of the assigned method (insert/update/delete) of the workflow step. |
| Revision filtering | If revision filtering is supported by both systems, you can activate the revision optimization here. Possible values are<br>- Use workflow default<br>- Use revision filter<br>- Do not use revision filter |
| Exception handling | This setting specifies if the workflow step stops at the first object that throws an error or continues to process and just log the failed records. During a synchronization you typically want to use the continue on error setting whereas during an ad hoc provisioning run, where you handle just one object, you want to get notified instantly if the operation was successful or not. |

| | |
|---|---|
| | The "continue on error" setting does not apply to all actions taken by the engine. If there are things fundamentally wrong such as an incorrect configuration or aborted connections during the synchronization, the engine will stop also.<br>In following scenarios the engine can continue working:<br>- A schema method for an object threw an error (i.e. due to incorrect/incomplete data)<br>- An object could not be reloaded (i.e. it was deleted from the target system in the meantime)<br>To review failures that were skipped during the synchronization you can review the synchronization log |
| Data Import | This flag is passed to the connector for each call. Currently only the One Identity Manager connector uses this setting to set the "Fullsync" variable as follows:<br><br>DataImport=True → Fullsync=False<br>DataImport=False → Fullsync=True |
| Description | The description of the workflow |
| Disabled | Disable the workflow step. It will not be executed and also not taken into consideration during the calculation of the synchronization execution plan |

## Processing settings

As described in the synchronization section, the engine creates three sets of items during the matching phase. Set "**A**" with objects (pairs) that are found in both systems, set "**B**" with objects that are only present in the target system and set "**C**" with objects that are only present in OneIM. Set "**A**" is also divided in the subsets "**A1**" which means "Matching objects in both systems that are equal according to the mapping rules" and subset "**A2**", which means "Matching objects in both systems that have property differences".

As we know, each connector exposes schema methods for each schema type. The processing section allows us, to define…

1. For which **set**…
2. … shall the engine execute which **schema methods** …
3. … in which **target system** …
4. … in which **order** …
5. … under which **condition**
6. … depending on the **synchronization direction?**

There are two visualizations for this section depending on the expert mode setting (menu "Database" → "Settings" → "Enable expert mode" of the Synchronization Editor.

With expert mode switched off, the configuration looks as follows:

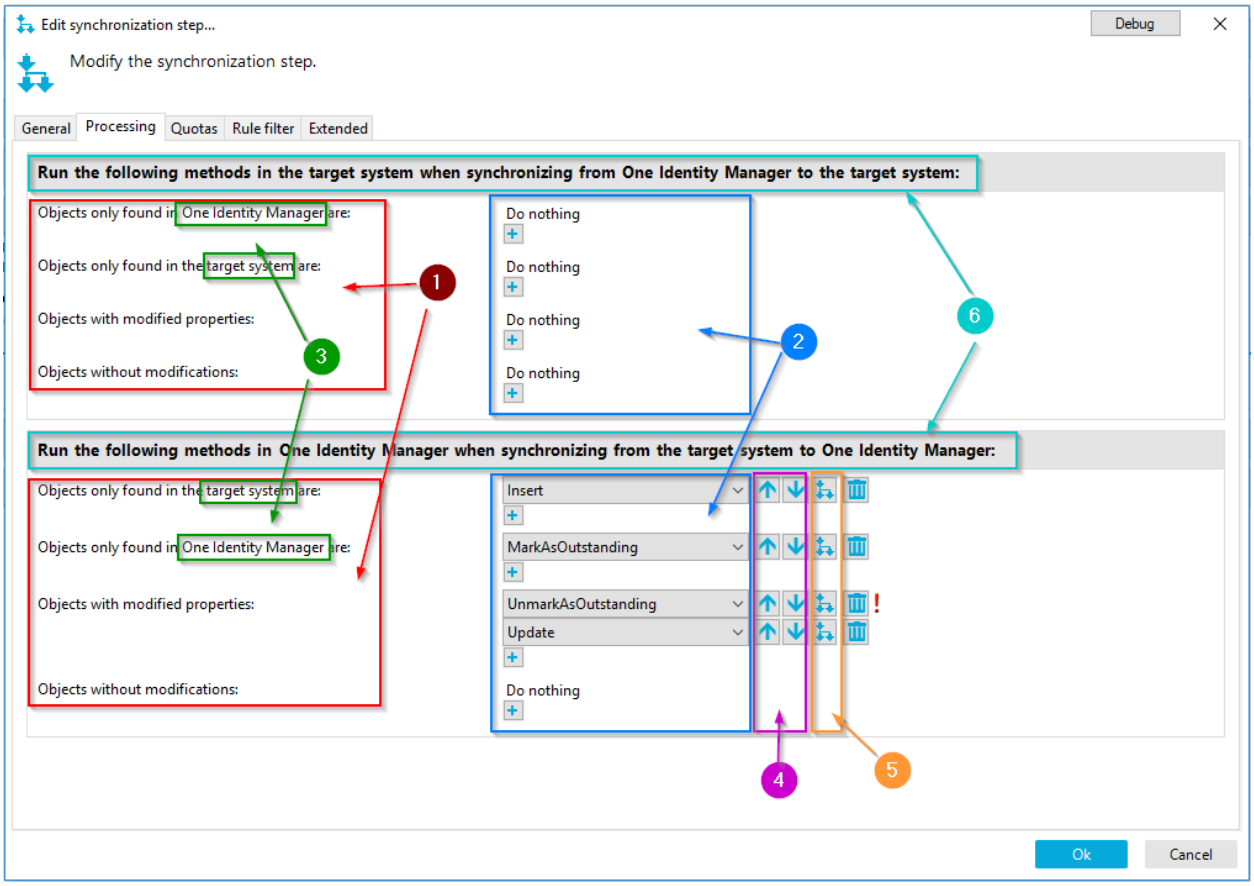



*Figure 89 Simple mode processing settings*

The set is for the processing settings (1) is represented in textual form as follows:

| Set | UI description |
|---|---|
| A1 | *Objects without modifications* |
| A2 | *Objects with modified properties* |
| B | *Objects only found in One Identity Manager* |
| C | *Objects only found in target system* |

The schema methods to execute for the different sets can be configured on the right hand side (2). The target system is also described in textual form (3). Depending on the synchronization direction (6) the method assignments for set A (A1/A2) applies to the One Identity Manager object (direction is "…to Identity Manager") or the target system object (direction is "to the target system"). The execution order can be modified with the "up"/"down" buttons (4) next to the method names. An execution condition can be configured with by clicking the icon (5) next to the order buttons. Methods will configured conditions will have an exclamation mark.

The second visualization in expert mode is as follows:

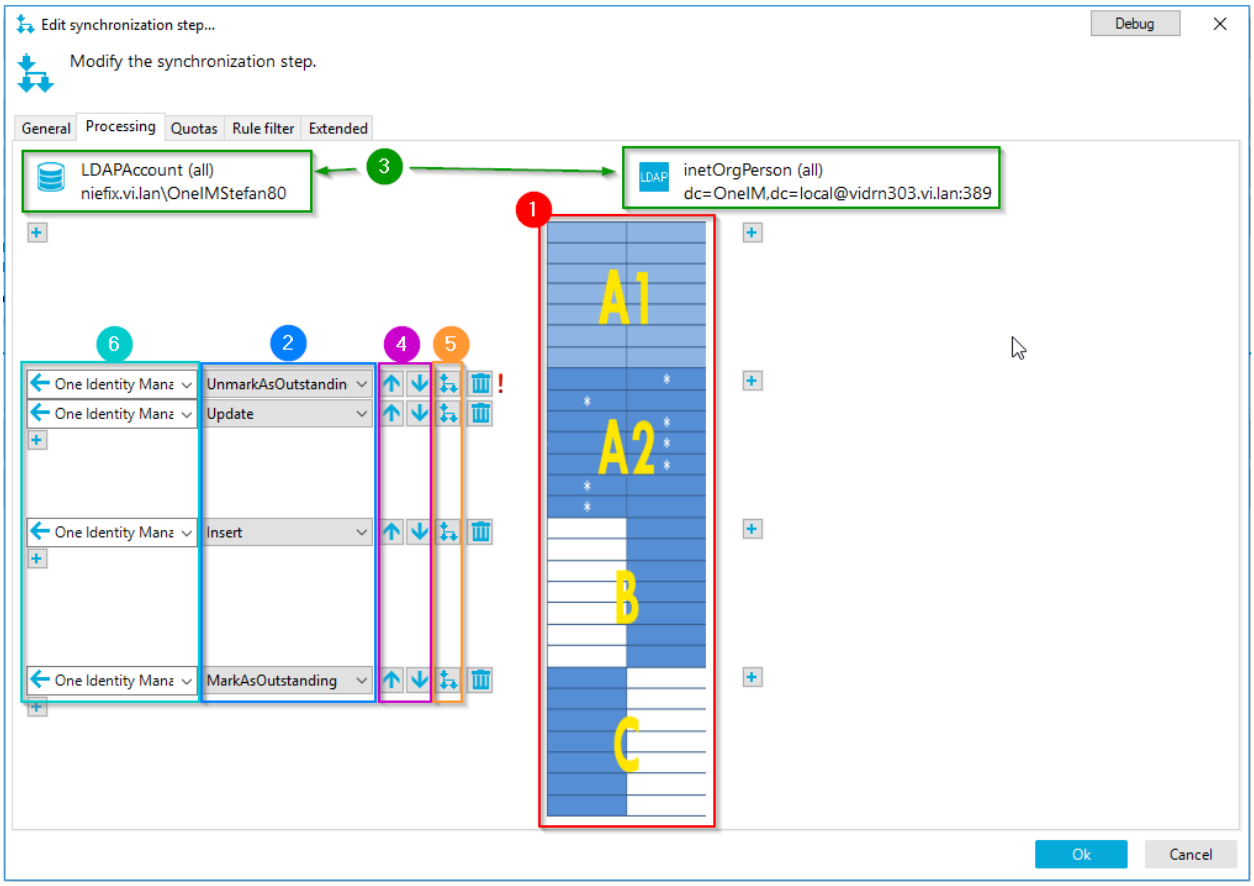



*Figure 90 expert mode processing settings*

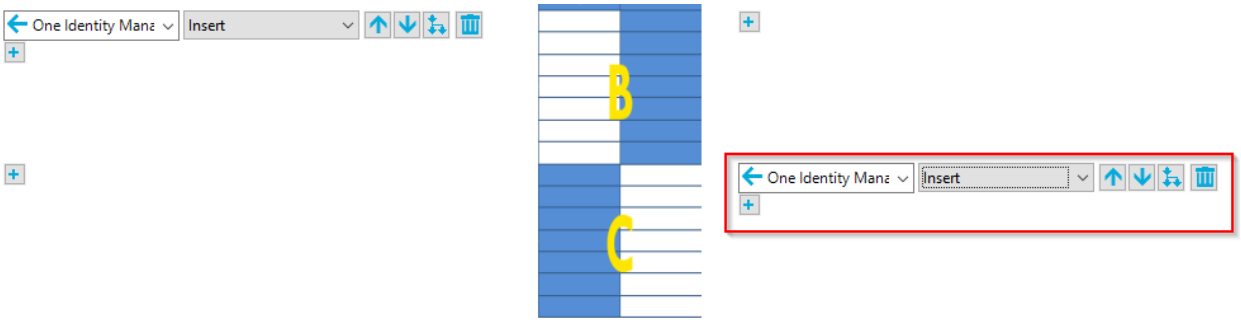
In this visualization the system is shown at the top (3). OneIM is always on the left side and the target system is right. The idle section visualizes the set that methods can be assigned to (1). The method to be executed for a set is configured at the corresponding system side (2). The synchronization direction selection for the method to execute is selected left to the method selection (6). The ordering of the methods (4) and the condition configuration (5) is similar to the simple mode visualization.

This visualization allows the configuration of "unusual" workflows. In the example above, objects of set "C" (that only exist in OneIM) shall be marked as "outstanding" (missing) if not found in the target system (LDAP). The configuration can be stated as:

"When synchronizing from LDAP to OneIM and you find an object in OneIM that is not present in LDAP, mark it as outstanding in OneIM"

If you want to republish them in LDAP you would need to add the schema method "Insert" on the LDAP side (right) for synchronization direction "One Identity Manager" instead of the

"MarkAsOutstanding" method assignment on the OneIM side. This configuration would look like this:




Written as a statement, the marked item would be:

"When synchronizing from LDAP to OneIM and you find an object in OneIM that is not present in LDAP, insert it to LDAP"

This visualization also allows you to define generic processing settings that work in either direction since the synchronization direction is fixed during runtime. See the following example:

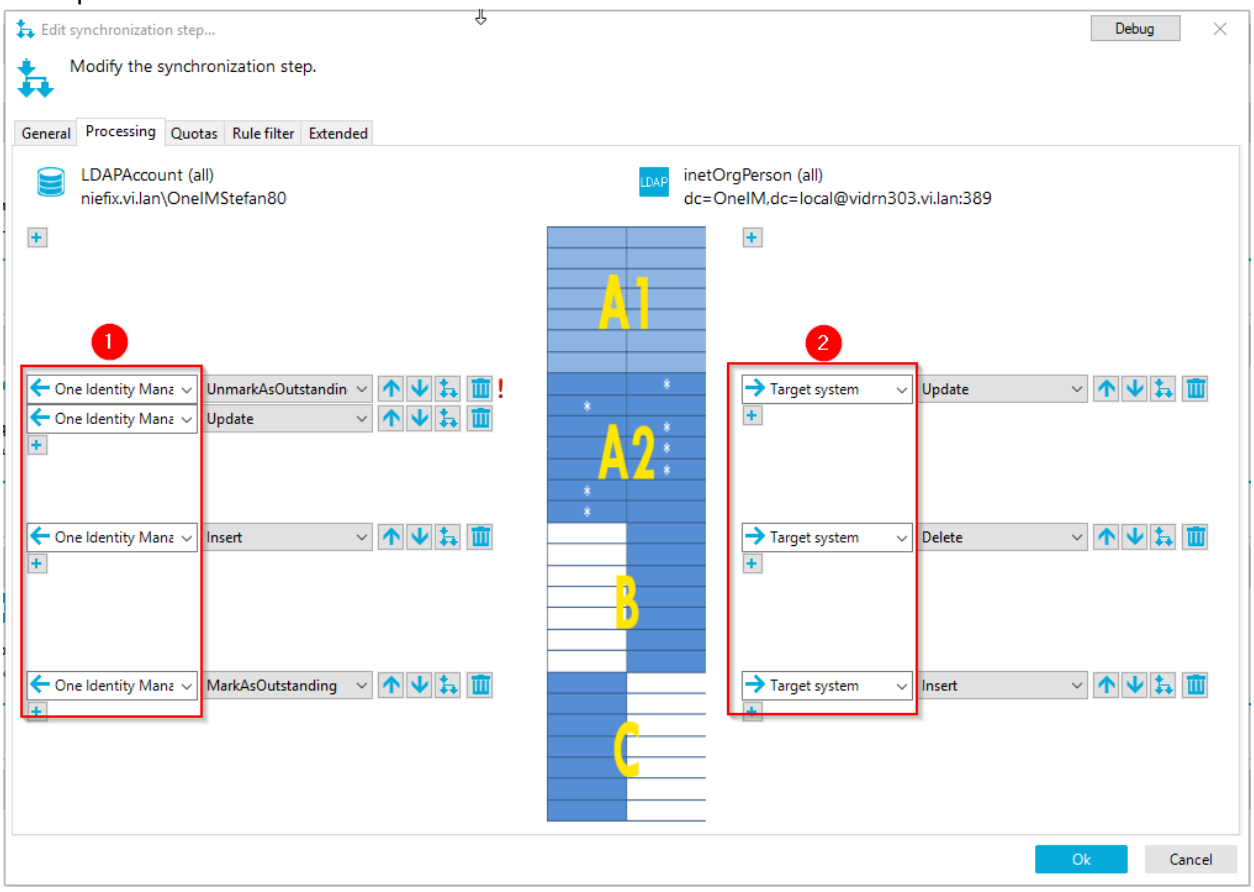


*Figure 91 Example of generic processing settings*

The corresponding statements are:

When synchronizing data to One Identity Manager…

… “UnmarkAsOutstanding” and “Update” the OneIM object for pairs found in both systems (set “A”) that have property differences (“A2”).

… “Insert” the object in OneIM if the object is only present in the target system (set “B”)

… Call the “MarkAsOutstanding” method for the OneIM object, for objects that only exist in OneIM and are not found in the target system (set “C”).

When synchronizing data **to the target system...**

... "Update" the target system object for pairs found in both systems (set "A") that have property differences ("A2").

... "Delete" the target system object if the object is only present in the target system but not in OneIM (set "B").

... "Insert" the target system object if the object is only present in OneIM but not in the target system (set "C").

However, keep in mind that some of the configuration possibilities do not make any sense. The following example shows such a configuration. As stated, the synchronization direction defines in which direction data is transferred for elements that exist in both directions (set "A"). Let us look at the following configuration

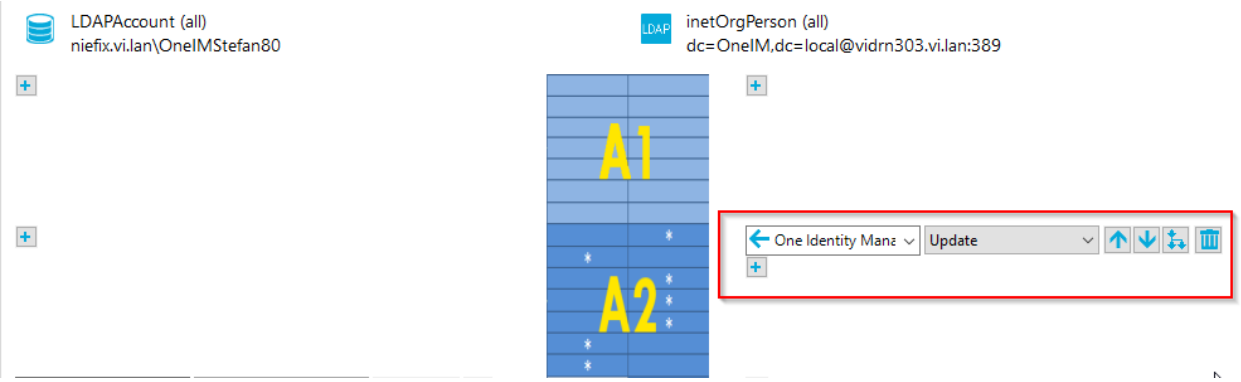



*Figure 92 Misconfiguration*

The data is transferred from the target system to OneIM but the update is called on the target system side. Since no data was changed due to the synchronization direction on this side, the update would simply do nothing.

Caution: When you use the "expert mode" for configuring the processing settings do not edit workflows with "expert mode" disabled.

## Method execution conditions

As stated the method assignments in the processing settings can be executed conditionally. To set a condition click the "tree" icon next to the assigned method in the processing settings as shown below.

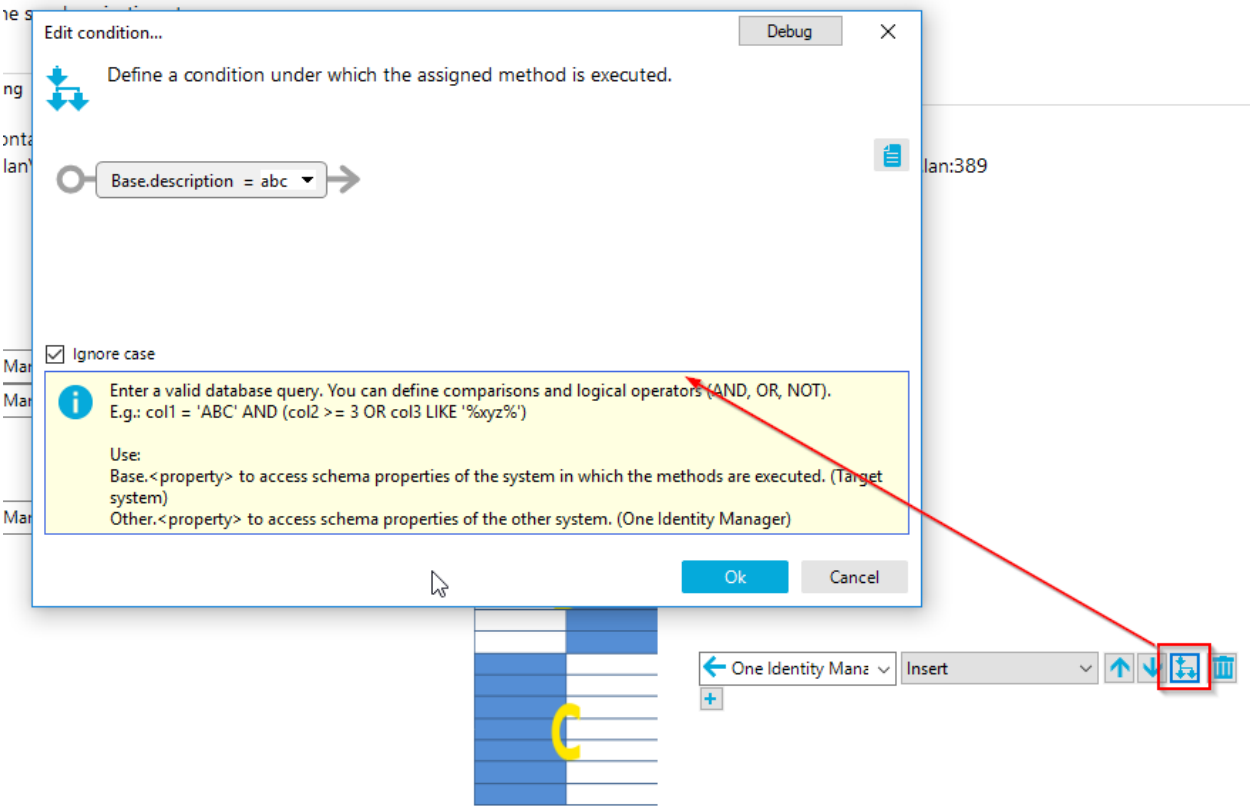



*Figure 93 Method execution condition*

The items that are available for defining the conditions are limited to the properties of the involved objects on both sides. The object that actually has the method assigned is the "**Base**" object. The object in the other system is the "**Other**" object. Accessing a property is done by either choosing it from the list in the graphical mode shown above or by typing

```
[Base|Other].PropertyName
```

The condition evaluation will be executed **after** the mapping was executed. This has the following implication. As shown in the example above, although the object does not yet exist in the target system, you can already use the "Base" object side since the new object already exists in memory.

## Quotas

Quotas are used to limit the number of method calls during a workflow execution in relation to the total amount of objects in the system. This shall prevent data loss or unwanted method executions in case of misconfiguration or target system unavailability. The latter is usually ensured by the connector so quotas are only an engine level safety feature.

Quotas can be configured per step in the "Quota" section of a workflow step. The following screenshot shows a quota definition for either side. For OneIM (1) the engine will stop if

more than for more than 20% of the objects in OneIM the methods “Delete” or “MarkAsOutstanding would be executed or more than 80% of the objects would be updated. On target system side (B) the delete quota is 20% of the total objects in the system and the update quota is 80%.

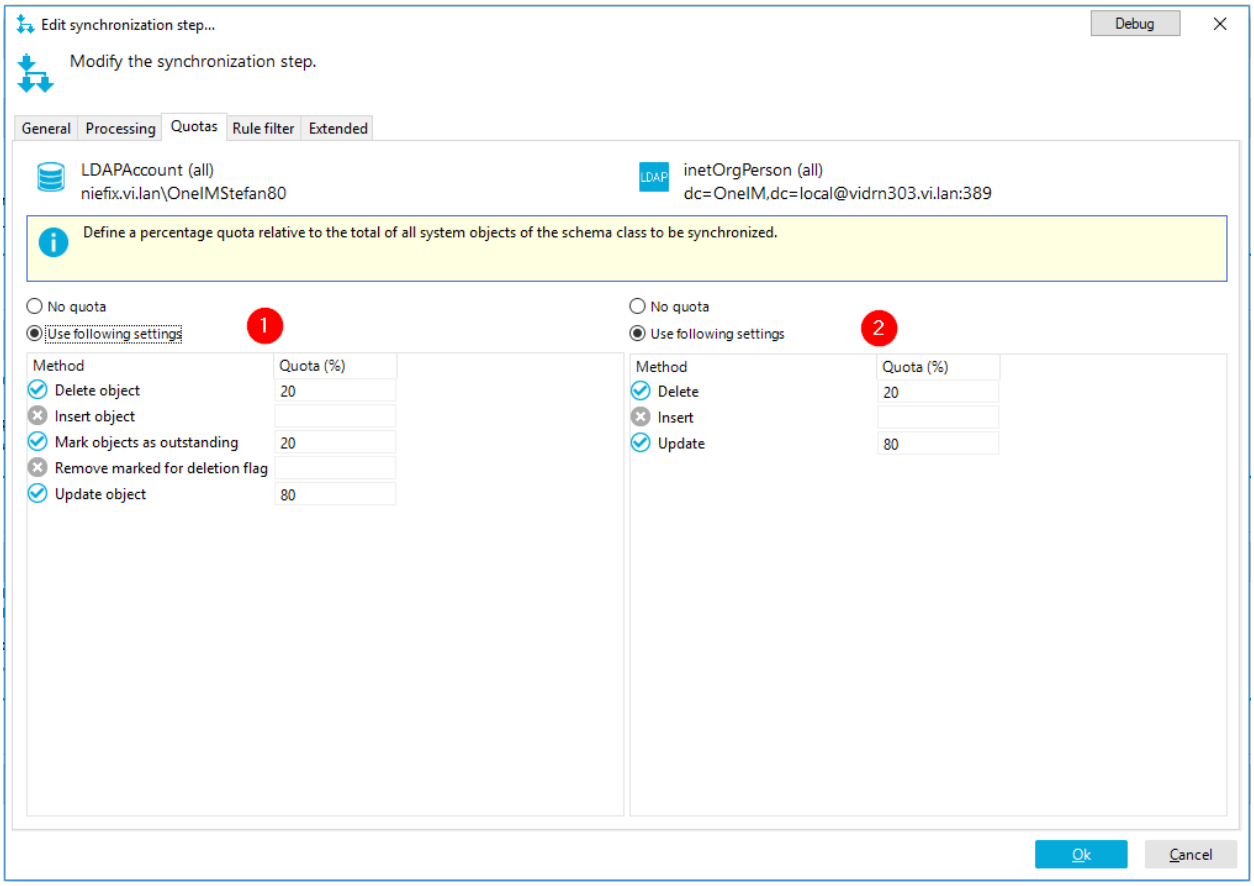



*Figure 94 Quota setting*

The percentage applies to the set of objects within the synchronization scope and is not applied during ad hoc provisioning of single objects.

## Rule Filter

As the name implies, the rule filter allows you to include or exclude some of the mapping rules of the mapping associated to the workflow when executing the step. The filter can be designed as include or exclude list. In the following example, the mapping rule to transfer the description is excluded. All other rules will be executed.

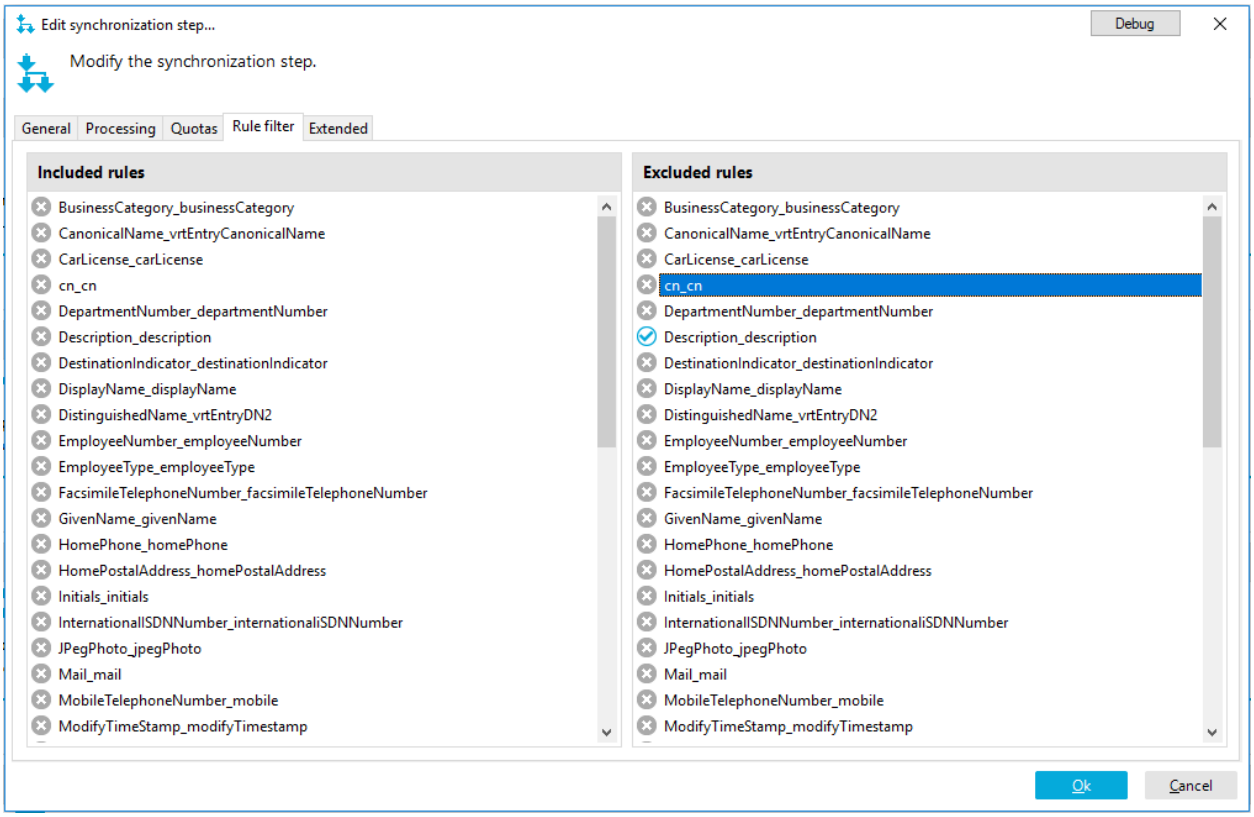



*Figure 95 Rule filter*

The manual configuration of those filters is rarely used. If you want to disable a mapping rule, you should set the direction of the rule to "Do not map" in the mapping. However, this feature is internally used by the automatic dependency resolution feature to create the separate steps.

## Performance settings (extended)

This section allows you to modify the overall synchronization behavior.

> Caution: To understand the performance optimization approaches described here, it is highly recommended to read through the synchronization section first..

### Performance/Memory factor

This setting influences the size of the partitions that are created after the matching phase of the synchronization. A bigger partition (slider to the right) will result in higher memory usage since more objects are fully loaded at the same time. On the other hand, if (and only if) the API used by the connector supports bulk loading of multiple objects, increasing the partition size **may** result in better performance since less target system calls are required.

A smaller partition side (slider to the left) will decrease the partition size and by doing this also the number of fully loaded objects. This will cause a decrease memory usage. Performance is only substantially affected, if the connector supports bulk processing.

**Reload threshold**

This setting allows the default reload threshold setting that was inherited from the workflow/start configuration to be changed.

## Workflow Wizard

The workflow wizard assists you during the workflow creation. It can be opened in the "Workflows" section by clicking the "wand" icon. First step is to choose a name and a description for the workflow.

On the "Assign mapping" page, you can select the mappings you want to create workflow steps for in the first column by clicking the check icons. If revisions are supported by both connectors involved, you can choose if you would like to enable revision filtering for the step. The "Part of hierarchy" column highlights those mappings that have the "Hierarchy synchronization" flag set. If you uncheck those mappings, you will get a warning when proceeding that workflow might not be able to handle some of the selected mappings if they rely on imported hierarchy information.

On the following screenshot the mappings for "GroupOfNames" and "GroupOfUniqueNames" were disabled. Therefore, workflow steps will only be created for "InetOrgPerson", "GroupOfNames" and "GroupOfUniqueNames". Revision filtering will only be applied for the inetOrgPerson mapping.

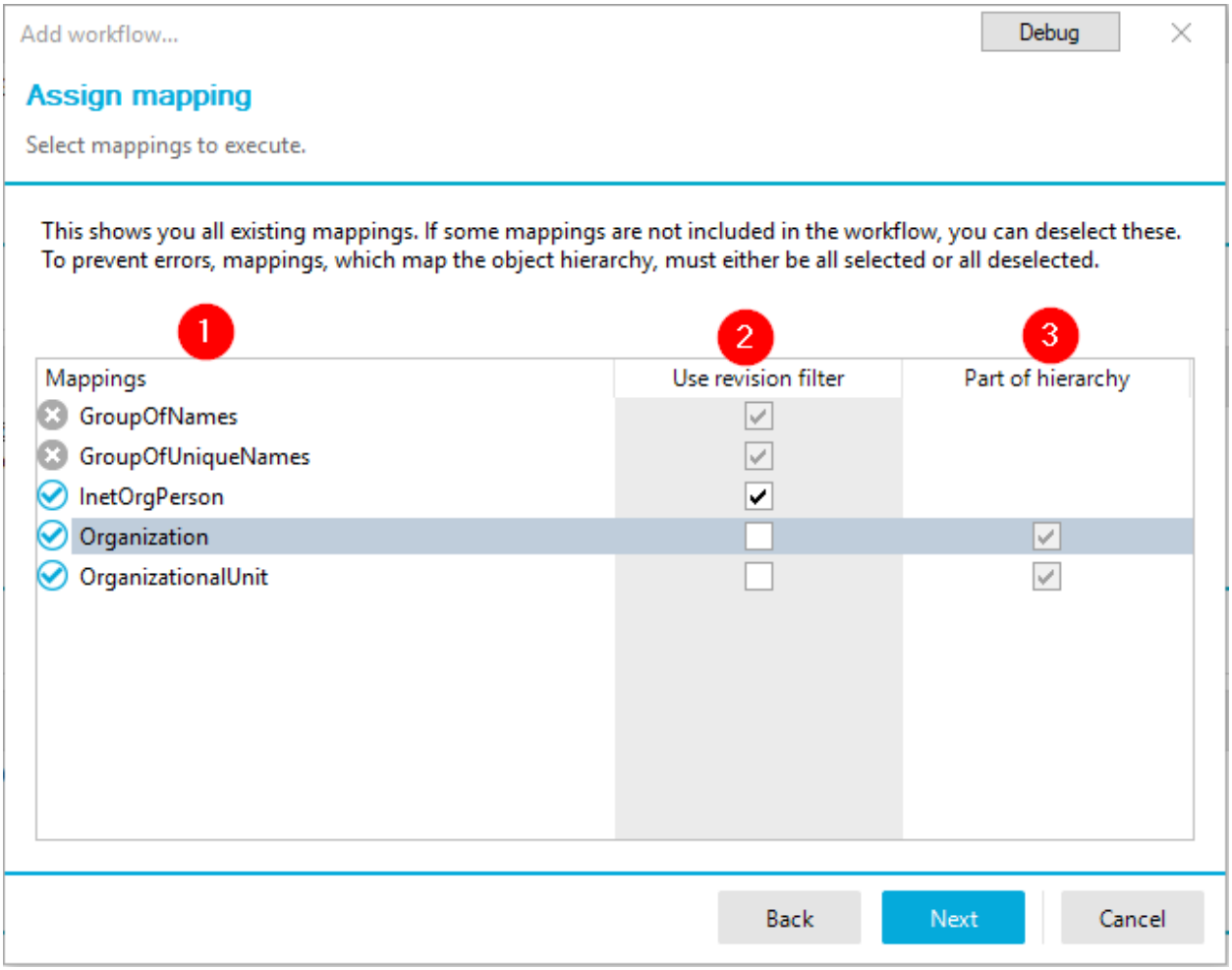



*Figure 96 Workflow wizard mapping selection*

The "Synchronization behavior for intersections" will configure the methods that will be called for objects that are present in both systems as determined during the matching phase. (set "A2"). A method can either be assigned to the One Identity Manager side or the target system side and will implicitly set the synchronization direction for the step. In the following example the "Update" method will be executed for the database objects.

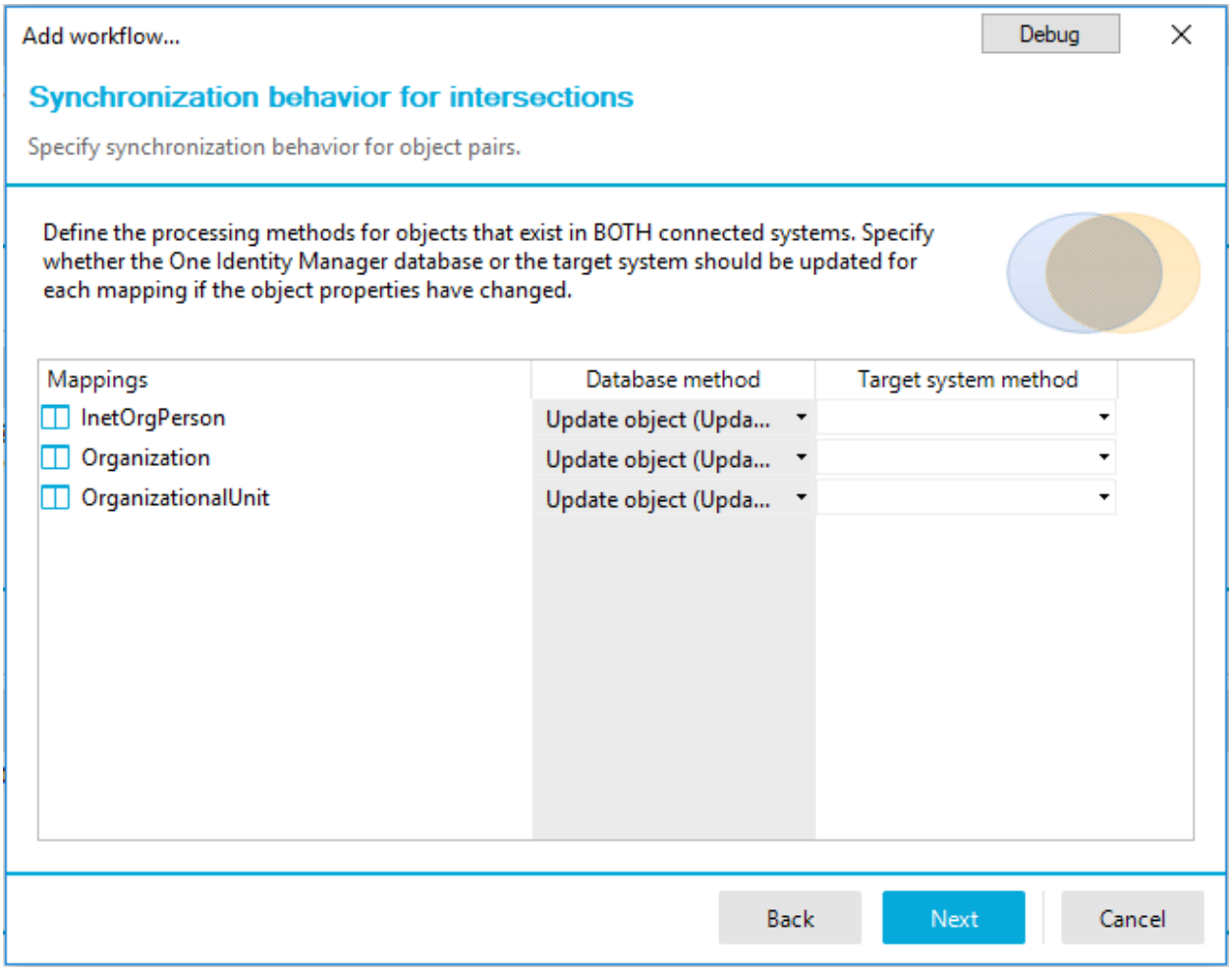



*Figure 97 Workflow wizard intersection configuration*

The next page will configure what shall be done with objects that only exist in One Identity Manager meaning they are not present in the target system. In our example they shall be republished to the target system so that the "Insert" method is assigned to the "target system" side.

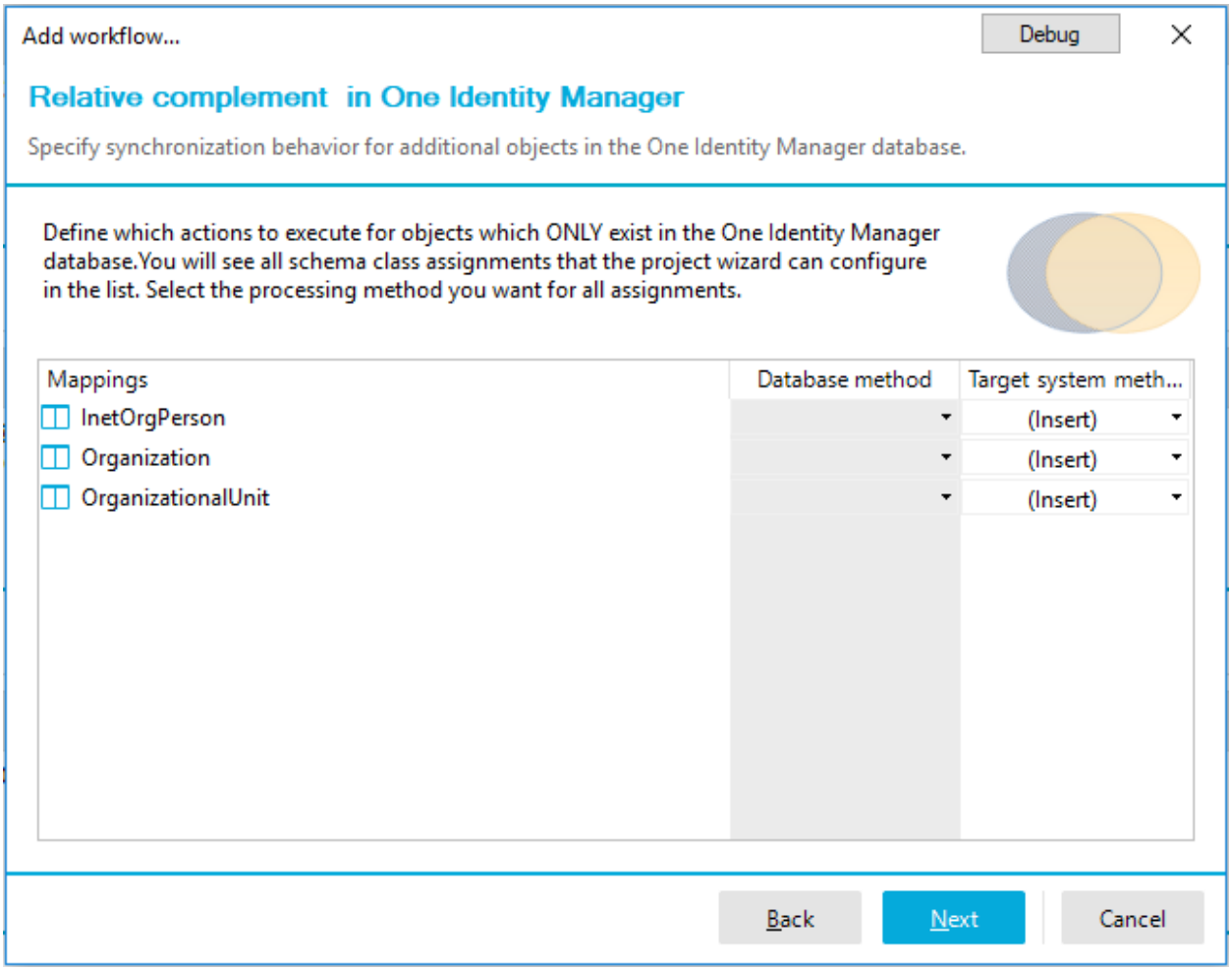



*Figure 98 methods for objects only available in Identity Manager*

The last page will configure methods for objects that are in the target system but not in Identity Manager. In our example, we want them to be imported to OneIM.

As a result we get a workflow with 3 steps. Each is step is configured equally and will import new objects as well as updated objects from the target system to OneIM. If an object is missing in the target system it will be re provisioned to the target system.

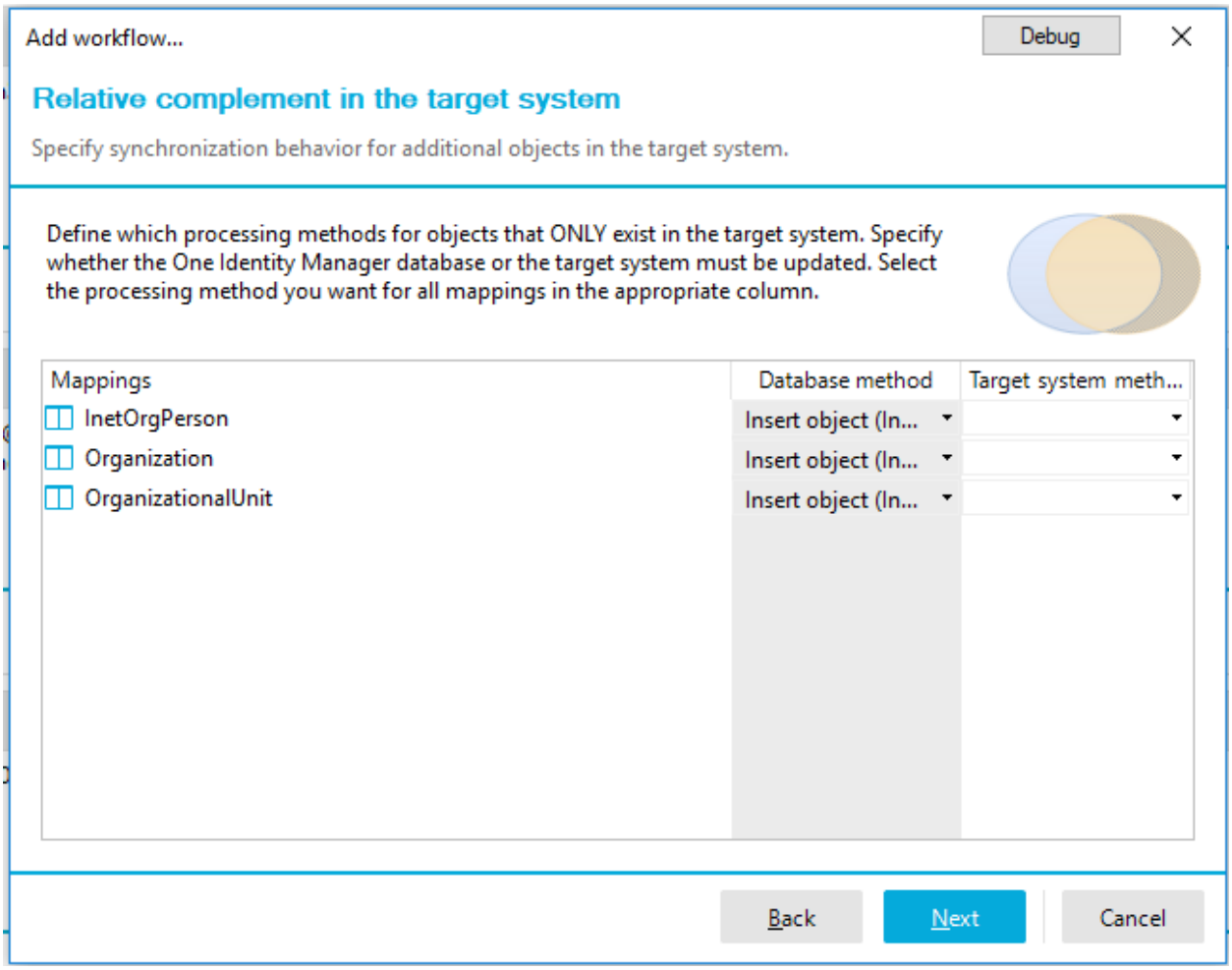



*Figure 99 methods for objects only available in target system*

## Running Workflows

In this section we need to distinguish the usage type of the workflow. As a best practice, the out of the box synchronization templates come with two preconfigured workflows.

The "Initial synchronization" or sync workflow is executed on a scheduled basis and processes all objects as described in the synchronization section. It is configured to import data from the target system to OneIM.

The second workflow is exclusively used for ad hoc provisioning and called "Provisioning". It will only be executed by the One Identity Manager processing engine during the "ad hoc projection" processes of the "ProjectorComponent" for single objects.

However, from a synchronization engine point of view there is no difference between the two and the engine could also run the "Provisioning" workflow and process all data. This would be a "publish all objects to the target system" sync which people usually do not want.

This section will focus on running the workflows that are supposed to process all objects of the target system/OneIM as described in the synchronization section. Those workflows require a startup configuration that can be configured in the "Configuration" → "Start up configurations" section of the Synchronization Editor.

Click the "new" button in the toolbar

The startup configuration configures the start parameters or environment for a workflow. If the workflow is not fully preconfigured in terms of revision filtering and/or synchronization direction, you can set the corresponding defaults at a startup configuration. In regards to the OneIM process orchestration integration a schedule can be specified that the OneIM processing engine uses to execute the workflow on a regular basis. Furthermore, the variable set that shall be active during the workflows runtime needs to be specified.

# Variables (variable sets)

Variables can be used at numerous places in a synchronization project such as connection parameters, mapping conditions, schema properties etc. They can be found in the "Variables" section of the "Configuration" pane.

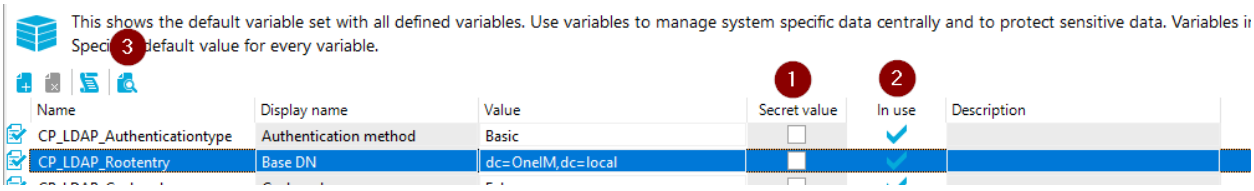



*Figure 100 Variable section*

If variables are marked as "Secret value" (1) they will be encrypted. The "In use" indicator (2) is set, if the variable is used somewhere in the synchronization project. To find those places, you can click the "Show usage…" icon (3) in the menu.

## Types of Variables

Variables can either have a fixed value or can be calculated by a variable script. This script can perform lookups in the OneIM database and access other variables (i.e. to combine values).

To define a fixed value variable, click the "Add a variable" (1) icon in the "Configuration → Variables" section and enter its name, description, value etc. in the corresponding line.

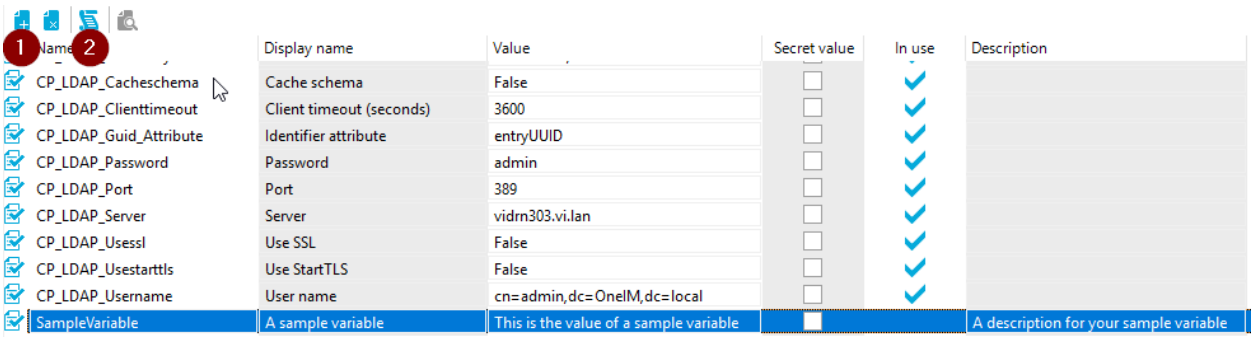



*Figure 101 Definition of a new variable*

To convert a variable to a scripted variable, select the variable you want to convert in from the list and click the "Convert to script" (2) icon. A scripted variable has access to other variables and the OneIM database. The following example shows a script that populates the value of a variable with the contents of the OneIM connection parameter *"TargetSystem\LDAP\PersonAutoFullSync"* but you can also access any other database table with this method.

```
Imports VI.Projector.Connection


Dim query = SystemQuery
.From("DialogConfigParm")
.Filter("FullPath='TargetSystem\LDAP\PersonAutoFullSync'")
.Select("Value")
Dim cpobj as ISystemObject = args.QueryDatabase(query).Result.First()
return cpobj.GetValue("Value").AsString
```

Hint: Be very careful with scripted variables. Depending on the script, the calculation of the variable content can be very expensive.

Also, when querying a system in a scripted variable, make sure that the schema type queried is excluded during the schema shrink process. The shrink process cannot automatically detect the usage of schema types used in scripted variables.

Hint: Another example of how a scripted variables can be used to dynamically determine connection data can be found here: https://connect.oneidentity.com/products/identity-manager/w/knowledge-base/628

## Usage

When using a variable, it needs to be enclosed in $ characters such as $yourVariable$.

## Variable sets

During the initialization of the engine runtime environment, the variables are initialized and made available for all other components. By defining variable sets, it is possible to switch the runtime to another set of variables. This allows a bunch of use cases like using the same synchronization project to synchronize multiple systems of the same type. There is always a default variable sets that the other sets are derived from. Variables that are not overridden in the derived set will keep the value from the default variable set. The following example shows a variable set that overrides the CP_LDAP_Rootentry variable which will cause the runtime to connect to a different base entry.

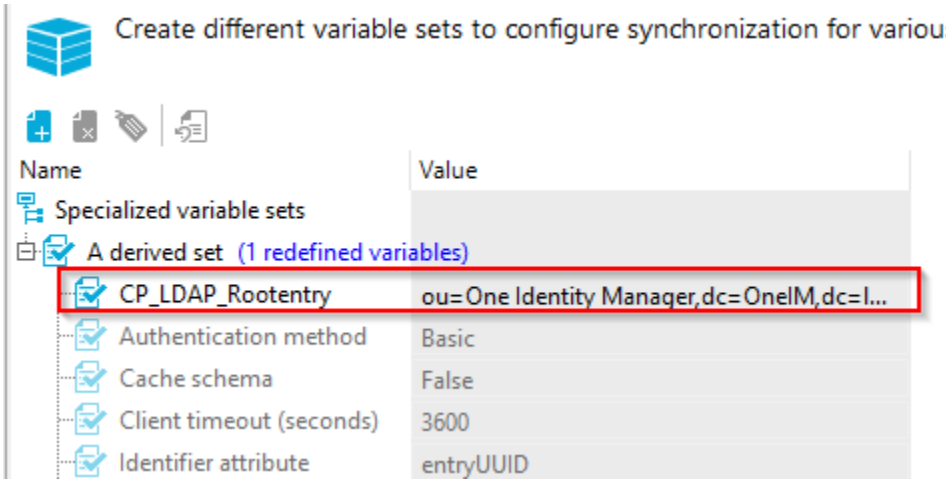



*Figure 102 overridden variable*

The Synchronization Editor usually initializes its runtime with the default variable set but you can switch to any other variable set in the main toolbar. All other operations (i.e. browsing the target system) will now use a runtime that is initialized with the selected variable set.

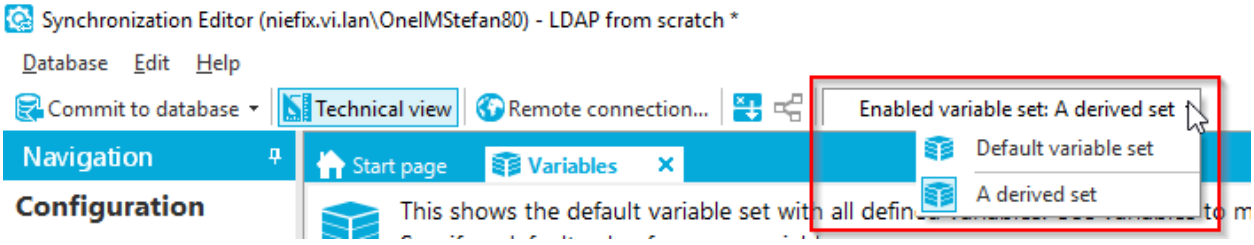



*Figure 103 Switching the variable set*

# Base objects

Base objects or sometimes referred to as "root" objects are objects in OneIM that represent one instance of a connected target system i.e. a LDAP directory, an Active Directory domain, a SAP client etc. In order to link a synchronization project to such a base object, base objects can be registered in the configuration section of the Synchronization Editor. The base objects of the Synchronization Editor also contain some additional information that is required by the synchronization engine and the OneIM process orchestration.

**A synchronization project can handle multiple root objects (system instances).** This is done by creating multiple variable sets with different connection parameters within one project. Then, those variable sets are assigned to a base object registration along with the default OneIM job server that shall handle the OneIM processes for the system. The following illustration shows a OneIM database with 3 LDAP domains and 2 job servers. Two of the domains (OneIM and AnotherLDAP) are handled by synchronization project "LDAP 1" since both systems share the same schema and workflows. Domain "SuperLDAP" however uses different workflows and was therefore configured in a separate synchronization project "LDAP 2".

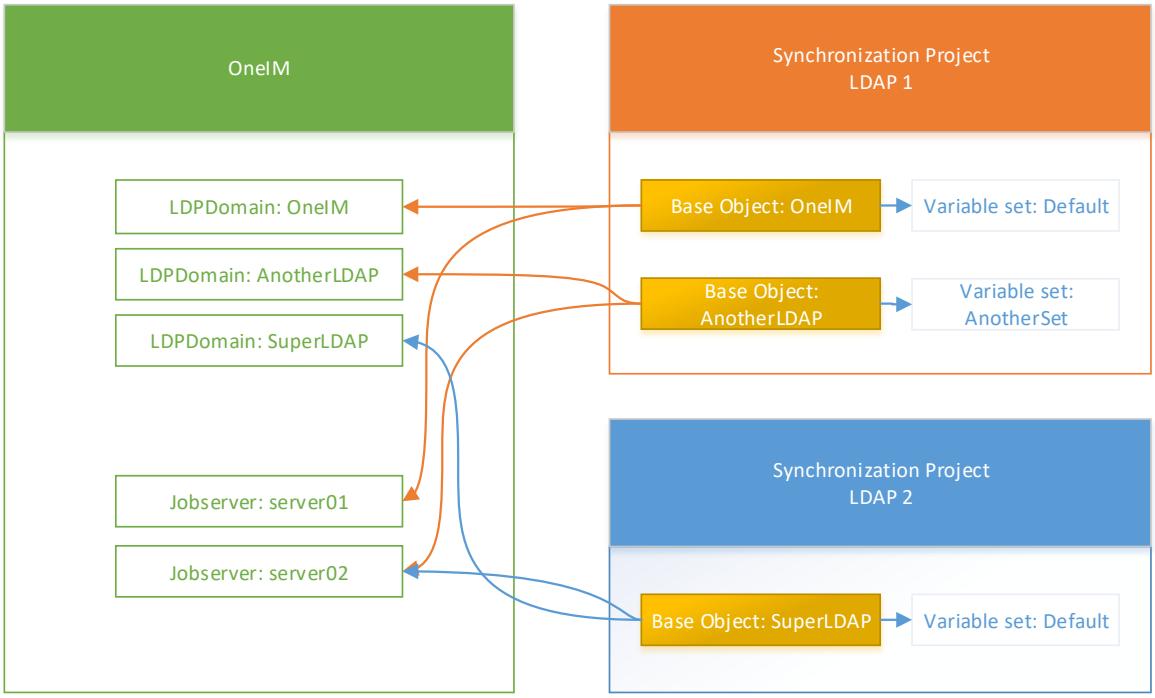



*Figure 104 Root object concept*

Hint: Just by defining a root object the Synchronization Editor cannot automatically separate data of the several LDAP systems in the OneIM tables. For this purpose, a synchronization scope needs to be configured.

Base objects are stored in the DPRRootObjectConnectionInfo table and are used for several OneIM process orchestration tasks.

Synchronization project templates of target systems that are supported out of the box usually handle root objects automatically. This means during the project creation, they create the base object in OneIM (i.e. a LDAP domain) and also configure a Synchronization Editor base object. In those scenarios, you do not need to create all those objects manually as was done in this walkthrough.

Besides a synchronization project template, out of the box supported systems usually come with a so called "base object template". These are used by the root object wizard in the Synchronization Editor.

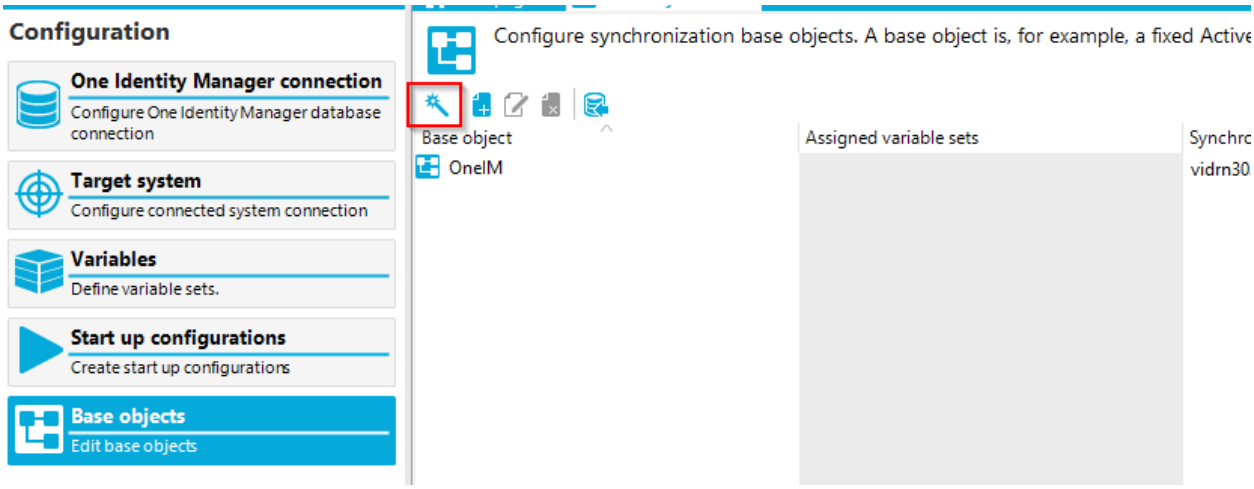



*Figure 105 Base object wizard*

If such a template is available, it will perform the following workflow:

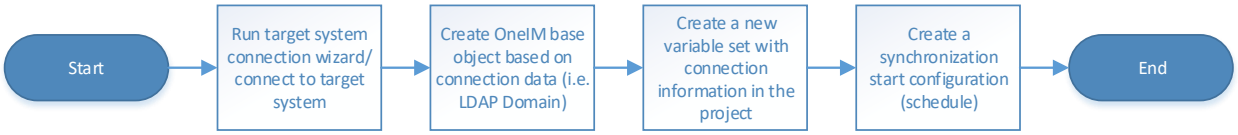



*Figure 106 Base object workflow*

## Systems without base objects

There are scenarios in OneIM where no base object is available. If you want to import "person" records to OneIM there is no real base object like a domain available since there are no "instances" of different "person" systems modelled. In those scenarios, you can create the anchor "base" object in the Synchronization Editor but instead of picking a record

from a certain table as base object, you select the table itself as shown on the following screenshot. The “Person” table is used as “base object”

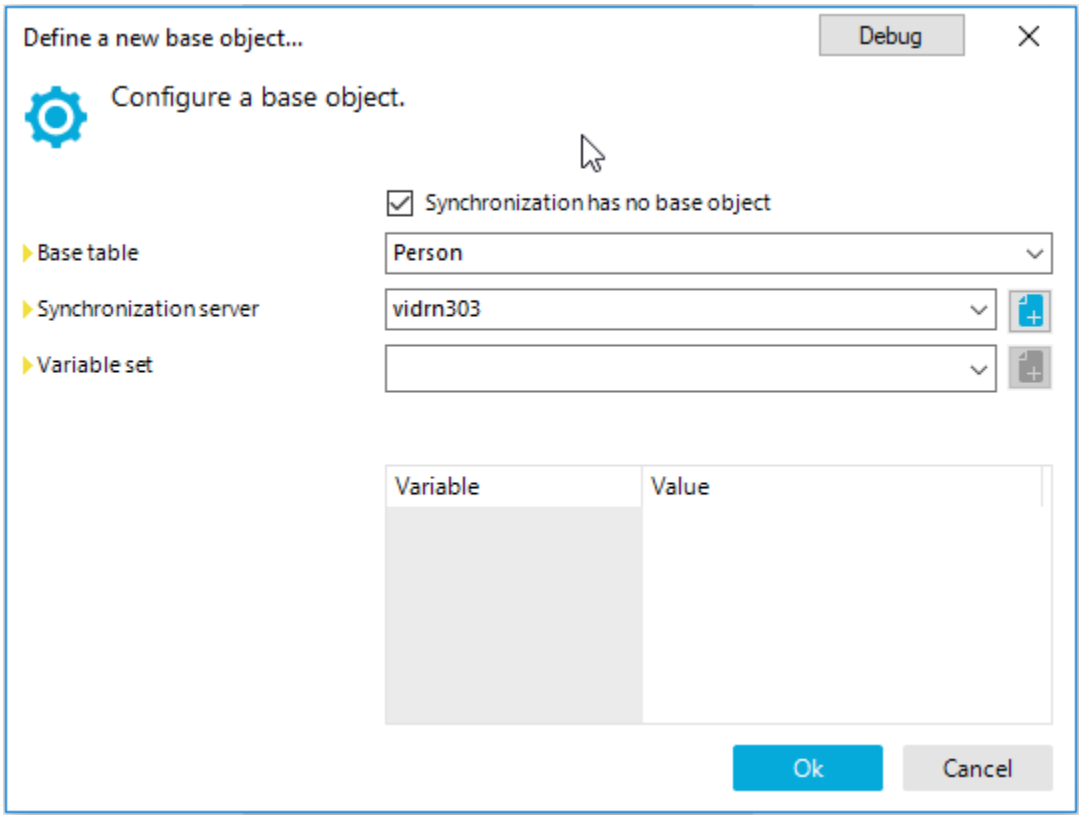



*Figure 107 Configure Person table as base object*

This feature can be used to populate those parts of the data model that do not support different “instances”.

## Synchronization scope

A synchronization scope limits the objects that are handled during a synchronization based on a given scope filter. Almost every synchronization project requires a scope filter for the OneIM connection since the data model of OneIM usually allows multiple instances of a target system of the same type to be synched to the same OneIM tables. Allthough this is the most common use case, the synchronization engine allows the definition of scopes for each connection.

Scopes can be configured in the “Configuration” section of the Synchronization Editor. Open the connection you want to specify a scope for and select “Edit scope”.

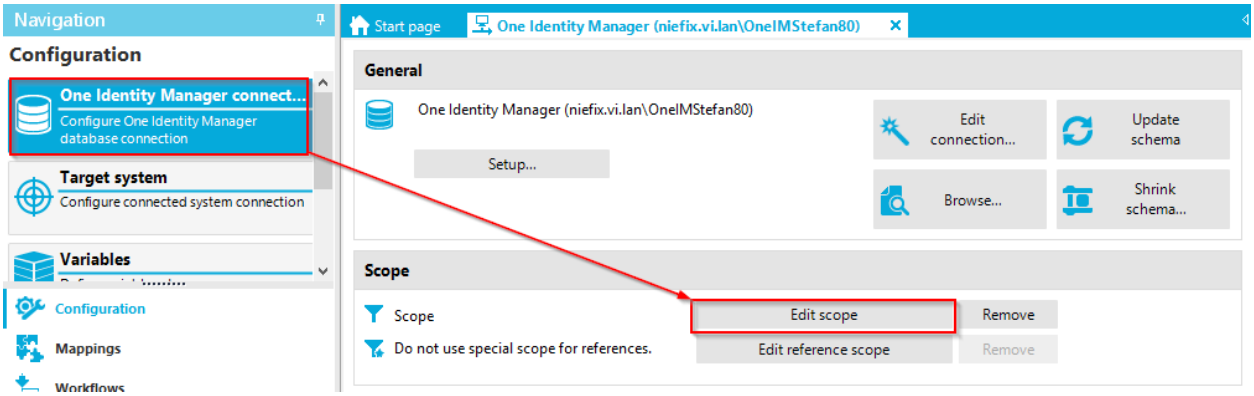



*Figure 108 Scope configuration*

The first thing you can define is the name of the scope. Since this setting is only used during logging, you can leave the name as is. The "scope" section (1) shows all schema types of the selected system in hierarchy form.

If the list is empty or incomplete you have either shrunk the schema already or schema types without hierarchy information are filtered. The "show all objects" option will make schema types without hierarchy information visible. To show the schema types that were removed during the schema shrink operation, you need to update the schema for the connection.

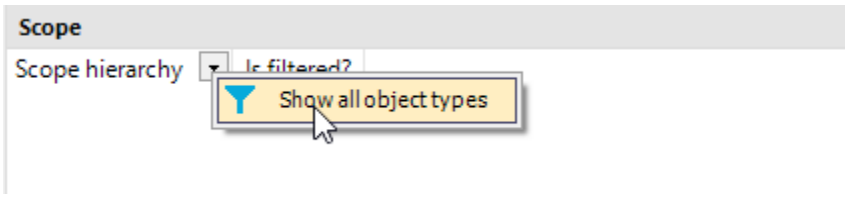



*Figure 109 Show all objects option*

There are two types of hierarchy within a scope definition.

## Logical hierarchy

The logical hierarchy describes the relation of the schema types in an object/data model. It searches the schematypes properties for a reference property that contains a reference target that is marked as "Scope reference". There can only be one of those references defined per schema type.

Since this is a bit abstract in theory let us look at the OneIM connector and the LDAP tables.

For the OneIM LDAP data model, the domain (LDPDomain) is the top level entity. Users (LDAPAccount) and groups (LDAPGroup) are within a domain. Same for containers (LDAPContainer). Users and groups can also exist in containers which, if present, would be the primary hierarchy step. The model for this example looks like this:

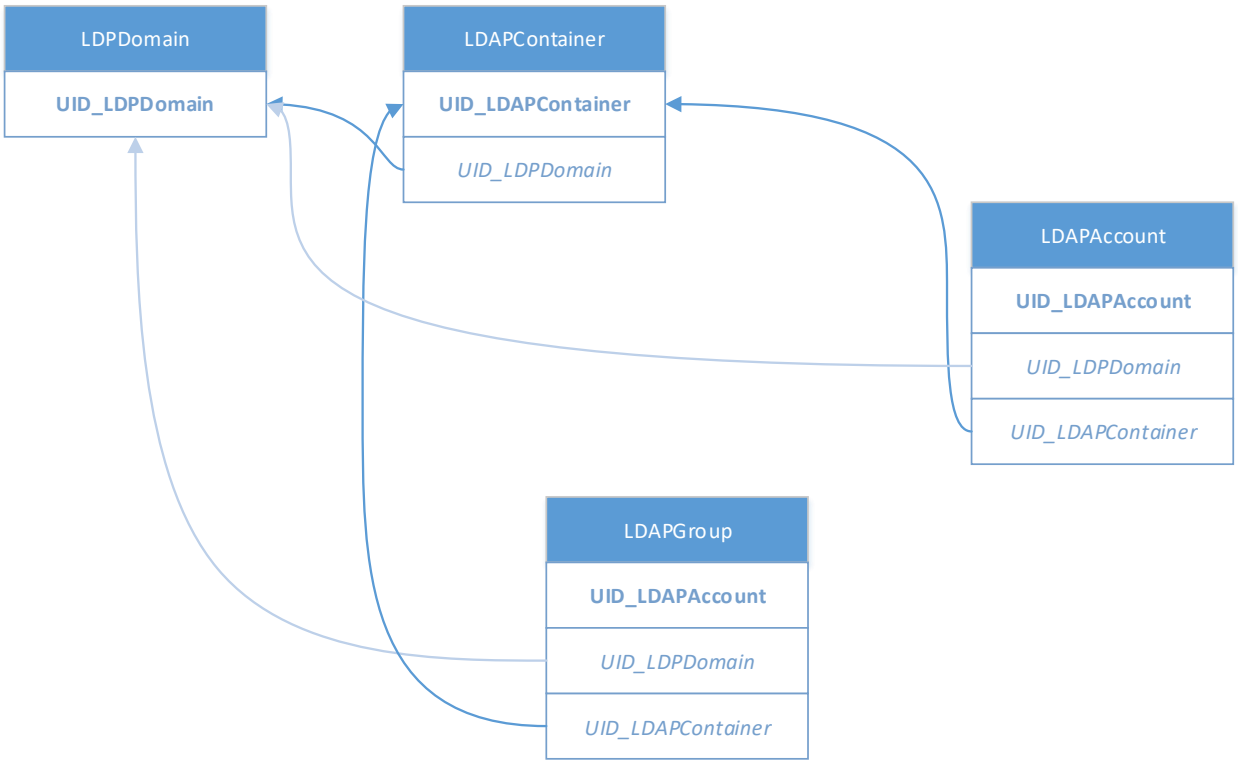



*Figure 110 LDAP logical hierarchy*

As you can see, there are multiple reference targets for LDAPGroup and LDAPAccount that could be marked as "scope reference" (UID_LDPDomain and UID_LDAPContainer). Since this is not allowed, the OneIM connector constructs a virtual property called "vrtScopeParentReference". It uses the information stored in the "DialogTable" table in the "ScopeReferenceColumns" column. This column contains a comma separated list of foreign key columns that construct the logical structure. The ordering represents the weighting of the link. The column mentioned first is the strongest. When dealing with an object instance, the OneIM connector iterates through the given properties mentioned in ScopeReferenceColumns and picks the first that has a value set. So for LDAPAccount, UID_LDAPContainer will win over UID_LDPDomain if set. Other connectors may use other approaches.

The following screenshot shows the scope hierarchy in the Synchronization Editor for the OneIM schema types dealing with the LDAP tables (note that LDPMachine also exist with the same links as LDAPGroup and LDAPAccount):

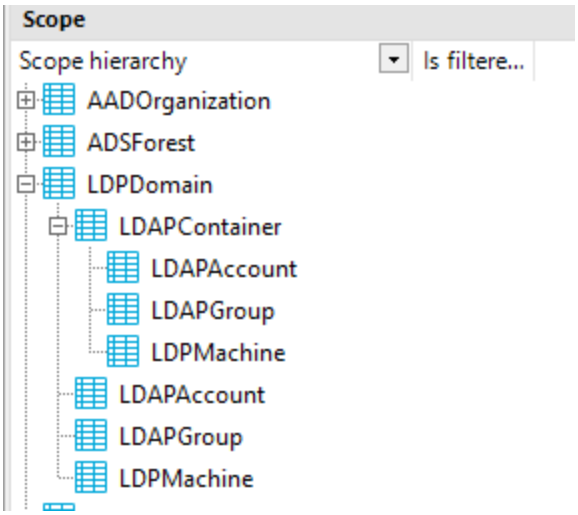



*Figure 111 LDAP scope hierarchy*

As you can see LDAPAccount and LDAPGroup are listed twice. Once directly under LDPDomain and once under LDAPContainer. By leveraging this information a scope shall be created that contains all objects of a domain. In this case, you just need to define a filter for LDPDomain that will automatically effect the logical subordinate schema types.

The filtering works the same way as the filter of the "Generic schema class". After defining the system filter as "distinguishedName = '$CP_LDAP_RootEntry$'" and the Object filter as "`DistinguishedName='$CP_LDAP_Rootentry$'`", the scope hierarchy view changes to:

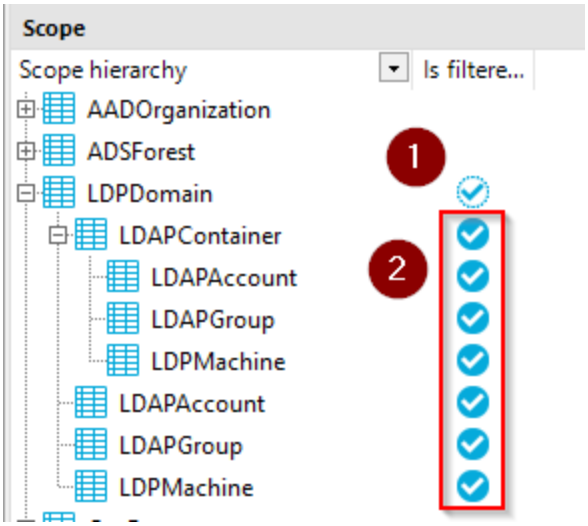



*Figure 112 Active scope filter*

The "Is filtered" column is checked for all LD(A)P* schema types now. LDPDomain (1) has a filter assigned directly whereas LDAPContainer, LDAPAccount, LDAPGroup… inherit the filter from LDPDomain (2) based on their reference properties.

Hint: If available, use system filters when setting scopes. This will speed up the synchronization.

Object hierarchy

In contrast to the logical hierarchy, the object hierarchy is based on object instances instead of the structure defined by the schema. The LDAP connector for example exposes the directory tree for scope selection and allows you to define include/exclude rules for single entries or complete subtrees. If a connector supports this type of scope, the scope editor will have an additional section.

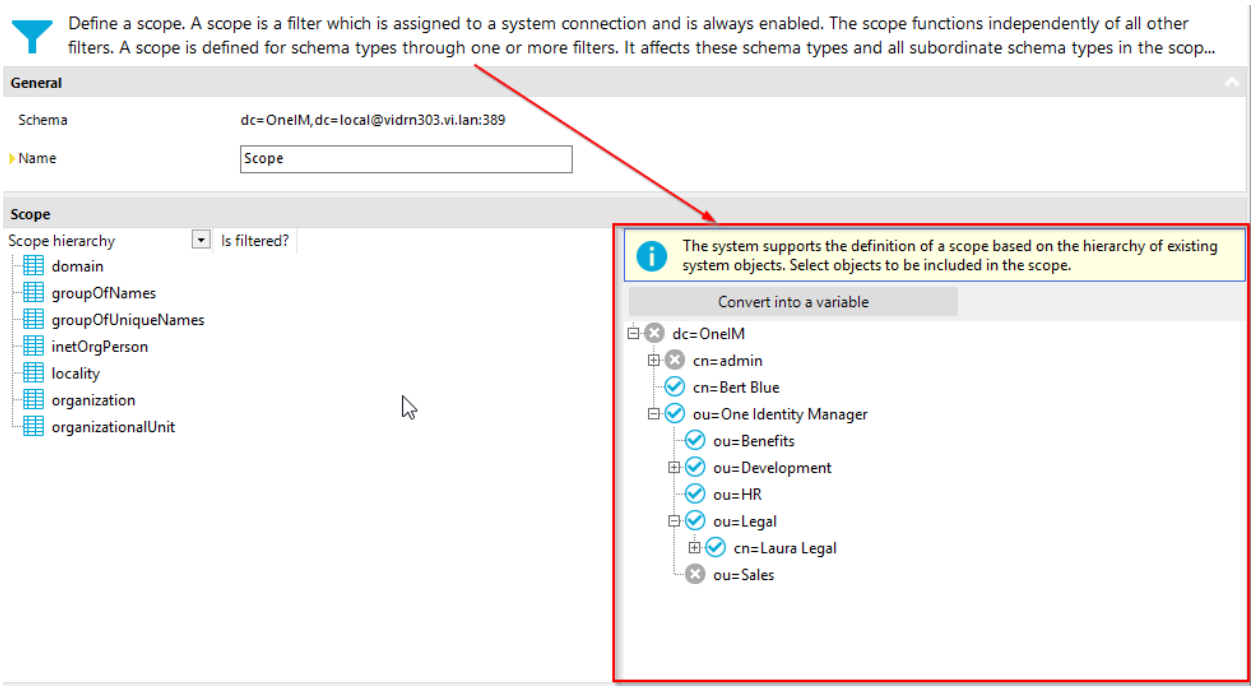



*Figure 113 Object hierarchy scope definition*

As already stated, this scope works using an include/exclude list. If this list is empty, it is assumed that the root element is included (and therefore all subordinate entries). A list entry (include/exclude) automatically affects all subordinate entries without them having to appear explicitly in the include exclude list. That means:

Hint:
If parent object is excluded → all children objects are excluded implicitly
If parent object is included → all children objects are included implicitly

In the above example, the include exclude list would be:

| Object | Type |
|---|---|
| dc=OneIM | Exclude |
| cn=Bert Blue | Include |
| ou=One Identity Manager | Include |
| ou=Sales | Exclude |

When using this type of scope, you need to keep in mind that new objects created in the target system are automatically imported or ignored, depending on their (in)direct parent and its include/exclude configuration. The visualization in the Synchronization Editor does not have an indicator if a node is implicitly included/excluded or was explicitly configured.

## Reference Scope

A reference scope defines the boundaries for objects that are reference targets. In a closed separated system the reference scope usually equals the regular synchronization scope. In some cases i.e. Active Directory however, you might have objects (i.e. groups) that are referencing other objects (i.e. users) that are from a different system (domain). While the Active Directory connection itself might not be able to load and resolve those references, the reference resolution on OneIM side might require a second synchronization project to synchronize the "foreign" domain.

The following illustration shows the scopes of a synchronization project for "Domain 1". In this domain is a group "Sales" with members from "Domain 1" and "Domain 2". While the regular synchronization scope just covers objects from "Domain 1", the reference scope allows objects from "Domain 1" and "Domain 2".

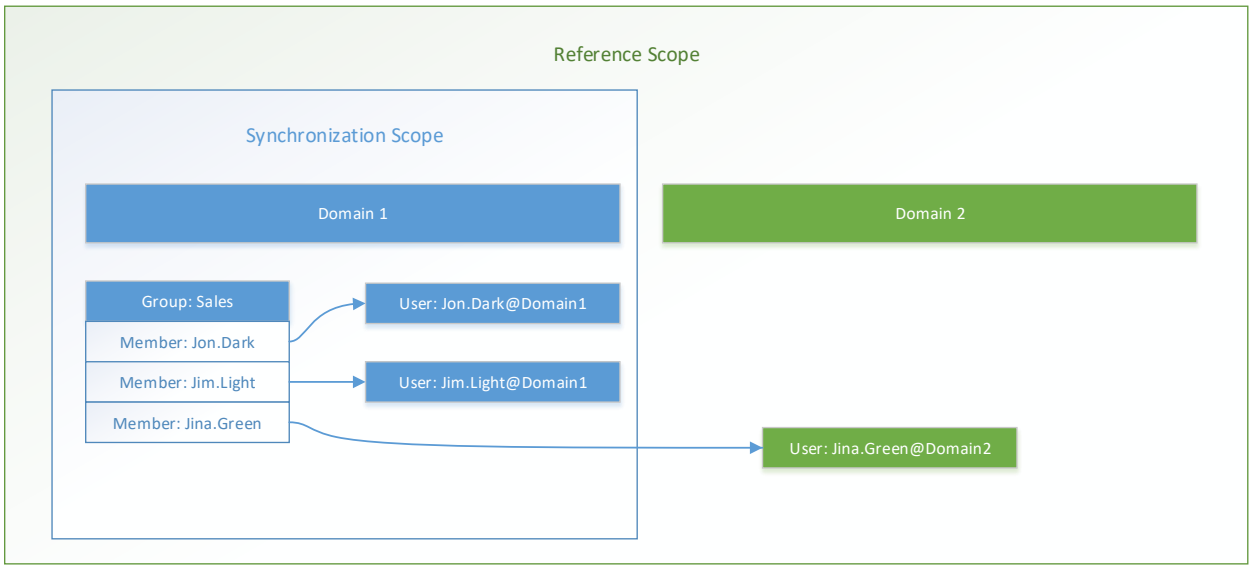



*Figure 114 Reference scope*

Hint: If no reference scope is defined, the reference scope equals the synchronization scope.

## SystemObjectData

SystemObjectData is the name of the internal structure that represents objects of a target system that are exposed by the connectors including their current state. The target system connectors are responsible for translating the native data from the system into this generalized form. During the synchronization, those SystemObjectData instances are passed back and forth by the engine. Thus, they also may contain modifications that were done along the way. A SystemObjectData usually only contains the properties (data) that were actually requested by the engine plus the properties that are required to identify (load) the object and its displays. Modifications to SystemObjectData instances are stored parallel to the data currently loaded until the connector actually carried out the changes. After the connector has carried out a modification, it has to “apply” the modifications which means, the modification data needs to be merged with the actual properties and be cleared from the data structure afterwards.

Example:

The following Diagram show the SystemObjectData lifecycle for an update to the description property for an LDAP inetOrgPerson object. The DistinguishedName property is used as a key property.

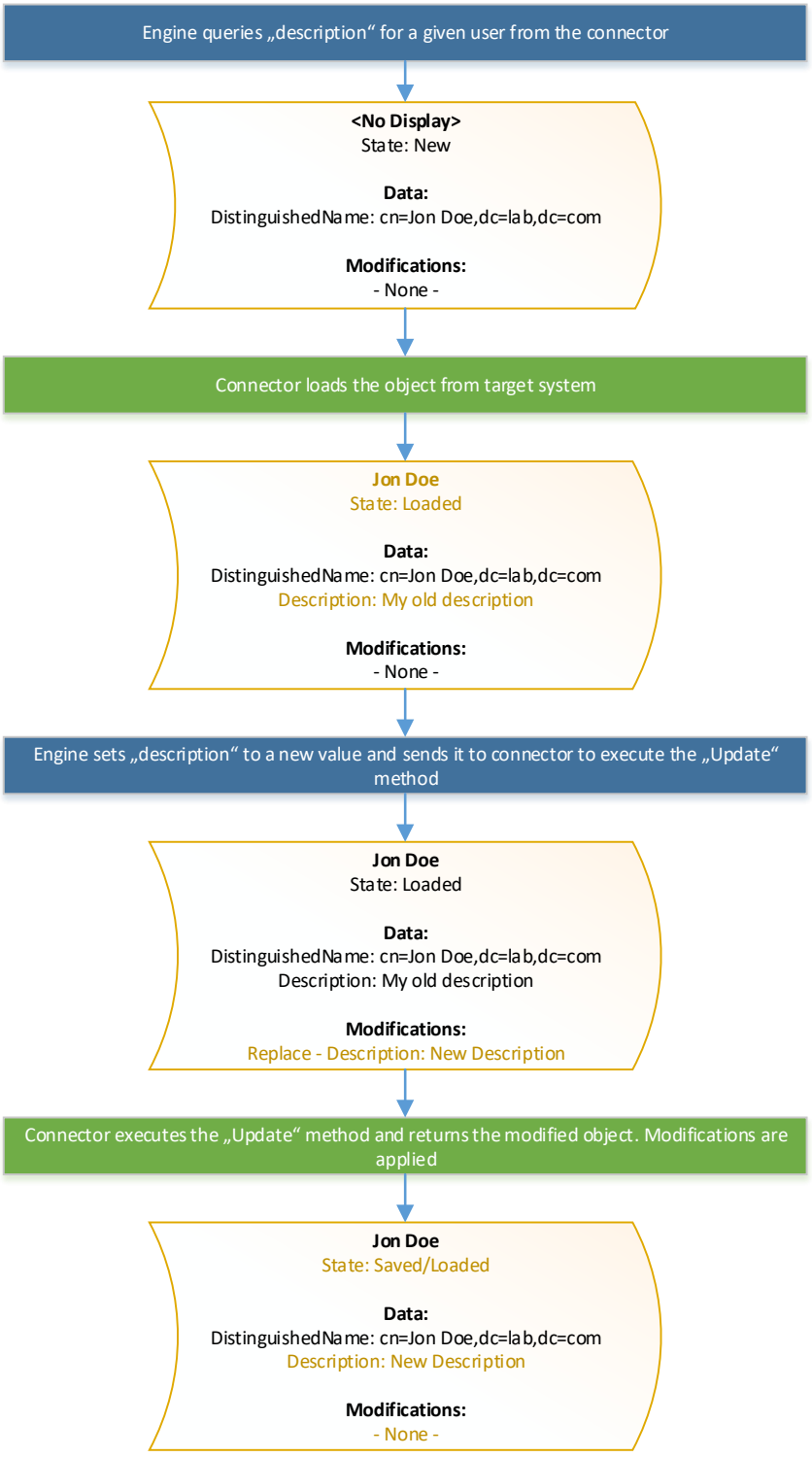



*Figure 115 SystemObjectData lifecycle*

# Logging

Information about what happened during a synchronization or provisioning can be found and configured in multiple ways. The first, most high level kind of logging is the Synchronization log which can be found in the “Logs” section of the Synchronization Editor.

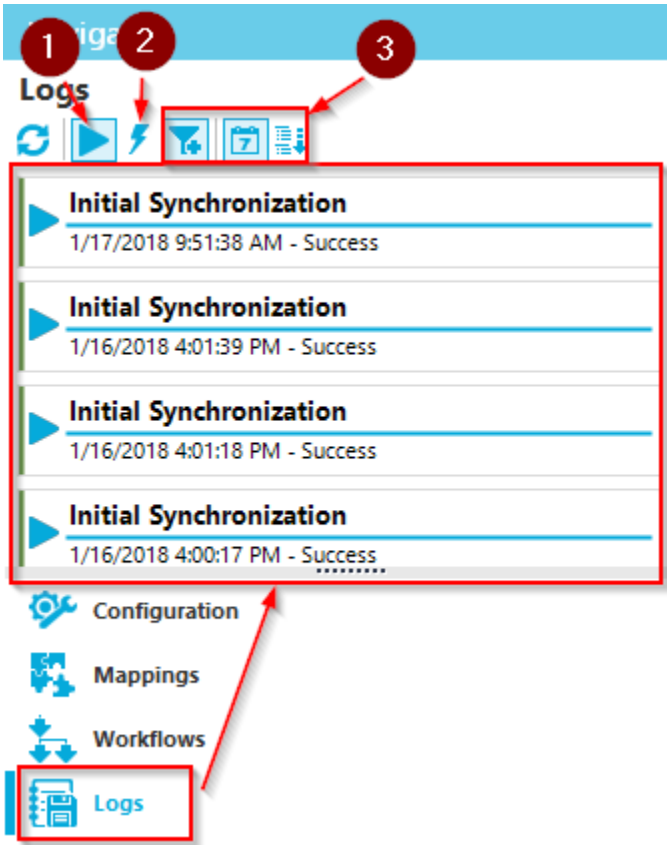
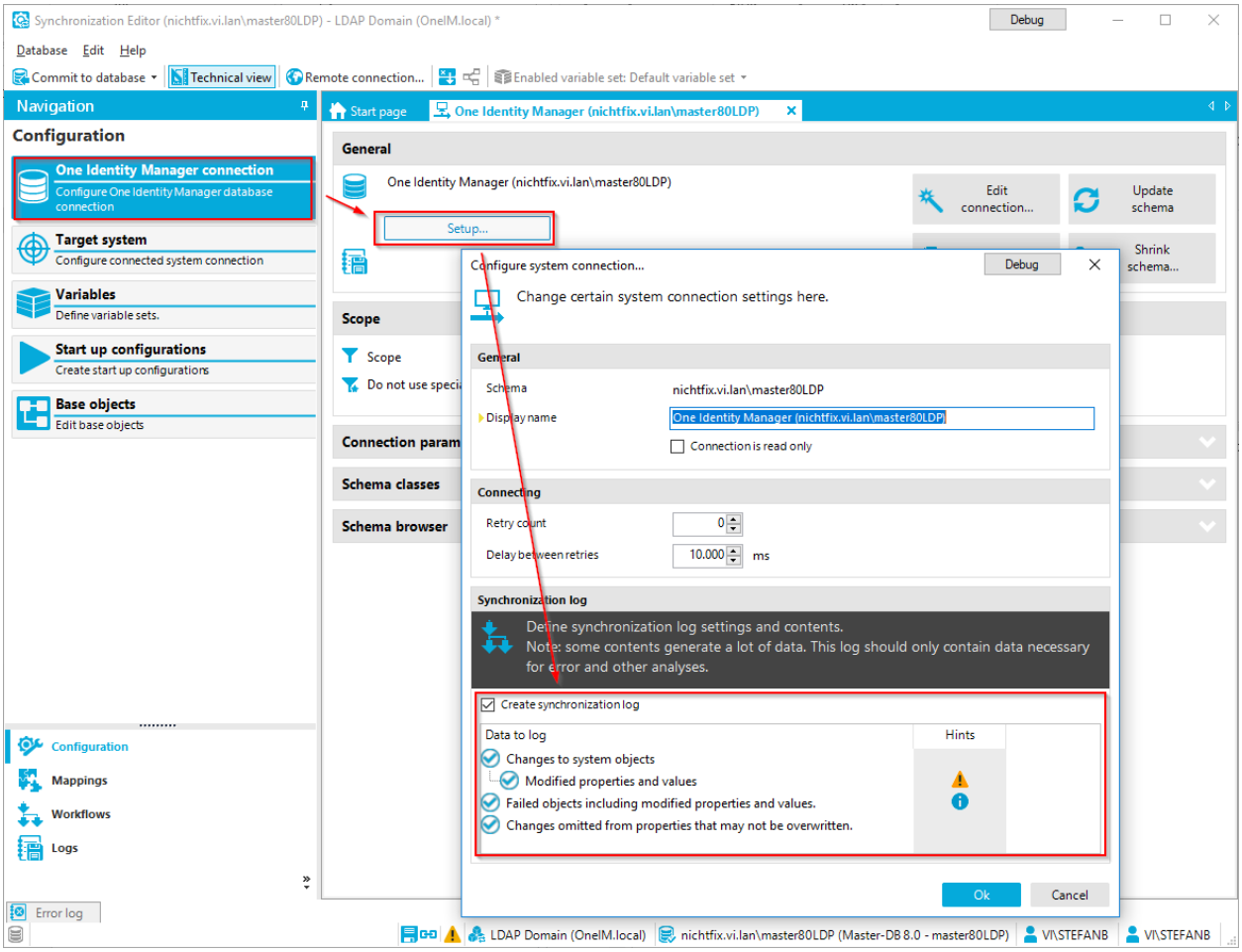



*Figure 116 Synchronization logs*

Logs can be filtered in various ways. The default setting (1) will show you the logs of synchronization processes. The setting (2) will show the logs of provisioning processes for single objects. Since there are usually a lot of these, this setting is turned off by default. Other settings (3) allow you to only show the logs of the last 24 hours and/or sort the result list by execution time or result state.

## Log settings

Log settings can be configured per connection involved in the synchronization project. To do so, choose the connection that you want to configure and click the setup button. The logging options appear in the “Synchronization log” section.




Enabling the different options affects the overall performance. The following options are available:

| Option | Description | Performance impact |
|---|---|---|
| Changes to system objects | Logs the number of executions of methods (i.e. Insert/Update/Delete) for each schema type processed. | Low (1 additional database write operation per object) |
| Modified properties and values | Logs the changes of properties for each object | High (multiple additional database write operations per object) |
| Failed objects including modified properties and values | If turned on, the engine dumps the entire object that created an error into the log along with the exception when the workflow execution is running in “break on error” mode. If turned off, only the error message will be logged. | Medium (multiple additional database write operations per object but only applicable for workflows running with “break on error”). |

| | | |
|---|---|---|
| Changes omitted from properties that may not be overwritten | Logs cancelled write attempts of value comparison rules that are configured to prevent overwriting of values. | Low (1 additional database write operation per prevented overwrite) |

## Reviewing the logs

A synchronization log is rendered as a report in the Synchronization Editor. The data source for this report are the OneIM database tables starting with DPRJournal*. In its most basic form, it has 4 sections.

**"Synchronization log"**

Information about the project and workflow that was executed as well as the used variable set and the result state.

| Synchronization log | |
|---|---|
| Creation time | Mittwoch, 17. Januar 2018 09:51:38 |
| Synchronization project | LDAP Domain (OneIM.local) |
| Synchronization context | Full |
| Synchronization workflow | Initial Synchronization |
| Variable set | default variable set |
| Synchronization state | Success |

*Figure 117 synchronization log section*

**"Synchronization journal settings"**

This section contains the log settings that were configured/active during runtime.

**"Synchronization log messages"**

This section can be split into several sub sections depending on the configured logging settings.

***Synchronization progress***

This is one of the most important sections when it comes to performance analysis. This section contains the workflow steps that were executed along with the number of "processed elements" and the processing time of the step. A processed element is not necessarily a single object of the system. Depending on the context, this can also refer to a whole schema type i.e. when a connector is updating the revision information for a certain type.

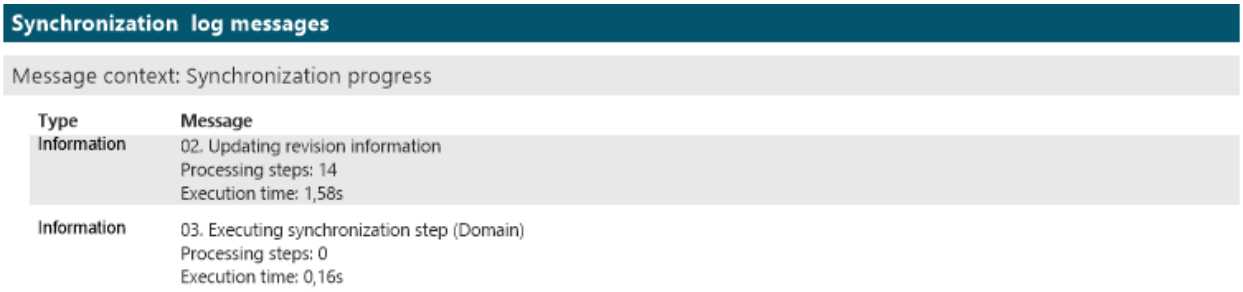



*Figure 118 Synchronization progress log messages*

Overwrite protection

For value comparison mapping rules that are configured not to overwrite a value, messages are logged, when the rule prevents overwriting data.

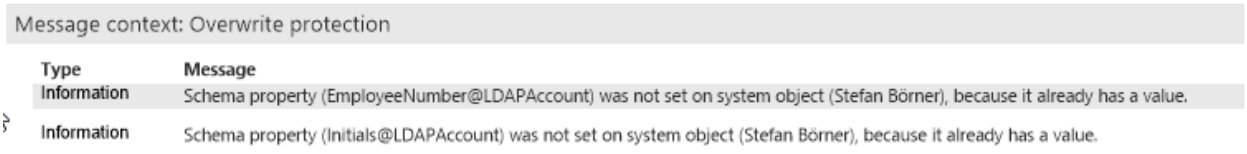



*Figure 119 Overwrite protection message example*

Rogue Change

Messages logged by the rogue change feature are logged in this section.

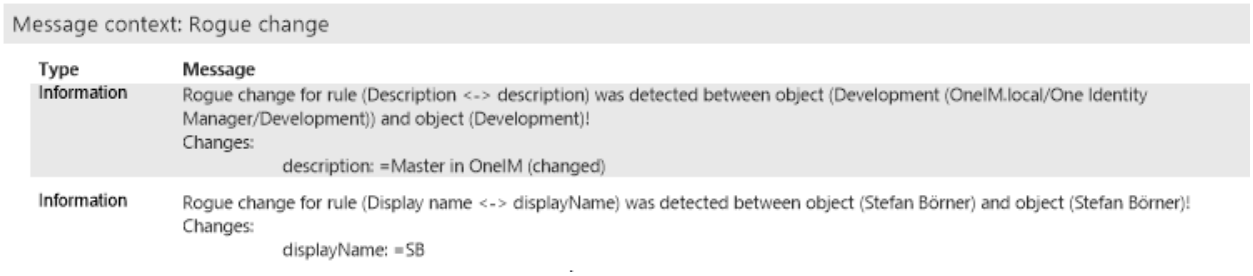



*Figure 120 Rogue change log message example*

## Processed objects

This section is only populated, if the log option “Changes to system objects” was configured for the target connection. It contains the number of schema method calls for each schema type.

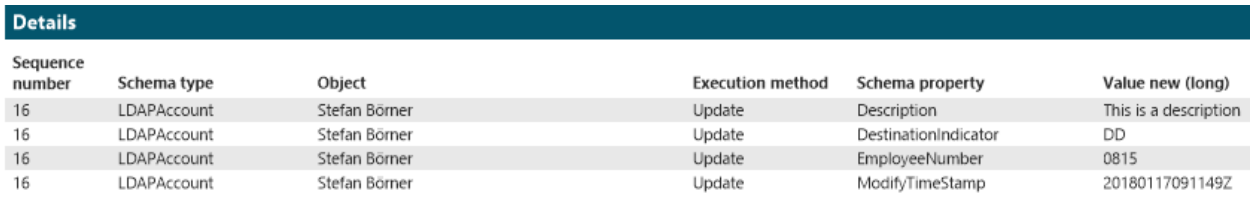
**Processed objects**

| Schema type | Execution method | Count |
|---|---|---|
| LDAPAccount | Update | 1 |

*Figure 121 Processed objects log*

**Details**

This section is only populated, if the log option "Modified properties and values" was configured for the target connection. It contains the actual property changes along with the corresponding schema method call for each object handled during the synchronization/provisioning execution.



**Details**

| Sequence number | Schema type | Object | Execution method | Schema property | Value new (long) |
|---|---|---|---|---|---|
| 16 | LDAPAccount | Stefan Börner | Update | Description | This is a description |
| 16 | LDAPAccount | Stefan Börner | Update | DestinationIndicator | DD |
| 16 | LDAPAccount | Stefan Börner | Update | EmployeeNumber | 0815 |
| 16 | LDAPAccount | Stefan Börner | Update | ModifyTimeStamp | 20180117091149Z |

*Figure 122 Details log*

# Extended logging

Sometimes the capabilities of the synchronization engines public available logging are not enough to locate a certain issue. In this case, you can enable debug logging or traces using the globallog.config that is located in the installation directory of OneIM. The globallog.config file only sets the settings for the current machine. If you want the log to be enabled for a process that is running on a OneIM jobserver, you need to modify the file there.

The globallog.config is a configuration file for Nlog (http://nlog-project.org/) in XML format. To enable debug and or trace for all OneIM components (including the synchronization engine), open the file with a text editor.

The default log target is a logfile that can be written for each application. The log will be written to

```
<user>\AppData\Local\One Identity\One Identity Manager\<application>
```

The most common applications to look for are

| | |
|---|---|
| SynchronizationEditor | To debug issues that occur inside the Synchronization Editor application i.e. when browsing the target system |
| StdioProcessor | To debug issues that occur on a job server during a scheduled synchronization or a provisioning job |

Log messages have a certain level that is specified by the developer of the application that raises the messages. Those can be:

| Level | Meaning/Use |
|---|---|
| Trace | Very detailed usually produces huge amount of log data. |
| Debug | Significantly less detailed as trace but still not recommended to for permanent use. |
| Info | "High" level information messages. |
| Warning | Messages that do not cause the system to fail but can be an indicator of a potential problem |
| Error | Errors that were thrown by a software component. |
| Fatal | Serious errors with a huge impact on the application. |

The default log file contains all messages with level Info (or above). However that level can be adjusted by changing the line

```
<logger name="*" minlevel="Info" writeTo="logfile"/>
```

to `<logger name="*" minlevel="Trace" writeTo="logfile"/>`
or `<logger name="*" minlevel="Info" writeTo="logfile"/>`

in the globallog.config file.

Caution: Changing the default log level setting applies to all OneIM applications and can produce very large files as well as have impact on the overall performance. Changing the log level should only be done temporarily.

When running processes on the OneIM processing server, you can also limit trace logging to the stdioprocessor.exe process that executes the synchronization processes. To do so, add a rules section to the nlog section of the stdioprocessor.exe.config file as follows:

```
<nlog autoReload="true" …>
  <variable name="appName" value="ProjectorBrowser"/>
  <include file="${basedir}/globallog.config" ignoreErrors="true"/>
    <rules>
          <logger name="*" minlevel="Trace" writeTo="logfile"/>
    </rules>
</nlog>
```

For more information on NLog and other log targets, consult the Nlog documentation available here under https://github.com/nlog/nlog/wiki.

## (Component-)Debug Mode of the OneIM processing service

If processes are executed by an OneIM processing server, you can configure the server to log some additional information about the process steps it executes. The log information is written using the NLog framework but the messages that are captured are covered by the preconfigured logging rules since they are logged with severity “Info”.

While the “Debug mode” settings causes the service to log common processing information such as process parameters and detailed results, the “Component Debug Mode” is just passed to the implementation of the OneIM processing information. The component implementation then decides whether additional information needs to be logged or not.

The component debug mode is only used by components developed in earlier versions of OneIM (6.x and prior) and can be considered obsolete.

# Projector Component Concepts

The OneIM projector component is the software layer that links the synchronization framework with the OneIM process orchestration engine. Besides the functionality to start synchronization workflows that were configured in the Synchronization Editor, it also allows ad hoc CRUD operations on single objects.

# Projector Component Process Tasks

This section will cover the tasks of the "ProjectorComponent". Tasks that are not mentioned here are for internal use and should not be required to be run in a custom process.

You may review these in Designer->Proces Orchestration->Process Components->ProjectorComponent

## AdhocProjection

This task executes a workflow for a given object. It is used in processes that are listening to the insert/update/delete events of the target system tables in OneIM.

This task is separated into 4 sub tasks. AdHocProjection, AdHocProjectionSingle, AdHocProjectionX86 and AdHochProjectionSingleX86. They differ in the runtime environment as well in the maximum number of parallel processes that the processing engine is allowed to process.

The runtime environment can be forced to 32 bit mode. Usually that runtime switch is done by the engine transparently depending on the information returned by the connector. However, this produces a lot of overhead (WCF communication) that is not required when dealing with small amounts of data such as during an ad hoc provisioning. In those scenarios you can use the X86 version of the AdHocProjection task.

| | **Max. parallel processes** | **Force 32 bit runtime** |
|---|---|---|
| AdHocProjection | 5 | No |
| AdHocProjectionX86 | 5 | Yes |
| AdHocProjectionSingle | 1 | No |
| AdHocProjectionSingleX86 | 1 | Yes |

The parameters of the task are as follows.

| **Parameter** | **Mandatory** | **Description** |
|---|---|---|
| AuthenticationString | Yes | The OneIM authentication the component uses to perform database operations |
| ConnectionProvider | Yes | The OneIM connection provider a.k.a. database system implementation to use |

| | | |
|---|---|---|
| ConnectionString | Yes | The connection string of the OneIM database |
| CausingEntityPatch | Yes | The OneIM entity that was modified. Usually constructed by the "DPR_WrapObjectForProjection" script |
| UID_DPRProjectionConfiguration | Yes | The OneIM UID of the workflow to run |
| UID_DPRSystemVariableSet | Yes | The OneIM UID of the variableset of the synchronization project to use |
| ForceSyncOf | False | Contains a list of properties that the engine shall transfer in addition to the changes contained in the patch. This is especially useful when a source property of a mapping is determined by a property value of a referenced object. (object reference property) |
| MemberShipActionPartitionSize | False | The size of the partition that shall be used when writing multi-valued references in merge mode. |
| OverrideVariables | False | Allows to override variables of the synchronization project for the time of the ad hoc process execution. The variables need to be specified in connectionstring form. i.e. "variable1=value1; variable=value2" |

Output parameters

| Parameter | Description |
|---|---|
| TargetSystemConnectError | If an error occurred during the connect phase of the processing, this error will be returned on this out parameter. |
| UID_DPRJournal | The UID of the log (DPRJournal) associated to the provisioning process. |

## FullProjection

The FullProjection task executes a full synchronization workflow on an OneIM processing server. It is usually not required to create your own workflows since there is a default workflow for all start up configurations (DPRProjectionStartInfo) available out of the box which can be executed either by starting the workflow in the Synchronization Editor or based on a schedule. The parameters of this task are:

| Parameter | Mandatory | Description |
|---|---|---|
| AuthenticationString | Yes | The OneIM authentication the component uses to perform database operations |

| | | |
|---|---|---|
| ConnectionProvider | Yes | The OneIM connection provider a.k.a. database system implementation to use |
| ConnectionString | Yes | The connection string of the OneIM database |
| UID_DPRProjectionStartInfo | Yes | The OneIM UID of the start-up configuration to execute |
| OverrideVariables | No | Allows to override variables of the synchronization project for the time of the ad hoc process execution. The variables need to be specified in connectionstring form. i.e. "variable1=value1; variable=value2" |
| TargetSystemConnectError | No | If an error occurred during the connect phase of the processing, this error will be returned on this out parameter. |

Output parameters

| Parameter | Description |
|---|---|
| TargetSystemConnectError | If an error occurred during the connect phase of the processing, this error will be returned on this out parameter. |

## Check Condition

This task allows you to check, if a property based condition is fulfilled or not. This can be used in scenarios where you need to wait for a certain property to get modified by another software component. The parameters are as follows:

| Parameter | Mandatory | Description |
|---|---|---|
| AuthenticationString | Yes | The OneIM authentication the component uses to perform database operations |
| ConnectionProvider | Yes | The OneIM connection provider a.k.a. database system implementation to use |
| ConnectionString | Yes | The connection string of the OneIM database |
| CausingEntityPatch | Yes | The OneIM entity to be used for checking the condition. The engine will try to load the matching target system object. |
| UID_DPRProjectionConfiguration | Yes | A workflow of the synchronization project to use. The workflow itself is not required during the execution but since DPR_GetAdHocData returns a workflow instead of the synchronization project, specifying a workflow is more convenient. |

| | | |
|---|---|---|
| UID_DPRSystemVariableSet | Yes | The OneIM UID of the variableset of the synchronization project to use |
| ConditionOptions | No | This parameter can be used to switch to case insensitive comparison when evaluating the condition. If case-insensitive evaluation shall be enabled set the value to: `"CompareCaseInsensitive=true"` otherwise `"CompareCaseInsensitive=false"` |
| FailIfDbObjectMissing | No | If the OneIM object is not available anymore during process step runtime, the process step will only use the values of supplied in the CausingEntityPatch for condition evaluation. Property values that are not contained in the patch will be treated as empty. |

The condition syntax is the same as for mapping conditions. Usable operands are all properties of the target system schema type accessed by their name.

Example Jobparameter:

```
Value = "CN= 'jon doe' AND (employeeType like 'Contractor' Or employeeType like
'Intern')"
```

Hint: When using fixed values that contain a ' character, make sure you escape the value by doubling it. You may also use the OneIM script DPR_FormatConditionValue as follows:

```
Value = "CN= " + DPR_FormatConditionValue("jon doe") + " AND (employeeType like
'Contractor' Or employeeType like 'Intern')"
```

## Check Properties

The check properties task was developed for compatibility reasons to version 6. It checks if the properties of the specified OneIM object and the target system are actually equal by running an ad hoc provisioning simulation with a given workflow and checking for potential modifications. The task fails, if differences were found. The properties that were identified as different are part of the errormessage.

The parameters are:

| Parameter | Mandatory | Description |
|---|---|---|
| AuthenticationString | Yes | The OneIM authentication the component uses to perform database operations |
| ConnectionProvider | Yes | The OneIM connection provider a.k.a. database system implementation to use |
| ConnectionString | Yes | The connection string of the OneIM database |
| CausingEntityPatch | Yes | The OneIM entity to be checked |
| UID_DPRProjectionConfiguration | Yes | A workflow of the synchronization project to use. |
| UID_DPRSystemVariableSet | Yes | The OneIM UID of the variableset of the synchronization project to use |
| OverrideVariables | No | Allows to override variables of the synchronization project for the time of the ad hoc process execution. The variables need to be specified in connectionstring form. i.e. "variable1=value1; variable=value2" |

## ObjectExists

This tasks allows you to check if a given OneIM object can be matched with an object from the target system according to the given workflow. The parameters are

| Parameter | Mandatory | Description |
|---|---|---|
| AuthenticationString | Yes | The OneIM authentication the component uses to perform database operations |
| ConnectionProvider | Yes | The OneIM connection provider a.k.a. database system implementation to use |
| ConnectionString | Yes | The connection string of the OneIM database |
| CausingEntityPatch | Yes | The OneIM entity to be searched for |
| UID_DPRProjectionConfiguration | Yes | A workflow of the synchronization project to use. |
| UID_DPRSystemVariableSet | Yes | The OneIM UID of the variableset of the synchronization project to use |
| OverrideVariables | No | Allows to override variables of the synchronization project for the time of the ad hoc process execution. The variables need to be specified in connectionstring form. i.e. "variable1=value1; variable=value2" |

## MaintainDatastore

This task allows you to run a full datastore maintenance. Usually the datastore maintenance is handled transparently by the synchronization workflow. However, you can disable datastore maintenance and create a dedicated process for this.

| Parameter | Mandatory | Description |
|---|---|---|
| AuthenticationString | Yes | The OneIM authentication the component uses to perform database operations |
| ConnectionProvider | Yes | The OneIM connection provider a.k.a. database system implementation to use |
| ConnectionString | Yes | The connection string of the OneIM database |
| UID_DPRProjectionStartInfo | Yes | The start configuration for which the datastore maintenance shall be executed. |
| RetryCycles | Yes | Retry cycles per datastore entry |

# Provisioning process operations

You may review these in Designer →"Proces Orchestration" → "Provisioning process operations". A provisioning process operation or "DPRObjectOperation" links the workflow of a synchronization project to table in the OneIM database. The target system connection of the synchronization project must also be specified. This information is used to find the corresponding base object in the OneIM database. Furthermore the link needs to have a name to reference it in the OneIM processes. Out of the box synchronization projects usually create 3 links (Insert/Update/Delete) that point to the same workflow to allow a flexible reconfiguration in case you want to define a different workflow for those operations.

Those links can be configured in the Designer application when navigating to "Process orchestration" → "Provisioning process orchestration". A example configuration can be found in the section "configure ad hoc provisioning" of the LDAP walkthrough.

# Setting up (custom) target system tables

In order to process data changes originated in OneIM through the OneIM processing engine, there are several configuration settings available that allow the configuration of different scenarios. Common settings can be configured independent of the fact if a table is used in target system synchronization or not. In addition to that, there are settings that are specific for tables that are involved in target system synchronization. The following sections will go into the details of the settings, where to find them.

## Common settings

Common settings can be configured in the "One Identity Manager Schema" section of the Designer application. The following settings are not all customizable but are required to understand the functionality of the OneIM process generation. The following screenshot shows where to configure the settings of the table itself (3), its columns (4) and the settings of its relations (5).

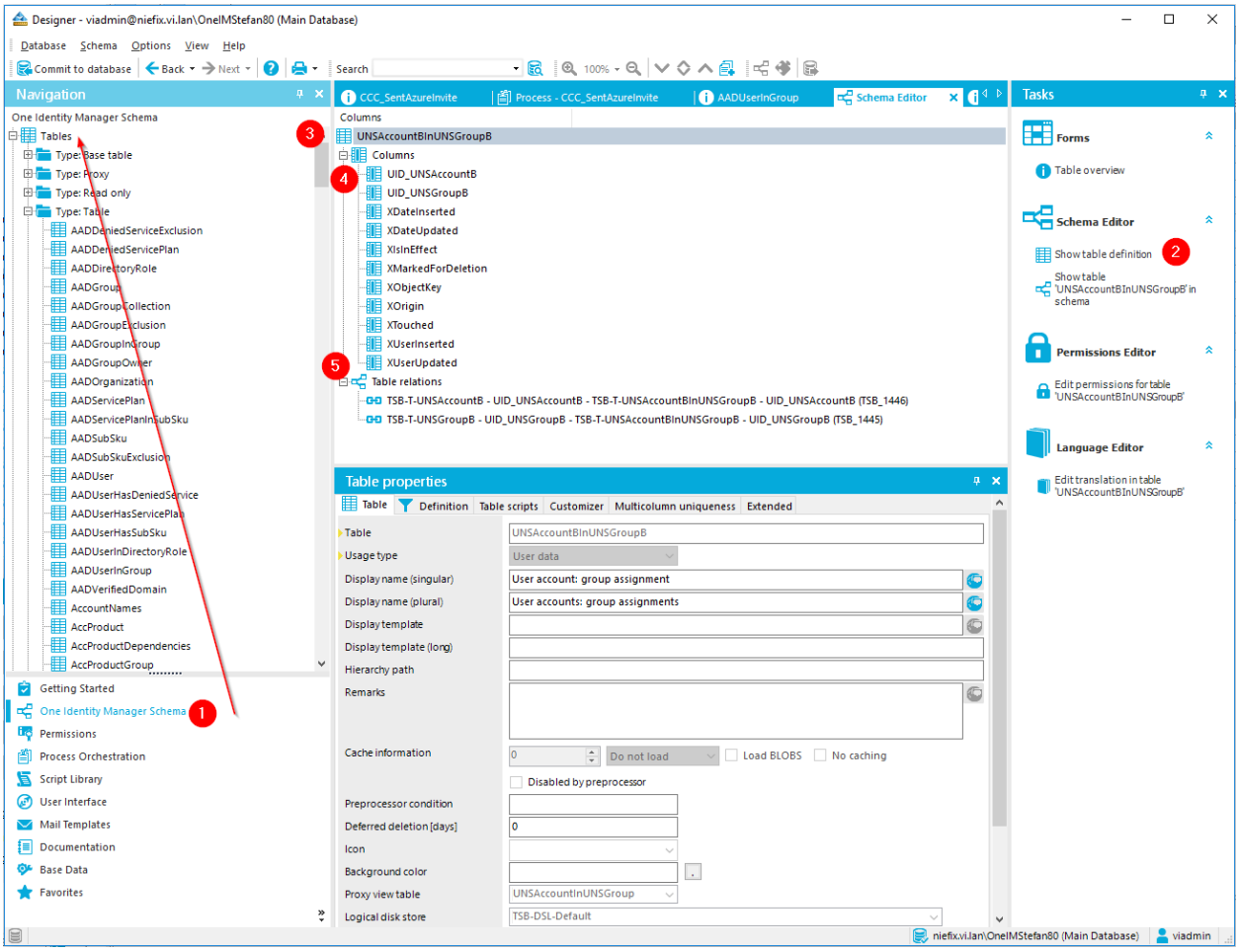



*Figure 123 OneIM Database Schema Configuration*

## Assign by event

This setting can be configured for all OneIM M:N table. It causes that whenever a record becomes active or gets disabled an "Assign" or "Revoke" event is generated. If M:N record is active or not, can be determined by looking at its "XIsInEffect" flag. If it is set to true, the relation is to be considered as existing. If it is false, the record exists but the relation is to be considered as non-existent. XIsInEffect can for example be set to false, when the Person (and with it, its accounts) get temporarily deactivated. When the Person gets enabled again,

XIsInEffect is set to true again. XIsInEffect of new M:N records is usually set to true instantly.

This setting is relevant for synchronization scenarios, where the M:N table (LDAPAccountInLDAPGroup) is directly mapped to corresponding M:N schema type in the target system (DBUserGroupRelations) as illustrated below.

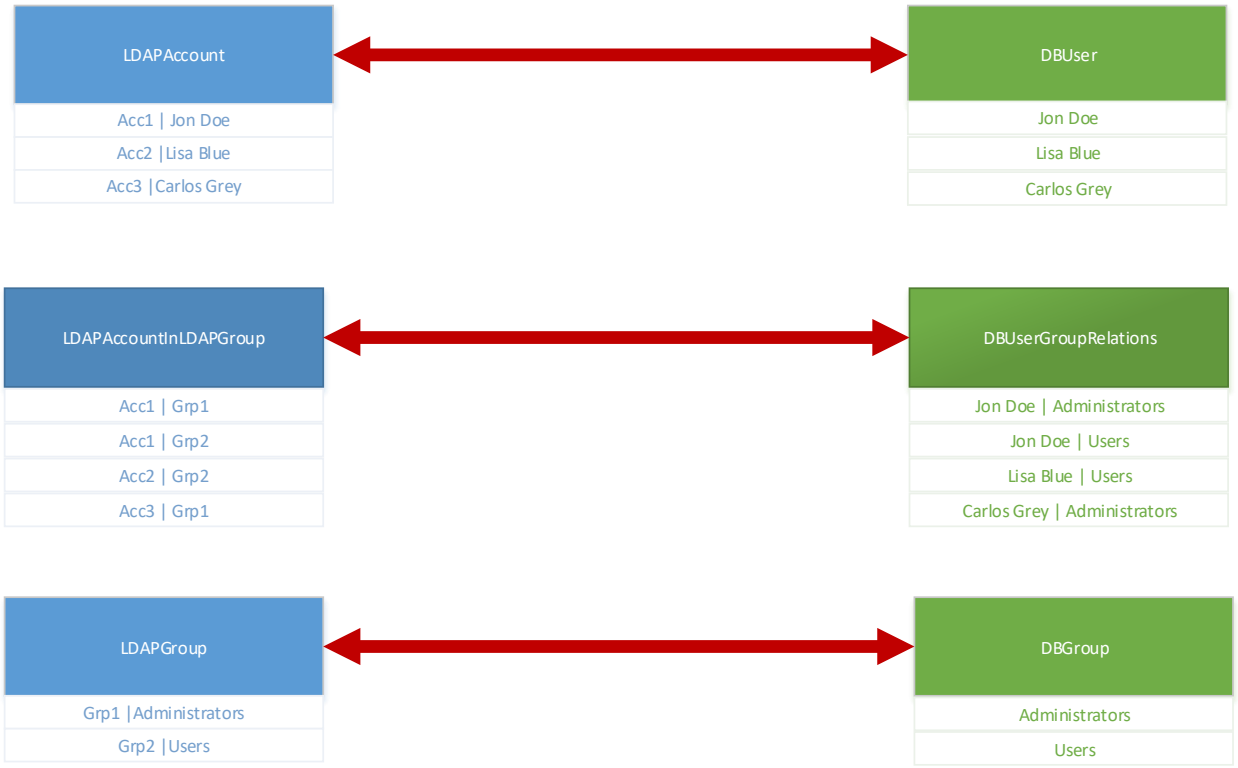



*Figure 124 Direct M:N mapping*

It is relevant because in this scenario because OneIM processes need to be configured and generated for changes of those M:N records and to do so, events are required.

This setting can be configured in the table properties of the M:N table in the "Designer" application.

## Update dependencies modification date (XDateSubItem)

XDateSubItem is the name of a column that is available for some of the OneIM tables that represent group-style objects that can have other objects as members. Those memberships are stored in M:N tables that point to the group, and to the member. Those two relations are displayed in the Designer. The relation pointing to the "group" object has a setting named "Update dependencies modification date" that indicates that whenever a change to a M:N record happens, the value of the column XDateSubItem of the group object needs to be updated. This update then causes the "update" process of the group object to start which

will ultimately publish the changed membership to the target system. This setting is not custom alterable.

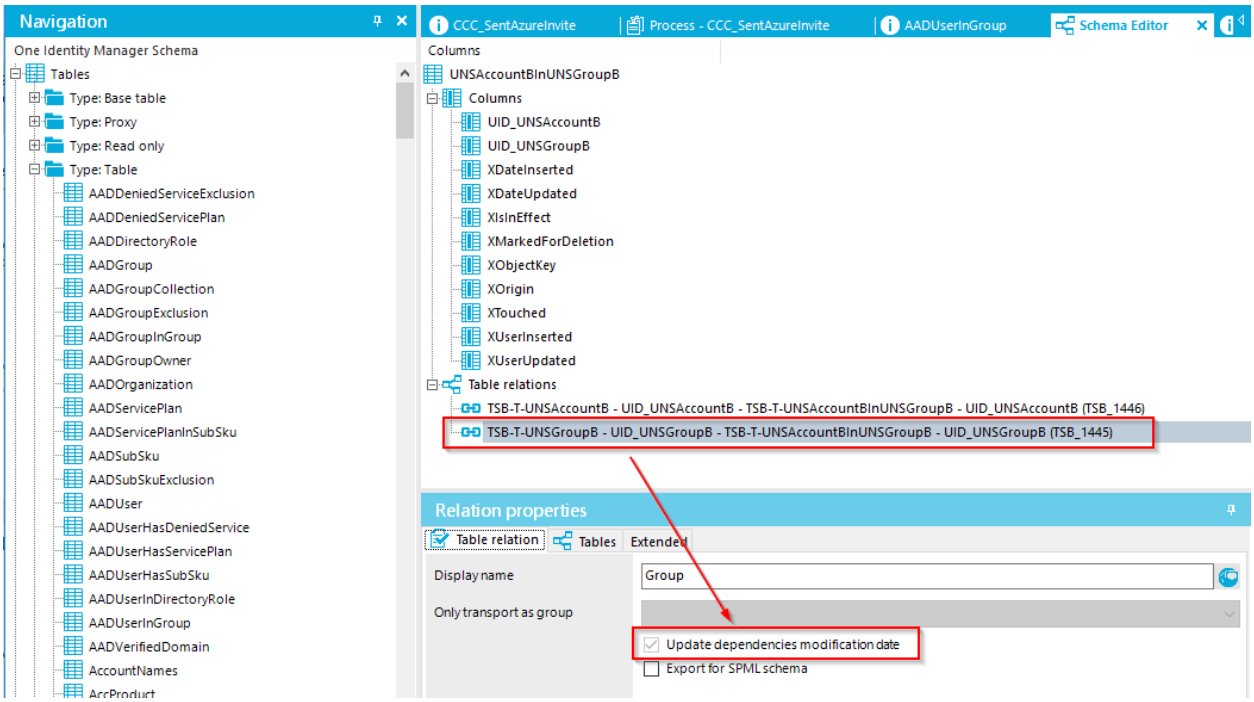



*Figure 125 XDateSubItem setting*

A M:N record usually becomes effective, when a new record is inserted. OneIM not necessarily deletes a record from an M:N table to remove a membership. Instead the

## Target system specific settings

In addition to the configuration capabilities of described in the "Common settings" section, there are features specific for OneIM processes that are dealing with target system synchronization like the "target system synchronization" nodes in the Manager navigation. First of all OneIM needs to know which types of target systems are available. The types of available target systems are stored in the database table "DPRNameSpace". Each target system module like AD or LDAP provide such a record out of the box. If you want to create an own target system type for a custom target system, you can do that in the

To configure a new custom target system type follow the instructions below.

Custom target system type definitions can be created in the "Custom target systems" (1) section of the Manager application. From there navigate to "Basic configuration data" (2) → "Target system types". (3)

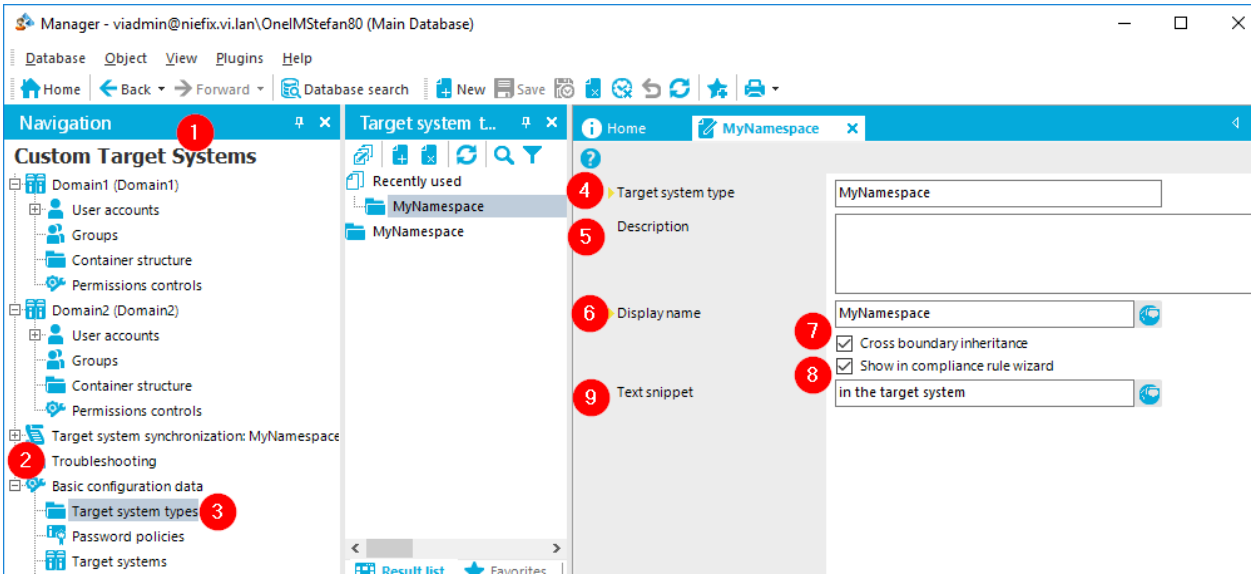



*Figure 126 Create custom target system*

When creating a new custom system type specify an identifier (4) and a display name (6). A description (5) is optional. The other settings have the following meaning:

**Cross boundary inheritance**

If this setting is enabled, OneIM allows creating cross system memberships for target system instances of this type. If in the above scenario two domains (D1 and D2) are available of target system type "MyNamespace", this setting would allow the creation of a membership between a group in D1 and a User in D2.

**Show in compliance rule wizard**

If this setting is enabled, the target system will be available in the compliance rule wizard of OneIM.

**Text snippet**

This text snippet is shown in the filter designer wizard when selecting items from the system i.e. when creating a dynamic role.

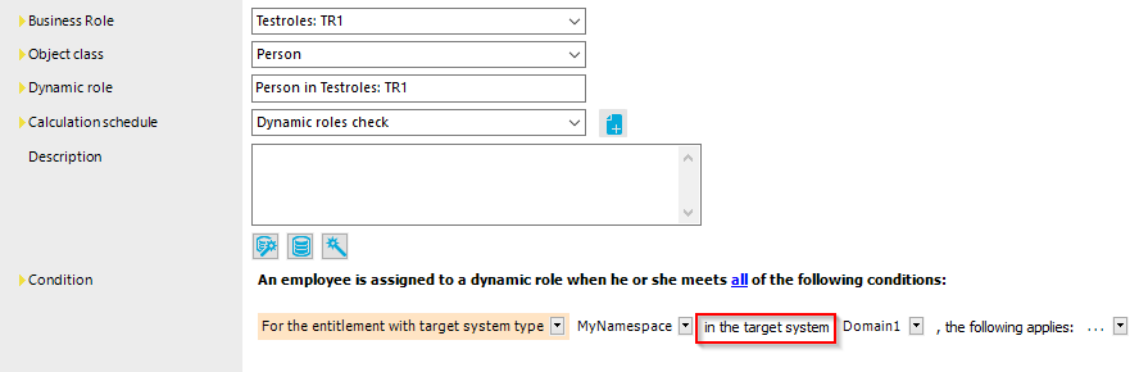



*Figure 127 Text snippet location*

Once the target system type is configured you need to assign the tables that belong to the system. To do so, start the "Assign synchronization tables" task for the custom target system type and assign the OneIM tables that are used by the system type to store data.

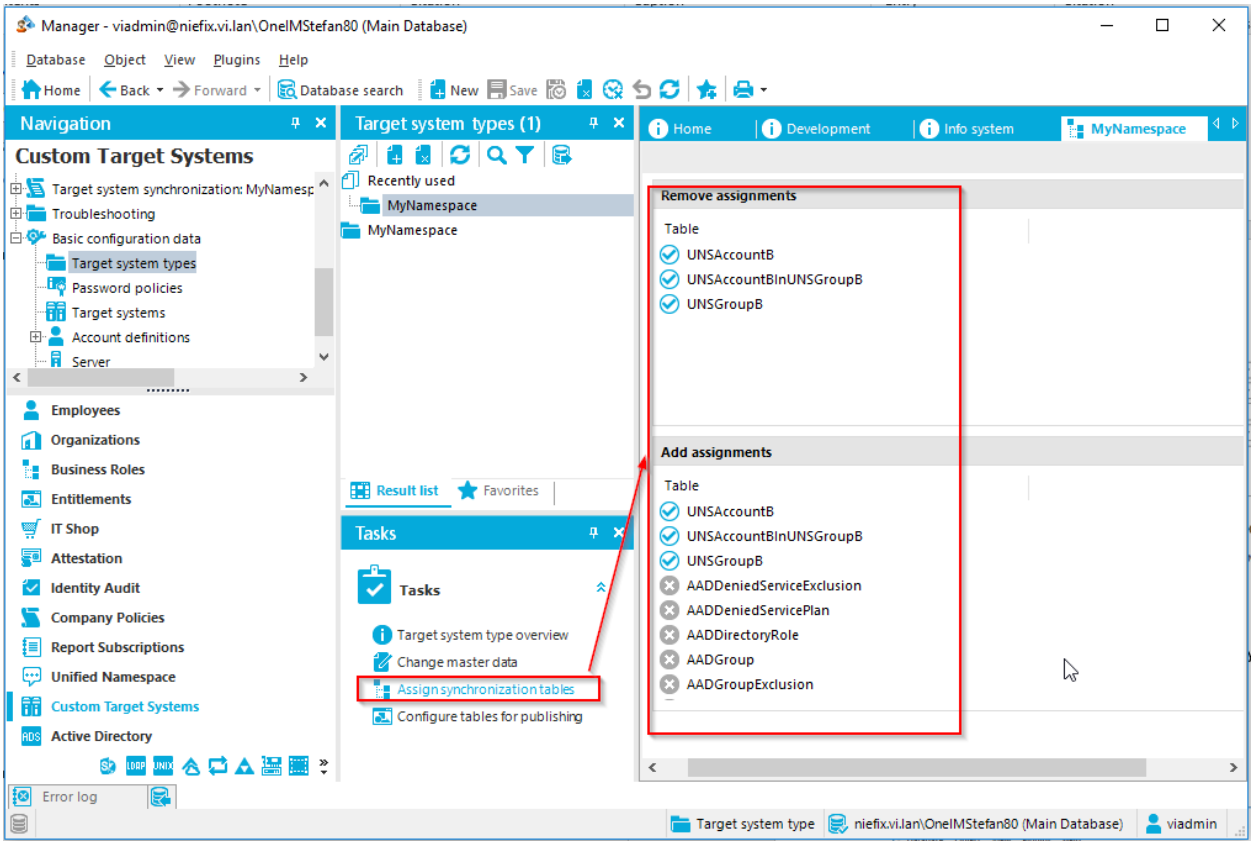



*Figure 128 custom target system type table assignment*

Once the tables are assigned, several settings can be configured for them. To do so, select the "Configure tables for publishing" task for the custom target system type.

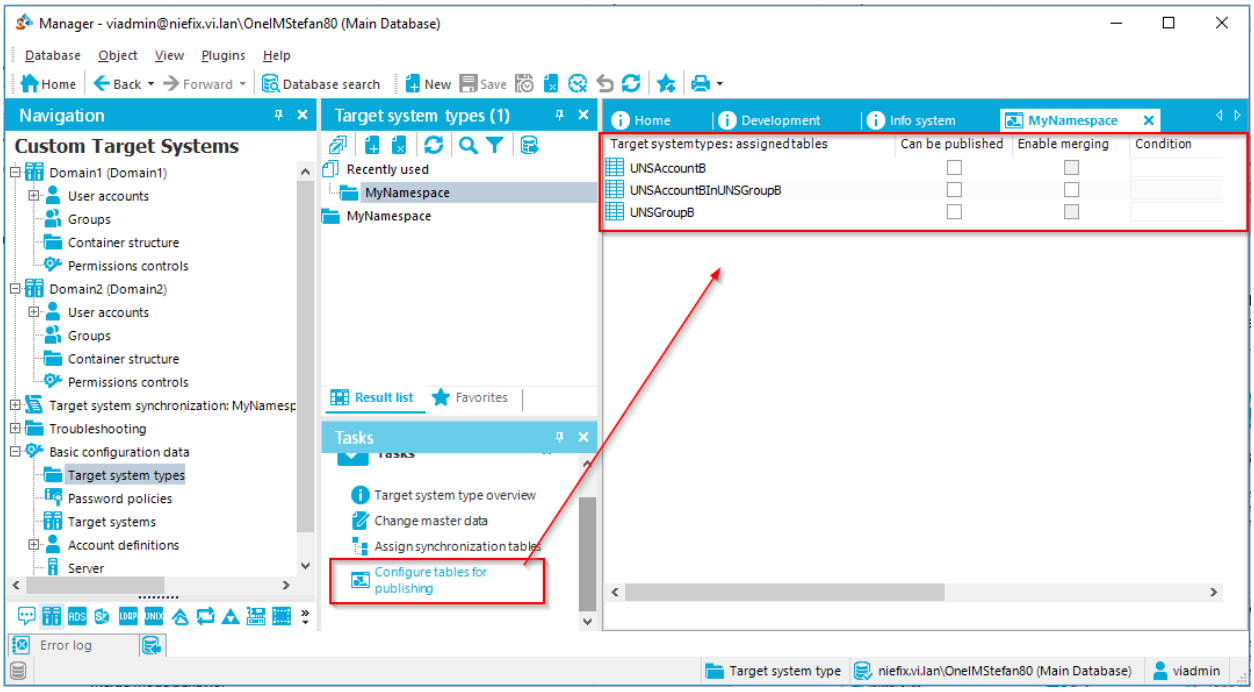



*Figure 129 configure tables for publishing*

The following settings are available:

## Can be Published

The "Can be published" setting enables the "publish" option in the target system for the selected OneIM table.

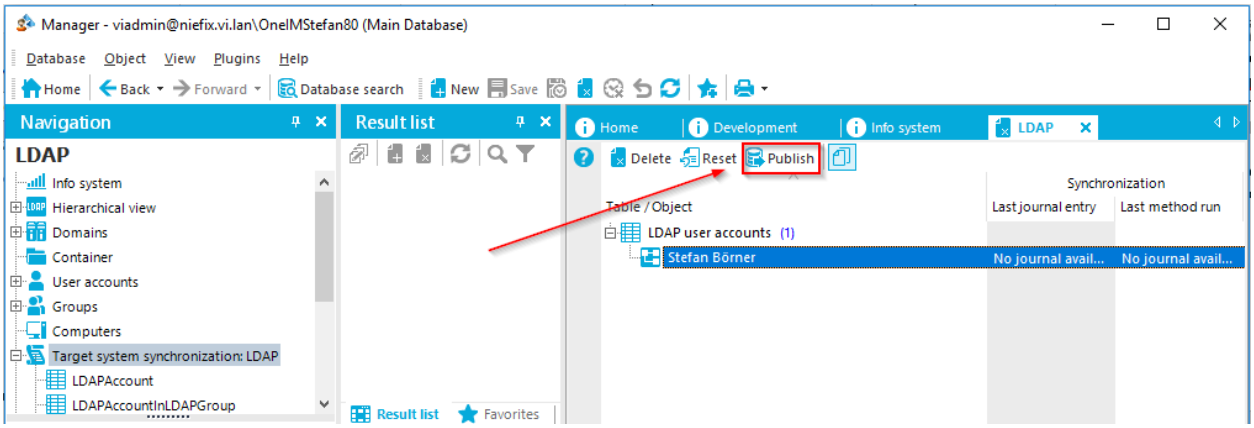



*Figure 130 Publish option*

## Enable Merging (Merge Mode)

The merge mode is a feature that allows the ad hoc projection to handle multi-valued references like group memberships more granularly. Without merge mode, the data

situation in OneIM will be published to the target system as described in the "multi reference mapping rule" section. However, in many environments a parallel manual administration of the target system is in place. If an ad hoc provisioning job runs before a manual administration was synchronized to OneIM, the manual change would be reverted. To prevent this behavior, you can enable the "merge mode" for M:N tables in OneIM.

Hint: For out of the box target system modules (e.g. AD, LDAP) this setting is preconfigured. The default is that merge mode is enabled to not destroy manual changes in the target system.

By doing so the OneIM behavior is different as follows:

**Default Behavior**
A membership change in OneIM is either an insert or a delete operation in a M:N table. By default, this triggers a database internal process which increases the XDateSubItem value of the "hosting" object (i.e. the group). This increase is done via an update using the object layer which triggers the OneIM processing engine and ultimately an ad hoc provisioning process. Any multi reference mapping rule will process the data as described in the corresponding section which will overwrite all manual changes in the target system after the last synchronization.

**Merge mode behavior**
When merge mode is active in addition to the insert or update to the M:N table, OneIM stores the "change" in a dedicated table called DPRMemberShipAction. A change of XDateUpdated and therefore a provisioning process is generated too. When merge mode is enabled, the ad hoc projection task directly sends "add" and "remove" modifications to the connector. By only executing the "add" and "remove" modifications of DPRMembersShipAction, any manual change done in the target system remains "as is". Once the ad hoc projection task is finished it clears the DPRMemberShipAction entries for the handled group object.

Hint: Group objects that still have unprocessed operations in DPRMemberShipAction will not be updated during full synchronizations since this would revert the unprocessed membership change in OneIM. Those group objects are mentioned in the synchronization log.

Hint: The majority of target systems is preconfigured to use the merge mode by default. Older versions 7.1.x did not enable the feature by default. After upgrading, you need to enable it manually

In case there are multiple operations for one group object, the maximum number of operations sent to the connector at a time is limited by the "MemberShipActionPartitionSize" parameter of the ad hoc provisioning task. If there are more than this number of changes to process, the connector will be called multiple times. If an error occurs during processing the partition size is decreased and the operations are retried (down to a partition size of 1). "Add" and "Remove" items that the connector was not able to publish will remain in the DPRMemberShipAction table. As described this blocks the synchronization of the group object until the error is resolved. To prevent endless blocking a daily cleanup process ("**DPR_MemberShipActions_RemoveOrphanedEntries**") will forcibly remove entries.

## Mark objects as outstanding

See Manager →"Target System (e.g. ADS, LDAP)" → "Target system synchronization", as shown in the screen shot below.

Mark objects as "outstanding" is a schema method or feature of the OneIM components and always related to OneIM records. It allows you to tag objects as "missing" during a synchronization instead of instantly deleting them from the database. This feature is useful in multiple scenarios:

1. It prevents accidental deletion of OneIM objects in case of insufficient matching rule-, scope- or schema class definitions.

2. It allows you to repair (republish) objects that were natively deleted from the target system. A native change may happen by accident and may be against policy. OR, it may be an urgent action by a target system administrator. This feature allows the administrator to gracefully handle all these cases.

The MarkAsOutstanding method is exposed for each OneIM table (schema type) that includes the column **"XMarkedForDeletion".** This is a bitmask (integer) property. The bits have the following meaning:

| **Value (Bit Position)** | **Mask** | **Description** |
|---|---|---|
| 2^0=1 (**1**) | 0000000**1** | Delayed deletion in progress |
| **2^1=2 (2)** | **0000001****0** | **Missing in target system** |
| 2^2=4 (**3**) | 00000**1**00 | Object is locked |
| 2^3=8 (**4**) | 0000**1**000 | Object is being published (currently not in use) |

When executing the "MarkAsOutstanding" schema method, the connector sets the corresponding bit. Those objects can be found in the "**Target system synchronization**" section of the target system in the OneIM Manager application.

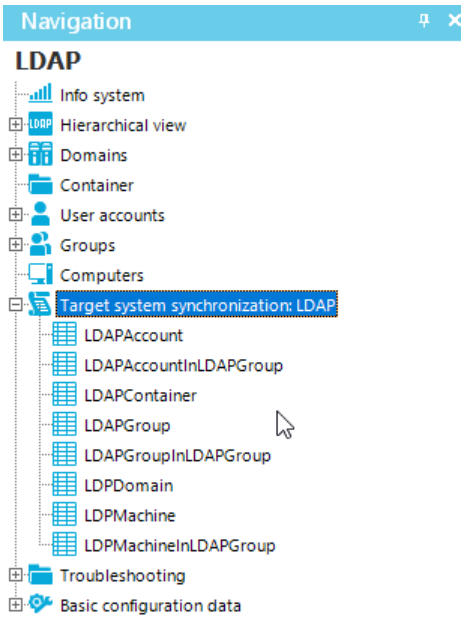



*Figure 131 Target system synchronization*

Hint: Out of the box synchronization project templates usually are configured to mark objects as outstanding instead of deleting them by default. Also note that the behavior for memberships is configured at the "members of m:n schema types" virtual property.

The elements found in this list are those objects that were marked as missing. You have several options as to what to do with them:

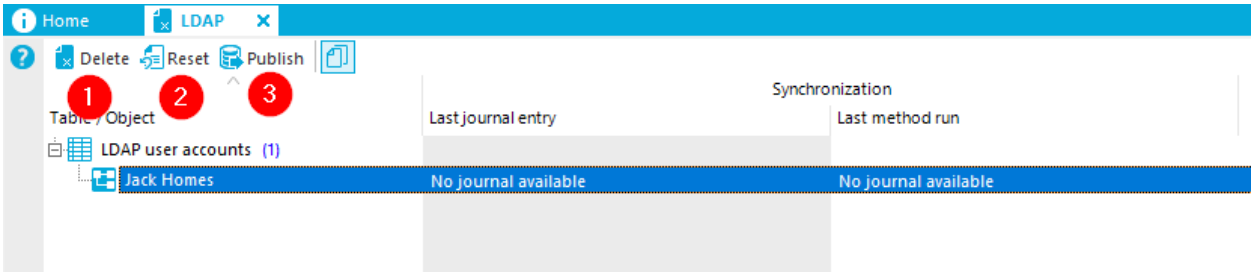



*Figure 132 Options for outstanding objects*

| Option | Description |
|---|---|
| Delete (1) | This option will delete the object from the OneIM database. To do so, the same restrictions apply as for a normal delete. That means you cannot delete objects that are referenced by other objects with a relation set to "delete restrict" etc. |
| Reset (2) | This simply removes the mark as outstanding bit from the object in OneIM. |
| Publish (3) | This option calls a provisioning process to re-publish the object to the target system. |

It is sometimes required to implement dedicated provisioning processes that handle a "publish" (provisioning) executed for outstanding objects. To distinguish between a regular insert provisioning job and a re-publish, a connection variable is set when an object is published. This variable is called "ManageOutStanding". Furthermore, the event that gets fired is not "Insert" but "HandleOutstanding".

The reason why "HandleOutstanding" is slightly different from a regular "Insert" is the volatile data that is not permanently stored in OneIM such as user passwords. When a new user account is created in OneIM, the account table password field is set to the initial password value. Once the provisioning process has successfully created the account in the target system, the password field is emptied in OneIM. Similar behavior is implemented for updating an accounts password. This also means that a new password needs to be generated when re-publishing an account that was deleted from the target system. To achieve this, several of the technologies described in this document are used.

A common used pattern can be found in the out of the box LDAP process "LDP_Account_HandleOutstanding" which is triggerd by the event "HandleOutstanding". It creates a new initial random password. When calling the AdHocProjection task, a synchronization project variable "DefaultUserPassword" is overridden with the generated value. The LDAPAccount schema type has fixed value virtual property "vrtInitialPassword" that contains the value of the DefaultUserPassword variable (and therefore at runtime the new generated initial password). Then, there are two mapping rules that write the password in the target system.


*Figure 133 Password handling during publish*

UserPassword → vrtPassword transfers the content of the LDAPAccount.UserPassword column to the target system if it actually has a value. In case that column is empty (during "publish") the second rule is used that maps the content of the variable and therefore the password that was generated in the prescript.

# Reference

This section contains detailed information about specific implementations of certain synchronization framework items.

## Synchronization projects

By default, synchronization projects are stored in the OneIM database. The "root" table for a project is DPRShell. If you want to have an overview about how the table relations are in detail, you can open the Designer and enable the "Show system data" option in the settings menu. Then navigate to "One Identity Manager Schema" → "System tables" → "Type: Table" → "DPRShell" and click the "Show table 'DPRShell' in schema" task.

However, the synchronization engine has a separate "configuration interface" for storing its data so that is also possible to store this information elsewhere.

Hint: If the Synchronization Editor is launched in debug mode (start the SynchronizationEditor.exe with the –D option), you can also store the configuration items that are completely independent from the rest of the OneIM manager suite, to a *.projshell file.

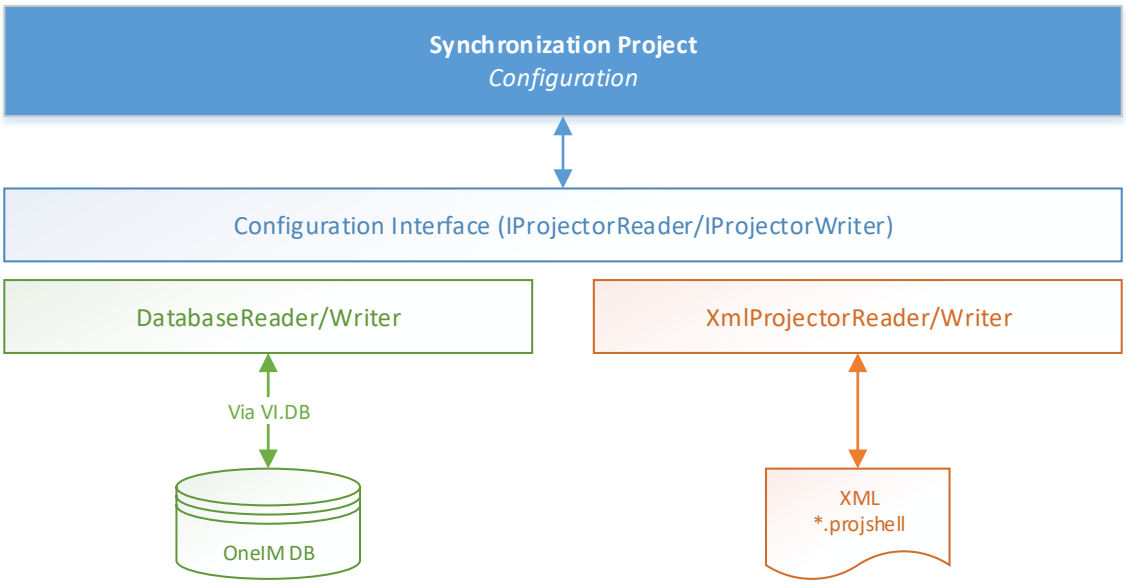



*Figure 134 Synchronization Project storage*

# Virtual property types

This section describes the different types of virtual properties that are currently shipped with the product. Further types of properties might be added in future versions.

## Bit Mask

This property allows you to split an Integer property that represents a bitmask into multiple boolean properties or vice versa. It operates in 2 modes:

**Mode 1**: Expose a virtual boolean property based on a bit position in the bitmask from a source property. If the Boolean needs to be negated, you can use the "Negate value" option.

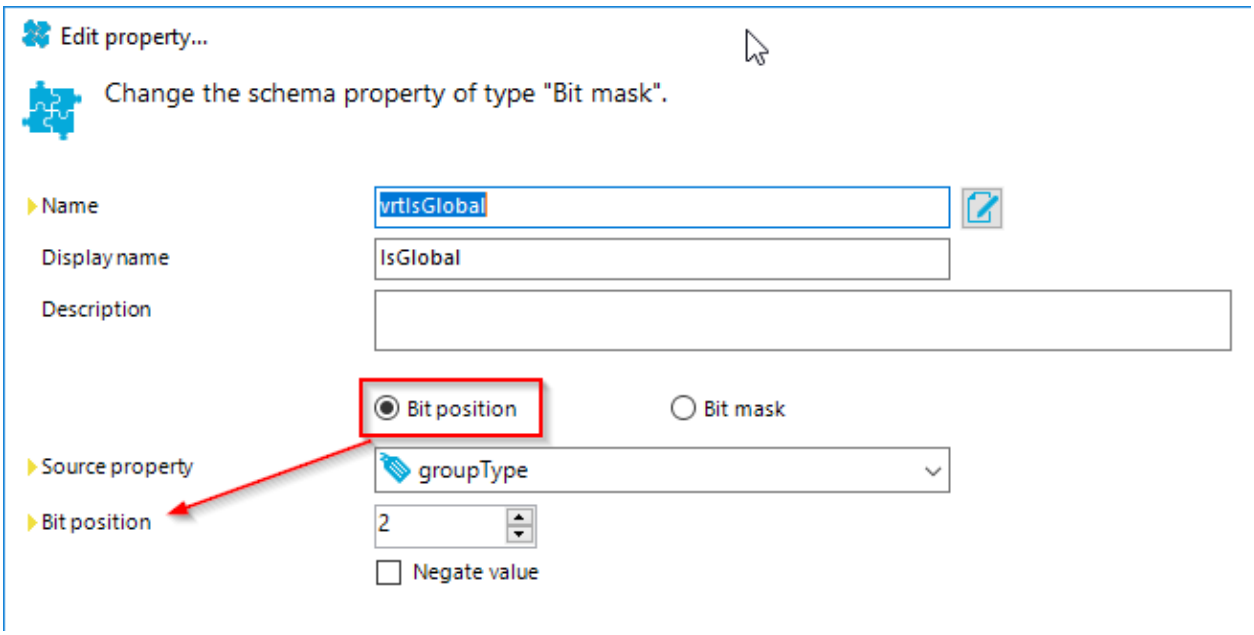



*Figure 135 Bitmask property mode "Bit position"*

**Mode 2:** Combine a set of boolean properties to a bitmask by specifying the bit position of each boolean property. The following rules apply when using this mode:

Hint: The following rules apply when using the "bit mask" mode:

A bit position can be only used once

The virtual property must handle all bit positions of the bit mask to be written, otherwise it will just set undefined bits to 0

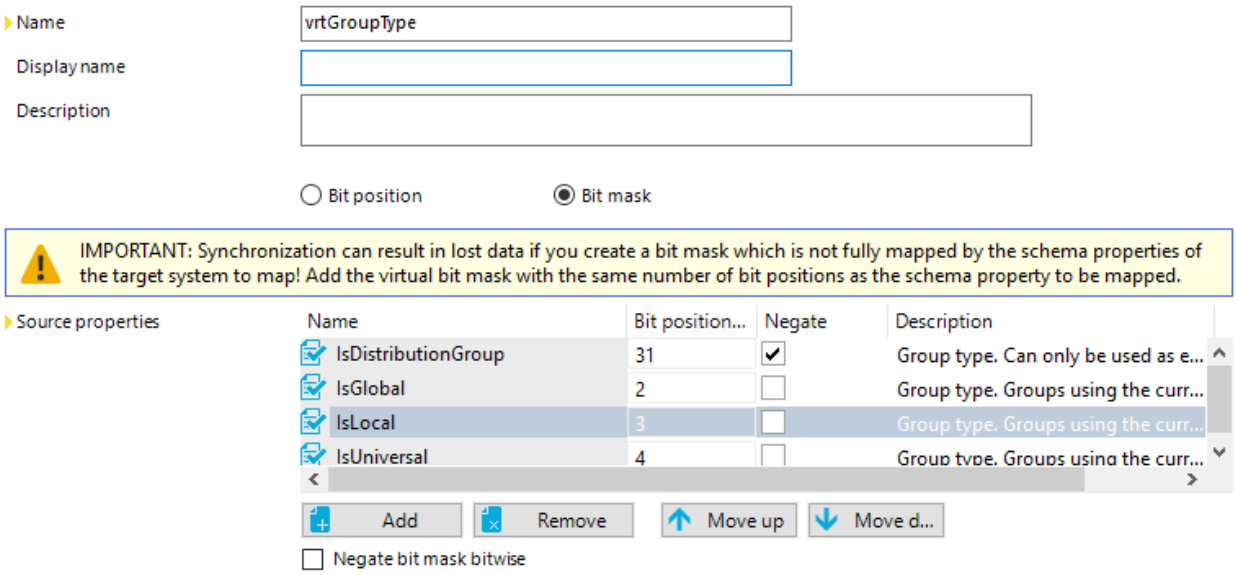



*Figure 136 Bitmask property (mode bit mask)*

**Example:**

Let us look at the groupType attribute from the Active Directory schema[1]. It is a bitmask that can have the following values:

| Value (Bit Position) | Mask | Description |
|---|---|---|
| 2^0=1 (**1**) | 00000000 00000000 00000000 0000000**1** | System Group |
| 2^1=2 (**2**) | 00000000 00000000 00000000 0000001**0** | Domain Global Scope |
| 2^2=4 (**3**) | 00000000 00000000 00000000 0000**1**00 | Domain Local Scope |
| 2^3=8 (**4**) | 00000000 00000000 00000000 0000**1**000 | Universal Scope |
| 2^4=16 (**5**) | 00000000 00000000 00000000 000**1**0000 | APP_BASIC group (for authorization manager) |
| 2^5=32 (**6**) | 00000000 00000000 00000000 00**1**00000 | APP_QUERY group (for authorization manager) |
| 2^31=2147483648 (**32**) | **1**0000000 00000000 00000000 00000000 | Security group if bit is set, if not it is a distribution group |

In OneIM we need to populate the properties IsLocal, IsSecurity, IsDistribution, IsGlobal and IsUniversal of the ADSGroup schema type.

So first thing to think of is, which mode can we use and this determines on which side, the virtual property has to be created.

**Idea A (use mode 1):**

Create virtual vrtIsLocal, vrtIsSecurity, vrtIsDistribution, vrtIsGlobal and vrtIsUniversal bit mask properties on the target system side that are using the native groupType property as base property.

Disadvantage: multiple virtual properties to define

**Idea B (use mode 2):**

Create a virtual vrtGroupType property on OneIM side that combines IsLocal, IsSecurity, IsDistribution, IsGlobal and IsUniversal to one Bit mask. Let us check if we can stick to the "Mode 2" rules here:

*Rule: "A bit position can only be used once"*

[1] https://msdn.microsoft.com/en-us/library/ms675935(v=vs.85).aspx

IsSecurity and IsDistribution share the bit position 32 by negation. Unfortunately the virtual property does not support shared bit positions that only differ in negation. As a workaround, you could handle just one property (IsDistribution) and set the other (IsSecurity) by an OneIM template.

Rule met? → Yes (with some customization)

*Rule: "The virtual property must handle all bit positions of the bit mask to be written"*

We need to set the values for bit positions 1, 2, 3, 4, 5, 6 and 32. Based on the properties that are available in OneIM, we could only set the values for bit positions 2 (IsGlobal), 3 (IsLocal), 4(IsUniversal) and 32(IsDistribution/IsSecurity) meaning any write operation would overwrite the bit positions 1, 5 and 6 with 0.

Rule met? → **No!**

Conclusion: go with idea 1

## Data conversion

The virtual data conversion property allows you to convert data between one basic datatype to another. Depending on the source and target datatype, conversion options such as culture code, time zones etc. are available. The general principle is the selection of a base property that specifies the source data type. Then choose the target data type and specify conversion options that apply to this type.

Examples:

*Convert a date to string with a specific output format*

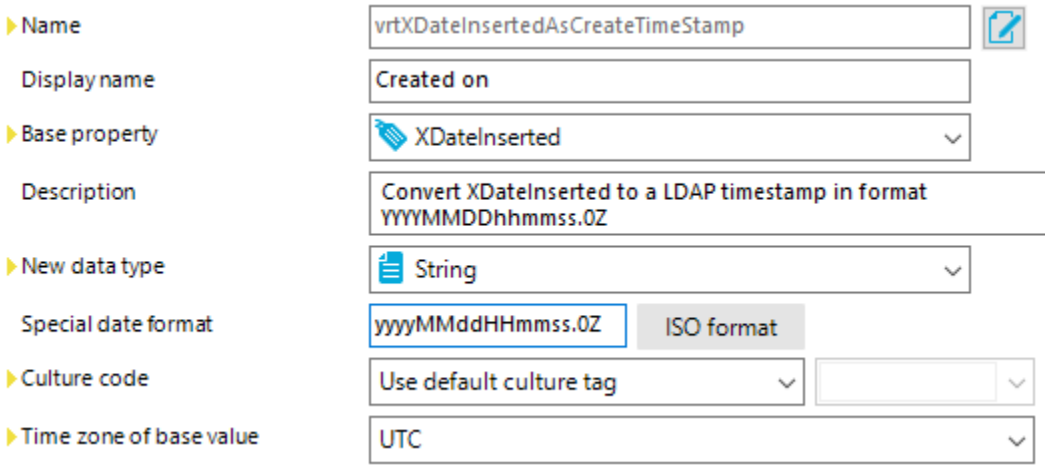



*Figure 137 Data conversion property (date to string)*

*Convert a Boolean property to string and specify custom values for true and false*

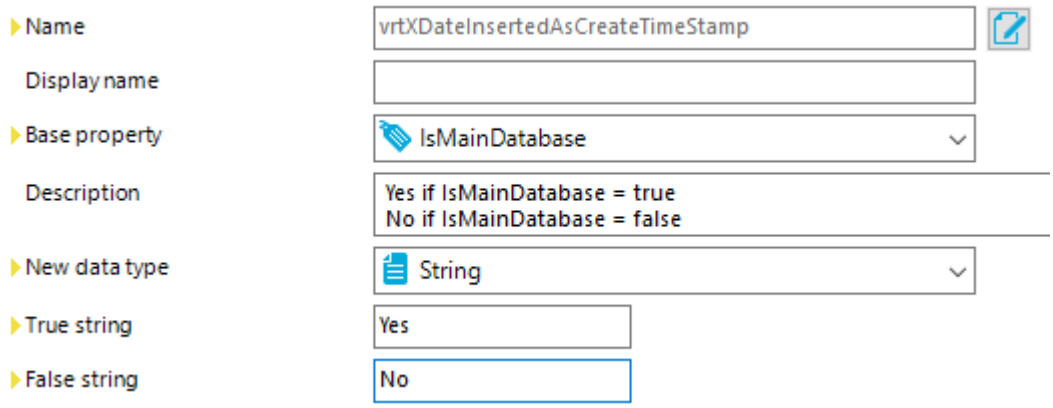



*Figure 138 Data conversion property (boolean to custom string)*

Convert a string to a Boolean value with custom values for true and false

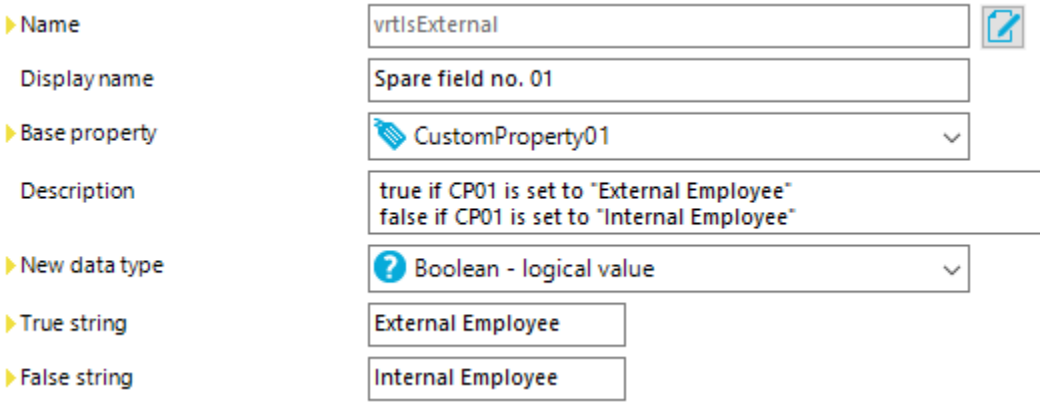



*Figure 139 Data conversion property (string to boolean)*

Convert a string given in a localized form to a date

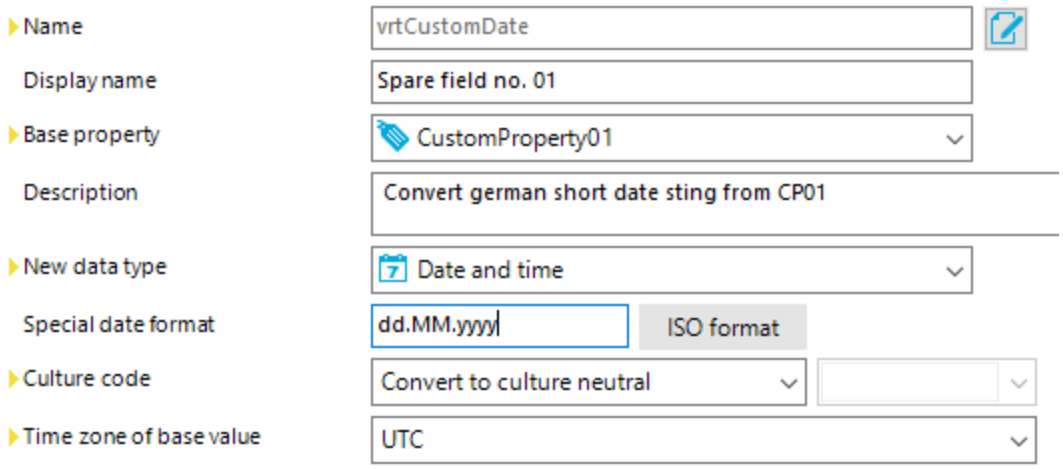



*Figure 140 Data conversion property (string to date)*

## Data mapping

The data mapping allows you to "translate" terms of one system to the terms of the other systems. OneIM for example, has the concept of limited values which is a list of allowed values for a property. In some cases the items or terms on that list differ slightly from the terms that are available in the target system. In those scenarios you can use the data mapping virtual property to map the term of OneIM to the appropriate term of the connected system. You can also specify the behavior for undefined values returned by a system. Those can be either handled as error, transferred as they are without conversion or ignored completely.

Example:

A field in the target system contains the type of address and can have the values "undefined", "home" and "work". In OneIM for CustomProperty01 limited values were defined as follows "personal", "company". The virtual property configuration looks like this:

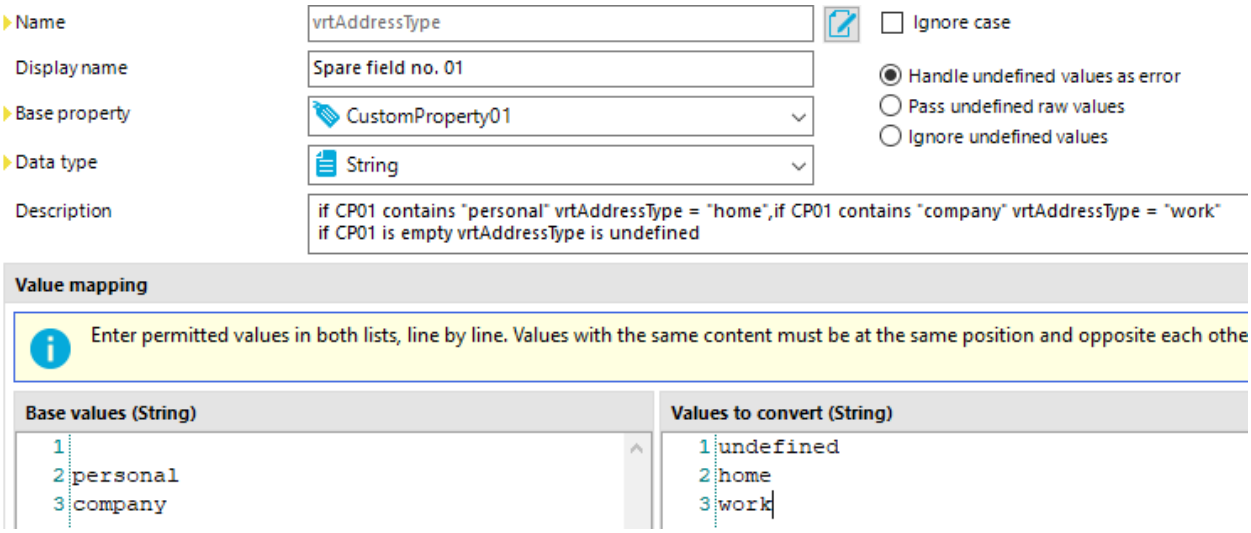



*Figure 141 Data mapping property*

## Fixed value

As the name implies, fixed value properties provide a constant value. The value itself can be determined in different ways. Those ways can be chosen in the "Constant type" list. The property can be marked as "multi-valued". However that is just for compatibility reasons. The resulting property will then be an array containing exactly one element.

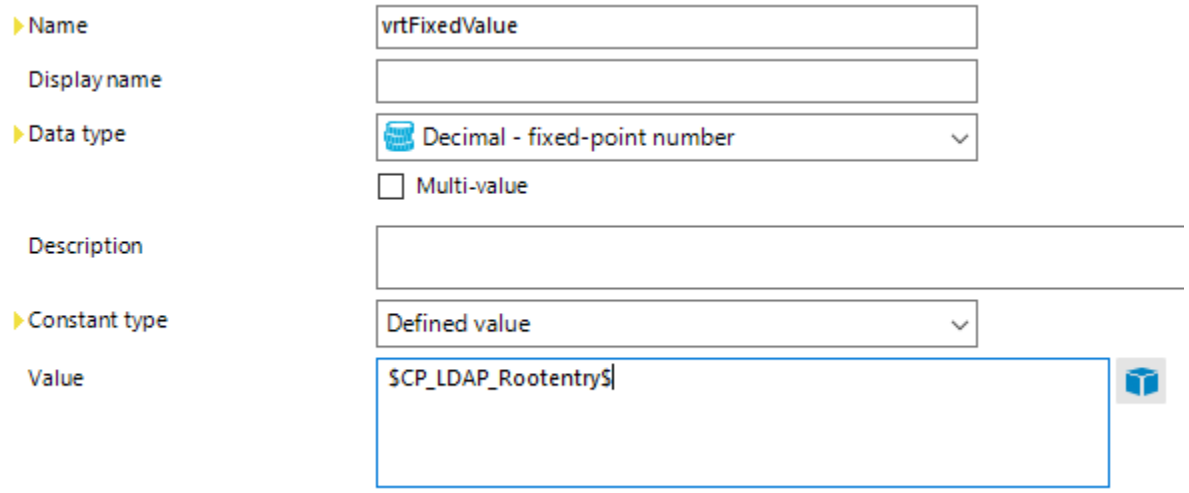



*Figure 142 Fixed value property*

Constant types

| Option | Description |
|---|---|
| Current date and time (UTC) | The date and time the property was read by a workflow |

| | |
|---|---|
| Current date (UTC) | The date the property was read by a workflow |
| Defined Value | A fixed value. You can also use a variable as source. Use the $variable$ notation. |
| Empty Value | Empty value to clear property contents of the mapped property |
| Environment Variable (Pattern) | A system environment variable i.e. %PATH% |
| Random Integer | A random integer |
| UID | A random UID |

## Format defined property

A format defined property allows the creation of combined properties based on a pattern.

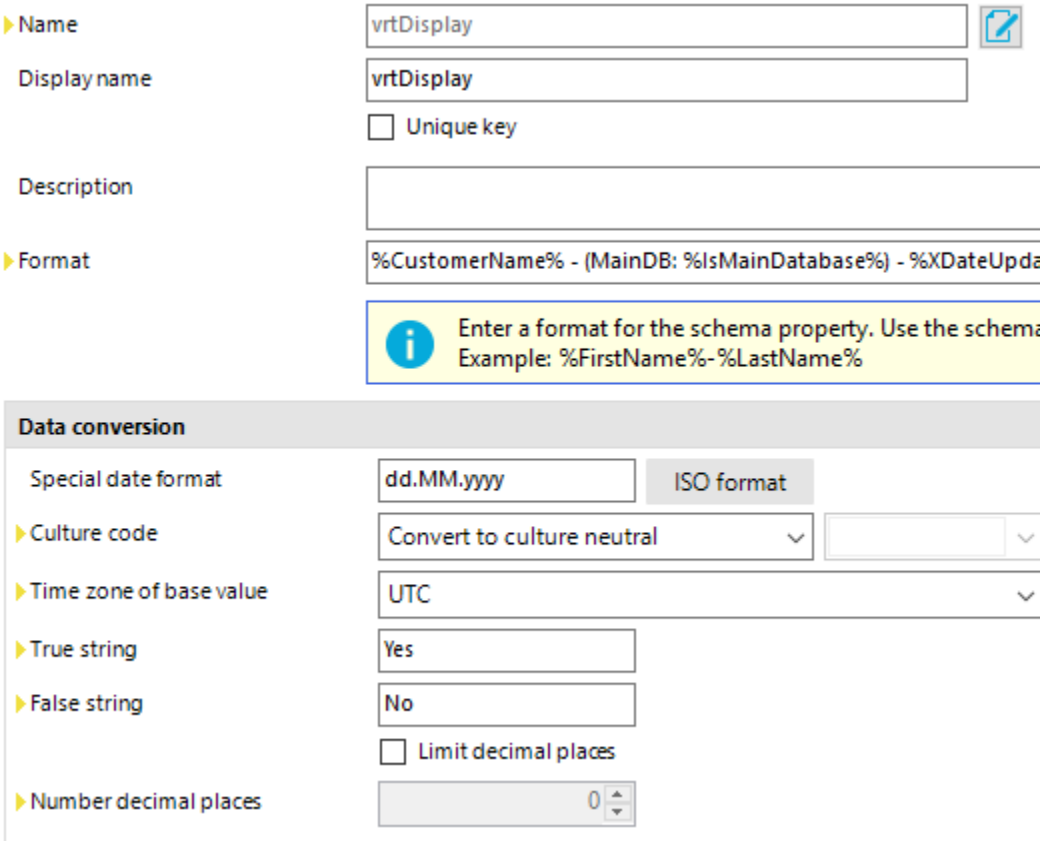



*Figure 143 Settings of a format defined property*

Besides the default settings such as Name, Display name and Description, the following configuration options are available:

| Option/Setting | Description |
|---|---|
| Unique key | Defines that the resulting virtual property uniquely identifies a single object in the system |
| Format | Contains the pattern that is used to populate the property value. The names of the used schema properties need to be enclosed in % characters. |
| Special date format | If datetime values are used in the format, this setting specifies the datetime format that should be used in the resulting string |
| Culture code | Specify the culture that should be used for data conversion. This also applies to used properties of type float, decimal, datetime and integer |
| Time zone of base value | Specify the time zone, date values shall be converted to. |
| True string | Value that shall be displayed if a Boolean true becomes part of the property value |
| False string | Value that shall be displayed if a Boolean false becomes part of the property value |
| Number decimal places | The maximum number of decimal places if a decimal or float value becomes part of the property value |

*Figure 144 Data conversion options*

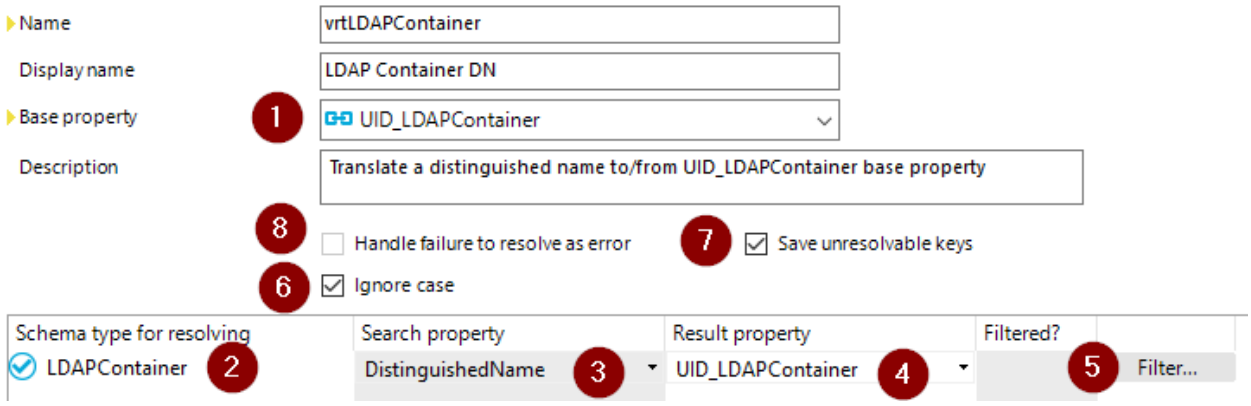
Hint: You can only use single valued properties in the format pattern of a format defined property. If you need to use a multi-valued property value you can create another virtual property based on the mvp converter and use this virtual property in the format defined property.

## Key resolution

The key resolution property, when written, uses a given value to search for a referenced object. It searches through the configured target schema types one by one, and looks, if it can find the value in the specified search property of the target schema type. If found, the result property is used to populate the base property the virtual property operates on. Reading a key resolution property works vice versa.

The main use case for such a virtual attribute is the foreign key handling in OneIM. A common example is the translation of a reference value (i.e. a distinguished name) into a XObjectKey (for dynamic foreign keys) or a UID (for regular foreign keys).

The following screenshot contains the settings of the key resolution virtual property.




Besides (display-)name and description, the following settings are available

| Setting | Description |
|---|---|
| Base property (1) | Selection of a property of the schema type you actually want to set. |
| Schema type for resolving (2) | This list contains the potential lookup schema types (schema types, the base property can point to) |
| Search property (3) | When the value of the key resolution property is written, this value is searched for in the search property of the potential target schema type. When the value of the key resolution property is read, the content of this property of the referenced object is returned. |
| Result property (4) | When the value of the key resolution property is written, the result property of the potential target schema type will be written to the base property. When the value of the key resolution property is read, the base property value will be used to search in the result property of the potential schema type to obtain the original value. |
| Filter (5) | To limit the set of potential objects of a target schema type, you can set a filter. This filter works the same way as the schema class filter of the generic schema class. |
| Ignore case (6) | Defines that the searches performed in the potential schema types are to be executed case insensitive. |
| Save unresolvable keys (7) | Defines that if a key cannot be resolved to an object of the type configured for |

| | |
|---|---|
| | resolution, then the key is nevertheless stored in the datastore for references to prevent data loss. |
| Handle failure to resolve as error (8) | Defines that unresolvable keys are to be treated as critical error. This will cause a synchronization to step independent of the configured ExceptionBehavior. |

The following illustration shows the schematic data flow, when a key resolution property is written.

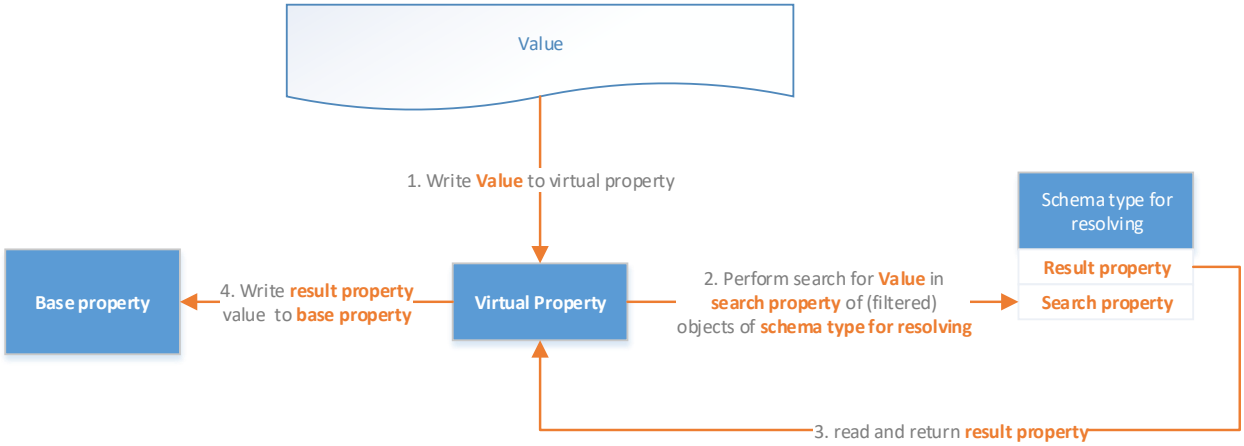



*Figure 145 Writing a key resolution property (schema)*

Example: Writing a distinguished name of a container to a virtual key resolution property that fills UID_LDAPContainer accordingly.

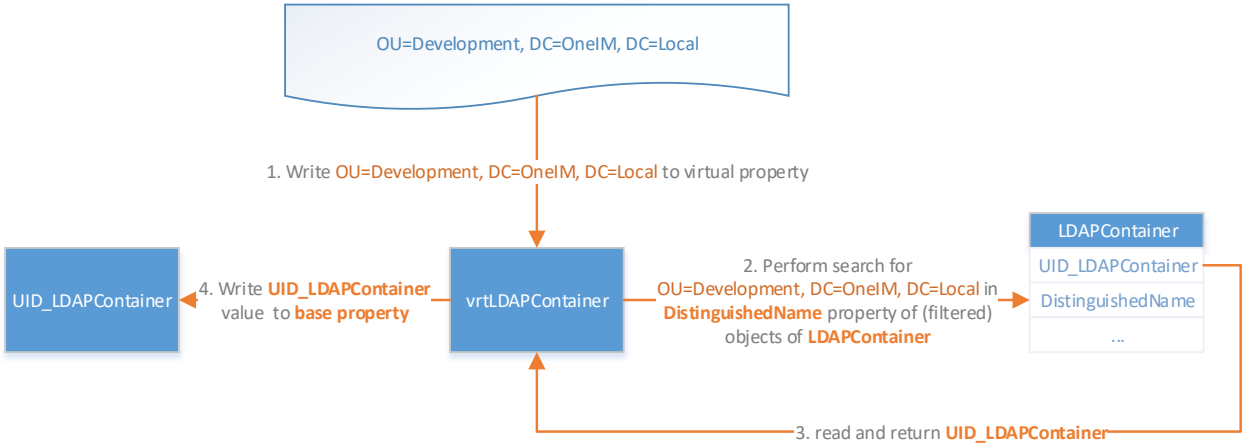



*Figure 146 Writing a key resolution property (example)*

As described, reading a key resolution property works vice versa.

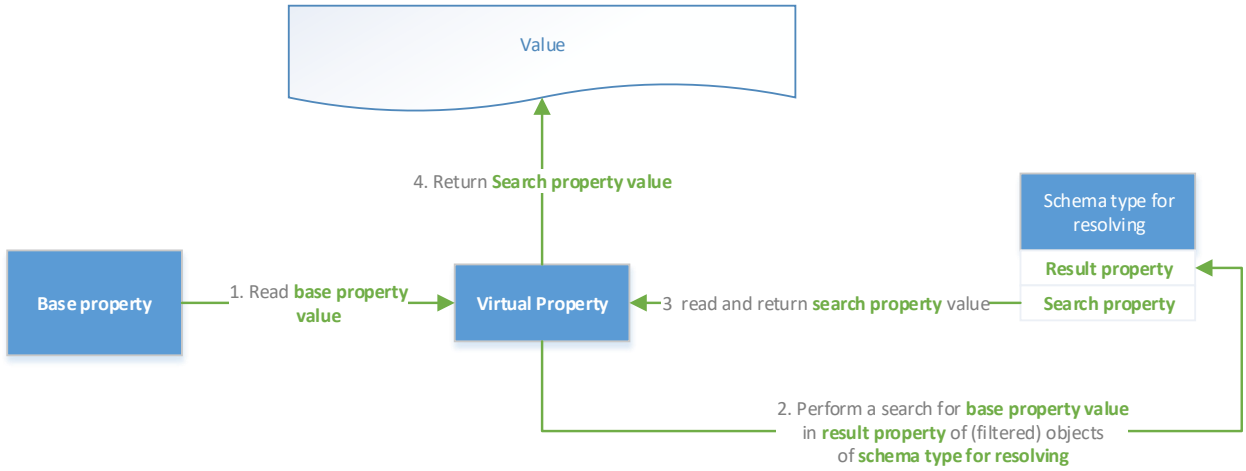



*Figure 147 Key resolution property read (schema)*

And the example accordingly

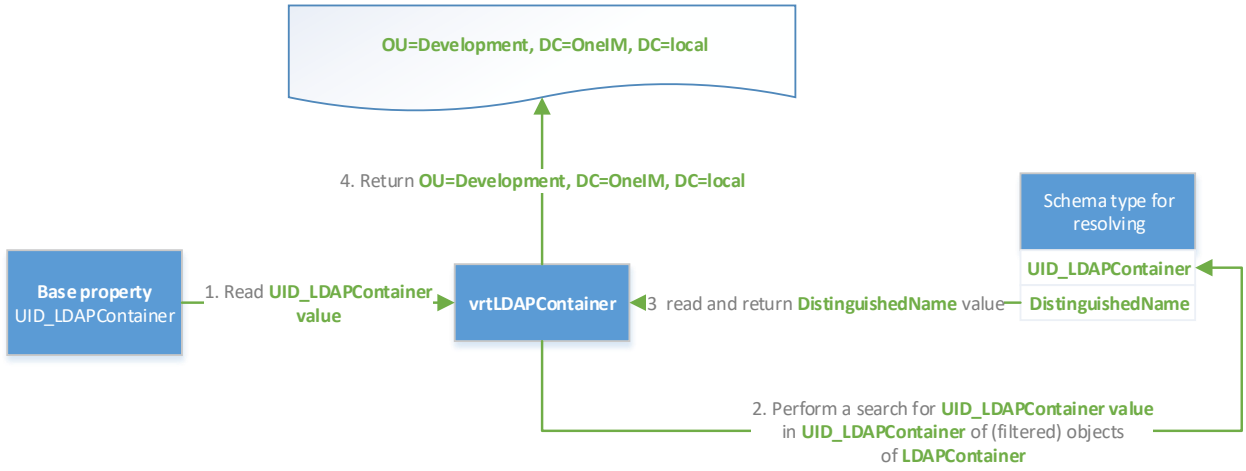



*Figure 148 Key resolution property read (example)*

## Key resolution by reference

The key resolution by reference is very similar to the key resolution property. This property can only be used for regular "foreign key" references that have exactly one target schema type. Since it uses the reference information exposed by the schema you just need to specify the base property and the search property. The overall process of resolving data at runtime is the same as for the key resolution property.

The following configuration settings are available for the key resolution by reference virtual property:

| Setting | Description |
|---|---|
| Base property | Selection of a property of the schema type you actually want to set. |
| Search property | When the value virtual property is written, this value is searched for in the search property of the target schema type. When the value of the virtual property is read, the content of this property of the referenced object is returned. |
| Ignore case | Defines that the search performed in the potential target type is to be executed case insensitive. |
| Save unresolvable keys | Defines that if a given value cannot be found in the objects of the schema types configured for resolution, the value is stored in the datastore for references to prevent data loss. |
| Handle failure to resolve as error | Defines that unresolvable keys are to be treated as critical error. This will cause a synchronization to step independent of the configured ExceptionBehavior. |
| Filter | To limit the set of potential objects of a target schema type, you can set a filter. This filter works the same way as the schema class filter of the generic schema class. |

Example:

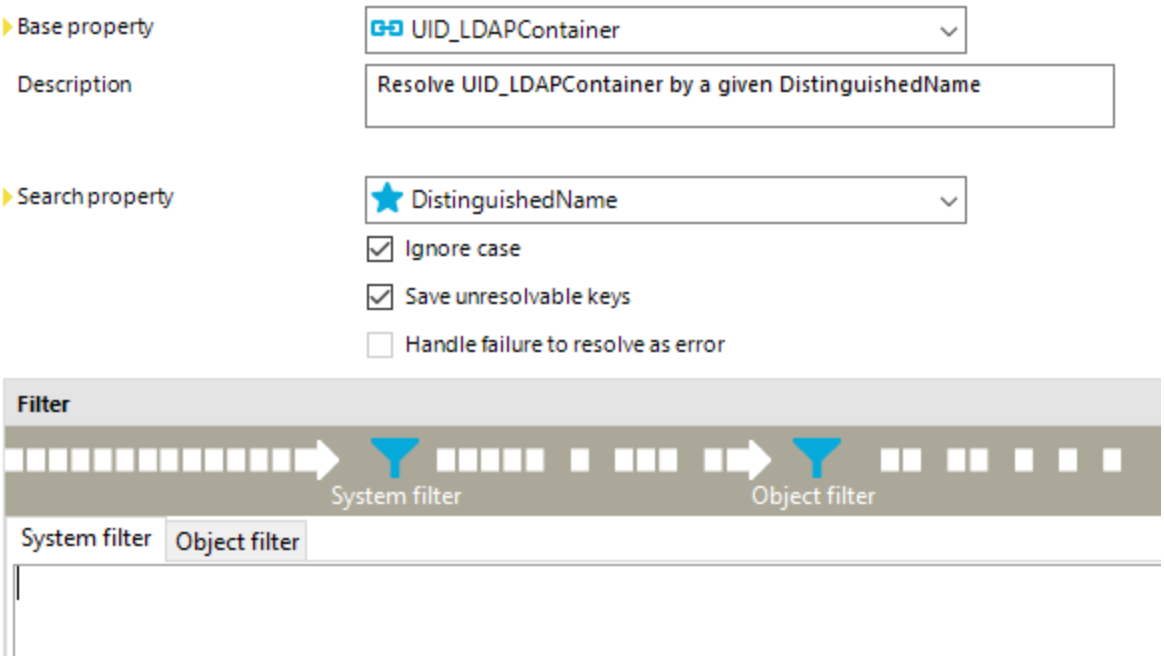



*Figure 149 Key resolution by reference example*

## Members of M:N schema types

There are usually two major concepts when a system exposes some kind of membership relation like users in groups. Database based systems usually use a M:N table that contains records, consisting of two references (the user and the group). So that means, memberships are represented as separate entities. In other systems however, such relations are exposed as a multi-valued property of an object. In LDAP for example, the "groupOfNames" schema type has a property called "member" that is contains the distinguished names of all group members.

When synchronizing data between two systems that are each using a different approach you would have to map a whole schema type (the M:N type) with the multi-valued reference property of the counterpart in the other system.

The following illustration should clarify the issue:

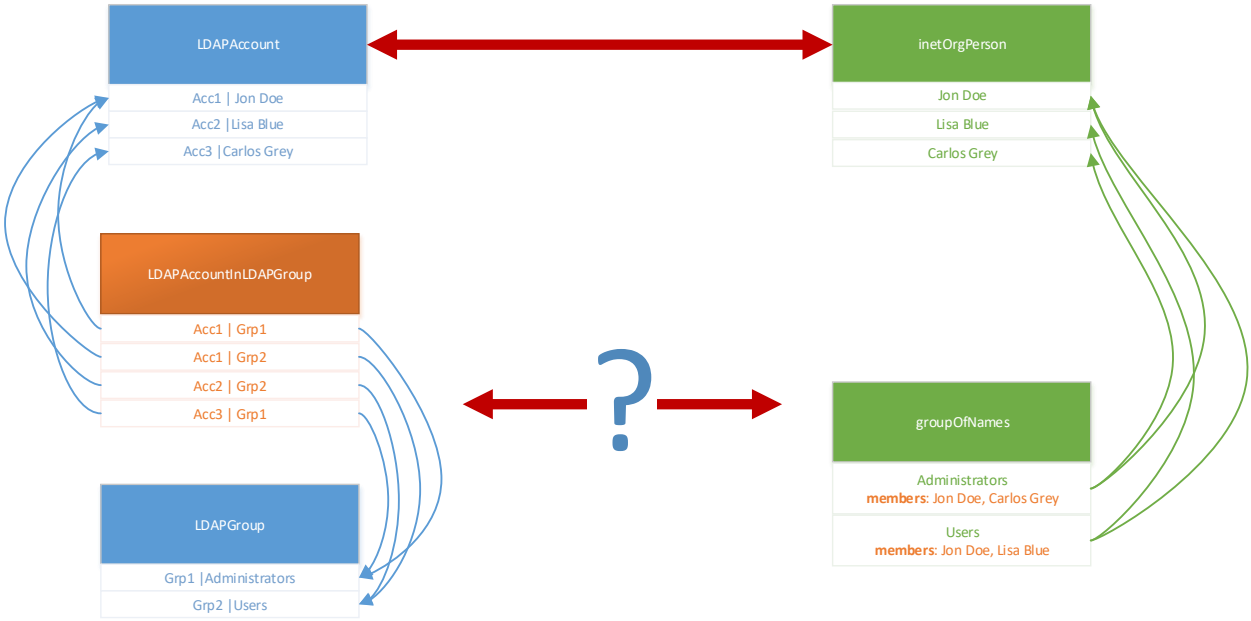



*Figure 150 Multi reference mapping issue*

To resolve this issue, the OneIM approach in this scenario is to simulate a multi-valued reference property for the system that has those references stored in a separate schema type. In the above scenario, this is the OneIM database that needs such a virtual attribute.

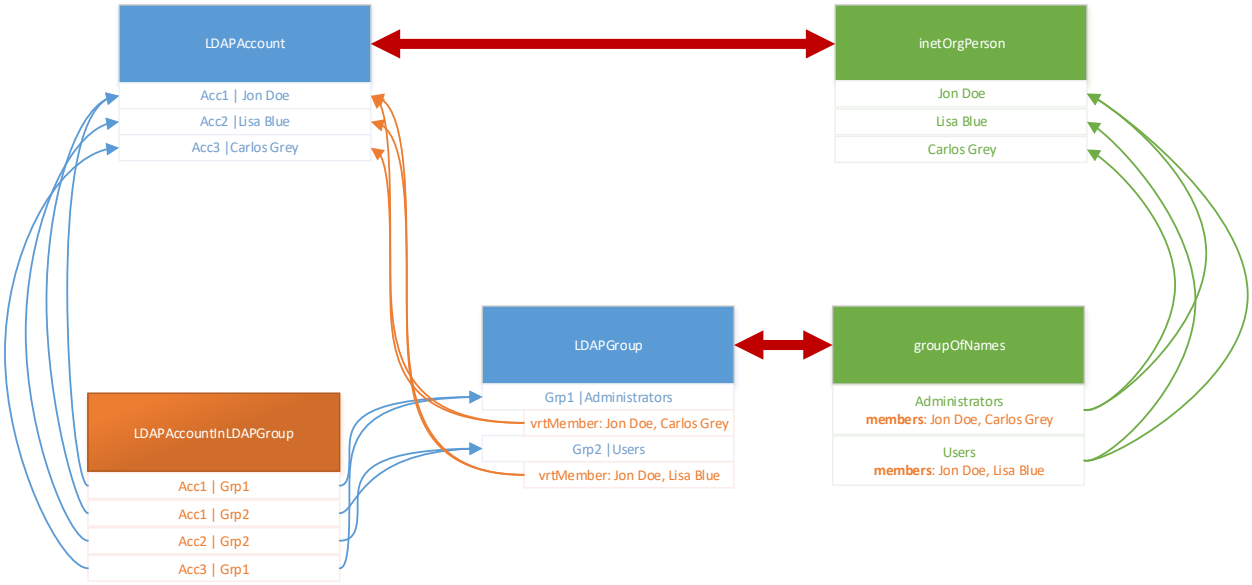



*Figure 151 Multi reference mapping solution approach*

As you can see, the LDAPGroup schema type now has the additional virtual property called "vrtMember". It contains the "member of the m:n schema type" (this is where the property name comes from). The virtual property uses the data stored in LDAPAccountInLDAPGroup

(the M:N schema type) to expose the data of this table an "own" property. This makes it compatible with the representation in LDAP so that LDAPAccount can be mapped to inetOrgPerson and LDAPGroup can be mapped to groupOfNames.

The "Member of M:N schema types" property can expose multiple M:N schema types via a single property. For LDAP, you would at least need the contents of LDAPAccountInLDAPGroup as well as LDAPGroupInLDAPGroup to create a virtual property that works similar to the actual LDAP representation.

## Configuration Options for the M:N Schema type

Now let us take a look at the configuration settings of this virtual property:

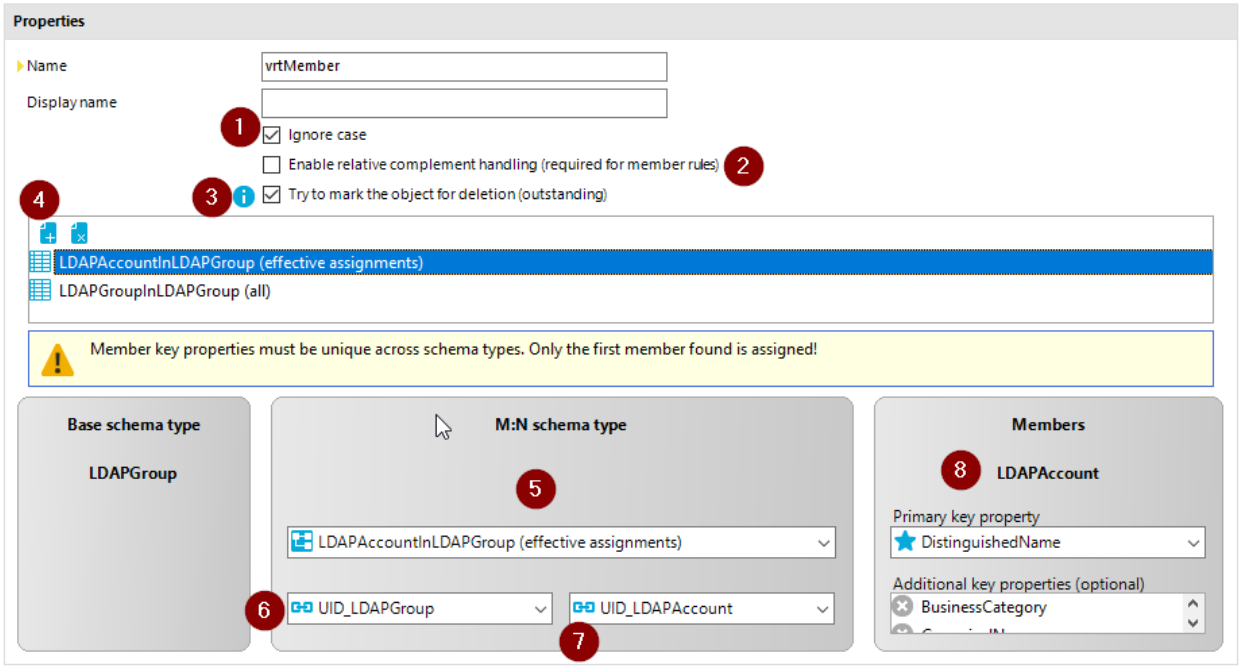



*Figure 152 "Members of M:N schema types" settings*

### Ignore Case

After specifying a name and a display name as for every other virtual property, you can enable the "Ignore case" option (1). This will make the engine perform comparisons (i.e. when searching for the primary key property (8)) case insensitive.

**Try to mark the object for deletion (outstanding)**

The "try to mark the object for deletion (outstanding)" has the effect that any value that is removed from the virtual property will not delete the object of the underlying m:n schema type but try to call a method "MarkAsOutstanding" if present. If the method is not present, a delete will be executed as fallback. Remember, this applies only to objects on the OneIM side: the point is that if memeberships are removed natively on the target system, then we see that in OneIM, allowing us to take action such as republishing the objects to the target system if we are authoritative. See also the section Enable Merging (Merge Mode)

The merge mode is a feature that allows the ad hoc projection to handle multi-valued references like group memberships more granularly. Without merge mode, the data situation in OneIM will be published to the target system as described in the "multi reference mapping rule" section. However, in many environments a parallel manual administration of the target system is in place. If an ad hoc provisioning job runs before a manual administration was synchronized to OneIM, the manual change would be reverted. To prevent this behavior, you can enable the "merge mode" for M:N tables in OneIM.

Hint: For out of the box target system modules (e.g. AD, LDAP) this setting is preconfigured. The default is that merge mode is enabled to not destroy manual changes in the target system.

By doing so the OneIM behavior is different as follows:

**Default Behavior**

A membership change in OneIM is either an insert or a delete operation in a M:N table. By default, this triggers a database internal process which increases the XDateSubItem value of the "hosting" object (i.e. the group). This increase is done via an update using the object layer which triggers the OneIM processing engine and ultimately an ad hoc provisioning process. Any multi reference mapping rule will process the data as described in the corresponding section which will overwrite all manual changes in the target system after the last synchronization.

**Merge mode behavior**

When merge mode is active in addition to the insert or update to the M:N table, OneIM stores the "change" in a dedicated table called DPRMemberShipAction. A change of XDateUpdated and therefore a provisioning process is generated too. When merge mode is enabled, the ad hoc projection task directly sends "add" and "remove" modifications to the

connector. By only executing the "add" and "remove" modifications of DPRMembersShipAction, any manual change done in the target system remains "as is". Once the ad hoc projection task is finished it clears the DPRMemberShipAction entries for the handled group object.

Hint: Group objects that still have unprocessed operations in DPRMemberShipAction will not be updated during full synchronizations since this would revert the unprocessed membership change in OneIM. Those group objects are mentioned in the synchronization log.

Hint: The majority of target systems is preconfigured to use the merge mode by default. Older versions 7.1.x did not enable the feature by default. After upgrading, you need to enable it manually

In case there are multiple operations for one group object, the maximum number of operations sent to the connector at a time is limited by the "MemberShipActionPartitionSize" parameter of the ad hoc provisioning task. If there are more than this number of changes to process, the connector will be called multiple times. If an error occurs during processing the partition size is decreased and the operations are retried (down to a partition size of 1). "Add" and "Remove" items that the connector was not able to publish will remain in the DPRMemberShipAction table. As described this blocks the synchronization of the group object until the error is resolved. To prevent endless blocking a daily cleanup process ("**DPR_MemberShipActions_RemoveOrphanedEntries**") will forcibly remove entries.

Mark objects as outstanding.

**Specifying the schema classes and schema types used in the mapping**
The list of M:N schema types that is handled by this virtual property can be edited in the section shown on the screenshot below.

Since mapping is always executed between schema classes instead of the types, you need to pick a class of a m:n schema type there. OneIM sometimes exposes two schema classes for m:n schema types:

**<SchemaType name> (all):** This is the master class containing all objects

**<SchemaType name> (effective assignments):** This class contains only those elements of the M:N schema type that are currently supposed to be "active" or "effective". Inactive m:n records can exist for relations referring to accounts that are currently deactivated by a business process in OneIM such as a temporary deactivation of an employee. They can be identified by the presence of the column "XIsInEffect". As a rule of thumb, you generally

want the synchronization framework to provision and synchronize the actual data in the systems. Therefore as a general rule, use the … *(effective assignments)* class if available.

To add a new m:n schema type to handle, click the "new" icon and proceed to area "M:N schema type" (5). Select the M:N schema class (type) you want to handle from the list.

The schema type must contain two references. In the list to the left (6), pick the reference property that is pointing to the object that you are currently configuring the virtual property for (this is referred to as the "Base schema type"). In the list to the right (7), pick the reference that is pointing to the actual member.

The next task is to configure the search attributes of the schema type of the potential members (8). Those are the properties of the target schema type that will be used for the actual object to reference to. You can specify multiple search properties but need to select one as primary. The property specified as primary property will be returned, when data is read from the virtual attribute. If the selected M:N schema type can point to multiple targets, search attributes need to be configured for each one.

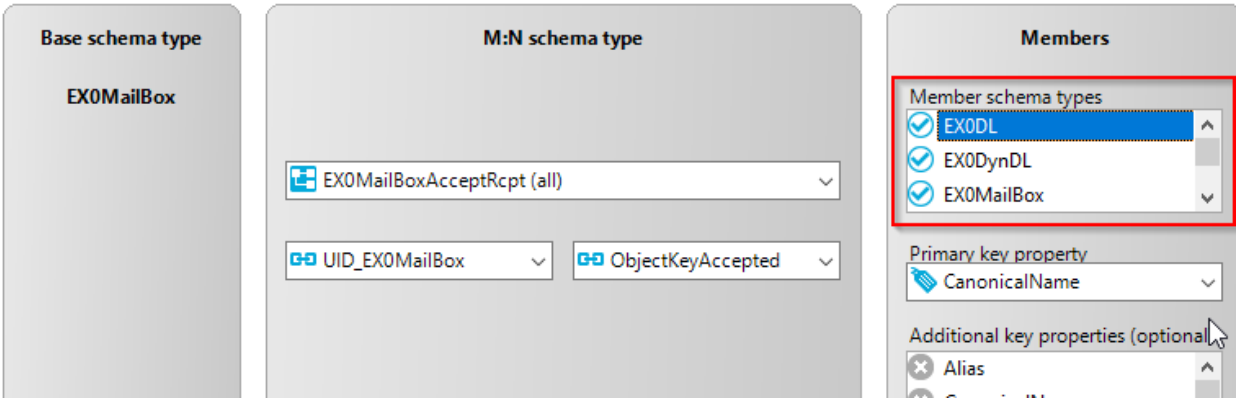
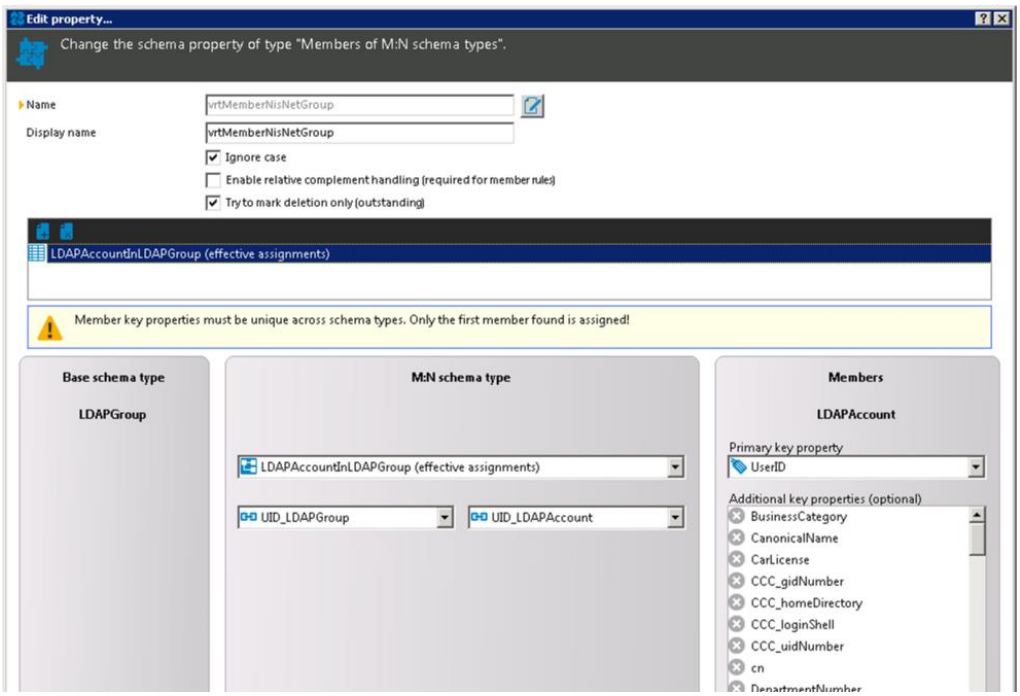



*Figure 153 M:N schema type (EX0MailboxAcceptRcpt) with multiple targets*

**Enable relative complement handling**

The last but very important setting is the "Enable relative complement handling" option. It basically defines the datatype of the virtual property. If not set, the property will be exposed as a multi-valued string property. If the option is set, the values read and written to the property are of type SystemObjectData. The use of SystemObjectData allows you to incorporate the virtual property in a "Multi reference mapping rule". This type of rule has some advanced features such as being able to add/remove single property values when

synchronizing instead of replacing all property values every time.




]

## MVP converter

MVP stands for multi-valued property. This converter allows a variety of operations that can be performed to handle those properties. Supported operations are:

## Extract single Value

This mode extracts a single value from a multi-valued property and exposes the result as virtual single valued property. The data type is defined by the base property. To configure the property this way, choose the option “Extract single value” (1) and select the base multi-valued property (2) you want to convert. Then, select the position of the item you want to expose (3+4).

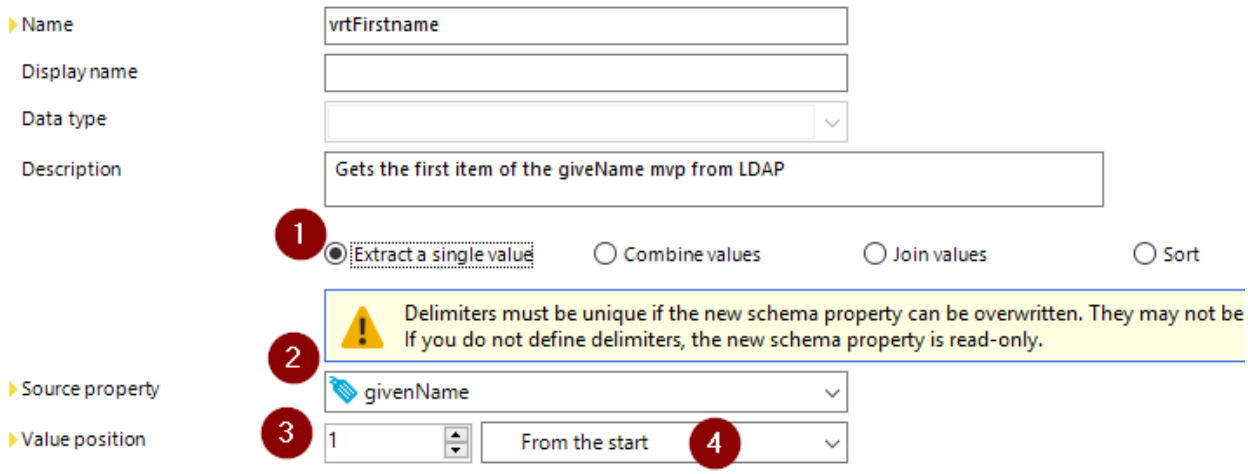



*Figure 154 Multivalued property extract single value*

## Combine values

This mode combines a set of single valued properties to one multi-valued property. Select mode "Combine values" (1) first. Then specify the properties (3) to combine along with their position (4) in the resulting multi-valued property as well as the data type (2) for the resulting property.

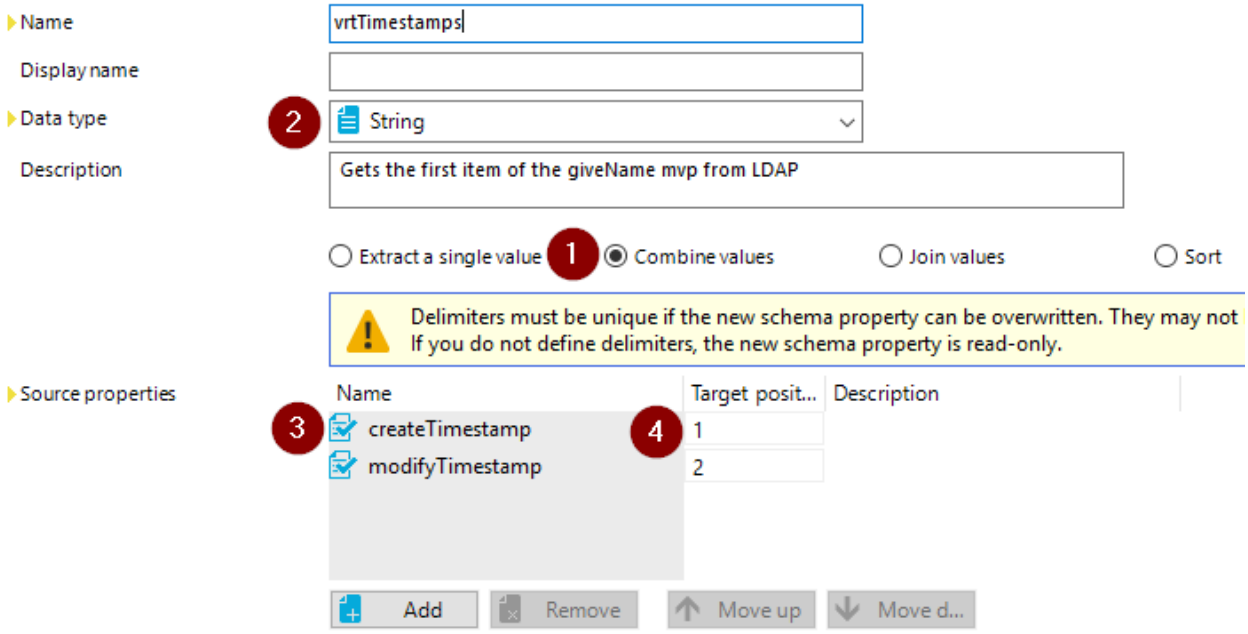



*Figure 155 Multivalued property combine values*

## Join values

The join values (1) mode is used to combine the values of a multi-valued base property (2) to a single valued property by appending each value divided by a given delimiter (3). Empty items can be ignored if required (4). If the base value is not of type string, a culture can be specified for the conversion. (5)

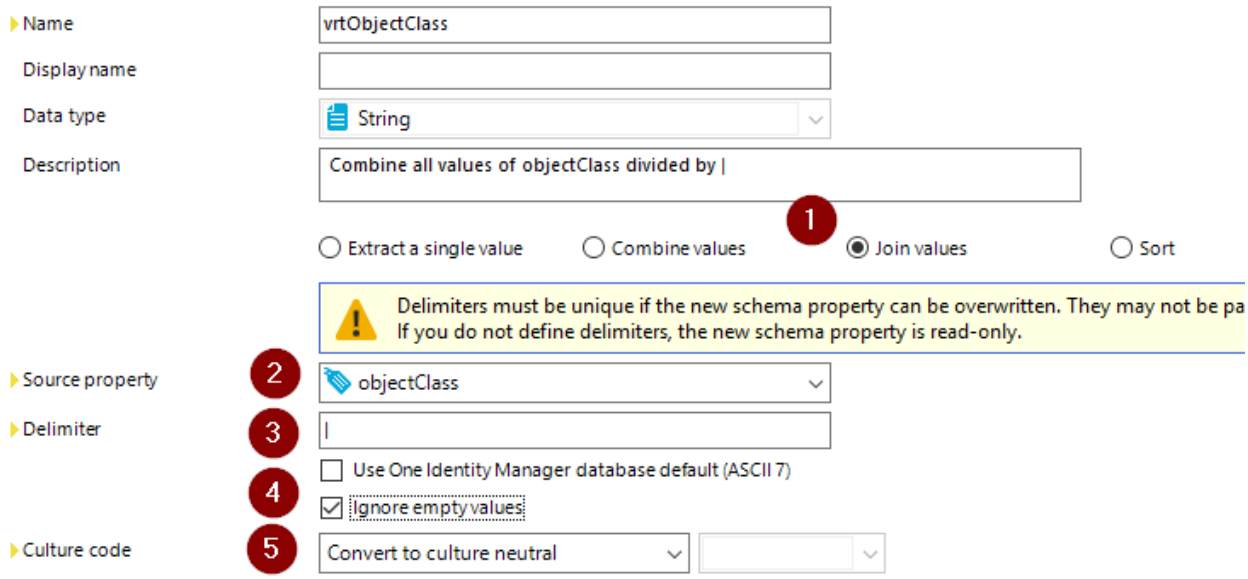
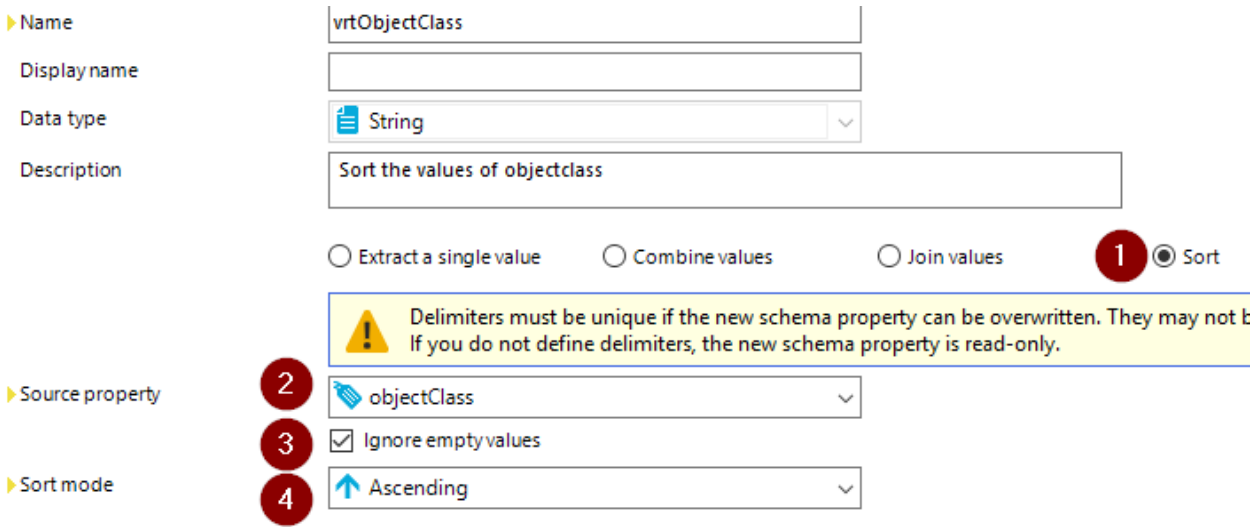



*Figure 156 Multivalued property join values*

## Sort

Sometimes it is required to have the values of a multi-valued property sorted. This mode (1) can be used to achieve this. It returns a multi-valued property as well. Specify the multi-valued property (2) to be sorted along with the option whether to ignore empty values or not (3). The last setting is the sort order (4).




## Object reference

The Object reference property works similar to the ObjectWalker in OneIM. Based on a given expression (route) it returns a value of a referenced object. Instances of this type of virtual property are always read only. The references used in the expression are also limited to those, who only have exactly one target defined. The route however can contain multiple hops.

The following example returns the "Locality long name" property of a person's department locality. The virtual property is defined for the schema type "Person".

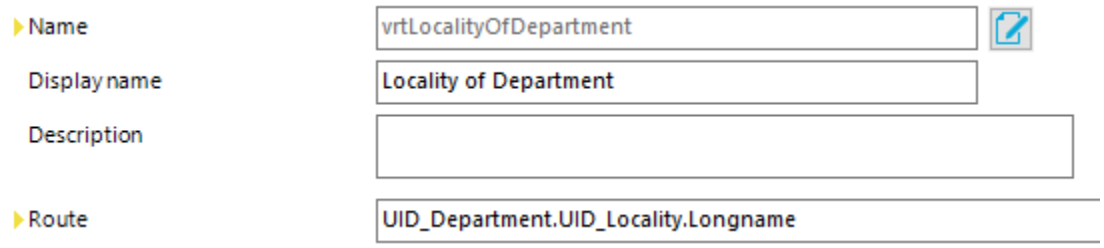



*Figure 157 Object reference property*

Since the property implementation needs to execute multiple potentially expensive queries, the extensive use of this property may have a performance impact.

## Property join

This virtual property implementation allows you to join several properties of a schema type in a given order. Those properties can be used as unique keys. The components are

concatenated and delimited by a given delimiter string or character. The resulting property is always of type “String”.

The following settings are available for this type of property

| **Option/Setting** | **Description** |
|---|---|
| Unique key | Defines that the resulting virtual property uniquely identifies a single object in the system |
| Delimiter | The string or character that shall be used to delimit the components |
| Properties to join | Contains properties that shall be joined together. The order is determined by the order in the table. It can be changed via “Move up” and “Move down” buttons. |
| Special date format | If datetime values are used in the format, this setting specifies the datetime format that should be used in the resulting string |
| Culture code | Specify the culture that should be used for data conversion. This also applies to used properties of type float, decimal, datetime and integer |
| Time zone of base value | Specify the time zone, date values shall be converted to. |
| True string | Value that shall be displayed if a Boolean true becomes part of the property value |
| False string | Value that shall be displayed if a Boolean false becomes part of the property value |
| Number decimal places | The maximum number of decimal places if a decimal or float value becomes part of the property value |

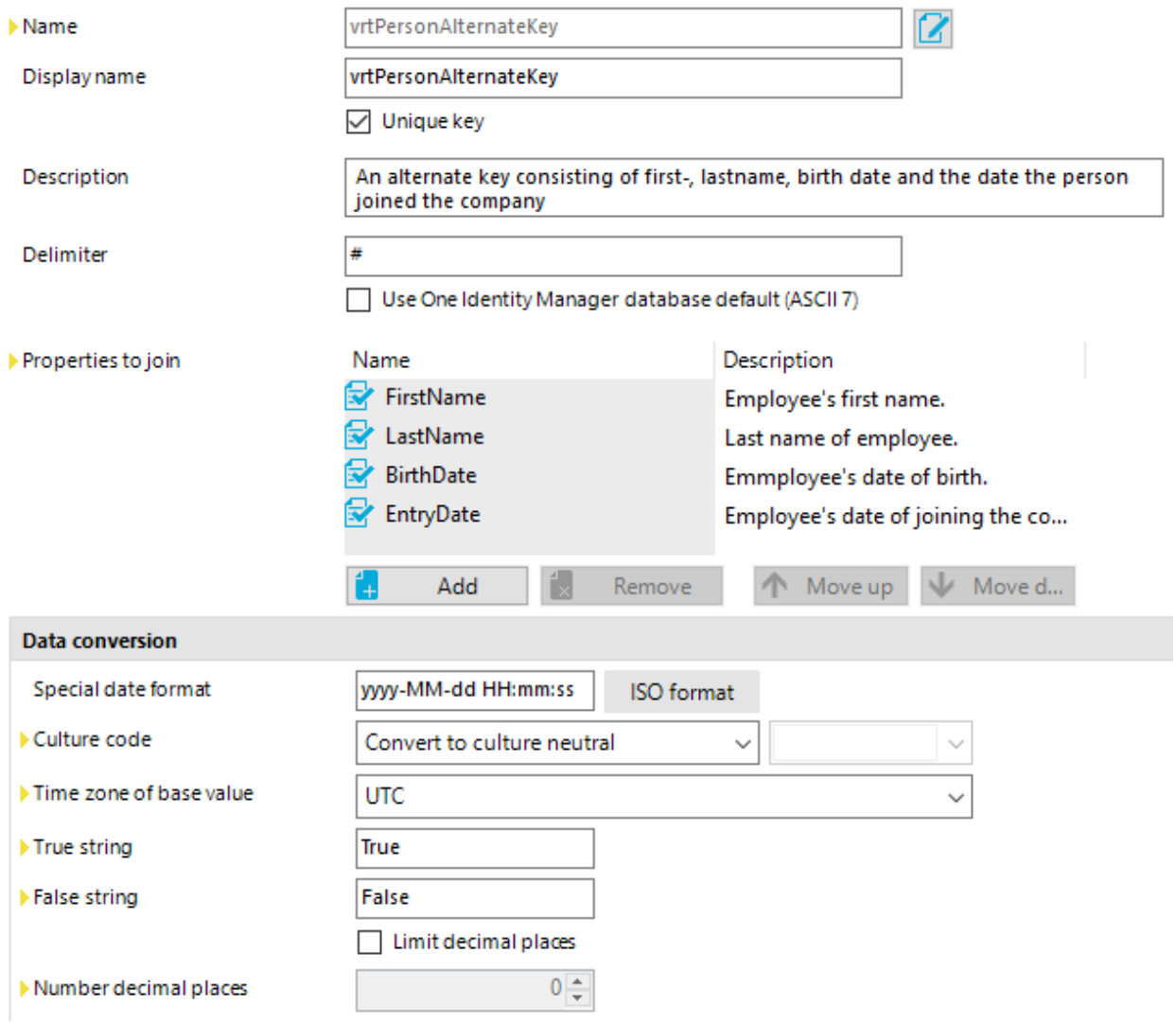



*Figure 158 Property join example*

## Script property

The script property can be used, if none of the other virtual properties can fulfill a given conversion task. It is very powerful since it offers the whole variety of the .NET framework features plus some language extensions only available within the synchronization framework. However, when using the virtual script property keep in mind that:

- In contrast to the other virtual property implementations, there is no optimization like caching etc. happening in the background for operations that are executed by the script property.
- Inefficient scripts can have a huge impact on the overall synchronization performance

When creating a script property you need to specify the data type of the property as well as whether it is multi-valued or not. Depending on your choice you can then implement a script for reading and writing the property value. Depending on the existence of the read/write script, the property will be exposed as read only/write only/read and write in the schema.

The script code itself needs to be written in C# or VB.NET depending on the scripting language choice that was taken during the synchronization project creation. Out of the box synchronization projects usually use VB.NET.

To load additional DLL files, use the `references` keyword.

Example:

References VI.TSUtils.dll

Imports VI.TargetSystem.Base.Utils.LDAP

## Read Script

The read script implements the code that is called when the property value shall be read. The script has the following signature

```
Public Function GetValue(
ByRef schemaProperty as ISchemaProperty,
ByVal options as SchemaPropertyGetValueOption) As YourSelectedDataType
```

The `schemaProperty` parameter can be used to access the containing SchemaType in the script if required (`schemaProperty.SchemaType`).

The `options` parameter is a data structure that contains information on "how" data shall be retrieved. The only relevant (non-internal) property of that structure is the `options.GetOldValueIfPresent` Boolean. It implies that the property shall calculate its value based on the old (original) values of the properties used in the script. Your script should evaluate this option and act accordingly.

> Hint: In some older scripts you will find the use of a `useOldValues` variable. This variable is obsolete but still available and contains the contents of `options.GetOldValueIfPresent`. You should always use the "options" instance in your scripts.

To access a property within a read script, use the $ notation `$PropertyName$`. To access the old value of such a property use `$PropertyName[o]$`. To automatically convert the datatype, you can use the following notation `$PropertyName:DataType$` i.e `$PropertyName:Int$`. The list of supported datatypes for this notation is `Int`, `Bool`, `Float`, `Decimal`, `DateTime`, `String` and `Binary`.

You can also can combine data conversion and oldvalue usage as follows:
`$PropertyName[o]:String$`

The following example calculates a display name of a person, based on first- and last name.

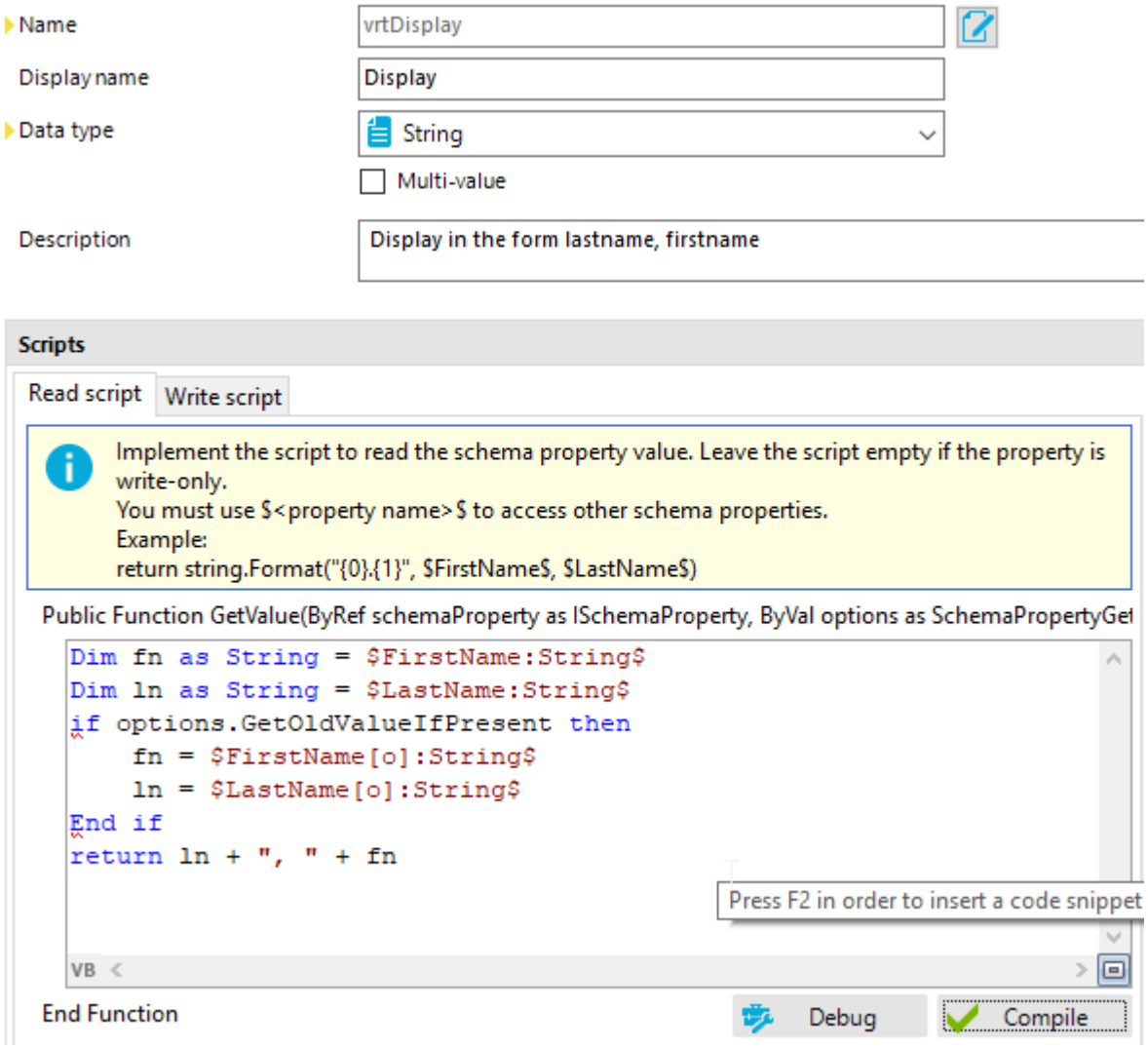



*Figure 159 Script property example (read)*

Hint: The "Compile" button can be used to check your script for syntactical correctness.

## Write Script

The write script implements the routine for writing data to the virtual property. If it is not present, the virtual property is marked as read only. The signature of the method is:

```
Public Function SetValue(
ByRef schemaProperty as ISchemaProperty,
ByRef value as YourSelectedDataType,
ByRef options as SchemaPropertySetValueOption)
```

Again, the `schemaProperty` parameter can be used to access the containing SchemaType in the script if required (`schemaProperty.SchemaType`). The `options` parameter is present, but empty in preparation for possible upcoming features.

For accessing (reading) other properties of the schema type, the same rules as for the read script apply. To actually write a value to a property, the := operator was introduced and has to be used as follows:

$PropertyName$ := <value>

The following write script example implements the "write" operation for the corresponding read script example of the previous section.

Hint: A more complex example for the use of a script property can be found here https://connect.oneidentity.com/products/identity-manager/w/knowledge-base/503/how-to-use-a-script-property-for-a-conditional-mapping

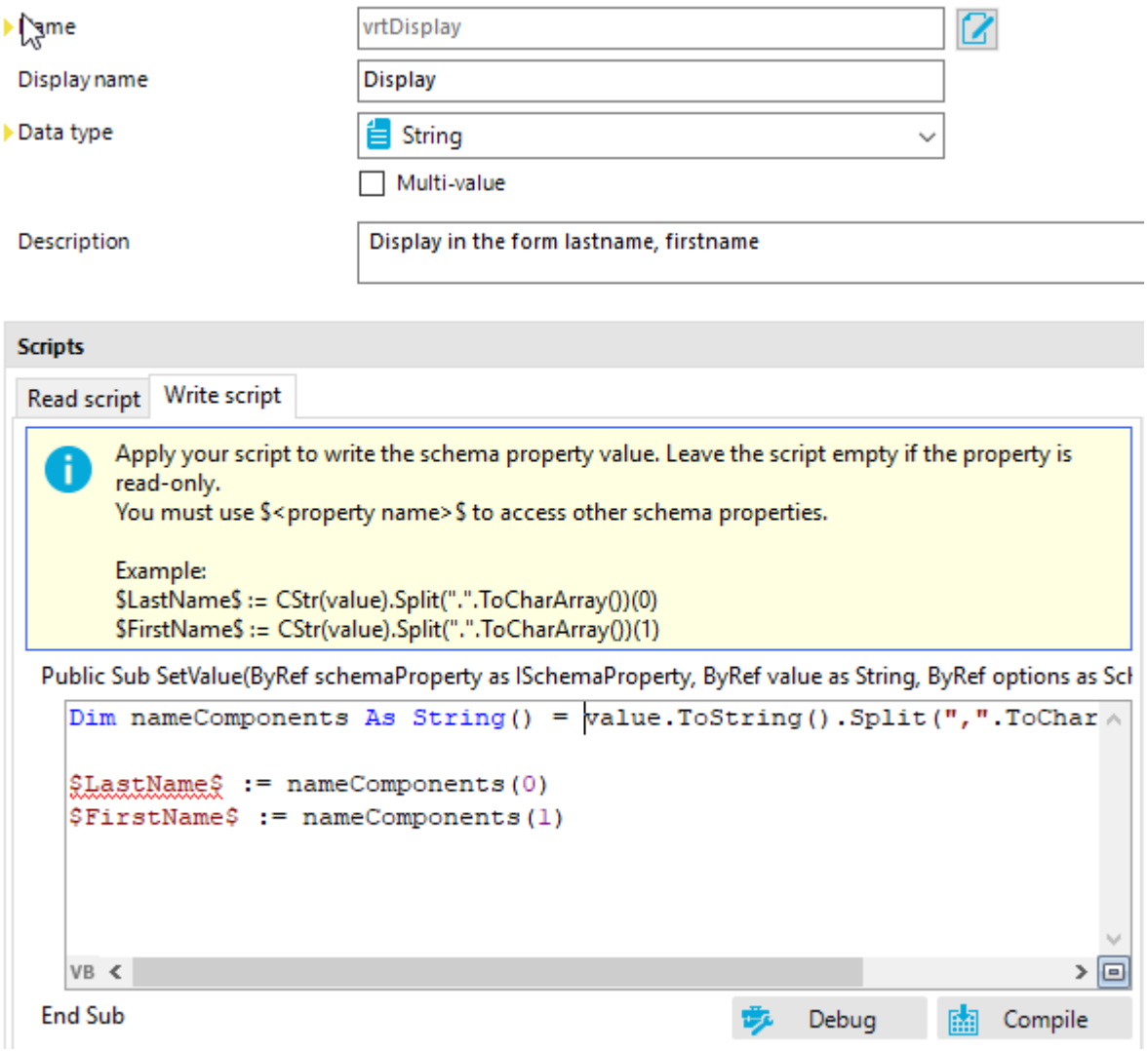



*Figure 160 Script property example (write)*

## Debugging scripts

To use the "Debug" function next to the "Compile" button, you will need an installed Visual Studio on the machine that is running the Synchronization Editor. When activated the Synchronization Editor will create a debug solution in the <user>\Documents\OneIM.Projector.Scripting directory and launch Visual Studio. If such a solution already exists, the Synchronization Editor will overwrite it.

Hint: Make sure to save/commit your virtual property to database before debugging. Otherwise debugging will not work.

When Visual Studio is starting, you might get a message regarding not consistent line endings. That can be confirmed.

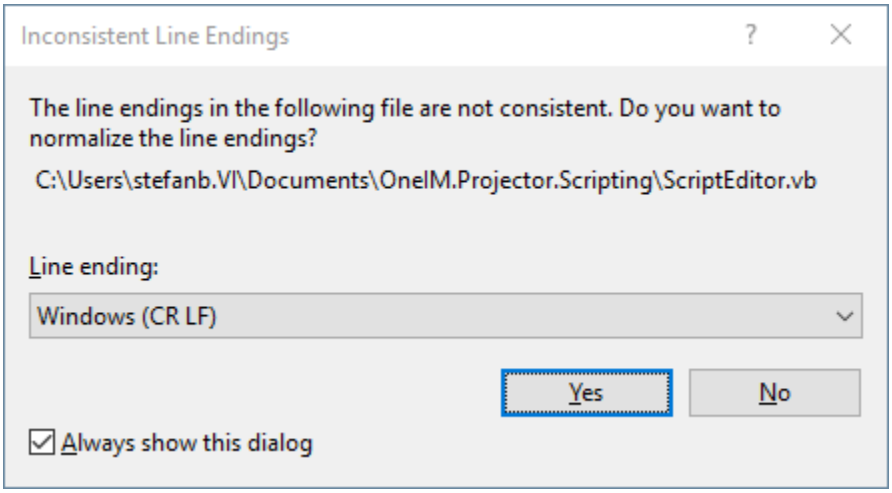



*Figure 161 Line endings popup*

Once started, depending on your chosen script language you will see a ScriptEditor.vb/cs file containing the scripts of your synchronization project. You can now set breakpoints and use the "Debug" → "Start debugging" menu to launch a new instance of the Synchronization Editor. Now perform an operation that will cause the script to fire (i.e. open a virtual property in the target system browser) and your breakpoint should trigger.

Note that the script code in ScriptEditor.vb/.cs will contain replacements of the property access expressions (`$<PropertyName>$`, `:=`, `[o]`, `:)`. The `useOldValues` alias is also declared.

Writing back/editing scripts from Visual Studio to the Synchronization Editor is not supported.

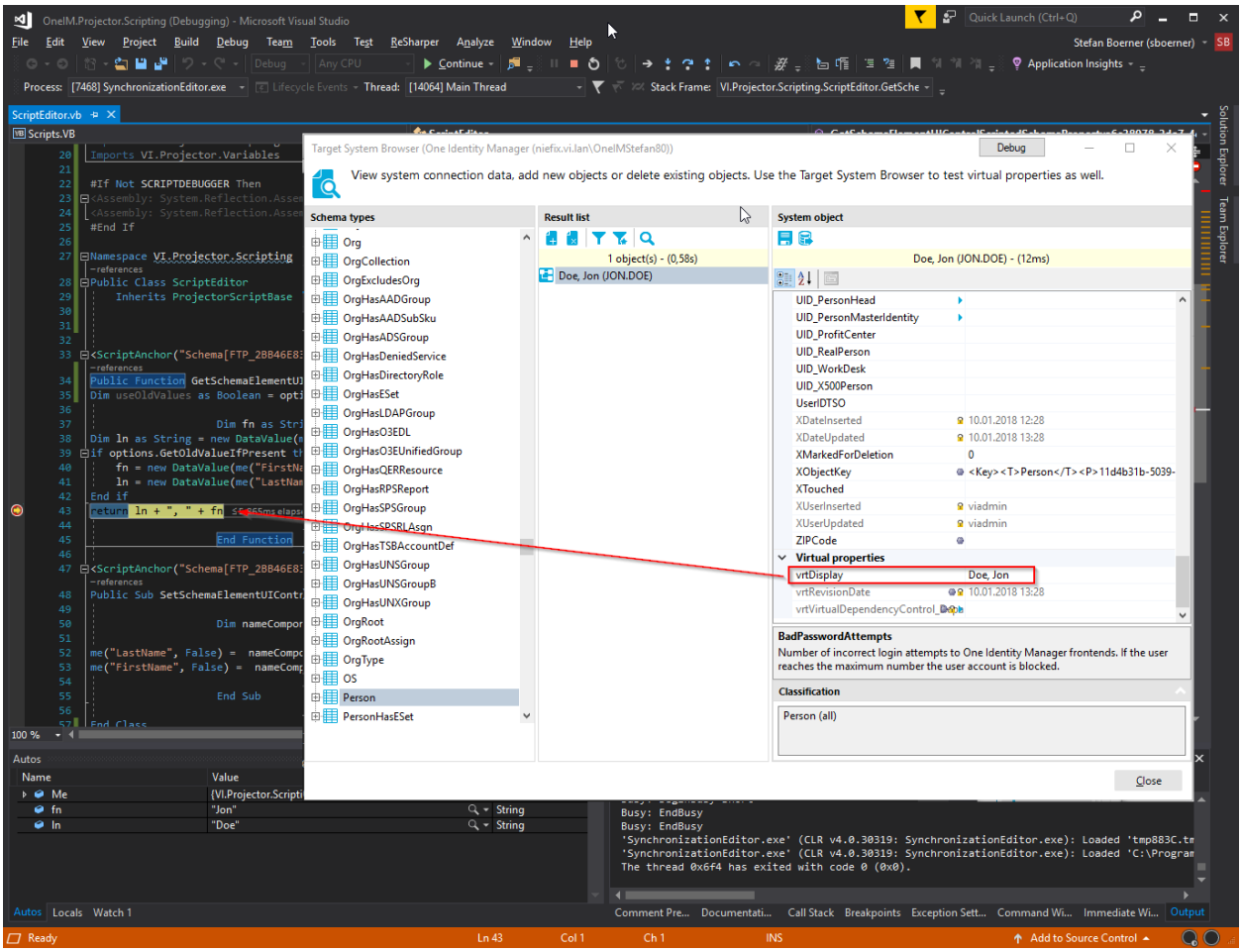



*Figure 162 Debug a scripting property*

# Dialog scripts

This section contains information about the OneIM scripts that are involved within synchronization processes. They are mainly used for interaction between the OneIM processing engine and the synchronization engine.

## DPR_FormatConditionValue

This script helps formatting values that are used in the CheckCondition task of the ProjectorComponent to escape characters that otherwise would cause an invalid condition Syntax.

### Input arguments

| | |
|---|---|
| Input as String | The value to be converted |

**Output**

A string containing the formatted value

## DPR_GetAdHocData

This is the core script used during ad hoc provisioning processes in OneIM.

**Input arguments**

| | |
|---|---|
| rootObjectkey as String | The XObjectKey of the "Domain" or "System" the object that gets a process generated resides in |
| targetSystemType as String | The identifier (type) of the system we want to provision to. The system type is exposed by the connector as part of the schema. For LDAP it is "LDAP". The exact system type identifier can be found in the Synchronization Editor by navigating to "Configuration" → "Target system" → "Schema browser". Click the root node of the schema tree and review the property "System". This string contains system information exposed by the connector in connection string syntax.<br><br>Example:<br>`DisplayName="Lightweight Directory Access Protocol";Id="dc=OneIM,dc=local@vidrn303.vi.lan:389";Type=LDAP;SubType=;Version=1;Capabilities=SupportsRevisions`<br><br>The key/value pair "Type" contains the string that needs to be passed for this parameter. |
| targetSystemVersion as String | An additional version for the target system type if available. For SharePoint for example, the target system type is "SharePoint" and the available versions are "2010", "2013" and "2016"<br>The version can also be found in the "System" attribute of the schema as described above. The key/value pair "Version" contains the version information. Note that connectors usually expose some kind of (generic) version information. Only pass a value for this parameter if multiple versions are available. |
| operationName as String | The name of the provisioning process operation that shall be executed |
| baseObject as IEntity | The OneIM a process shall be generated for |

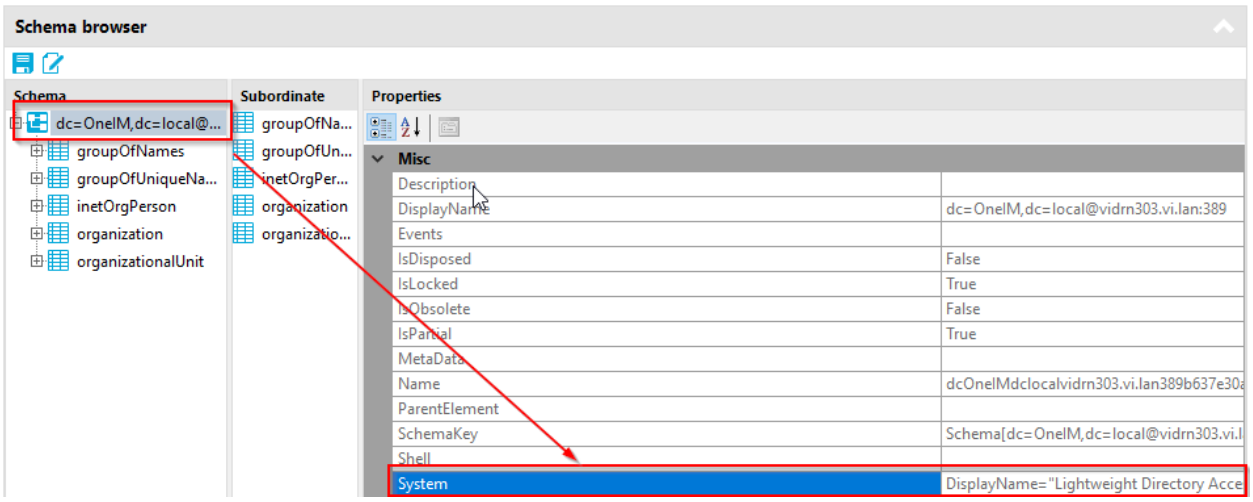



*Figure 163 System information*

**Output**

A dictionary containing the following items

| | |
|---|---|
| ProjectionConfigUID | The ID of the workflow that needs to be executed according to selected provisioning object operation |
| VariableSetUID | The ID of the variable set in the synchronization project that needs to be used |
| ExecutionServerUID | The ID of the OneIM processing server that is configured to process the operation |

**Further information**

During the generation of OneIM processes, it searches the appropriate synchronization project instance and workflows that have to be used to publish changes for a given OneIM object. To gather this information it needs to find the appropriate base object first (based on the parameters rootObjectKey, targetSystemType and targetSystemVersion. Once a base object is found it continues to search the provisioning process operation based on the target connection (stored in the base object), the operation name passed (parameter parameterName) and the OneIM table of the entity that the OneIM process is currently generated for (parameter baseObject).

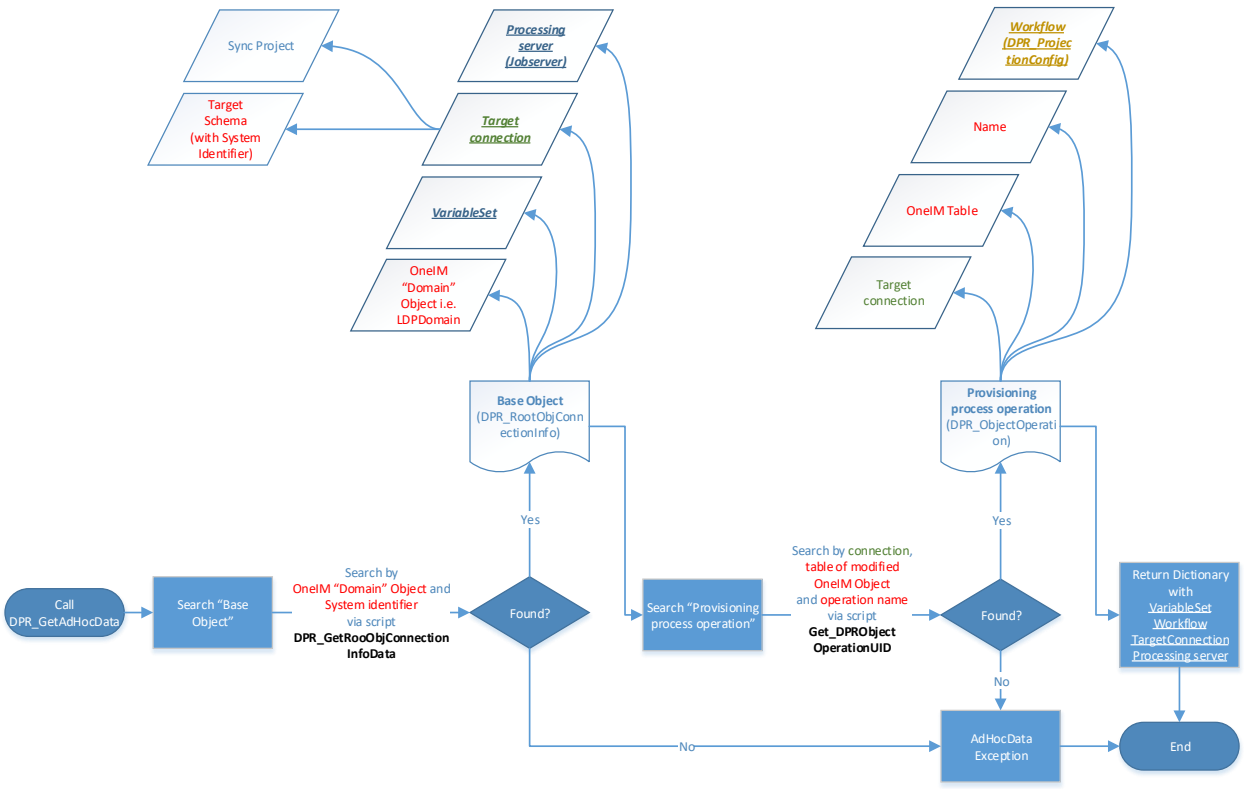



*Figure 164 DPR_GetAdHocData flowchart*

# DPR_GetDPRObjectOperationUID

This script is used by the DPR_GetAdHocData script and returns a provisioning process operation.

### Input arguments

| | |
|---|---|
| tablename as String | Name of the DialogTable |
| operationName as String | The name of the provisioning process operation that shall be executed |
| uidDPRSystemConnection as String | ID of a target system connection |

### Output

The ID of the provisioning object operation.

## DPR_GetDPRRootObjectConnectionInfoUID

Searches the base object for the given OneIM target system root (i.e. a domain). Since a target system root can be used in multiple synchronization projects (if they connect to different systems!), the type of system need to be specified as well. This script is used in DPR_GetAdHocData.

**Input arguments**

| | |
|---|---|
| xObjectKeyRoot as String | OneIM targetsystem root object XObjectKey (i.e. of a domain) |
| systemType as String | The target system identifier (i.e. "ADS" or "LDAP") to be used for searching a synchronization project. |
| systemVersion as String | Some connectors are applicable for different versions of a target system (i.e. SharePoint). In this case, the version need to be specified (i.e. "2010", "2013" …) |

**Output**

The ID of the base object.

## DPR_GetVariableValue

Returns the value of a variable of the given variable set.

**Input arguments**

| | |
|---|---|
| UID_DPRSystemVariableSet as String | The ID of the target variable set |
| VariableName as String | The variable name |

**Output**

The value of the variable.

## DPR_NeedExecuteWorkflow

This script is used during process generation in OneIM and determines for a given OneIM entity, if the changes that has been done it require the given workflow to be run in order to publish the changes to the system.

**Input arguments**

| | |
|---|---|
| uidWorkflow as String | The ID of the Workflow to be used in the check |
| target as IEntity | The OneIM entity to check |
| ignoreColumns as String() | Columns that shall not be taken into consideration |

**Output**

True, if the workflow needs to be executed, otherwise false.

## DPR_WrapObjectForProjection

Creates the XML representation of an entity (patch) to be used in an AdHocProjection task.

**Input arguments**

| | |
|---|---|
| sourceEntity as IEntity | The OneIM entity to create a XML patch for |
| includeColumns as String() | Columns that shall be included independent of the fact if they were changed or not |

**Output**

The XML representation (entity patch) of the OneIM entity

## DPR_GetColumnsHandledByWorkflow

Returns the columns of a given OneIM table that are used in the given workflow.

**Input arguments**

| | |
|---|---|
| uidProjectionConfig as String | The OneIM entity to create a XML patch for |
| includeColumns as String() | Columns that shall be included independent of the fact if they were changed or not |

**Output**

The XML representation (entity patch) of the OneIM entity

## DPR_Append2ConnectionString

Extends an existing connection string by a given key/value pair. Those strings are used in multiple places throughout the synchronization processes. The "OverrideVariables" parameter of the AdHocProjection i.e. need to be specified in connection string syntax.

**Input arguments**

| | |
|---|---|
| currentString as String | The current connection string |
| theKey as String | The key of the key/value pair to add |
| theValue as String | The value of the key/value pair to add |

**Output**

The modified connection string.

# Further Reading/Troubleshooting

As you can see, synchronization in OneIM is a very complex topic. There are a lot of resources regarding this technology. Some of them (and where to find them) are mentioned in the following list:

1. **Official OneIM documentation**

The official documentation of OneIM is available offline on your installation media or online via the support portal.

https://support.oneidentity.com/identity-manager/8.0/technical-documents

The "Target system Synchronization Reference Guide" contain general information about the synchronization technology. There are also sections available regarding topics that were not covered in this document such as patching synchronization projects.

Furthermore, for each target system module you will find information on the installation prerequisites (software, users, rights etc.) as well as a guide on how to configure the synchronization based on the out of the box synchronization templates.

> Hint: It is highly recommended to read through the module manual for the target system you are going to connect to OneIM to successfully plan and deploy the synchronization components

2. **One Identity Manager Knowledge Base**

https://support.oneidentity.com/identity-manager/kb

The knowledge base contains articles regarding the whole OneIM product. Articles are often inspired by customer requests/issues.

3. **One Identity Manager Forum**

https://www.quest.com/community/products/one-identity/f/identity-manager

The public One Identity Manager forum is the place for users of OneIM to get in touch with each other and discuss deployment scenarios as well as issues/challenges regarding the product. This forum is available to everyone.

4. **Connect**

https://connect.oneidentity.com/

The "Connect" platform is a community website for One Identity employees, partners and registered customers. Besides a forum it also contains a wiki with content contributed by OneIM users as well as a "Best practice" section.

Hint: The following content is very valuable when for troubleshooting synchronization issues: https://connect.oneidentity.com/products/identity-manager/w/knowledge-base/296/troubleshooting-synchronization-projects

5. **One Identity Youtube Channel**

https://www.youtube.com/channel/UCCDrBomtYlPFM1coxYknuuA

The OneIM youtube channel offers a variety of video tutorials for all OneIM product features. Although there is no dedicated section or playlist for synchronization framework videos, there are a lot of such videos out there.

# Glossary/Terminology

This section contains the terminology used in the context of the OneIM synchronization framework. Due to the history of the product, there are several synonyms for the same thing. Further terms are introduced in the sections they are first mentioned in.

**Synchronization**

A bulk transfer of data between two systems done by the synchronization engine. If not explicitly specified it refers to the bulk transfer of data from a target system to One Identity Manager.

**Provisioning, Ad hoc operation**

The process of modifying a single object in the target system based on a change in OneIM (e.g. insert/update/delete an entry).

**Left-hand side**

A synchronization always involves two systems. One of them is OneIM which is displayed on the left side when creating mappings in the Synchronization Editor. When it comes to specifying the direction of a data transfer operation, "*Left-hand side*" or "*to the left*" means to or in One Identity Manager.

**Right-hand side**

The counterpart to Left-hand side. It refers to the target system that is involved. When it comes to specifying the direction of a data transfer operation, "*Right-hand side*" or "*to the Right*" means to or in the target system.

# Index